DEVOPS ENGINEERING MASTERY

OBSERVABILITY ENGINEERING

Prometheus, Grafana, Logging, Distributed Tracing, Alerting & Production Monitoring

VOLUME 10 • FIRST EDITION

```
nahid@technova:~$ curl -s localhost:9090/-/healthy
✓ Prometheus is Healthy
nahid@technova:~$ promtool check config prometheus.yml
✓ SUCCESS — 12 rules loaded
nahid@technova:~$ rate(http_requests_total{status=~"5.."}[5m])
```

Prometheus • Grafana • Loki • OpenTelemetry • MERN

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Observability Engineering

DEVOPS ENGINEERING MASTERY — Volume 10

Prometheus, Grafana, Logging, Distributed Tracing, Alerting ও Production Monitoring

First Edition — Published 9 November 2026.

Hands-on environment: Prometheus, Grafana, Loki, Grafana Alloy, OpenTelemetry Collector, Jaeger and Grafana Tempo, with Node.js 20, Docker Compose and a local Kubernetes cluster (kind / Minikube), and a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud-provider comparisons (AWS, Microsoft Azure, Google Cloud, IBM Cloud) are high-level and operational; provider details vary by region and plan, and official provider documentation is the authority — no pricing or service limits are stated in this book.

How to use this book. Read the parts in order — each part builds on the previous one. Keep a Docker-ready machine beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy a Prometheus, Grafana or OpenTelemetry configuration blindly; read the explanation under each field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on a production system: use a disposable virtual machine or an isolated lab environment, and never paste a real secret into an example — use the fictional placeholders shown. Never put a real password, token or API key into a log line, a metric label or a dashboard.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test system, finding the root cause in your own metrics, logs and traces, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Monitoring changes should first be practised in an isolated lab, never directly on a production system. The author and publisher accept no liability for data loss or service disruption caused by running commands without a backup or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and networking, has studied Linux server administration, CI/CD, Docker, Kubernetes, Terraform and Ansible, and wants to understand how production systems behave — to measure them with Prometheus, visualise them with Grafana, read them through logs, follow a request through traces, and respond to incidents with calm, evidence-based judgment.

Learning outcomes

By the end of this book you will be able to: explain the difference between monitoring and observability and design a telemetry strategy from SLI, SLO and error budget; model and query metrics with Prometheus and PromQL, and design alerts that are actionable instead of noisy; build Grafana dashboards that help engineers decide, not merely display graphs; design centralised, structured logging with Loki and LogQL and compare it with Elastic/OpenSearch; instrument Node.js applications with OpenTelemetry, collect traces with the Collector, and analyse them in Jaeger and Tempo; correlate metrics, logs and traces to find root causes in microservices; monitor Docker and Kubernetes workloads; manage alert fatigue and on-call work; and design an enterprise observability platform with retention, cost control, access control and incident runbooks.

সূচিপত্র — Contents

PART I — Observability Foundations

Chapter 01 — Monitoring vs Observability — Concepts and Engineering Principles 7

Chapter 02 — The Three Pillars — Metrics, Logs, and Traces 17

Chapter 03 — Production Monitoring Architecture and Telemetry Data Flow 27

Chapter 04 — SLIs, SLOs, SLAs, Error Budgets, and Golden Signals 37

Chapter 05 — Monitoring Strategy, Instrumentation, and Telemetry Collection 47

PART II — Metrics and Prometheus Engineering

Chapter 06 — Metrics Fundamentals — Counters, Gauges, Histograms, and Summaries 58

Chapter 07 — Prometheus Architecture and Data Collection 69

Chapter 08 — PromQL — Query Language and Metrics Analysis 78

Chapter 09 — Exporters, Node Exporter, and Application Instrumentation 87

Chapter 10 — Alertmanager, Alert Rules, Routing, and Notifications 95

Chapter 11 — Kubernetes Monitoring with Prometheus and Service Discovery 104

PART III — Visualization and Logging Engineering

Chapter 12 — Grafana Architecture, Dashboards, and Visualization 115

Chapter 13 — Grafana Variables, Alerts, Provisioning, and Data Sources 127

Chapter 14 — Centralized Logging Architecture and Structured Logging 138

Chapter 15 — Loki, Grafana Alloy, and LogQL 148

Chapter 16 — ELK Stack, Elastic Stack, and OpenSearch Logging 159

PART IV — Distributed Tracing and Advanced Observability

Chapter 17 — Distributed Tracing Fundamentals and Trace Context 172

Chapter 18 — OpenTelemetry Architecture, SDKs, and Collector 180

Chapter 19 — Jaeger, Grafana Tempo, and Trace Analysis 187

Chapter 20 — Microservices Observability and Telemetry Correlation 194

Chapter 21 — Application Performance Monitoring and Profiling 200

PART V — Production Observability Engineering

Chapter 22 — Incident Detection, Alert Fatigue, and On-call Engineering 208

Chapter 23 — Observability for Docker, Kubernetes, and Cloud Platforms 215

Chapter 24 — Enterprise Observability Architecture, Reliability, and Production Operations 222

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (180+ Terms) • Prometheus / PromQL / LogQL / OpenTelemetry References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *Observability Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, Prometheus, PromQL, LogQL and OpenTelemetry references, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

Observability Foundations

Monitoring বনাম Observability, তিনটি স্তম্ভ — Metrics, Logs, Traces — টেলিমেট্রি ডেটা প্রবাহ, SLI/SLO/SLA ও Error Budget, মনিটরিং কৌশল ও Instrumentation।

অধ্যায় ১ • PART I — Observability Foundations

Chapter 1 — Monitoring vs Observability — Concepts and Engineering Principles

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Monitoring এবং Observability-এর মধ্যে মৌলিক পার্থক্য ব্যাখ্যা করতে পারবেন। আপনি বুঝবেন কেন শুধু Monitoring দিয়ে আধুনিক Distributed System চালানো যায় না, Observability কেন সিস্টেমের একটি বৈশিষ্ট্য (property) আর Monitoring কেন একটি কাজ (activity)। আপনি Known-Unknowns ও Unknown-Unknowns-এর ধারণা, White-box ও Black-box পর্যবেক্ষণের পার্থক্য, এবং TechNova-র MERN অ্যাপ্লিকেশনে এই ধারণাগুলো কীভাবে প্রয়োগ হয় তা বলতে পারবেন।

- Monitoring ও Observability-এর সংজ্ঞা নিজের ভাষায় ব্যাখ্যা করা।
- কোন প্রশ্নের উত্তর Monitoring দিতে পারে আর কোন প্রশ্নের জন্য Observability দরকার, তা চিহ্নিত করা।
- একটি প্রোডাকশন ঘটনায় শুধু ড্যাশবোর্ড দেখে কেন মূল কারণ (root cause) পাওয়া যায় না, তা বিশ্লেষণ করা।
- TechNova-র API ধীর হয়ে গেলে কী কী Telemetry দরকার হবে, তার তালিকা তৈরি করা।

১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

DevOps-এর মূল প্রতিশ্রুতি হলো দ্রুত পরিবর্তন আর স্থিতিশীল প্রোডাকশন — একসঙ্গে। কিন্তু পরিবর্তন মানেই অনিশ্চয়তা। TechNova যখন প্রতিদিন নতুন কোড deploy করে, তখন প্রতিটি deploy একটি প্রশ্ন তৈরি করে: সিস্টেম কি এখনো ঠিকভাবে কাজ করছে? Monitoring সেই প্রশ্নের পূর্বনির্ধারিত উত্তর দেয় — CPU কত, Memory কত, Error Rate কত। কিন্তু প্রোডাকশনে সবচেয়ে ব্যয়বহুল ঘটনাগুলো পূর্বনির্ধারিত প্রশ্নের বাইরে ঘটে। একজন ব্যবহারকারী বলে, "চেকআউট বোতামে ক্লিক করলে কিছুই হয় না" — অথচ সব সার্ভারের CPU স্বাভাবিক, সব Health Check সবুজ। এই মুহূর্তে Monitoring নীরব, কারণ কেউ তাকে এই প্রশ্নটি আগে জিজ্ঞেস করতে শেখায়নি।

Observability এই শূন্যস্থান পূরণ করে। এটি সিস্টেমকে এমনভাবে তৈরি করে যাতে অজানা প্রশ্নও জিজ্ঞেস করা যায়। DevOps ইঞ্জিনিয়ারের কাজ শুধু সার্ভার চালু রাখা নয় — সিস্টেমের ভেতরে কী ঘটছে তা যেকোনো মুহূর্তে জানার ক্ষমতা তৈরি করা। যে দল Observability-তে বিনিয়োগ করে, তারা ঘটনা (incident) দ্রুত বোঝে, ভুল deploy দ্রুত Rollback করে এবং ব্যবহারকারীর অভিযোগ আসার আগেই সমস্যা ধরে। যে দল করে না, তারা অন্ধকারে হাতড়ায়।

> **REAL WORLD:** TechNova-র এক শুক্রবার রাতে অর্ডার API ধীর হয়ে যায়। Grafana ড্যাশবোর্ডে CPU ৩৫%, Memory ৬০% — সব স্বাভাবিক। কিন্তু ব্যবহারকারীরা টাইমআউট পাচ্ছিল। পরে দেখা যায় MongoDB-র একটি Query-তে Index অনুপস্থিত, ফলে Collection Scan হচ্ছিল। CPU স্বাভাবিক ছিল কারণ ডাটাবেস অপেক্ষা করছিল Disk I/O-র জন্য, গণনা করছিল না। এই ঘটনাই এই অধ্যায়ের মূল শিক্ষা: স্বাভাবিক Resource Metric মানেই সুস্থ সিস্টেম নয়।

১.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায় বুঝতে আপনার Linux কমান্ড লাইনের মৌলিক ধারণা, HTTP Request/Response কীভাবে কাজ করে, এবং Node.js/Express অ্যাপ্লিকেশনের সাধারণ গঠন জানা থাকলে সুবিধা হবে। Prometheus বা Grafana আগে ব্যবহার না করলেও চলবে — এখানে ধারণাগত ভিত্তি তৈরি হবে, টুলের বিস্তারিত ব্যবহার পরের অধ্যায়গুলোতে আসবে।

- Linux: `systemctl`, `curl`, লগ ফাইল পড়া।
- HTTP: Status Code (200, 404, 500, 502, 504), Latency, Timeout।
- অ্যাপ্লিকেশন: Frontend (React), API (Express/Node.js), Database (MongoDB) — তিন স্তরের ধারণা।
- DevOps ভিত্তি: Deploy, Rollback, Health Check কী — এগুলো জানা থাকলে ভালো।

১.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে শুরু করি: একটি কম্পিউটার সিস্টেম হলো এমন একটি যন্ত্র যার ভেতরের অবস্থা (internal state) আমরা সরাসরি দেখতে পাই না। আমরা শুধু তার বাইরের আচরণ দেখি — Response আসে কি না, কত দেরিতে আসে, কী Error দেয়। Observability-এর মূল প্রশ্ন হলো: বাইরের পর্যবেক্ষণ (output) থেকে ভেতরের অবস্থা কতটা অনুমান করা যায়?

এই ধারণাটি এসেছে Control Theory থেকে। একটি সিস্টেম Observable তখনই, যখন তার Output দেখে তার Internal State নির্ণয় করা যায়। একটি কালো বাক্সে যদি শুধু একটি বাতি থাকে — জ্বলছে বা নিভছে — তাহলে আপনি জানেন বাক্সটি চালু কি না, কিন্তু জানেন না ভেতরে কোন অংশটি গরম হচ্ছে, কোন Queue জমছে, কোন Database Query আটকে আছে। বাতির সংখ্যা বাড়ানো, অর্থাৎ Telemetry বাড়ানো, মানে ভেতরের অবস্থা আরও স্পষ্টভাবে জানা।

Monitoring কী

Monitoring হলো পূর্বনির্ধারিত প্রশ্নের উত্তর দেওয়ার ব্যবস্থা। আপনি আগে থেকেই ঠিক করেন: CPU ৮০% ছাড়ালে Alert দাও, Error Rate ৫% ছাড়ালে Alert দাও, Disk ৯০% পূর্ণ হলে Alert দাও। এটি Known-Unknowns-এর জগৎ — আপনি জানেন কী ভুল হতে পারে, শুধু জানেন না কখন হবে। Monitoring সেই "কখন" ধরার যন্ত্র।

Observability কী

Observability হলো সিস্টেমের বৈশিষ্ট্য — অজানা প্রশ্ন জিজ্ঞেস করার ক্ষমতা। আপনি আগে থেকে জানেন না কী ভুল হবে; কিন্তু যখন কিছু অদ্ভুত ঘটে, তখন আপনি সিস্টেমকে নতুন প্রশ্ন করতে পারেন এবং উত্তর পেতে পারেন। এটি Unknown-Unknowns-এর জগৎ। Observability তৈরি হয় Instrumentation থেকে — কোড এমনভাবে লেখা হয় যাতে প্রতিটি Request, প্রতিটি Query, প্রতিটি সিদ্ধান্তের চিহ্ন (Telemetry) রেখে যায়।

> **CONCEPT:** Monitoring হলো "আমি জানি কী দেখতে হবে" — Observability হলো "আমি যেকোনো কিছু জিজ্ঞেস করতে পারি"। Monitoring একটি Dashboard; Observability একটি অনুসন্ধানের ক্ষমতা। ভালো Monitoring ছাড়া Observability অসম্পূর্ণ, কিন্তু শুধু Monitoring দিয়ে আধুনিক সিস্টেম বোঝা যায় না।

Known-Unknowns বনাম Unknown-Unknowns

ধরন	অর্থ	উদাহরণ (TechNova)	সমাধান
Known-Knowns	জানি এবং জানি যে জানি	CPU বেশি হলে সার্ভার ধীর হয়	Dashboard, Alert

Known-Unknowns	জানি কী জানি না	Deploy-এর পর Error Rate বাড়বে কি না	Monitoring, Alert
Unknown-Unknowns	জানিই না কী জানি না	একটি নির্দিষ্ট পেমেন্ট Gateway শুধু একটি শহরের ব্যবহারকারীর জন্য ব্যর্থ হচ্ছে	Observability, Tracing, বিস্তারিত Telemetry

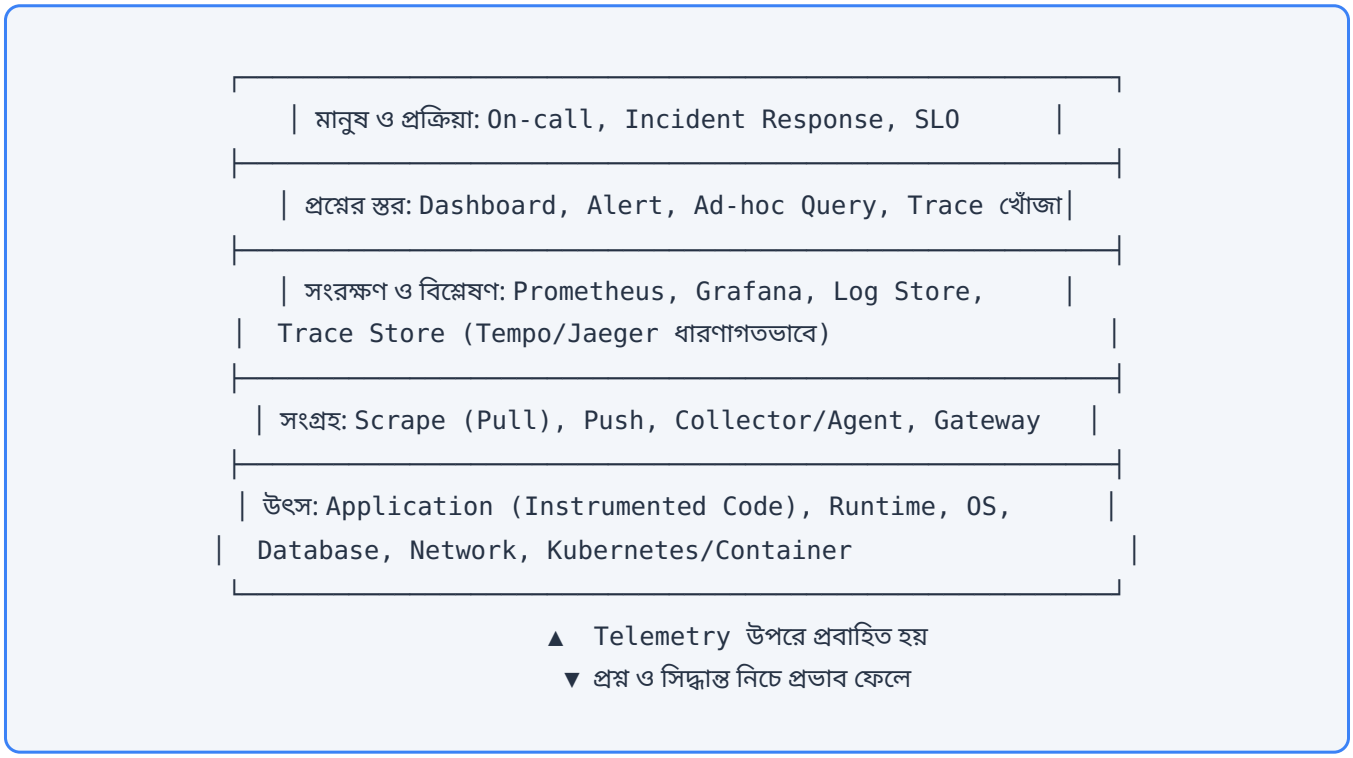
প্রোডাকশন ঘটনার সবচেয়ে কঠিন অংশটি তৃতীয় শ্রেণিতে পড়ে। সেজন্যই Telemetry এমনভাবে সংগ্রহ করতে হয় যাতে পরে নতুন প্রশ্ন করা যায় — High-cardinality ডেটা, Structured Log, Distributed Trace — এগুলো Unknown-Unknowns ধরার হাতিয়ার।

White-box বনাম Black-box পর্যবেক্ষণ

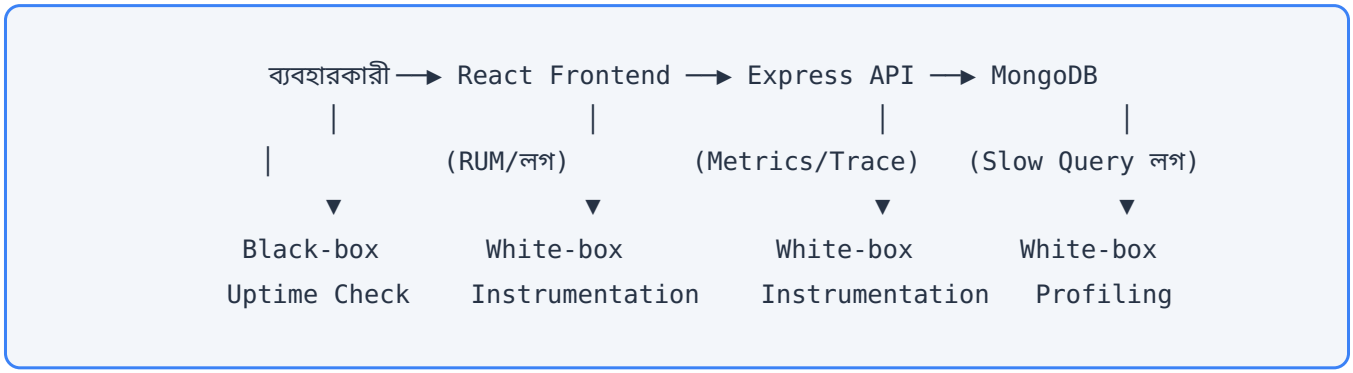
Black-box পর্যবেক্ষণ বাইরে থেকে দেখে: বাইরে থেকে Request পাঠিয়ে Response মাপে — Uptime Check, Synthetic Monitoring। এটি বলে সিস্টেম কাজ করছে কি না, কিন্তু কেন ব্যর্থ হচ্ছে তা বলে না। White-box পর্যবেক্ষণ ভেতর থেকে দেখে: কোডের ভেতরে Instrumentation বসিয়ে প্রতিটি Function, Query, Queue-র অবস্থা প্রকাশ করে। Observability মূলত White-box; Monitoring দুটোই ব্যবহার করে। TechNova-তে দুটোই দরকার: বাইরের Health Check বলবে API ডাউন কি না, ভেতরের Trace বলবে কোন Database Query আটকে আছে।

১.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র

Monitoring ও Observability-এর সম্পর্ক বোঝার সবচেয়ে ভালো উপায় হলো একটি স্তরচিত্র। নিচে দেখুন — নিচের স্তরগুলো Telemetry তৈরি করে, উপরের স্তরগুলো সেই Telemetry ব্যবহার করে প্রশ্নের উত্তর দেয়।



লক্ষ করুন: Monitoring সাধারণত উপরের দুই স্তরে বাস করে — Dashboard ও Alert। Observability পুরো স্তম্ভজুড়ে — উৎসে Instrumentation না থাকলে উপরে যত সুন্দর Dashboardই বানান, তা মিথ্যা নিরাপত্তা দেবে। একটি সিস্টেমের Observability তার সবচেয়ে দুর্বল স্তরের সমান।



১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Monitoring-এর সীমা কোথায়

Monitoring কাজ করে Aggregate-এর উপর: গড় Latency, মোট Error সংখ্যা, গড় CPU। কিন্তু গড় প্রায়ই মিথ্যা বলে। TechNova-র API-র গড় Latency ১২০ মিলিসেকেন্ড দেখা যেতে পারে, অথচ ৫% ব্যবহারকারী ৮ সেকেন্ড অপেক্ষা করছে — কারণ গড়ে সেই ৫% হারিয়ে যায়। এজন্য Percentile (p50, p95, p99) দরকার, এবং তার চেয়েও বেশি দরকার সেই ৫% কারা, কোন Endpoint, কোন অঞ্চল, কোন সংস্করণ — সেই ভাঙন (breakdown)। Monitoring যা আগে থেকে ভাঙন ঠিক করে রেখেছে, শুধু তা-ই দেখাতে পারে।

Observability-এর তিনটি ক্ষমতা

প্রথমত, **অনুসন্ধানযোগ্যতা (Explorability)** — পূর্বনির্ধারিত Dashboard ছাড়াও নতুন প্রশ্ন করা যায়। দ্বিতীয়ত, **সংযোগ (Correlation)** — একটি Metric-এর স্পাইক থেকে সংশ্লিষ্ট Log ও Trace-এ ঝাঁপ দেওয়া যায়, একই Trace ID বা Request ID ধরে। তৃতীয়ত, **প্রসঙ্গ (Context)** — প্রতিটি সংকেতের সঙ্গে পর্যাপ্ত Label ও Attribute থাকে যাতে "কোন ব্যবহারকারী, কোন সংস্করণ, কোন অঞ্চল" প্রশ্নের উত্তর মেলে। এই তিনটি না থাকলে আপনার কাছে শুধু সংখ্যা আছে, বোঝাপড়া নেই।

Telemetry কী

Telemetry হলো সিস্টেম থেকে সংগৃহীত দূরবর্তী পরিমাপের ডেটা — Metrics, Logs, Traces-এর সমষ্টি। শব্দটি এসেছে Tele (দূর) + Metry (পরিমাপ) থেকে। গুরুত্বপূর্ণ বিষয়: Telemetry স্বয়ংক্রিয়ভাবে তৈরি হয় না; কোডে Instrumentation লিখতে হয়, Agent বসাতে হয়, Log Format ঠিক করতে হয়। Observability কোনো টুল কিনে পাওয়া যায় না — এটি একটি Engineering অনুশীলন, টুল শুধু তাকে সহায়তা করে।

স্বাভাবিক CPU/Memory অথচ অসুস্থ সিস্টেম — কেন?

এই প্রশ্নের উত্তরেই Monitoring ও Observability-এর পার্থক্য স্পষ্ট হয়। CPU ও Memory হলো Resource Saturation-এর মাপকাঠি, কিন্তু অনেক ব্যর্থতা Resource-এর নয়: (১) ডাটাবেসে Index না থাকলে Query Disk-এ আটকে থাকে, CPU অলস বসে থাকে; (২) Downstream সার্ভিস ধীর হলে API Thread Pool অপেক্ষায় আটকে থাকে — CPU কম, কিন্তু Throughput শূন্য; (৩) Connection Pool শেষ হয়ে গেলে নতুন Request Queue-তে আটকে থাকে; (৪) Memory Leak ধীরে ধীরে বাড়ে, কিন্তু Leak ধরা পড়ার আগ পর্যন্ত Memory "স্বাভাবিক" দেখায়; (৫) একটি নির্দিষ্ট Tenant বা অঞ্চলের জন্য সমস্যা হলে সামগ্রিক গড় তা ঢেকে দেয়। এজন্য Golden Signals (Latency, Traffic, Errors, Saturation) একসঙ্গে দেখতে হয়, শুধু Resource Metric নয়।

> **WARNING:** "সবুজ ড্যাশবোর্ড" মানেই সুস্থ সিস্টেম নয়। সবচেয়ে বিপজ্জনক অবস্থা হলো যখন Dashboard সবুজ আর ব্যবহারকারীরা কষ্টে আছে — তার মানে আপনার Monitoring ভুল জিনিস মাপছে।

১.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

এই অধ্যায়ে ধারণা প্রধান, তবু একটি ছোট হাতে-কলমে পরীক্ষা ধারণাটি দৃঢ় করে। TechNova-র API সার্ভারে (203.0.113.10) প্রথমে দেখুন সিস্টেম কী বলে, তারপর ভাবুন সিস্টেম কী বলছে না।

```
# সার্ভারে লগ ইন করে প্রাথমিক স্বাস্থ্য দেখা (নমুনা আউটপুট — চিত্রণমূলক)
ssh deploy@203.0.113.10
uptime
# uptime: load average কম দেখাতে পারে, অথচ API ধীর — কারণ load শুধু CPU বলে

curl -s -o /dev/null -w "HTTP %{http_code} — %{time_total}s\n" https://api.technova.example/
health
# Black-box পরীক্ষা: বাইরে থেকে Response সময় মাপা; ভেতরের কারণ জানা যায় না

curl -s https://api.technova.example/health | head -c 500
# Health endpoint কী ফেরত দেয় দেখুন — শুধু {"status":"ok"} হলে তা অপর্যাপ্ত

ss -tulpn | grep -E '3000|27017'
# কোন Port-এ API (3000) ও MongoDB (27017) শুনছে, তা যাচাই
```

উপরের প্রতিটি কমান্ড একটি ভিন্ন স্তরের তথ্য দেয়। `uptime` Resource বলে, `curl` বাইরের আচরণ বলে, কিন্তু কোনোটিই বলে না কোন Database Query ধীর। সেই তথ্য পেতে Instrumentation ও Structured Log দরকার — যা পরের অধ্যায়গুলোতে আসবে।

```
# একটি ধীর Request-এর চিহ্ন খুঁজতে (ধারণাগত — বিস্তারিত পরের অধ্যায়ে)
# ১) API লগে ধীর Request খুঁজুন (duration বেশি যেগুলো)
# ২) একই Request ID দিয়ে Database লগে সংশ্লিষ্ট Query খুঁজুন
# ৩) Trace থাকলে সেই Request-এর Span গাছ (waterfall) দেখুন
# এই তিন ধাপই Observability: একই ঘটনার তিনটি দৃষ্টিকোণ
```

১.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

এই অধ্যায়ের উদাহরণটি ধারণাগত — একটি Health Endpoint কেমন হওয়া উচিত, যাতে Black-box ও White-box দুটোই কাজ করে। শুধু `{"status":"ok"}` ফেরত দেওয়া Health Check বিপজ্জনক, কারণ Database ডাউন থাকলেও তা "ok" বলতে পারে।

```
// TechNova API — গভীর Health Check-এর ধারণা (Node.js / Express sketch)
// ফাইল: src/routes/health.js (চিত্রণমূলক)
const express = require('express');
const router = express.Router();
const mongoose = require('mongoose');

router.get('/health', async (req, res) => {
  // Liveness: প্রক্রিয়াটি বেঁচে আছে কি না (সহজ, দ্রুত)
  res.json({ status: 'ok', service: 'technova-api', version: '1.4.0' });
});
```

```
router.get('/ready', async (req, res) => {
  // Readiness: নির্ভরতাগুলো প্রস্তুত কি না (Database, Cache)
  const dbState = mongoose.connection.readyState; // 1 = connected
  if (dbState !== 1) {
    return res.status(503).json({ status: 'not_ready', db: 'down' });
  }
  res.json({ status: 'ready', db: 'up' });
});

module.exports = router;
```

এখানে `/health` হলো Liveness Probe — প্রক্রিয়া বেঁচে আছে কি না। `/ready` হলো Readiness Probe — ট্রাফিক নেওয়ার মতো অবস্থায় আছে কি না। দুটোকে গুলিয়ে ফেললে Load Balancer অসুস্থ সার্ভারেও ট্রাফিক পাঠাবে, অথবা সুস্থ সার্ভারকে বারবার Restart করবে। Monitoring-এর প্রথম পাঠ: কী মাপছেন, তা ঠিকমতো সংজ্ঞায়িত করুন।

১.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি: TechNova-র শুক্রবার সন্ধ্যায় Flash Sale চলছে। ১৯:৪০-এ Support-এ অভিযোগ আসে — "অর্ডার করতে গেলে পেজ ঘুরতেই থাকে।" On-call ইঞ্জিনিয়ার Grafana খোলেন: API সার্ভারের CPU ৪০%, Memory ৫৫%, Disk ৩০% — সব সবুজ। Error Rate-ও মাত্র ১%। তিনি ভাবেন সমস্যা ব্যবহারকারীর নেটওয়ার্কে। কিন্তু অভিযোগ বাড়তে থাকে।

আধা ঘণ্টা পর তিনি Log খোলেন: `checkout` Endpoint-এর p99 Latency ৯ সেকেন্ড, অথচ গড় ২০০ মিলিসেকেন্ড। মাত্র ৮% Request আক্রান্ত — যারা একটি নির্দিষ্ট Payment Gateway ব্যবহার করছিল। Gateway-টি ধীরে Response দিচ্ছিল, API-র Thread Pool সেই অপেক্ষায় আটকে যাচ্ছিল, নতুন Request Queue-তে জমছিল। CPU কম ছিল কারণ Thread-গুলো কাজ করছিল না, অপেক্ষা করছিল। সমাধান ছিল Gateway-তে Timeout কমানো ও Circuit Breaker বসানো।

এই ঘটনার শিক্ষা তিনটি: (১) গড় নয়, Percentile দেখুন; (২) সামগ্রিক নয়, ভাঙন (Endpoint, Gateway, অঞ্চল) দেখুন; (৩) Resource Metric নয়, Golden Signals দেখুন। Monitoring এই তিনটির কোনোটিই স্বয়ংক্রিয়ভাবে দেয় না — এগুলো Observability-এর নকশা (design) থেকে আসে।

১.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **প্রশ্নের তালিকা লিখুন:** TechNova-র জন্য লিখুন — "ব্যবহারকারী কী কী অভিজ্ঞতা পেতে পারে যা খারাপ?" (ধীর চেকআউট, ব্যর্থ লগইন, অনুপস্থিত অর্ডার)। প্রতিটি অভিজ্ঞতার পেছনে কোন সংকেত দরকার, তা লিখুন।
2. **Known-Unknowns চিহ্নিত করুন:** যেসব ব্যর্থতা আপনি আগে থেকে জানেন (Database ডাউন, Disk পূর্ণ, Deploy ভুল) — সেগুলোর জন্য Alert ও Dashboard তৈরি করুন। এটি Monitoring।
3. **Unknown-Unknowns-এর জন্য জায়গা রাখুন:** প্রতিটি Request-এ Trace ID ও Structured Log যোগ করার পরিকল্পনা করুন, যাতে অজানা ঘটনায় অনুসন্ধান করা যায়। এটি Observability-এর ভিত্তি।
4. **Health Check দুই ভাগ করুন:** `/health` (Liveness) ও `/ready` (Readiness) আলাদা করুন, যেমন ১.৮-এ দেখানো হয়েছে।
5. **একটি ঘটনা অনুশীলন করুন:** ইচ্ছাকৃতভাবে একটি ধীর Endpoint তৈরি করে দেখুন — গড় Latency কি তা ধরতে পারে? p99 পারে? কোন ভাঙন দরকার? ফলাফল লিখে রাখুন।
6. **পর্যালোচনা করুন:** আপনার বর্তমান Dashboard-এ কোন প্রশ্নের উত্তর নেই, তার তালিকা করুন। সেগুলোই আপনার Observability ঘাটতি।

১.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল ধারণা	আসলে কী
"Monitoring আর Observability একই জিনিস"	Monitoring হলো কাজ, Observability হলো সিস্টেমের বৈশিষ্ট্য। Monitoring Observability-এর একটি ব্যবহার।
"টুল কিনলেই Observability আসে"	টুল ছাড়া Instrumentation না থাকলে টুলের কিছু দেখার নেই। প্রথমে কোডে চিহ্ন রাখুন।
"CPU/Memory স্বাভাবিক মানেই সব ঠিক"	অনেক ব্যর্থতা (ধীর Downstream, Pool শেষ, Index নেই) Resource Metric-এ ধরা পড়ে না।
"গড় Latency যথেষ্ট"	গড় সংখ্যালঘুর কষ্ট লুকায়। p95/p99 ও ভাঙন দেখুন।
"Health Check সবুজ মানেই সিস্টেম সুস্থ"	অগভীর Health Check Database ডাউন থাকলেও সবুজ দেখায়। Readiness আলাদা করুন।
"Log বেশি মানেই Observability বেশি"	অগোছালো Log অনুসন্ধানকে কঠিন করে। Structured, Correlated Log দরকার।

১.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Observability বাড়াতে গিয়ে গোপনীয়তা হারানো যাবে না। Log ও Trace-এ কখনো Password, Token, API Key, Credit Card নম্বর, বা ব্যক্তিগত তথ্য (PII) লেখা যাবে না। TechNova-র লগে ব্যবহারকারীর Email বা ঠিকানা পুরো লেখা উচিত নয় — প্রয়োজনে Mask করুন বা Hash করুন। Error Log-এ Request Body পুরো ছাপানো বিপজ্জনক, কারণ তাতে Password থাকতে পারে।

Telemetry সংরক্ষণেও Access Control দরকার। Production Log ও Trace দেখার অধিকার শুধু On-call ও নির্দিষ্ট দলের থাকা উচিত, সবার নয়। Grafana Dashboard শেয়ার করার সময় খেয়াল রাখুন — কোনো Dashboard-এ সংবেদনশীল Label (যেমন ব্যবহারকারীর Email) না থাকে। নমুনা কনফিগারেশনে কখনো আসল Secret লিখবেন না — সবসময় `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করুন।

> **WARNING:** একটি ভুল Log লাইন — যেমন Authorization Header পুরো ছাপানো — পুরো সিস্টেমের নিরাপত্তা ভেঙে দিতে পারে। Instrumentation লেখার সময় প্রতিটি Field জিজ্ঞেস করুন: "এটি কি সংবেদনশীল?"

১.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Telemetry-এরও খরচ আছে। প্রতিটি Request-এ বিস্তারিত Log লিখলে Disk ভরে যায়, Network ব্যস্ত হয়, এমনকি অ্যাপ্লিকেশন ধীর হয়। আবার Telemetry কম রাখলে ঘটনার সময় অন্ধ হয়ে যান। ভারসাম্য হলো: গুরুত্বপূর্ণ ঘটনা সবসময় রেকর্ড করুন, সাধারণ ঘটনা Sampling করুন, আর ব্যয়বহুল বিস্তারিত (যেমন পুরো Request Body) শুধু Error-এর সময় রেকর্ড করুন।

খরচের তিনটি উৎস মনে রাখুন: (১) সংগ্রহের খরচ — CPU ও Network; (২) সংরক্ষণের খরচ — Disk ও Retention; (৩) অনুসন্ধানের খরচ — মানুষের সময়। একটি অগোছালো Log ব্যবস্থায় তৃতীয় খরচটিই সবচেয়ে বড় — ঘটনার সময় ইঞ্জিনিয়ার ঘণ্টার পর ঘণ্টা খুঁজতে থাকেন। তাই শুরু থেকেই Structured Log ও সঠিক Label-এ বিনিয়োগ করা আসলে খরচ বাঁচায়।

১.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরবর্তী ধাপ
Dashboard সবুজ, ব্যবহারকারী অভিযোগ করছে	ভুল Metric মাপা হচ্ছে; গড় Percentile ঢাকছে	p95/p99 দেখুন; Endpoint ও অঞ্চল ভাঙুন
ঘটনার কারণ খুঁজে পাওয়া যাচ্ছে না	Correlation নেই — Log ও Metric-এর মধ্যে Trace ID নেই	Request ID দিয়ে Log খুঁজুন; Trace যোগ করুন
Alert আসে কিন্তু সব False Positive	Threshold ভুল; সামগ্রিক Metric-এ Alert	Alert-এর সংজ্ঞা পর্যালোচনা করুন; Burn Rate ভাবুন (অধ্যায় ৪)
Health Check সবুজ, Database ডাউন	অগভীর Health Check	Readiness Probe-এ নির্ভরতা যাচাই যোগ করুন
Log আছে কিন্তু খুঁজে পাওয়া যায় না	Unstructured Log, Timezone বা Clock Skew	Structured Log ও UTC Timestamp নিশ্চিত করুন

TROUBLESHOOTING: কোনো ঘটনায় প্রথম প্রশ্নটি হওয়া উচিত "ব্যবহারকারী কী অনুভব করছে?" — তারপর "কোন Golden Signal তা প্রতিফলিত করছে?" — তারপর "কোন ভাঙনে সমস্যাটি সীমাবদ্ধ?" এই ক্রম মেনে চললে অন্ধ অনুসন্ধান কমে।

১.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ব্যবহারকারীর অভিজ্ঞতা থেকে শুরু করুন — কোন Golden Signal খারাপ হলে ব্যবহারকারী কষ্ট পায়, তা আগে ঠিক করুন। প্রতিটি Request-এ Trace ID দিন। Structured Log লিখুন। Percentile ব্যবহার করুন। Dashboard-এর পাশাপাশি Ad-hoc অনুসন্ধানের ক্ষমতা রাখুন।

অ্যান্টি-প্যাটার্ন: শুধু Infrastructure Metric (CPU/Memory/Disk) দিয়ে Dashboard ভরিয়ে ফেলা; গড় Latency-কে একমাত্র সত্য মানা; Log-এ সংবেদনশীল তথ্য লেখা; Alert-এর সংখ্যা এত বাড়ানো যে কেউ আর Alert পড়ে না (Alert Fatigue); ঘটনার পর Telemetry যোগ করা — ঘটনার আগেই তা থাকা উচিত।

PRO TIP: প্রতিটি নতুন Feature-এর সঙ্গে সঙ্গে জিজ্ঞেস করুন: "এটি ভাঙলে আমরা কীভাবে জানব, আর কীভাবে কারণ খুঁজব?" উত্তরটি কোডের সঙ্গে সঙ্গেই লিখুন — পরে যোগ করা Telemetry প্রায়ই অসম্পূর্ণ থাকে।

১.১৬ অধ্যায়ের সারাংশ

Monitoring হলো পূর্বনির্ধারিত প্রশ্নের উত্তর — Known-Unknowns ধরার ব্যবস্থা। Observability হলো সিস্টেমের বৈশিষ্ট্য — অজানা প্রশ্নও জিজ্ঞেস করার ক্ষমতা, যা Instrumentation থেকে আসে। Black-box পর্যবেক্ষণ বাইরের আচরণ বলে, White-box ভেতরের অবস্থা বলে — দুটোই দরকার। স্বাভাবিক CPU/Memory মানেই সুস্থ সিস্টেম নয়, কারণ অনেক ব্যর্থতা Resource-এর নয়। গড় Latency বিভ্রান্তিকর — Percentile ও ভাঙন দেখতে হয়। TechNova-র ঘটনাগুলো দেখায়: যে দল শুধু Dashboard দেখে, তারা ব্যবহারকারীর কষ্টের কারণ খুঁজে পায় না; যে দল Telemetry দিয়ে অনুসন্ধান করতে পারে, তারাই দ্রুত মূল কারণে পৌঁছায়।

১.১৭ মূল প্রযুক্তিগত পরিভাষা

Monitoring: পূর্বনির্ধারিত Metric ও Threshold-এর ভিত্তিতে সিস্টেমের অবস্থা পর্যবেক্ষণ ও Alert দেওয়ার কাজ।

Observability: সিস্টেমের ভেতরের অবস্থা তার বাইরের Telemetry থেকে অনুমান করার ক্ষমতা; সিস্টেমের একটি বৈশিষ্ট্য।

Telemetry: সিস্টেম থেকে সংগৃহীত দূরবর্তী পরিমাপ — Metrics, Logs ও Traces-এর সমষ্টি।

Known-Unknowns: যে অনিশ্চয়তাগুলো সম্পর্কে আমরা জানি যে জানি না — যেমন Deploy-এর পর Error বাড়বে কি না।

Unknown-Unknowns: যে সমস্যাগুলো সম্পর্কে আমরা জানিই না যে জানি না — অপ্রত্যাশিত, নতুন ধরনের ব্যর্থতা।

White-box Monitoring: কোডের ভেতরে Instrumentation দিয়ে অভ্যন্তরীণ অবস্থা প্রকাশ করে পর্যবেক্ষণ।

Black-box Monitoring: বাইরে থেকে Request পাঠিয়ে সিস্টেমের আচরণ মেপে পর্যবেক্ষণ; ভেতরের কারণ জানা যায় না।

Instrumentation: কোডে Telemetry তৈরি করার ব্যবস্থা যোগ করা — Metric, Log ও Trace নির্গমন।

Golden Signals: Latency, Traffic, Errors ও Saturation — ব্যবহারকারীর অভিজ্ঞতা প্রতিফলিত করে এমন চারটি মূল সংকেত।

Percentile (p95/p99): Latency-র বণ্টন বোঝার মাপকাঠি — p99 মানে ৯৯% Request এর চেয়ে দ্রুত সম্পন্ন হয়েছে।

Liveness / Readiness: Liveness মানে প্রক্রিয়া বেঁচে আছে কি না; Readiness মানে ট্রাফিক নেওয়ার মতো প্রস্তুত কি না।

Alert Fatigue: অতিরিক্ত বা ভুল Alert-এর কারণে দল Alert-কে গুরুত্ব না দেওয়ার অবস্থা।

১.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Monitoring ও Observability-এর মধ্যে মৌলিক পার্থক্য কী? নিজের ভাষায় ব্যাখ্যা করুন।
2. Known-Unknowns ও Unknown-Unknowns-এর একটি করে TechNova-র উদাহরণ দিন।
3. CPU ৩৫% ও Memory ৬০% থাকা সত্ত্বেও API ধীর হতে পারে — এমন তিনটি কারণ লিখুন।
4. White-box ও Black-box পর্যবেক্ষণের পার্থক্য কী? প্রতিটির একটি উদাহরণ দিন।
5. গড় Latency কেন বিভ্রান্তিকর হতে পারে? p99 কীভাবে সাহায্য করে?
6. Liveness ও Readiness Probe গুলিয়ে ফেললে কী সমস্যা হতে পারে?
7. Observability কেন "টুল কিনে পাওয়া যায় না" — ব্যাখ্যা করুন।
8. একটি ঘটনায় প্রথমে কোন প্রশ্নটি করা উচিত — "CPU কত?" নাকি "ব্যবহারকারী কী অনুভব করছে?" — যুক্তি দিন।

১.১৯ ব্যবহারিক অনুশীলন

1. TechNova-র MERN অ্যাপের জন্য ব্যবহারকারীর পাঁচটি খারাপ অভিজ্ঞতার তালিকা লিখুন (যেমন: ধীর চেকআউট, ব্যর্থ লগইন) এবং প্রতিটির পেছনে কোন Golden Signal খারাপ হবে তা লিখুন।
2. আপনার নিজের একটি ছোট Express অ্যাপে `/health` ও `/ready` Endpoint আলাদা করে বাস্তবায়ন করুন; Database সংযোগ বিচ্ছিন্ন করে দেখুন `/ready` 503 দেয় কি না।
3. একটি ধীর Endpoint ইচ্ছাকৃতভাবে তৈরি করুন (যেমন ৩ সেকেন্ড `setTimeout`) এবং গড় Latency বনাম p99 কীভাবে আলাদা আচরণ করে, তা পর্যবেক্ষণ করে লিখুন।
4. আপনার বর্তমান কোনো Dashboard বা লগ ব্যবস্থা থাকলে তার ঘাটতির তালিকা করুন: কোন প্রশ্নের উত্তর সেখানে নেই?
5. একটি কাল্পনিক ঘটনার Incident Timeline লিখুন: অভিযোগ আসা থেকে মূল কারণ পাওয়া পর্যন্ত কোন কোন Telemetry কোন ধাপে সাহায্য করত।

অধ্যায় ২ • PART I — Observability Foundations

Chapter 2 — The Three Pillars — Metrics, Logs, and Traces

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Observability-র তিনটি স্তম্ভ — Metrics, Logs ও Traces — প্রতিটির সংজ্ঞা, শক্তি ও সীমা ব্যাখ্যা করতে পারবেন। আপনি বলতে পারবেন কোন প্রশ্নের উত্তর কোন স্তম্ভ দিতে পারে আর কোনটি পারে না, কীভাবে তিনটি একসঙ্গে Correlation করে একটি ঘটনার পূর্ণ চিত্র দেয়, এবং TechNova-র MERN অ্যাপে প্রতিটি স্তম্ভ কোথায় ব্যবহার করতে হয়।

- Metrics, Logs ও Traces-এর সংজ্ঞা ও পার্থক্য ব্যাখ্যা করা।
- প্রতিটি স্তম্ভের সীমা চিহ্নিত করা — যেমন Metrics কেন "কেন" বলতে পারে না।
- একই Trace ID / Request ID দিয়ে তিনটি স্তম্ভকে জোড়া দেওয়ার পদ্ধতি বোঝা।
- Structured Log ও Structured Event-এর ধারণা প্রয়োগ করা।

২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

তিনটি স্তম্ভকে তিনটি ভিন্ন ইন্দ্রিয় ভাবুন: Metrics হলো থার্মোমিটার — শরীরের তাপমাত্রা বলে, কিন্তু কোথায় সংক্রমণ তা বলে না। Logs হলো ডায়েরি — কী ঘটেছে তার বিবরণ দেয়। Traces হলো মানচিত্র — একটি Request কোথা দিয়ে গেল, কোথায় সময় কাটাল, তা দেখায়। শুধু থার্মোমিটার দিয়ে রোগ নির্ণয় করা যায় না, শুধু ডায়েরি দিয়ে জ্বর মাপা যায় না। প্রোডাকশন ঘটনায় তিনটিই একসঙ্গে দরকার।

TechNova-তে প্রায়ই দেখা যায় দলগুলো শুধু একটি স্তম্ভে বিনিয়োগ করে — হয় শুধু Grafana Dashboard, নয়তো শুধু Log ফাইল। ফলে ঘটনার সময় তারা অর্ধেক অন্ধ থাকে। Metric স্পাইক দেখে তারা জানে কিছু ভুল হচ্ছে, কিন্তু কোন Request, কোন ব্যবহারকারী, কোন Query — তা জানতে Log ও Trace দরকার। উল্টোদিকে, Trace ছাড়া শুধু Log পড়লে একটি Request-এর যাত্রাপথ জোড়া দেওয়া প্রায় অসম্ভব। এই অধ্যায় সেই তিনটির সমন্বয় শেখায়।

২.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১-এর Monitoring বনাম Observability ধারণা, HTTP Status Code, এবং JSON-এর মৌলিক ধারণা জানা থাকলে এই অধ্যায় সহজ হবে। Prometheus-এর Metric ধারণা (Counter, Gauge) এখানে পরিচিত করা হবে, আগে জানা আবশ্যক নয়। Node.js-এ `console.log` ব্যবহারের অভিজ্ঞতা থাকলে Structured Log-এর পার্থক্য বুঝতে সুবিধা হবে।

২.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি সিস্টেম সম্পর্কে আমরা তিন ধরনের প্রশ্ন করি। (১) "কত?" — কত Request এল, কত Error হলো, কত সময় লাগল? (২) "কী ঘটল?" — একটি নির্দিষ্ট মুহূর্তে ঠিক কী ঘটেছিল, কার জন্য, কী বার্তা নিয়ে? (৩) "কোথা দিয়ে গেল?" — একটি Request সিস্টেমের ভেতর দিয়ে কোন পথে গেল, কোথায় আটকাল? এই তিন প্রশ্নের উত্তরে তিনটি ভিন্ন ধরনের ডেটা দরকার — কারণ ডেটার আকৃতি প্রশ্নের আকৃতি নির্ধারণ করে।

Metrics — "কত?" প্রশ্নের উত্তর

Metrics হলো সময়ের সঙ্গে পরিবর্তিত সংখ্যাগত পরিমাপ। যেমন: প্রতি সেকেন্ডে কত Request, গড় Latency কত, কত শতাংশ Error। Metrics Aggregate করা হয় — হাজার হাজার Request-এর সংখ্যাকে একটি সংখ্যায় সংকুচিত করা হয়। এই সংকোচনই তার শক্তি (সস্তায় দীর্ঘদিন সংরক্ষণ করা যায়, দ্রুত গ্রাফ আঁকা যায়) এবং তার সীমা (ব্যক্তিগত Request-এর গল্প হারিয়ে যায়)। Metric বলে "কিছু ভুল হচ্ছে", কিন্তু "কার সঙ্গে, কেন" বলে না।

Logs — "কী ঘটল?" প্রশ্নের উত্তর

Logs হলো সময়ানুক্রমিক ঘটনার বিবরণ — সিস্টেম যা করেছে বা যা ঘটেছে তার রেকর্ড। প্রতিটি Log লাইন একটি নির্দিষ্ট মুহূর্তের একটি নির্দিষ্ট ঘটনা। Log-এর শক্তি তার বিস্তারিত বিবরণে: Error Message, Stack Trace, Request ID, ব্যবহারকারীর প্রসঙ্গ। তার সীমা: পরিমাণ — প্রোডাকশনে প্রতি সেকেন্ডে হাজার হাজার লাইন তৈরি হয়, আর Unstructured Log (মানুষের পড়ার মতো মুক্ত বাক্য) যন্ত্র দিয়ে খোঁজা কঠিন।

Traces — "কোথা দিয়ে গেল?" প্রশ্নের উত্তর

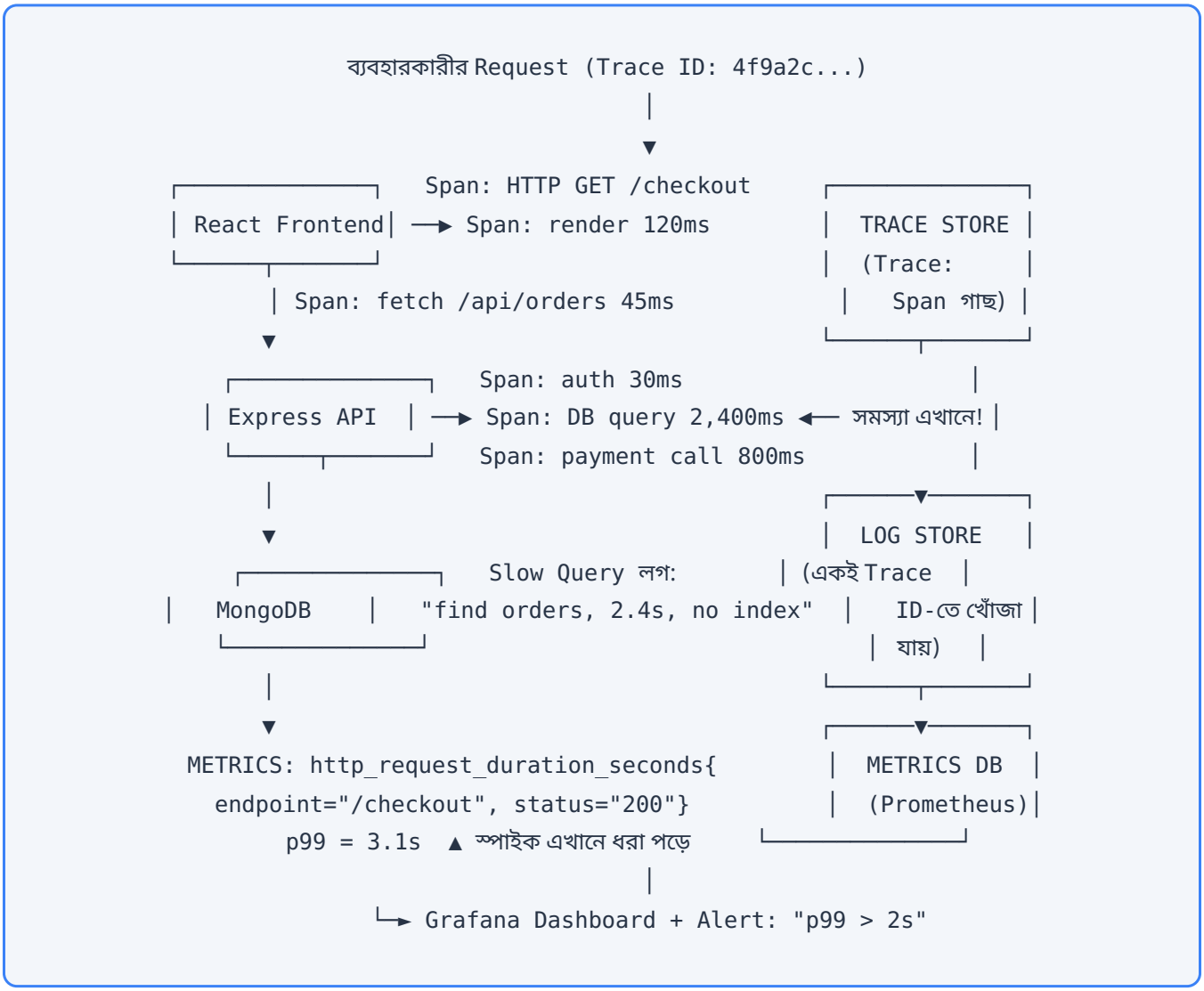
Traces হলো একটি Request-এর যাত্রাপথের রেকর্ড — Frontend থেকে API, API থেকে Database, API থেকে Downstream সার্ভিস — প্রতিটি ধাপ কত সময় নিল, কোন ধাপে Error হলো। একটি Trace অনেকগুলো Span-এর সমষ্টি; প্রতিটি Span একটি কাজের একক (যেমন একটি Database Query বা একটি HTTP Call)। Trace-এর শক্তি তার কার্যকারণ শৃঙ্খলে (causal chain): কোথায় সময় গেল, তা চোখে দেখা যায়। তার সীমা: খরচ ও জটিলতা — প্রতিটি Request Trace করলে সংরক্ষণ ও প্রক্রিয়াকরণের খরচ অনেক বাড়ে, তাই Sampling দরকার হয়।

> **CONCEPT:** Metrics বলে "কী পরিমাণে ভুল হচ্ছে", Logs বলে "ঠিক কী ঘটেছিল", Traces বলে "কোথায় সময় গেল ও কোন পথে"। তিনটি মিলে একটি ঘটনার পূর্ণ গল্প হয়; একটিও একা পূর্ণ গল্প দিতে পারে না।

Structured Events — তিনটির মিলনবিন্দু

আধুনিক ধারণা হলো Structured Event: প্রতিটি Request বা কাজকে একটি সমৃদ্ধ, কাঠামোবদ্ধ ঘটনা হিসেবে রেকর্ড করা, যাতে Metric-এর মতো সংখ্যা, Log-এর মতো বিবরণ ও Trace-এর মতো প্রসঙ্গ একসঙ্গে থাকে। বাস্তবে পুরো সিস্টেমকে এভাবে বানানো কঠিন, তাই তিনটি স্তম্ভকে Correlation ID (Trace ID / Request ID) দিয়ে জোড়া দেওয়াই বর্তমান সেরা অনুশীলন।

২.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



চিত্রটি লক্ষ করুন: একই ঘটনা তিন জায়গায় তিন রূপে আছে। Metric-এ p99 স্পাইক, Trace-এ Database Span-টি ২.৪ সেকেন্ড, Log-এ Slow Query বার্তা। Trace ID হলো সেই সুতো যা তিনটিকে বাঁধে। এই সুতো ছাড়া ইঞ্জিনিয়ার তিনটি আলাদা খুঁজতে বাধ্য হন — সময় নষ্ট হয়, ভুল সিদ্ধান্ত হয়।

বৈশিষ্ট্য	Metrics	Logs	Traces
প্রশ্ন	কত?	কী ঘটল?	কোথা দিয়ে গেল?
আকৃতি	সংখ্যা, সময়-সিরিজ	ঘটনার বিবরণ	Span-এর গাছ
খরচ	কম	মাঝারি–বেশি	বেশি (Sampling দরকার)
Retention	দীর্ঘ (মাস–বছর)	মাঝারি (দিন–সপ্তাহ)	সংক্ষিপ্ত (দিন)
শক্তি	Trend, Alert	বিস্তারিত কারণ	কার্যকারণ, বিলম্বের উৎস
সীমা	"কেন" বলে না	পরিমাণ বেশি, খোঁজা কঠিন	সব Request Trace করা ব্যয়বহুল

২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Metrics-এর প্রকারভেদ

Prometheus-জগতে চারটি মৌলিক Metric প্রকার আছে। **Counter** শুধু বাড়ে — যেমন মোট Request সংখ্যা, মোট Error সংখ্যা। **Gauge** বাড়ে-কমে — যেমন বর্তমান Memory ব্যবহার, সক্রিয় সংযোগ সংখ্যা। **Histogram** মানের বণ্টন মাপে — যেমন Latency কত Request ১০০ মিলিসেকেন্ডের নিচে, কত ৫০০-এর উপরে; এখান থেকে Percentile হিসাব করা যায়। **Summary** Histogram-এর মতো, কিন্তু Percentile ক্লায়েন্ট পক্ষে হিসাব করে। TechNova-র জন্য Histogram সবচেয়ে গুরুত্বপূর্ণ, কারণ Latency-র Percentile ছাড়া ব্যবহারকারীর অভিজ্ঞতা বোঝা যায় না।

Logs: Unstructured বনাম Structured

Unstructured Log মানুষের পড়ার মতো বাক্য: `2026-11-09 User Nahid logged in successfully`। এটি চোখে পড়তে ভালো, কিন্তু যন্ত্র দিয়ে খোঁজা কঠিন — "গত এক ঘণ্টায় কতজন লগইন ব্যর্থ হয়েছে" জানতে পুরো ফাইল পড়তে হয়। Structured Log JSON আকারে ক্ষেত্র (field) ভাগ করে দেয়: `{"level":"info","event":"login","user_id":"u123","success":true,"duration_ms":42}`। এখন যন্ত্র সহজে ফিল্টার করতে পারে: `event=login AND success=false`। প্রোডাকশনে সবসময় Structured Log লিখুন; মানুষের পড়ার সুবিধা Grafana বা Log Viewer দেবে, ফাইল ফরম্যাটকে তা হতে হবে না।

Traces: Span, Trace, Context Propagation

একটি Trace শুরু হয় যখন Request সিস্টেমে প্রবেশ করে। প্রতিটি কাজ একটি Span — নাম, শুরু-শেষ সময়, Attribute (যেমন `http.method`, `db.statement`), Status (ok/error) ও Parent Span-এর রেফারেন্স নিয়ে। Context Propagation মানে Trace ID-টি এক সার্ভিস থেকে আরেক সার্ভিসে HTTP Header (যেমন `traceparent`) দিয়ে পাঠানো — যাতে Frontend, API ও Database Query একই Trace-এর অংশ হয়। Propagation ভেঙে গেলে Trace ছিন্নভিন্ন হয়ে যায় — এটি সবচেয়ে সাধারণ Tracing ব্যর্থতা।

প্রতিটি স্তম্ভ কী পারে না

Metrics পারে না: একটি নির্দিষ্ট ব্যবহারকারীর নির্দিষ্ট Request-এর গল্প বলতে; Error-এর Stack Trace দেখাতে; "কেন এই Query ধীর" বলতে। Logs পারে না: দীর্ঘমেয়াদি Trend দেখাতে; প্রতি সেকেন্ডে কত Error হচ্ছে তা দ্রুত গ্রাফে দেখাতে; একটি Request-এর সম্পূর্ণ যাত্রাপথ এক নজরে দেখাতে। Traces পারে না: সিস্টেম-ব্যাপী Trend বলতে (Sampling-এর কারণে); দীর্ঘ Retention দিতে; প্রতিটি ছোট ঘটনার বিবরণ দিতে (সবকিছু Span করা হয় না)। এই সীমাগুলো জানা থাকলে আপনি সঠিক প্রশ্নের জন্য সঠিক স্তম্ভ বেছে নেবেন।

Correlation কীভাবে কাজ করে

Correlation-এর মূল উপায় তিনটি: (১) একই Request/Trace ID সব স্তম্ভে লেখা; (২) একই Label/Attribute (যেমন `service`, `endpoint`, `environment`) সব জায়গায় ব্যবহার; (৩) সময়ের মিল — একই সময়-জানালায় Metric স্পাইক, Log Error ও Trace-এর ধীর Span মিলিয়ে দেখা। Grafana-তে Exemplar নামে একটি সুবিধা আছে: Metric-এর সঙ্গে একটি Trace ID-র নমুনা জুড়ে দেওয়া যায়, ফলে গ্রাফের একটি বিন্দুতে ক্লিক করেই সংশ্লিষ্ট Trace খোলা যায়। এটি Correlation-এর সবচেয়ে ব্যবহারিক রূপ।

> **PRO TIP:** প্রতিটি Log লাইনে `trace_id` ও `request_id` Field যোগ করুন। এই একটি অভ্যাসই ঘটনার সময় অনুসন্ধানকে ঘণ্টা থেকে মিনিটে নামিয়ে আনে।

২.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

নিচে TechNova-র API-তে তিনটি স্তম্ভের ন্যূনতম বাস্তবায়নের ধারণা দেওয়া হলো — Node.js-এ Prometheus Metric, Structured Log ও Trace Context।

```
# Prometheus Metric Endpoint যাচাই (নমুনা আউটপুট — চিত্রণমূলক)
curl -s http://203.0.113.10:3000/metrics | grep -E 'http_request|process_'
# /metrics Endpoint থেকে Prometheus যা Scrape করে, তা দেখা
# http_request_duration_seconds_bucket — Histogram Bucket
# process_resident_memory_bytes — Gauge: বর্তমান Memory

# Structured Log-এর নমুনা (JSON, এক লাইনে একটি ঘটনা)
# {"level":"info","ts":"2026-11-09T10:15:30Z","service":"technova-api",
#  "event":"order_created","trace_id":"4f9a2c1e","user_id":"u123",
#  "order_id":"ord_8841","duration_ms":212,"status":"success"}
```

```
// Node.js — Structured Log + Metric + Trace ID একসঙ্গে (sketch)
// ফাইল: src/middleware/observability.js (চিত্রণমূলক)
const pino = require('pino');
const logger = pino({ level: 'info' }); // JSON Structured Log দেয়
const client = require('prom-client');

// Histogram: Latency-র বণ্টন মাপা
const httpDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request latency in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.05, 0.1, 0.3, 1, 2, 5], // 50ms থেকে 5s
});

function observability(req, res, next) {
  const start = Date.now();
  // প্রতিটি Request-এ Trace/Request ID তৈরি বা গ্রহণ
  req.traceId = req.headers['x-trace-id'] || generateId();
  res.on('finish', () => {
    const seconds = (Date.now() - start) / 1000;
    httpDuration.observe(
      { method: req.method, route: req.route?.path || 'unknown',
        status_code: res.statusCode }, seconds);
    logger.info({
      event: 'http_request', trace_id: req.traceId,
      method: req.method, route: req.path,
      status_code: res.statusCode, duration_ms: Date.now() - start,
    });
  });
  next();
}
module.exports = { observability, httpDuration };
```

লক্ষ করুন: একই Middleware তিনটি কাজ করে — Metric রেকর্ড করে, Structured Log লেখে, Trace ID বহন করে। Label-এ `route` ব্যবহার করা হয়েছে, `path` নয় — কারণ `/orders/8841` এর মতো Path-এ User ID থাকলে Cardinality বিস্ফোরিত হয় (অধ্যায় ৫-এ বিস্তারিত)।

২.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

একটি সম্পূর্ণ Structured Log লাইন ও একটি Trace Span-এর JSON রূপ পাশাপাশি দেখুন — লক্ষ করুন কীভাবে `trace_id` দুটোকে বাঁধে।

```
// ১) Structured Log — অর্ডার তৈরির ঘটনা
{
  "level": "info",
  "ts": "2026-11-09T10:15:30.212Z",
  "service": "technova-api",
  "env": "production",
  "event": "order_created",
  "trace_id": "4f9a2c1e7b3d4a55",
  "request_id": "req_9f21",
  "user_id": "u123",
  "order_id": "ord_8841",
  "amount_bdt": 2450,
  "duration_ms": 212,
  "status": "success"
}

// ২) একই Request-এর Trace Span (ধারণাগত JSON)
{
  "trace_id": "4f9a2c1e7b3d4a55",
  "span_id": "span_api_01",
  "parent_span_id": null,
  "name": "POST /api/orders",
  "service": "technova-api",
  "start_ms": 0,
  "duration_ms": 212,
  "attributes": { "http.method": "POST", "http.status_code": 200 },
  "children": [
    { "name": "mongodb insert orders", "duration_ms": 48 },
    { "name": "payment gateway call", "duration_ms": 120 }
  ]
}

// ৩) একই ঘটনার Metric (Prometheus exposition format)
http_requests_total{method="POST",route="/api/orders",status_code="200"} 15230
http_request_duration_seconds_bucket{route="/api/orders",le="0.5"} 14980
```

তিনটি রূপের মিল দেখুন: একই `trace_id`, একই `route`, একই সময়। কোনো ব্যবহারকারী অভিযোগ করলে Support প্রথমে Trace ID নেবে, তা দিয়ে Log খুঁজবে, Trace খুলবে, আর Metric দেখে জানবে এটি বিচ্ছিন্ন ঘটনা নাকি ব্যাপক সমস্যা।

২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি: TechNova-র Support-এ অভিযোগ — "আমার অর্ডারটি হয়নি, কিন্তু টাকা কেটেছে।" Order ID: ord_8841। শুধু Metric থাকলে ইঞ্জিনিয়ার বলতে পারতেন "আজ Error Rate ১.২%" — তাতে ওই ব্যবহারকারীর কোনো সাহায্য হয় না। শুধু Log থাকলে তিনি Order ID দিয়ে খুঁজে দেখতেন Payment সফল, কিন্তু Order তৈরি হয়নি — কিন্তু কেন, তা জানতে আরও খুঁজতে হতো।

তিনটি স্তম্ভ একসঙ্গে থাকায় ঘটনাটি এভাবে মেলে: Trace খুলে দেখা যায় Payment Span সফল (১২০ মিলিসেকেন্ড), তারপর Database Insert Span Error দিয়ে শেষ — Duplicate Key Error। Log-এ দেখা যায় একই Order ID দিয়ে দুবার Insert চেষ্টা হয়েছিল — Frontend-এর Retry-এর কারণে। Metric দেখায় গত এক ঘণ্টায় এই ধরনের Duplicate Error ৪০ বার হয়েছে — এটি বিচ্ছিন্ন নয়, একটি Pattern। সমাধান: Idempotency Key বসানো — একই Request দুবার এলেও Order একবারই তৈরি হবে। তিনটি স্তম্ভের প্রতিটি একটি করে সূত্র দিয়েছে; একটিও একা সমাধান দিতে পারত না।

২.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **Metrics শুরু করুন:** API-তে `/metrics` Endpoint যোগ করুন — Request Count (Counter) ও Latency Histogram। Prometheus দিয়ে Scrape করুন (বিস্তারিত অধ্যায় ৩ ও ৫-এ)।
2. **Structured Log চালু করুন:** `console.log`-এর মুক্ত বাক্যের বদলে JSON Log লিখুন — প্রতিটি লাইনে `level`, `ts`, `service`, `event`, `trace_id` থাকবে।
3. **Trace ID বহন করুন:** প্রতিটি Incoming Request-এ `x-trace-id` Header গ্রহণ করুন, না থাকলে তৈরি করুন; Downstream Call-এ (Database নয়, HTTP Call-এ) সেটি Forward করুন।
4. **Correlation পরীক্ষা করুন:** একটি Test Request পাঠিয়ে তার Trace ID দিয়ে Log খুঁজুন, Metric-এ তার প্রভাব দেখুন, Trace-এর Span গাছ দেখুন — তিনটি মেলে কি না যাচাই করুন।
5. **একটি ঘটনা মহড়া করুন:** ইচ্ছাকৃতভাবে একটি ধীর Query তৈরি করুন; Metric-এ p99 স্পাইক, Log-এ Slow Query, Trace-এ দীর্ঘ Span — তিনটি ধরা পড়ে কি না দেখুন।
6. **Retention ঠিক করুন:** Metrics ৩০ দিন, Logs ১৪ দিন, Traces ৭ দিন (Sampling-সহ) — এই ধরনের স্তর ঠিক করুন ও লিখে রাখুন।

২.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	কেন সমস্যা	সঠিক পথ
Log-এ মুক্ত বাক্য লেখা	যন্ত্র দিয়ে খোঁজা যায় না	JSON Structured Log
Metric Label-এ User ID / Order ID	Cardinality বিস্ফোরণ; Prometheus ধীর/ভারী	High-cardinality মান Log/Trace-এ রাখুন
Trace ID ছাড়া Log	Log ও Trace জোড়া দেওয়া যায় না	প্রতিটি Log-এ trace_id
সব Request-এর পূর্ণ Trace সংরক্ষণ	খরচ ও Storage বিস্ফোরণ	Sampling + Error-এর সব Trace রাখা
শুধু Error Log রাখা	সফল Request-এর প্রসঙ্গ হারায়	Info-স্তরে মূল ঘটনা, Debug প্রয়োজনে
Metrics-কে "সত্য" মানা	Aggregate ব্যক্তিগত গল্প লুকায়	Metric দিয়ে শুরু, Log/Trace দিয়ে গভীরে

২.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

তিনটি স্তম্ভেই সংবেদনশীল তথ্য ঢুকতে পারে। Log-এ Password, Token, Authorization Header কখনো লিখবেন না। Trace-এর Attribute-এ Request Body বা Query Parameter পুরো রাখবেন না — তাতে Email, ঠিকানা বা Token

থাকতে পারে। Metric Label-এ ব্যবহারকারীর পরিচয় (Email, ফোন) কখনো দেবেন না — Label সংরক্ষিত হয়, খুঁজে বের করা যায়, এবং Cardinality বাড়ায়।

Log ও Trace সংরক্ষণে Retention ও Access Control ঠিক করুন। Production Trace দেখার অধিকার সীমিত রাখুন। বাংলাদেশের প্রেক্ষাপটে ব্যবহারকারীর ব্যক্তিগত তথ্য সংরক্ষণের ক্ষেত্রে সর্বনিম্ন তথ্য (data minimisation) নীতি মানুন — যা দরকার শুধু তা-ই রাখুন, যতদিন দরকার ততদিনই রাখুন।

২.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Metrics সবচেয়ে সস্তা: Aggregate করা সংখ্যা অল্প জায়গা নেয়, দীর্ঘদিন রাখা যায়। Logs মাঝারি: প্রতিটি লাইন জায়গা নেয়, Volume বাড়লে খরচ বাড়ে — তাই Log Level ঠিক করুন (Production-এ Info, Debug নয়), আর অপ্রয়োজনীয় Log বন্ধ করুন। Traces সবচেয়ে ব্যয়বহুল: প্রতিটি Span সংরক্ষণ করতে হয় — তাই Sampling অপরিহার্য। একটি সাধারণ নিয়ম: সব Error Trace রাখুন, সফল Request-এর একটি নমুনা (যেমন ১০%) রাখুন, আর ধীর Request (যেমন ১ সেকেন্ডের বেশি) সব রাখুন — একে Tail-based Sampling বলে।

Instrumentation-এর নিজের খরচও আছে: প্রতিটি Request-এ Metric Observe করা, Log লেখা, Span তৈরি করা CPU নেয়। কিন্তু সাধারণত এই খরচ ১–৩% এর মধ্যে থাকে — আর ঘটনার সময় যে সময় বাঁচে, তার তুলনায় তা নগণ্য। তবু Synchronous Log লেখা (প্রতিটি Log-এর জন্য Disk অপেক্ষা) এড়িয়ে Asynchronous বা Buffered লেখা ব্যবহার করুন।

২.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরবর্তী ধাপ
Metric আছে, কিন্তু কোন Request তা জানা যায় না	Label-এ ভাঙন নেই; High-cardinality এড়াতে গিয়ে অতিরিক্ত Aggregate	Log/Trace-এ যান; Exemplar যোগ করুন
Log খুঁজে পাওয়া যায় না	Unstructured Log; Clock Skew; ভুল Timezone	UTC Timestamp, Structured Field যাচাই করুন
Trace ছিন্নভিন্ন (অর্ধেক Span অনুপস্থিত)	Context Propagation ভেঙেছে; কোনো সার্ভিস Header Forward করেনি	প্রতিটি Hop-এ traceparent Header যাচাই করুন
Prometheus ধীর / Memory বেশি	High-cardinality Label (User ID, Path)	Label তালিকা পর্যালোচনা করুন; Cardinality কমান
Log Volume হঠাৎ বেড়ে গেল	Debug Level চালু, বা Loop-এ Log	Log Level ও Rate যাচাই করুন

TROUBLESHOOTING: Trace ছিন্নভিন্ন হলে প্রথমে Header দেখুন: Frontend থেকে API-তে `traceparent` যাচ্ছে কি না, API থেকে Downstream-এ যাচ্ছে কি না। বেশিরভাগ Tracing ব্যর্থতা Instrumentation-এর নয়, Propagation-এর।

২.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Structured Log; প্রতিটি Log ও Span-এ Trace ID; Metric-এ কম Cardinality Label; Latency-র জন্য Histogram; Error Trace সব রাখা, সফল Trace Sampling করা; তিনটি স্তম্ভে একই নামকরণ (Service, Environment, Endpoint)।

অ্যান্টি-প্যাটার্ন: `console.log("here")` ধরনের অর্থহীন Log; Metric Label-এ User ID; Log-এ Password; সবকিছু Trace করার চেষ্টা (Over-instrumentation); তিনটি স্তম্ভে তিন রকম নামকরণ — ফলে Correlation অসম্ভব।

> **PRO TIP:** নতুন সার্ভিস শুরু করার সময় প্রথম দিনই তিনটি স্তম্ভের ন্যূনতম সেট বসান — একটি Counter, একটি Histogram, JSON Log, Trace ID। পরে যোগ করা Telemetry প্রায়ই অসম্পূর্ণ থাকে।

২.১৬ অধ্যায়ের সারাংশ

Metrics "কত?" বলে — Trend ও Alert-এর জন্য সস্তা ও শক্তিশালী, কিন্তু "কেন" বলে না। Logs "কী ঘটল?" বলে — বিস্তারিত কারণের জন্য অপরিহার্য, কিন্তু Unstructured হলে খোঁজা কঠিন। Traces "কোথা দিয়ে গেল?" বলে — বিলম্বের উৎস ও কার্যকারণ দেখায়, কিন্তু ব্যয়বহুল বলে Sampling দরকার। তিনটির মিলনবিন্দু হলো Correlation — একই Trace ID, একই Label, একই সময়। TechNova-র অর্ডার ঘটনা দেখায়: Metric Pattern ধরায়, Log কারণ দেখায়, Trace পথ দেখায় — তিনটি মিলেই সমাধান।

২.১৭ মূল প্রযুক্তিগত পরিভাষা

Metrics: সময়ের সঙ্গে পরিবর্তিত সংখ্যাগত পরিমাপ; Aggregate করা হয়, Trend ও Alert-এর জন্য ব্যবহৃত হয়।

Logs: সময়ানুক্রমিক ঘটনার বিবরণ; একটি নির্দিষ্ট মুহূর্তে কী ঘটেছিল তার রেকর্ড।

Traces: একটি Request-এর সিস্টেমজুড়ে যাত্রাপথের রেকর্ড; অনেক Span-এর সমষ্টি।

Span: Trace-এর একটি একক কাজ — যেমন একটি Database Query বা HTTP Call — নাম, সময় ও Attribute-সহ।

Counter: শুধু বাড়ে এমন Metric — যেমন মোট Request সংখ্যা।

Gauge: বাড়ে-কমে এমন Metric — যেমন বর্তমান Memory ব্যবহার।

Histogram: মানের বণ্টন মাপে এমন Metric — Latency-র Percentile হিসাবের ভিত্তি।

Structured Log: JSON-এর মতো ক্ষেত্র-ভাগ করা Log, যা যন্ত্র দিয়ে ফিল্টার ও খোঁজা যায়।

Correlation: একই Trace ID / Label / সময় দিয়ে Metrics, Logs ও Traces-কে জোড়া দেওয়া।

Context Propagation: Trace ID-কে এক সার্ভিস থেকে আরেক সার্ভিসে Header দিয়ে পাঠানো।

Sampling: সব Trace/Log সংরক্ষণ না করে একটি প্রতিনিধিত্বমূলক অংশ রাখা — খরচ নিয়ন্ত্রণের উপায়।

Exemplar: Metric-এর সঙ্গে জুড়ে দেওয়া একটি Trace ID-র নমুনা, যা গ্রাফ থেকে সরাসরি Trace-এ যেতে দেয়।

২.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Metrics, Logs ও Traces — প্রতিটি কোন প্রশ্নের উত্তর দেয়? একটি করে উদাহরণ দিন।
2. Metrics কেন "কেন ভুল হচ্ছে" বলতে পারে না? ব্যাখ্যা করুন।
3. Structured Log কেন Unstructured Log-এর চেয়ে প্রোডাকশনে ভালো?
4. Trace ও Span-এর সম্পর্ক কী? Context Propagation ভেঙে গেলে কী হয়?
5. Counter, Gauge ও Histogram-এর পার্থক্য কী? প্রতিটির একটি TechNova উদাহরণ দিন।
6. Correlation ছাড়া তিনটি স্তম্ভ থাকলে ঘটনার সময় কী সমস্যা হয়?
7. সব Request-এর পূর্ণ Trace সংরক্ষণ করা কেন অবাস্তব? Sampling কীভাবে সাহায্য করে?
8. Metric Label-এ User ID দেওয়া কেন ভুল? কোথায় User ID রাখা উচিত?

২.১৯ ব্যবহারিক অনুশীলন

1. একটি ছোট Express অ্যাপে `prom-client` দিয়ে Request Counter ও Latency Histogram যোগ করুন; `/metrics` খুলে Bucket-গুলো দেখুন।
2. `console.log`-এর বদলে JSON Structured Log লেখে এমন একটি Logger বসান; প্রতিটি লাইনে `trace_id` যোগ করুন।
3. একটি Test Request-এর Trace ID নিয়ে (ক) Log-এ সেটি খুঁজুন, (খ) Metric-এ তার প্রভাব দেখুন, (গ) কাগজে তার Span গাছ আঁকুন — তিনটি মেলে কি না লিখুন।
4. একটি ধীর Database Query ইচ্ছাকৃতভাবে তৈরি করুন; Metric, Log ও Trace — প্রতিটিতে তা কীভাবে ধরা পড়ে, তার বিবরণ লিখুন।
5. আপনার অ্যাপের Log থেকে তিনটি সংবেদনশীল Field খুঁজে বের করুন যা থাকা উচিত নয়, এবং সেগুলো Mask করার পরিকল্পনা লিখুন।

অধ্যায় ৩ • PART I — Observability Foundations

Chapter 3 — Production Monitoring Architecture and Telemetry Data Flow

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Telemetry Pipeline-এর পূর্ণ প্রবাহ ব্যাখ্যা করতে পারবেন: Instrumentation থেকে Collection, Processing, Storage, Query/Visualization ও Alerting পর্যন্ত। আপনি Pull বনাম Push মডেলের পার্থক্য, Agent/Collector/Gateway-এর ভূমিকা, Buffering ও Backpressure কীভাবে ডেটা হারানো ঠেকায়, এবং Retention Tier কীভাবে খরচ ও অনুসন্ধানক্ষমতার ভারসাম্য রাখে তা বলতে পারবেন। TechNova-র জন্য একটি সম্পূর্ণ Monitoring Architecture-এর চিত্র আঁকতে পারবেন।

- Telemetry Pipeline-এর ছয়টি ধাপের নাম ও কাজ ব্যাখ্যা করা।
- Pull ও Push মডেলের সুবিধা-অসুবিধা তুলনা করা।
- Agent, Collector ও Gateway-এর ভূমিকা আলাদা করা।
- Backpressure, Buffering ও Retention Tier-এর নকশা সিদ্ধান্ত নেওয়া।

৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

অনেক দল Telemetry সংগ্রহ শুরু করে টুল বসিয়ে, কিন্তু Pipeline না বুঝে। ফলাফল: অ্যাপ্লিকেশন থেকে ডেটা বেরোয়, কিন্তু কোথাও হারিয়ে যায়; Dashboard খালি থাকে; Alert আসে না; আর ঘটনার সময় দেখা যায় গত তিন দিনের Log-ই নেই। Pipeline হলো Observability-র রক্তসংবহনতন্ত্র — একটি ধাপ ভেঙে গেলে পুরো শরীর অচল।

TechNova যখন একটিমাত্র সার্ভারে চলত, তখন Log ফাইল পড়লেই চলত। এখন Frontend, API, Database, Payment Gateway, Cache — পাঁচটি উৎস থেকে Telemetry আসে। কোনো কেন্দ্রীয় Pipeline ছাড়া এই ডেটা ছড়িয়ে-ছিটিয়ে থাকে, আর ঘটনার সময় ইঞ্জিনিয়ার পাঁচটি সার্ভারে SSH করে খুঁজতে থাকেন। একটি সু-নকশাকৃত Pipeline সেই ছড়ানো ডেটাকে এক জায়গায় এনে, প্রক্রিয়া করে, সংরক্ষণ করে এবং প্রশ্নের উত্তর দেওয়ার মতো করে তোলে। এই অধ্যায় সেই নকশা শেখায়।

৩.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১ ও ২-এর ধারণা (Monitoring বনাম Observability, তিনটি স্তম্ভ), HTTP ও TCP-র মৌলিক ধারণা, এবং Linux-এ Service চালানোর অভিজ্ঞতা (`systemctl`) জানা থাকলে এই অধ্যায় সহজ হবে। Prometheus ও Grafana-র নাম শুনে থাকলেই চলবে — কনফিগারেশন এখানে ধাপে ধাপে ব্যাখ্যা করা হবে।

৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Telemetry তৈরি হয় যেখানে কাজ হয় (Application, OS, Database), কিন্তু প্রশ্ন করা হয় অন্য জায়গায় (Dashboard, Alert, অনুসন্ধান)। এই দুই জায়গার মধ্যে ডেটাকে ভ্রমণ করতে হয় — সংগ্রহ, প্রক্রিয়া, সংরক্ষণ — এবং প্রতিটি ভ্রমণে ডেটা হারাতে পারে, বিলম্বিত হতে পারে, বা বিকৃত হতে পারে। Pipeline-এর কাজ হলো এই ভ্রমণকে নির্ভরযোগ্য করা: ডেটা যাতে না হারায়, সময়মতো পৌঁছায়, এবং প্রশ্নের উপযোগী আকারে থাকে।

দ্বিতীয় নীতি: প্রতিটি ধাপ একটি Trade-off। বেশি বিস্তারিত ডেটা মানে ভালো অনুসন্ধান, কিন্তু বেশি খরচ। দীর্ঘ Retention মানে পুরোনো ঘটনা বিশ্লেষণ করা যায়, কিন্তু বেশি Storage খরচ। Real-time Alert মানে দ্রুত সাড়া, কিন্তু বেশি False Positive-এর ঝুঁকি। ভালো Architecture এই Trade-off-গুলো সচেতনভাবে বেছে নেয়, দুর্ঘটনাবশত নয়।

Pull বনাম Push — দুটি দর্শন

Pull মডেল: কেন্দ্রীয় সার্ভার (যেমন Prometheus Server) পর্যায়ক্রমে প্রতিটি Target-এর কাছে যায় এবং ডেটা টেনে নেয় (Scrape)। Target শুধু একটি HTTP Endpoint (`/metrics`) প্রকাশ করে। সুবিধা: কেন্দ্রীয় নিয়ন্ত্রণ — কোন Target থেকে কত ঘন ঘন নেওয়া হবে, তা এক জায়গায় ঠিক করা যায়; Target ডাউন থাকলে Scrape ব্যর্থ হয়, যা নিজেই একটি সংকেত। অসুবিধা: Firewall/NAT-এর পেছনের Target-এ পৌঁছানো কঠিন; খুব অল্পস্থায়ী কাজ (Short-lived Job) শেষ হওয়ার আগেই Scrape না-ও হতে পারে।

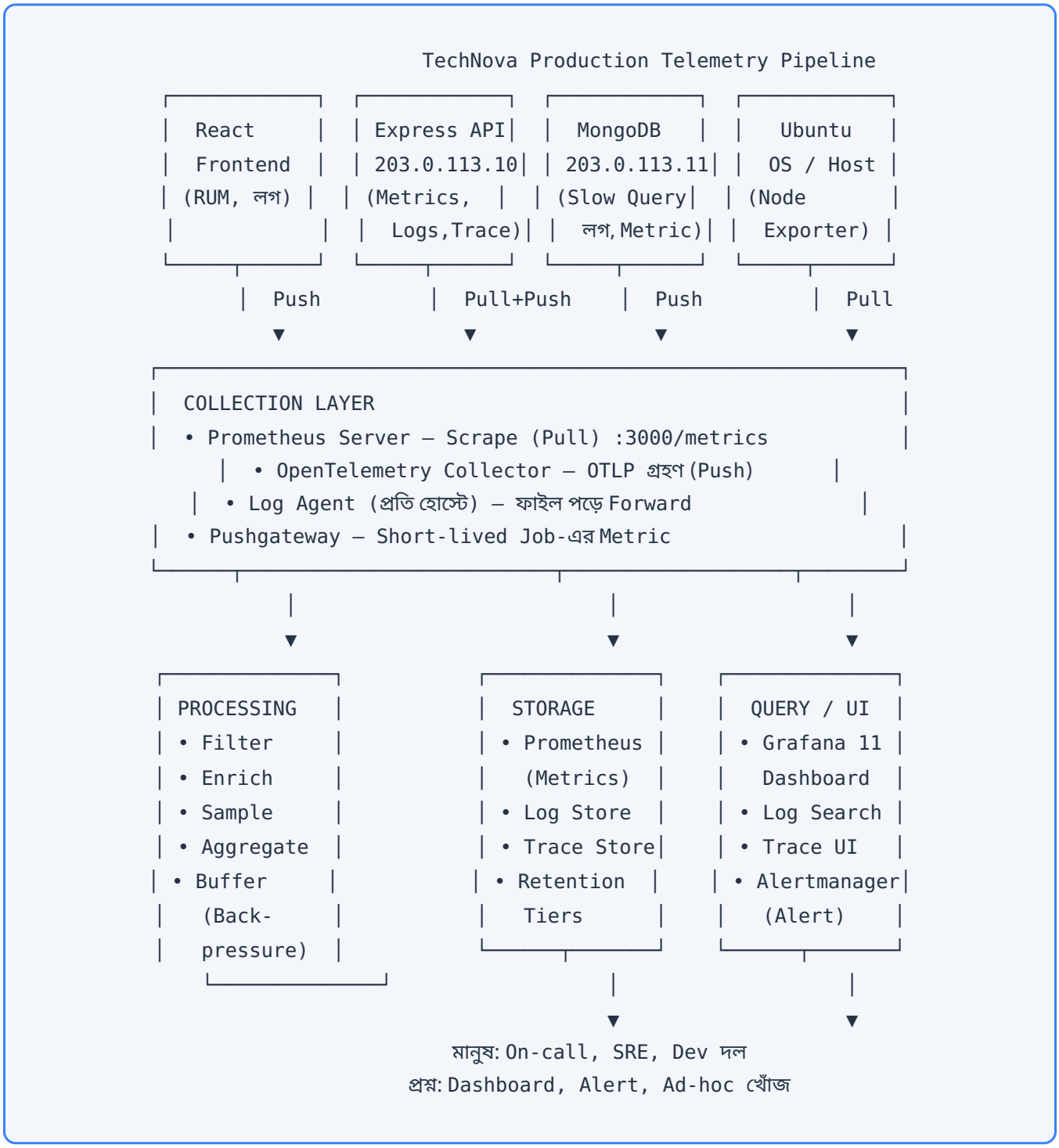
Push মডেল: প্রতিটি উৎস নিজে থেকে ডেটা পাঠায় — Log Agent Log পাঠায়, Application Trace পাঠায়, Short-lived Job শেষে Metric Push করে। সুবিধা: Firewall পেরিয়ে বাইরে পাঠানো সহজ; অল্পস্থায়ী কাজও ডেটা পাঠাতে পারে; Real-time-এর কাছাকাছি। অসুবিধা: উৎসের সংখ্যা বাড়লে কেন্দ্রীয় দিককে অনেক সংযোগ সামলাতে হয়; একটি উৎস হঠাৎ বেশি ডেটা পাঠালে কেন্দ্রীয় দিক চাপে পড়ে (এখানেই Backpressure দরকার)। বাস্তবে দুটোই ব্যবহৃত হয়: Prometheus-এ Metrics Pull, Logs ও Traces Push — এটাই TechNova-র জন্য প্রস্তাবিত মিশ্রণ।

> **CONCEPT:** Pull মানে "আমি এসে নিয়ে যাব", Push মানে "তুমি পাঠিয়ে দাও"। কোনটি ভালো তা নির্ভর করে কে নিয়ন্ত্রণ করবে, Firewall কেমন, আর কাজটি কতক্ষণ বাঁচে তার উপর। একটি নয়, দুটোর মিশ্রণই বাস্তবসম্মত।

Agent, Collector, Gateway — তিনটি ভূমিকা

Agent উৎসের পাশে বসে — প্রতিটি সার্ভারে একটি ছোট প্রোগ্রাম যা Log ফাইল পড়ে, System Metric সংগ্রহ করে, বা Application থেকে Telemetry নেয়। **Collector** অনেক Agent থেকে ডেটা নিয়ে একত্র করে, Filter করে, Enrich করে (যেমন Service নাম, Environment Label যোগ করে) এবং সামনে পাঠায়। **Gateway** হলো প্রবেশদ্বার — বাইরের নেটওয়ার্ক থেকে আসা Push ডেটা গ্রহণ করে, প্রমাণীকরণ (Authentication) যাচাই করে, Rate Limit করে, এবং ভেতরের Storage-এ পাঠায়। Prometheus-জগতে Pushgateway একটি Gateway-এর উদাহরণ — Short-lived Job-এর Metric গ্রহণ করে। OpenTelemetry Collector একটি Collector-এর উদাহরণ — Metrics, Logs, Traces তিনটিই গ্রহণ ও প্রক্রিয়া করতে পারে।

৩.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



এই চিত্রের মূল বার্তা: উৎস অনেক, কিন্তু প্রবাহ একমুখী ও স্তরবদ্ধ। প্রতিটি স্তরের একটি দায়িত্ব আছে। Collection স্তর ডেটা হারাতে দেবে না; Processing স্তর ডেটাকে প্রশ্নের উপযোগী করবে; Storage স্তর খরচ ও Retention-এর ভারসাম্য রাখবে; Query স্তর মানুষকে উত্তর দেবে। কোনো স্তর বাদ দিলে পরের স্তর ভেঙে পড়ে।

৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Instrumentation → Collection → Processing → Storage → Query → Alerting

Instrumentation: কোড ও সিস্টেমে Telemetry তৈরির ব্যবস্থা (অধ্যায় ২ ও ৫)। **Collection:** Telemetry সংগ্রহ — Scrape বা Push, Agent বা সরাসরি। **Processing:** সংগৃহীত ডেটাকে ব্যবহারযোগ্য করা — অপ্রয়োজনীয় Field বাদ দেওয়া (Filter), Label যোগ করা (Enrich), নমুনা নেওয়া (Sampling), প্রাথমিক Aggregate (যেমন ১ মিনিটের গড়)। **Storage:** Time-series Database (Metrics), Index-সহ Log Store, Trace Store — প্রতিটির নিজস্ব Retention। **Query/Visualization:** PromQL (Prometheus Query Language), Log Query, Trace Search, Grafana Dashboard। **Alerting:** Query-এর ফলাফল Threshold ছাড়ালে Notification — Alertmanager-এর মাধ্যমে।

Buffering ও Backpressure

হঠাৎ ট্রাফিক বাড়লে (Flash Sale, বা কোনো Bug-এর কারণে Log বিস্ফোরণ) Collection স্তর Storage-এর চেয়ে দ্রুত ডেটা পেতে পারে। তখন দুটি উপায়: **Buffering** — ডেটা সাময়িকভাবে Queue-তে (যেমন Disk-এ) জমিয়ে রেখে ধীরে ধীরে পাঠানো; **Backpressure** — উৎসকে জানানো "ধীরে পাঠাও" বা অগ্রাধিকার অনুযায়ী কিছু ডেটা বাদ দেওয়া। Buffer ছাড়া Pipeline-এ একটি ধাক্কায় ডেটা হারায় — আর ঘটনার ঠিক মুহূর্তের ডেটা হারানোই সবচেয়ে ক্ষতিকর। তাই Agent-এ Local Buffer (Disk Queue) রাখা সেরা অনুশীলন: নেটওয়ার্ক বা কেন্দ্রীয় দিক ডাউন থাকলেও Agent ডেটা ধরে রাখে, পরে পাঠায়।

Retention Tier — গরম, উষ্ণ, ঠান্ডা

সব ডেটা সমান দামি নয়। **Hot Tier** (সাম্প্রতিক ২৪–৭২ ঘণ্টা): দ্রুত Disk-এ, দ্রুত অনুসন্ধান — ঘটনার সময় এটাই ব্যবহৃত হয়। **Warm Tier** (১–৪ সপ্তাহ): সস্তা Storage-এ, ধীর অনুসন্ধান — Trend ও সাপ্তাহিক পর্যালোচনার জন্য। **Cold Tier** (মাস–বছর): Archive, যেমন Object Storage-এ Compressed — Compliance বা বার্ষিক বিশ্লেষণের জন্য। Metrics-এর ক্ষেত্রে Downsampling করা হয়: পুরোনো ডেটার বিস্তারিত Resolution কমিয়ে (যেমন ১৫ সেকেন্ড থেকে ৫ মিনিট) জায়গা বাঁচানো হয়। এই স্তরবিন্যাস ছাড়া Storage খরচ অনিয়ন্ত্রিতভাবে বাড়ে।

Service Discovery — Target কীভাবে খুঁজে পাওয়া যায়

Prometheus-কে বলতে হয় কোন Target Scrape করতে হবে। Static তালিকা ছোট পরিবেশে চলে, কিন্তু সার্ভার বাড়লে Service Discovery দরকার — যেমন DNS, File-based Discovery, বা Kubernetes Discovery। TechNova-র মতো ছোট পরিবেশে একটি YAML ফাইলে Target তালিকা রাখা ও Git-এ সংস্করণ নিয়ন্ত্রণ (Version Control) করাই যথেষ্ট ও নিরাপদ।

High Availability — Monitoring-এর Monitoring

Monitoring ব্যবস্থা নিজে ডাউন হলে আপনি অন্ধ। তাই Prometheus-এর দুটি Instance (HA Pair) চালানো, Grafana-র Database Backup নেওয়া, আর Alertmanager-এর নিজের স্বাস্থ্য পর্যবেক্ষণ করা দরকার — একে বলে "Who watches the watchers?"। অন্তত একটি বাইরের Uptime Check (Black-box) রাখুন যা আপনার Monitoring Stack-এর বাইরে থেকে API-র স্বাস্থ্য যাচাই করে — যাতে Monitoring ডাউন থাকলেও আপনি জানতে পারেন।

> **WARNING:** Monitoring Stack-কে প্রোডাকশন অ্যাপ্লিকেশনের একই সার্ভারে চালাবেন না। অ্যাপ্লিকেশন সার্ভার ডাউন হলে Monitoring-ও ডাউন হবে — ঠিক যখন তার সবচেয়ে বেশি দরকার। আলাদা হোস্ট (যেমন 203.0.113.20) ব্যবহার করুন।

৩.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

নিচে TechNova-র Prometheus Server-এর মূল কনফিগারেশন — প্রতিটি লাইনের ব্যাখ্যাসহ। এটি `prometheus.yml`-এর একটি বৈধ, ন্যূনতম উদাহরণ।

```
# /etc/prometheus/prometheus.yml (Ubuntu Server 24.04, Prometheus 3.x)
global:
  scrape_interval: 15s      # প্রতি 15 সেকেন্ডে Target Scrape করা হবে
  evaluation_interval: 15s  # প্রতি 15 সেকেন্ডে Alert Rule মূল্যায়ন
  external_labels:
    env: production         # সব Metric-এ 'production' Label যোগ হবে
    cluster: technova-bd    # কোন Cluster, তা চিহ্নিত করতে

# Alertmanager — Alert পাঠানোর ঠিকানা
alerting:
  alertmanagers:
    - static_configs:
        - targets: ['203.0.113.20:9093']

# Alert Rule ফাইল (অধ্যায় ৪-এ বিস্তারিত)
rule_files:
  - /etc/prometheus/rules/*.yml

# Scrape Target-সমূহ
scrape_configs:
  - job_name: 'technova-api'       # API সার্ভিসের কাজের নাম
    static_configs:
      - targets: ['203.0.113.10:3000']  # API-র /metrics Endpoint
        labels:
          service: 'technova-api'

  - job_name: 'node'               # হোস্ট-স্তরের Metric
    static_configs:
      - targets: ['203.0.113.10:9100', '203.0.113.11:9100']
        labels:
          service: 'node-exporter'

  - job_name: 'prometheus'         # নিজের স্বাস্থ্য (Self-monitoring)
    static_configs:
      - targets: ['localhost:9090']
```

```
# কনফিগারেশন যাচাই ও সার্ভিস চালু (নমুনা আউটপুট — চিত্রণমূলক)
promtool check config /etc/prometheus/prometheus.yml
# "SUCCESS" দেখালে কনফিগারেশন বৈধ

sudo systemctl restart prometheus
sudo systemctl status prometheus --no-pager | head -n 15
# Active: active (running) দেখুন

# Target-গুলো Scrape হচ্ছে কি না যাচাই
curl -s http://203.0.113.20:9090/api/v1/targets | head -c 800
# "health": "up" থাকা Target-গুলো সচল; "down" থাকলে কারণ দেখুন
```

৩.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

নিচে TechNova-র সম্পূর্ণ Pipeline-এর একটি সংক্ষিপ্ত কিন্তু বাস্তবসম্মত চিত্র — Prometheus কনফিগারেশন, OpenTelemetry Collector-এর ধারণা, এবং Grafana ডেটা সোর্স।

```
# OpenTelemetry Collector — ধারণাগত কনফিগারেশন (otel-collector.yaml)
receivers:
  otlp:
    protocols:
      http:
        endpoint: 0.0.0.0:4318   # Application থেকে Trace/Log গ্রহণ
      grpc:
        endpoint: 0.0.0.0:4317

processors:
  batch: {}                      # ছোট ছোট ডেটা জড়ো করে পাঠানো
  memory_limiter:
    limit_mib: 512               # Memory সীমা — Backpressure-এর অংশ

exporters:
  prometheus:
    endpoint: 0.0.0.0:9464       # Metrics Prometheus-এর জন্য প্রকাশ
  # Log ও Trace Exporter এখানে যোগ হবে (Storage অনুযায়ী)

service:
  pipelines:
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [prometheus]
```

```
# Grafana 11.x — Prometheus ডেটা সোর্স (UI-তে যোগ করার তথ্য)
# Name: Prometheus-Technova
# Type: Prometheus
# URL: http://203.0.113.20:9090
# Access: Server (Grafana সার্ভার থেকে Prometheus-এ যাবে)
# Scrape interval: 15s
# (নমুনা — আসল পাসওয়ার্ড বা Secret এখানে লিখবেন না)
```

এই কনফিগারেশনে লক্ষ করুন: Collector-এ `memory_limiter` আছে — এটি Backpressure-এর একটি রূপ। Memory সীমা ছাড়ালে Collector নতুন ডেটা গ্রহণ ধীর করে, ফলে নিজে Crash করে না। ছোট একটি সেটিং, কিন্তু Pipeline-এর স্থিতিশীলতায় বড় ভূমিকা রাখে।

৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি: TechNova-র Flash Sale-এর রাতে হঠাৎ Log Volume দশ গুণ বেড়ে যায় — একটি Bug-এর কারণে প্রতিটি Request-এ দশটি করে Error Log লেখা হচ্ছিল। Log Agent-এর Buffer ভরে যায়, Collector-এর Memory বাড়তে থাকে, আর Grafana-র Log অনুসন্ধান ধীর হয়ে যায়। কিন্তু API নিজে সচল ছিল — সমস্যা ছিল Pipeline-এ।

কারণ অনুসন্ধানে দেখা যায়: (১) Agent-এ Disk Buffer ছিল না — Memory Buffer ভরে গেলে ডেটা হারাচ্ছিল; (২) Collector-এ কোনো Rate Limit ছিল না; (৩) Log Level Production-এ Debug ছিল। সমাধান: Agent-এ Disk Queue চালু, Collector-এ Memory Limiter ও Sampling, আর Log Level Info-তে নামানো। শিক্ষা: Pipeline-এর প্রতিটি স্তরে

সীমা (Limit) ও Buffer থাকা উচিত — নইলে একটি উৎসের সমস্যা পুরো Pipeline-কে ফেলে দেয়। একে বলে Bulkhead — জাহাজের একটি কক্ষে পানি ঢুকলে যাতে পুরো জাহাজ না ডোবে।

৩.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **আলাদা হোস্ট ঠিক করুন:** Monitoring Stack (Prometheus + Grafana + Alertmanager) আলাদা হোস্টে (203.0.113.20) বসান — অ্যাপ্লিকেশন হোস্ট থেকে আলাদা।
2. **Prometheus বসান:** ৩.৭-এর কনফিগারেশন দিয়ে Prometheus চালু করুন; `promtool check config` দিয়ে যাচাই করুন।
3. **Node Exporter বসান:** প্রতিটি হোস্টে (203.0.113.10, 203.0.113.11) Node Exporter চালু করুন — হোস্ট-স্তরের Metric (CPU, Memory, Disk) পেতে।
4. **API-তে /metrics যোগ করুন:** অধ্যায় ২-এর Middleware ব্যবহার করে API-তে Metric Endpoint প্রকাশ করুন।
5. **Grafana যোগ করুন:** Prometheus-কে Data Source হিসেবে যোগ করুন; একটি Dashboard বানান — Request Rate, Error Rate, p99 Latency, CPU/Memory।
6. **Buffer ও Limit যাচাই করুন:** Collector/Agent-এ Disk Buffer ও Memory Limit আছে কি না দেখুন; না থাকলে যোগ করুন।
7. **ব্যর্থতার মহড়া করুন:** Prometheus বন্ধ করে দেখুন Agent ডেটা ধরে রাখে কি না; চালু করলে পেছনের ডেটা আসে কি না।

৩.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	পরিণতি
Monitoring Stack অ্যাপ্লিকেশনের একই সার্ভারে	অ্যাপ ডাউন হলে Monitoring-ও ডাউন — অন্ধ হয়ে যাওয়া
Buffer ছাড়া Pipeline	হঠাৎ চাপে ডেটা হারানো — ঘটনার মুহূর্তের ডেটাই নেই
সব ডেটা একই Retention-এ রাখা	Storage খরচ বিস্ফোরণ; পুরোনো বিস্তারিত ডেটা অপ্রয়োজনীয়
Scrape Interval খুব কম (যেমন ১ সেকেন্ড)	Prometheus ও Target-এর উপর অপ্রয়োজনীয় চাপ
Alertmanager ছাড়া Alert	Alert হারায়, Duplicate হয়, নীরব থাকে
Monitoring-এর নিজের স্বাস্থ্য না দেখা	Monitoring ডাউন থাকলে কেউ জানে না

৩.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Telemetry Pipeline-এ নিরাপত্তার তিনটি দিক আছে। প্রথমত, **পরিবহন (Transport):** Agent থেকে Collector, Collector থেকে Storage — সব সংযোগে TLS ব্যবহার করুন; অভ্যন্তরীণ নেটওয়ার্ক বলেও Plaintext ধরে নেবেন না। দ্বিতীয়ত, **প্রমাণীকরণ:** Push Endpoint-এ (যেমন OTLP Receiver, Pushgateway) Authentication ছাড়া খোলা রাখবেন না — নইলে যে কেউ ভুয়া Metric পাঠিয়ে Dashboard নষ্ট করতে পারে। তৃতীয়ত, **প্রবেশ নিয়ন্ত্রণ:** Prometheus ও Grafana-র UI-তে Authentication চালু করুন; `/metrics` Endpoint-এ সংবেদনশীল তথ্য না থাকে তা যাচাই করুন। Grafana-র Admin Password কখনো উদাহরণে আসলটি লিখবেন না — `CHANGE-ME-GRAFANA-ADMIN-...` ব্যবহার করুন।

৩.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Pipeline-এর খরচের বড় অংশ তিন জায়গায়: Scrape/Ingest (Network ও CPU), Storage (Disk, Retention), Query (Dashboard ও Alert Evaluation)। Scrape Interval ১৫ সেকেন্ড থেকে ৫ সেকেন্ডে নামালে Storage প্রায় তিন গুণ বাড়ে, কিন্তু ঘটনা ধরার গতি খুব বেশি বাড়ে না — বেশিরভাগ ক্ষেত্রে ১৫ সেকেন্ডই যথেষ্ট। Retention-এর ক্ষেত্রে Downsampling ব্যবহার করুন: সাম্প্রতিক ডেটা বিস্তারিত, পুরোনো ডেটা সংক্ষিপ্ত। Log-এর ক্ষেত্রে Index-এর খরচ মনে রাখুন — প্রতিটি Field Index করলে Storage বাড়ে; শুধু খোঁজা হবে এমন Field Index করুন।

৩.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরবর্তী ধাপ
Target "down" দেখাচ্ছে	Firewall, ভুল Port, অ্যাপ বন্ধ	Target হোস্ট থেকে `curl /metrics` করুন; Port যাচাই করুন
Metric আসছে কিন্তু Grafana খালি	ভুল Data Source, ভুল Query, Time Range	Prometheus UI-তে PromQL চালিয়ে যাচাই করুন
Log-এ ফাঁক (Gap) আছে	Buffer পূর্ণ, Agent বন্ধ, Clock Skew	Agent Log দেখুন; Disk Buffer যাচাই করুন
Prometheus Memory বাড়ছে	High Cardinality, কম Scrape Interval	Cardinality পরীক্ষা করুন; Retention দেখুন
Alert আসছে না	Alertmanager সংযোগ, Rule Evaluation বন্ধ	Prometheus-এ Rule Status দেখুন; Alertmanager UI যাচাই করুন

TROUBLESHOOTING: Pipeline-এ সমস্যা হলে ধাপে ধাপে যান: উৎসে ডেটা তৈরি হচ্ছে কি না → Agent/Collector পাচ্ছে কি না → Storage-এ পৌঁছেছে কি না → Query উত্তর দিচ্ছে কি না। মাঝের ধাপ এড়িয়ে সরাসরি Dashboard দোষ দেবেন না।

৩.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Monitoring Stack আলাদা হোস্টে; প্রতিটি স্তরে Buffer ও Limit; Retention Tier ও Downsampling; কনফিগারেশন Git-এ; Self-monitoring (Prometheus নিজের Metric); বাইরের Uptime Check।

অ্যান্টি-প্যাটার্ন: একটিমাত্র Prometheus Instance-এর উপর সব নির্ভরতা; Buffer ছাড়া Push; Scrape Interval অকারণে কমানো; Dashboard-এ সরাসরি Production Database Query; Alert Rule Git-এ না রাখা — ফলে কে কী বদলেছে জানা যায় না।

PRO TIP: Pipeline-এর প্রতিটি কনফিগারেশন ফাইল Git-এ রাখুন। ঘটনার পর "কী বদলেছে?" প্রশ্নের উত্তর Git History দেবে — অনুমান নয়।

৩.১৬ অধ্যায়ের সারাংশ

Telemetry Pipeline ছয় ধাপে চলে: Instrumentation, Collection, Processing, Storage, Query/Visualization, Alerting। Metrics-এর জন্য Pull (Prometheus Scrape) আর Logs/Traces-এর জন্য Push — দুটোর মিশ্রণই বাস্তবসম্মত। Agent উৎসের পাশে, Collector একত্র করে, Gateway প্রবেশদ্বার সামলায়। Buffering ও Backpressure হঠাৎ চাপে ডেটা হারানো ঠেকায়; Retention Tier (Hot/Warm/Cold) ও Downsampling খরচ নিয়ন্ত্রণ করে। Monitoring Stack আলাদা হোস্টে রাখা, Self-monitoring করা, আর কনফিগারেশন Git-এ রাখা — এগুলো ছোট সিদ্ধান্ত, কিন্তু ঘটনার সময় এগুলোই পার্থক্য গড়ে দেয়।

৩.১৭ মূল প্রযুক্তিগত পরিভাষা

Telemetry Pipeline: Telemetry-র উৎস থেকে ব্যবহারকারী পর্যন্ত প্রবাহের ধাপসমূহ — সংগ্রহ, প্রক্রিয়া, সংরক্ষণ, অনুসন্ধান, সতর্কতা।

Pull (Scrape): কেন্দ্রীয় সার্ভার Target-এর কাছে গিয়ে ডেটা টেনে নেওয়ার মডেল; Prometheus এভাবে কাজ করে।

Push: উৎস নিজে থেকে কেন্দ্রীয় দিকে ডেটা পাঠানোর মডেল; Logs ও Traces-এ সাধারণ।

Agent: উৎসের পাশে চলা ছোট প্রোগ্রাম যা Telemetry সংগ্রহ করে Forward করে।

Collector: অনেক উৎস থেকে ডেটা নিয়ে Filter, Enrich ও Forward করে এমন উপাদান; যেমন OpenTelemetry Collector।

Gateway: Push ডেটা গ্রহণের প্রবেশদ্বার; Authentication ও Rate Limit করে; যেমন Pushgateway।

Buffering: ডেটা সাময়িকভাবে Queue-তে জমিয়ে রেখে ধীরে ধীরে পাঠানো; হঠাৎ চাপ সামলায়।

Backpressure: নিচের স্তর চাপে পড়লে উপরের স্তরকে ধীরে পাঠাতে বলার ব্যবস্থা।

Retention Tier: ডেটার বয়স অনুযায়ী সংরক্ষণের স্তর — Hot (দ্রুত), Warm (সস্তা), Cold (Archive)।

Downsampling: পুরোনো ডেটার Resolution কমিয়ে Storage বাঁচানো।

Service Discovery: Scrape করার Target-গুলো স্বয়ংক্রিয়ভাবে খুঁজে পাওয়ার ব্যবস্থা।

PromQL: Prometheus-এর Query ভাষা; Metric থেকে প্রশ্নের উত্তর বের করতে ব্যবহৃত হয়।

৩.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Telemetry Pipeline-এর ছয়টি ধাপের নাম লিখুন এবং প্রতিটির কাজ এক লাইনে ব্যাখ্যা করুন।
2. Pull ও Push মডেলের পার্থক্য কী? কোন ক্ষেত্রে কোনটি ভালো?
3. Agent, Collector ও Gateway-এর ভূমিকা কীভাবে আলাদা?
4. Buffering না থাকলে হঠাৎ ট্রাফিক বাড়লে কী হয়?
5. Retention Tier (Hot/Warm/Cold) কেন দরকার? Downsampling কীভাবে সাহায্য করে?
6. Monitoring Stack অ্যাপ্লিকেশনের একই সার্ভারে রাখা কেন বিপজ্জনক?
7. Short-lived Job-এর Metric সংগ্রহে Pushgateway কেন দরকার?
8. "Who watches the watchers?" — এই প্রশ্নের বাস্তব অর্থ কী?

৩.১৯ ব্যবহারিক অনুশীলন

1. ৩.৫-এর চিত্রটি দেখে TechNova-র জন্য আপনার নিজের Pipeline চিত্র আঁকুন — উৎস, Collection, Storage ও Query স্তরসহ।
2. একটি Ubuntu সার্ভারে Prometheus বসিয়ে ৩.৭-এর কনফিগারেশন দিয়ে একটি Target Scrape করুন; Targets পৃষ্ঠায় `up` দেখুন।
3. Node Exporter বসিয়ে হোস্টের CPU ও Memory Metric Grafana-তে গ্রাফ আঁকুন।
4. Prometheus বন্ধ করে Agent/Collector ডেটা ধরে রাখে কি না পরীক্ষা করুন; চালু করলে ডেটা ফিরে আসে কি না লিখুন।
5. আপনার Pipeline-এর Retention পরিকল্পনা লিখুন: Metrics, Logs ও Traces প্রতিটি কতদিন রাখবেন, কোন Tier-এ — যুক্তিসহ।

অধ্যায় ৪ • PART I — Observability Foundations

Chapter 4 — SLIs, SLOs, SLAs, Error Budgets, and Golden Signals

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি SLI, SLO, SLA ও Error Budget-এর সংজ্ঞা ও পার্থক্য ব্যাখ্যা করতে পারবেন এবং সংখ্যা দিয়ে হিসাব করতে পারবেন — যেমন ৯৯.৯% Availability মানে মাসে কত মিনিট Downtime অনুমোদিত। আপনি Golden Signals (Latency, Traffic, Errors, Saturation), RED Method ও USE Method প্রয়োগ করে TechNova-র জন্য সঠিক SLI বেছে নিতে পারবেন এবং Error Budget Policy লিখতে পারবেন।

- SLI, SLO, SLA ও Error Budget-এর সংজ্ঞা ও সম্পর্ক ব্যাখ্যা করা।
- Availability শতাংশ থেকে অনুমোদিত Downtime হিসাব করা।
- Golden Signals দিয়ে Dashboard ও Alert নকশা করা।
- RED ও USE Method-এর পার্থক্য ও প্রয়োগ জানা।
- Error Budget শেষ হলে কী করতে হয়, তার নীতি লেখা।

৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

"সিস্টেম ঠিক আছে" — এই বাক্যটির কোনো অর্থ নেই, যতক্ষণ না "ঠিক" শব্দটি সংখ্যায় সংজ্ঞায়িত হয়। একটি দল বলে "আমাদের সিস্টেম ৯৯% আপ থাকে", আরেক দল বলে "আমাদের API দ্রুত" — দুটো বাক্যই অস্পষ্ট। SLI/SLO সেই অস্পষ্টতাকে সংখ্যায় রূপান্তর করে: কোন পরিমাপ, কত লক্ষ্য, কত সময়ের মধ্যে। এটি ছাড়া DevOps দল ও ব্যবসায়িক দল একে অপরকে বোঝে না — একজন বলে "আরও Feature চাই", আরেকজন বলে "আরও স্থিতিশীলতা চাই" — কিন্তু কেউ জানে না কতটা স্থিতিশীলতা যথেষ্ট।

Error Budget এই বিতর্কের সমাধান দেয়: এটি স্থিতিশীলতা ও গতির মধ্যে একটি সংখ্যাগত চুক্তি। Budget থাকলে Feature ছাড়া যায়; Budget শেষ হলে Feature থামিয়ে স্থিতিশীলতায় মন দিতে হয়। TechNova-র মতো ছোট দলে এটি বিশেষভাবে দরকারি — কারণ সেখানে একজন ইঞ্জিনিয়ারই Feature লেখেন আর On-call-ও সামলান। সংখ্যা ছাড়া তিনি প্রতিদিন এই টানাপোড়েনে ভোগেন; সংখ্যা থাকলে সিদ্ধান্ত স্পষ্ট হয়।

৪.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১–৩-এর ধারণা (Golden Signals-এর নাম, Metrics ও Percentile), শতাংশের মৌলিক হিসাব, এবং HTTP Status Code জানা থাকলে এই অধ্যায় সহজ হবে। PromQL-এর বিস্তারিত জানা আবশ্যক নয় — উদাহরণগুলো ধাপে ধাপে ব্যাখ্যা করা হবে।

৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: ব্যবহারকারী সার্ভারের CPU দেখে না, সে দেখে তার Request সফল হলো কি না, কত দ্রুত হলো। তাই নির্ভরযোগ্যতা (Reliability) মাপতে হবে ব্যবহারকারীর দৃষ্টিকোণ থেকে, সার্ভারের দৃষ্টিকোণ থেকে নয়। একটি সার্ভার ১০০% চালু থাকতে পারে, অথচ ব্যবহারকারীর Request ব্যর্থ হতে পারে — কারণ Database সংযোগ ভেঙেছে, বা একটি ভুল Deploy হয়েছে। তাই SLI সবসময় ব্যবহারকারীর অভিজ্ঞতার কাছাকাছি কিছু মাপে।

দ্বিতীয় নীতি: ১০০% নির্ভরযোগ্যতা অসম্ভব ও অপ্রয়োজনীয়। ১০০% চাইলে খরচ অসীম হয়, আর পরিবর্তনের গতি শূন্য হয়। প্রশ্নটি "কতটা নির্ভরযোগ্য?" — উত্তরটি ব্যবসা ও ব্যবহারকারীর প্রয়োজন থেকে আসে। একটি ব্যাংকিং লেনদেনের জন্য ৯৯.৯৯% দরকার হতে পারে, একটি অভ্যন্তরীণ Admin প্যানেলের জন্য ৯৯% যথেষ্ট হতে পারে। SLO সেই "কতটা" সংখ্যায় লেখে।

SLI — Service Level Indicator (পরিমাপ)

SLI হলো পরিমাপ — একটি সংখ্যা যা সেবার মান প্রকাশ করে। যেমন: "গত ৩০ দিনে সফল Request-এর অনুপাত" বা "p95 Latency"। SLI-এর দুটি অংশ: কী মাপা হচ্ছে (যেমন সফল Request-এর অনুপাত) এবং কীভাবে মাপা হচ্ছে (কোন Data Source, কোন Time Window)। ভালো SLI-এর বৈশিষ্ট্য: ব্যবহারকারীর অভিজ্ঞতার সঙ্গে সরাসরি সম্পর্ক, সহজে বোঝা যায়, এবং নির্ভরযোগ্যভাবে মাপা যায়।

SLO — Service Level Objective (লক্ষ্য)

SLO হলো SLI-এর লক্ষ্যমান — যেমন "গত ৩০ দিনে সফল Request-এর অনুপাত অন্তত ৯৯.৯% হবে"। SLO-তে তিনটি অংশ থাকে: SLI, লক্ষ্যমান (৯৯.৯%), এবং সময়কাল (৩০ দিন)। SLO একটি অভ্যন্তরীণ লক্ষ্য — এটি দলকে বলে কতটা ভালো হতে হবে। SLO ভঙ্গ হলে কেউ জরিমানা দেয় না; কিন্তু দল জানে যে স্থিতিশীলতায় মনোযোগ দিতে হবে।

SLA — Service Level Agreement (চুক্তি)

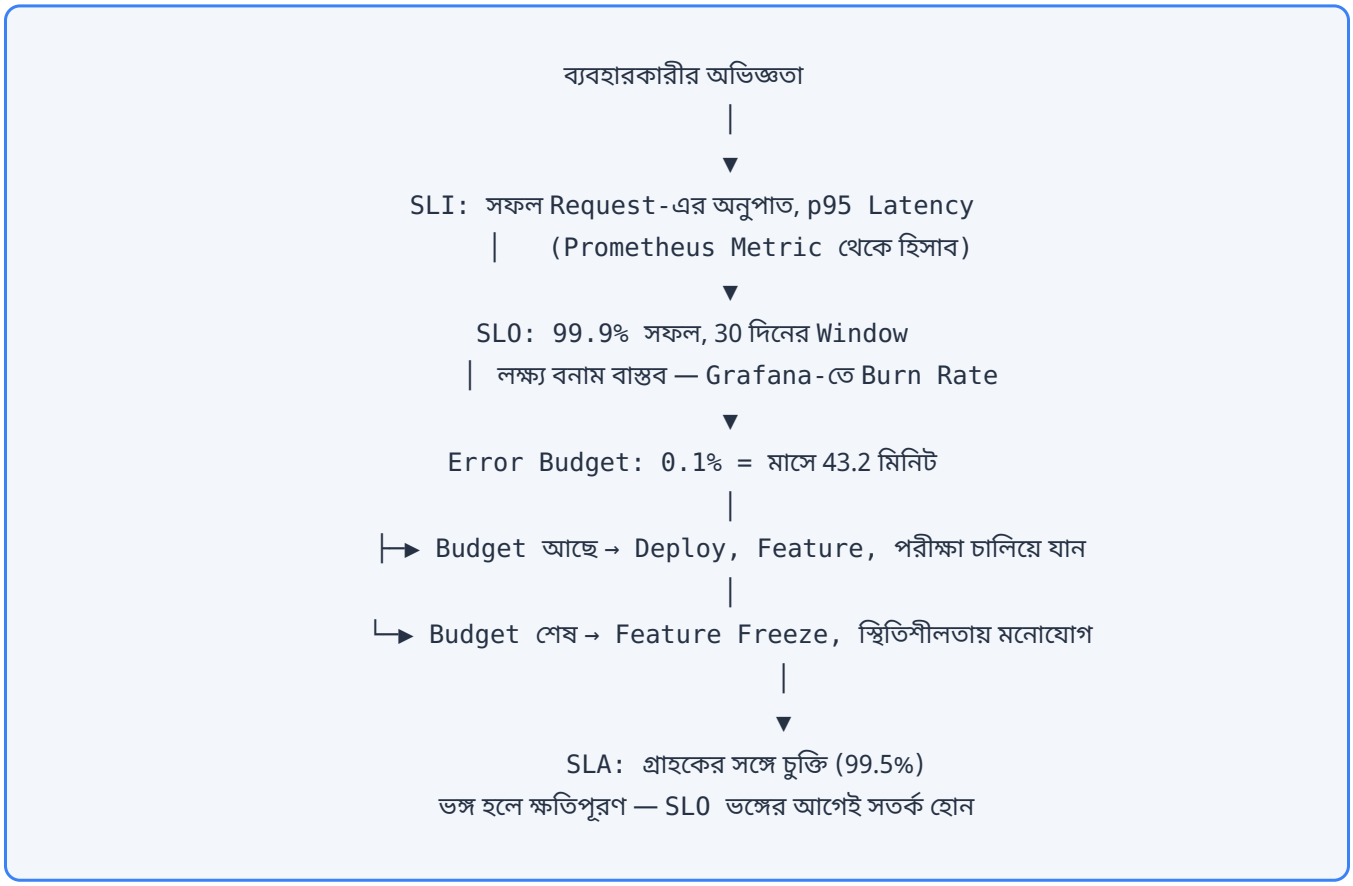
SLA হলো আইনগত বা বাণিজ্যিক চুক্তি — গ্রাহকের সঙ্গে লিখিত প্রতিশ্রুতি, যেখানে লক্ষ্য পূরণ না হলে আর্থিক ক্ষতিপূরণ (Credit) বা অন্য পরিণতি থাকে। SLA সাধারণত SLO-র চেয়ে শিথিল হয় — যেমন SLO ৯৯.৯% হলে SLA ৯৯.৫% হতে পারে — যাতে অভ্যন্তরীণ লক্ষ্য ভঙ্গ হওয়ার আগেই সতর্ক হওয়া যায়, কিন্তু চুক্তি ভঙ্গ না হয়। TechNova-র মতো ছোট কোম্পানিতে SLA প্রায়ই থাকে না; কিন্তু SLO থাকা উচিত, কারণ তা ছাড়া নির্ভরযোগ্যতা নিয়ে সিদ্ধান্ত নেওয়া যায় না।

Error Budget — অনুমোদিত ভুলের পরিমাণ

Error Budget হলো SLO-র বিপরীত দিক: ১০০% থেকে SLO বিয়োগ করলে যা থাকে। SLO ৯৯.৯% হলে Error Budget ০.১% — অর্থাৎ ১০০০টি Request-এর মধ্যে ১টি ব্যর্থ হতে পারে, বা মাসে ৪৩.২ মিনিট Downtime অনুমোদিত। এই Budget-ই Feature ও স্থিতিশীলতার মধ্যে বিনিময়ের মুদ্রা। Budget থাকলে ঝুঁকি নেওয়া যায় (নতুন Deploy, নতুন Feature); Budget শেষ হলে ঝুঁকি নেওয়া বন্ধ করতে হয়।

> **CONCEPT:** SLI হলো "আমরা কী মাপছি", SLO হলো "কতটা ভালো হতে চাই", SLA হলো "গ্রাহককে কী প্রতিশ্রুতি দিয়েছি", আর Error Budget হলো "কতটা ভুল করার অনুমতি আছে"। চারটি একসঙ্গে একটি সম্পূর্ণ চুক্তি তৈরি করে।

৪.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



ধারণা	প্রশ্ন	উদাহরণ (TechNova)
SLI	কী মাপছি?	সফল Checkout Request-এর অনুপাত
SLO	কতটা ভালো?	৩০ দিনে ৯৯.৯% সফল
SLA	গ্রাহককে কী প্রতিশ্রুতি?	৯৯.৫% (চুক্তি, ক্ষতিপূরণসহ)
Error Budget	কতটা ভুল চলবে?	০.১% — মাসে ৪৩.২ মিনিট

৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Downtime হিসাব — সংখ্যা দিয়ে

Availability শতাংশ থেকে অনুমোদিত Downtime হিসাব করা হয় ৩০ দিনের মাস ধরে (৩০ দিন = ৪৩,২০০ মিনিট = ২,৫৯২,০০০ সেকেন্ড)। হিসাবটি: অনুমোদিত Downtime = (১০০% − Availability) × ৪৩,২০০ মিনিট।

Availability (SLO)	Error Budget	মাসে অনুমোদিত Downtime (৩০ দিন)	বছরে অনুমোদিত Downtime
৯৯% (two nines)	১%	৪৩২ মিনিট = ৭.২ ঘণ্টা	৩.৬৫ দিন
৯৯.৯% (three nines)	০.১%	৪৩.২ মিনিট	৮.৭৬ ঘণ্টা
৯৯.৯৫%	০.০৫%	২১.৬ মিনিট	৪.৩৮ ঘণ্টা

৯৯.৯৯% (four nines)	০.০১%	৪.৩২ মিনিট	৫২.৬ মিনিট
৯৯.৯৯৯% (five nines)	০.০০১%	০.৪৩২ মিনিট = ২৫.৯ সেকেন্ড	৫.২৬ মিনিট

যাচাই: ৯৯.৯%-এর ক্ষেত্রে ০.১% × ৪৩,২০০ = ৪৩.২ মিনিট — ঠিক। ৯৯.৯৯%-এর ক্ষেত্রে ০.০১% × ৪৩,২০০ = ৪.৩২ মিনিট — ঠিক। এই হিসাবটি মনে রাখুন: প্রতিটি অতিরিক্ত "৯" মানে Downtime দশ ভাগের এক ভাগ — আর খরচ ও জটিলতা বহু গুণ বেশি। TechNova-র মতো ছোট দলের জন্য ৯৯.৯% একটি বাস্তবসম্মত লক্ষ্য; ৯৯.৯৯৯% চাইলে Multi-region, Automated Failover ও বড় দল দরকার — যা এই পর্যায়ে অপ্রয়োজনীয়।

কাজ করা উদাহরণ — TechNova Checkout

ধরুন গত ৩০ দিনে Checkout Endpoint-এ মোট Request এসেছে ১,০০০,০০০টি, এর মধ্যে সফল (HTTP 2xx) হয়েছে ৯৯৮,৫০০টি। তাহলে SLI = ৯৯৮,৫০০ ÷ ১,০০০,০০০ = ৯৯.৮৫%। SLO ৯৯.৯% হলে লক্ষ্য পূরণ হয়নি — ঘাটতি ০.০৫%, অর্থাৎ ৫০০টি অতিরিক্ত ব্যর্থ Request। Error Budget-এর হিসাব: অনুমোদিত ব্যর্থ Request = ০.১% × ১,০০০,০০০ = ১,০০০টি; বাস্তব ব্যর্থ = ১,৫০০টি — Budget ৫০০টি ছাড়িয়ে গেছে। অর্থাৎ এই মাসে Budget শেষ — নতুন ঝুঁকিপূর্ণ Deploy থামাতে হবে, স্থিতিশীলতায় মন দিতে হবে। এই হিসাবটি Grafana-তে স্বয়ংক্রিয়ভাবে দেখানো যায়, কিন্তু হাতে হিসাব করতে জানা দরকার — নইলে Dashboard-এর সংখ্যা বিশ্বাস করা যায় না।

Golden Signals — চারটি সংকেত

Google SRE বই থেকে আসা Golden Signals চারটি: **Latency** (Request সম্পন্ন হতে কত সময় লাগে — সফল ও ব্যর্থ আলাদাভাবে), **Traffic** (সিস্টেমে কত চাহিদা আসছে — Request per Second), **Errors** (ব্যর্থ Request-এর হার), **Saturation** (সিস্টেম কতটা পূর্ণ — CPU, Memory, Connection Pool, Queue)। চারটি একসঙ্গে দেখলে ব্যবহারকারীর অভিজ্ঞতার পূর্ণ চিত্র মেলে। শুধু Saturation (CPU/Memory) দেখলে অধ্যায় ১-এর ভুল হয় — সিস্টেম "স্বাভাবিক" দেখায়, অথচ ব্যবহারকারী কষ্টে থাকে। প্রতিটি সার্ভিসের Dashboard-এ চারটি Signal-ই থাকা উচিত।

RED Method — Request-কেন্দ্রিক সার্ভিসের জন্য

RED হলো **Rate** (প্রতি সেকেন্ডে কত Request), **Errors** (ব্যর্থ Request-এর সংখ্যা/হার), **Duration** (Request-এর সময় — Latency)। এটি Microservice ও API-র জন্য আদর্শ — TechNova-র Express API-র জন্য RED-ই প্রধান পদ্ধতি। RED-এর সুবিধা: তিনটি Metric-ই Request-কেন্দ্রিক, ব্যবহারকারীর অভিজ্ঞতার সঙ্গে সরাসরি সম্পর্কিত।

USE Method — Resource-কেন্দ্রিক উপাদানের জন্য

USE হলো **Utilization** (Resource কতটা ব্যবহৃত হচ্ছে), **Saturation** (Queue বা অপেক্ষা কতটা), **Errors** (Resource-স্তরের Error)। এটি Database, Disk, Network, CPU-র মতো Resource-এর জন্য আদর্শ। যেমন MongoDB-র জন্য: Utilization (CPU/Memory), Saturation (Connection Pool-এর অপেক্ষমাণ সংযোগ), Errors (ব্যর্থ Query)। RED ও USE একে অপরের প্রতিদ্বন্দ্বী নয় — RED সার্ভিসের জন্য, USE Resource-এর জন্য; TechNova-তে দুটোই দরকার।

SLI বাছাই — কী মাপবেন, কী মাপবেন না

ভালো SLI বাছাইয়ের নিয়ম: (১) ব্যবহারকারীর অভিজ্ঞতার কাছাকাছি — "Checkout সফল হলো কি না" ভালো SLI, "CPU ৮০%-এর নিচে" খারাপ SLI (এটি কারণ, ফলাফল নয়)। (২) কম সংখ্যক SLI — প্রতিটি সার্ভিসে ২–৩টি যথেষ্ট; বেশি SLI মানে বেশি Alert, বেশি বিভ্রান্তি। (৩) পরিমাপযোগ্য — যে SLI মাপা যায় না, তা SLO হতে পারে না। (৪) নিয়ন্ত্রণযোগ্য — দল যা নিয়ন্ত্রণ করতে পারে না (যেমন তৃতীয় পক্ষের Payment Gateway-এর নিজস্ব Downtime), তা SLI-তে সরাসরি না রেখে আলাদাভাবে ট্র্যাক করুন।

> **WARNING:** SLO-কে ১০০% বানাবেন না। ১০০% SLO মানে Error Budget শূন্য — অর্থাৎ কোনো Deploy, কোনো পরিবর্তন, কোনো ভুলের অনুমতি নেই। এটি দলকে পক্ষাঘাতগ্রস্ত করে। ৯৯.৯% বা ৯৯.৯৫% বাস্তবসম্মত; ১০০% নয়।

৪.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

নিচে TechNova-র Checkout SLI হিসাবের PromQL উদাহরণ — প্রতিটি লাইনের ব্যাখ্যাসহ। PromQL হলো Prometheus-এর Query ভাষা; এখানে শুধু ধারণা বোঝানো হচ্ছে, বিস্তারিত PromQL পরের Volume-এ আসবে।

```
# SLI: গত 30 দিনে সফল Request-এর অনুপাত (ধারণাগত PromQL)
# sum(rate(http_requests_total{route="/api/checkout",status_code=~"2.."}[30d]))
#   — সফল Request-এর হার (2xx Status)
# / sum(rate(http_requests_total{route="/api/checkout"}[30d]))
#   — মোট Request-এর হার
# ফলাফল: 0.9985 মানে 99.85% — SLO 99.9% পূরণ হয়নি

# Error Budget অবশিষ্ট (ধারণাগত):
# 1 - SLI = 1 - 0.9985 = 0.0015 (0.15% ভুল হয়েছে)
# অনুমোদিত ছিল 0.001 (0.1%) — অর্থাৎ Budget ছাড়িয়ে গেছে

# p95 Latency (সফল Request-এর জন্য):
# histogram_quantile(0.95,
#   sum(rate(http_request_duration_seconds_bucket{
#     route="/api/checkout",status_code=~"2.."}[5m])) by (le))
# ফলাফল সেকেন্ডে — যেমন 0.42 মানে p95 Latency 420ms
```

```
# হাতে হিসাব যাচাই (টার্মিনালে — চিত্রণমূলক)
python3 -c "print(0.001 * 43200)"
# 43.2 — 99.9% SLO-তে মাসে 43.2 মিনিট Downtime অনুমোদিত
python3 -c "print(0.0001 * 43200)"
# 4.32 — 99.99% SLO-তে মাসে 4.32 মিনিট
```

৪.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র জন্য একটি সম্পূর্ণ SLO সংজ্ঞা (YAML আকারে, নথি হিসেবে) ও একটি Prometheus Alert Rule — Error Budget দ্রুত শেষ হলে Alert দেবে।

```
# slo-technova.yaml — SLO সংজ্ঞা (নথি হিসেবে, Git-এ রাখুন)
service: technova-api
slos:
  - name: checkout-availability
    description: "Checkout Endpoint-এ সফল Request-এর অনুপাত"
    sli: "successful checkout requests / total checkout requests"
    target: 99.9          # শতাংশে
    window: 30d
    error_budget: 0.1     # শতাংশে — মাসে 43.2 মিনিট
  - name: checkout-latency
    description: "Checkout-এর p95 Latency 500ms-এর নিচে"
    sli: "p95 latency of successful checkout requests"
    target: 95            # 95% Request 500ms-এর নিচে
```

```
    threshold_ms: 500
    window: 30d
```

```
# /etc/prometheus/rules/technova-slo.yml — Error Budget Alert (বৈধ YAML)
groups:
  - name: technova-slo
    rules:
      - alert: CheckoutErrorBudgetBurning
        expr: |
          sum(rate(http_requests_total{route="/api/checkout",status_code!~"2.."}[5m]))
          /
          sum(rate(http_requests_total{route="/api/checkout"}[5m])) > 0.01
        for: 15m
        labels:
          severity: warning
          service: technova-api
        annotations:
          summary: "Checkout Error Rate 1% ছাড়িয়েছে (15 মিনিট ধরে)"
          description: "SLO 99.9% হলে 5 মিনিটে 1% Error মানে Budget দ্রুত শেষ হচ্ছে।"
```

এই Rule-টি বলে: ৫ মিনিটের Window-এ Checkout-এর Error Rate ১% ছাড়িয়ে ১৫ মিনিট ধরে থাকলে Alert দাও। ১% Error মানে Budget ১০ গুণ দ্রুত শেষ হচ্ছে (কারণ অনুমোদিত ০.১%) — তাই দ্রুত সাড়া দরকার। এটিই Burn Rate Alert-এর ধারণা: শুধু "Error বেশি" নয়, "Budget কত দ্রুত শেষ হচ্ছে" — তা দেখা।

৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি: TechNova-র Checkout SLO ৯৯.৯%। নভেম্বরের প্রথম সপ্তাহে একটি ভুল Deploy-এর কারণে ২০ মিনিট Checkout ব্যর্থ থাকে। Error Budget-এর হিসাব: ২০ মিনিট খরচ হয়েছে ৪৩.২ মিনিটের মধ্যে — প্রায় অর্ধেক Budget এক ঘটনায় শেষ। দল Rollback করে, ঘটনা মেটে। কিন্তু এরপর Product দল একটি বড় Feature Deploy করতে চায় — যাতে ঝুঁকি আছে।

Error Budget Policy অনুযায়ী সিদ্ধান্তটি স্পষ্ট: Budget-এর অর্ধেক শেষ, আর মাত্র ২৩.২ মিনিট বাকি। বড় ঝুঁকিপূর্ণ Deploy এখন করা যাবে না — প্রথমে Feature-টিকে ছোট ছোট ভাগে ভাগ করতে হবে, Canary Deploy করতে হবে, আর Rollback পরিকল্পনা প্রস্তুত রাখতে হবে। অথবা Feature-টি পরের মাস পর্যন্ত পিছিয়ে দিতে হবে, যখন Budget আবার পূর্ণ হবে। সংখ্যা ছাড়া এই সিদ্ধান্তটি ব্যক্তিগত বিতর্ক হতো; সংখ্যা থাকায় এটি নীতিগত সিদ্ধান্ত হয়েছে। এটাই Error Budget-এর আসল শক্তি — এটি বিতর্ককে সিদ্ধান্তে রূপান্তর করে।

৪.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **সার্ভিস বাছুন:** TechNova-র সবচেয়ে গুরুত্বপূর্ণ ব্যবহারকারী-যাত্রা বাছুন — Checkout। সেটির জন্য SLI লিখুন।
2. **SLI সংজ্ঞায়িত করুন:** "সফল" মানে কী (HTTP 2xx, ৫ সেকেন্ডের মধ্যে)? "মোট" মানে কী (সব Request, নাকি শুধু বৈধ Request)? লিখে রাখুন।
3. **SLO ঠিক করুন:** ৯৯.৯% (৩০ দিন) — বাস্তবসম্মত লক্ষ্য বাছুন; ১০০% নয়।
4. **Downtime হিসাব করুন:** ৪৩.২ মিনিট/মাস — এই সংখ্যাটি দলের সবাইকে জানান।
5. **Dashboard বানান:** Grafana-তে SLI, SLO, অবশিষ্ট Budget ও Burn Rate দেখান — চারটি Golden Signal-সহ।
6. **Alert বসান:** ৪.৮-এর মতো Burn Rate Alert — Budget দ্রুত শেষ হলে সতর্ক করুন, শুধু "Error বেশি" হলে নয়।

7. **Policy লিখুন:** Budget শেষ হলে কী হবে — Feature Freeze, নাকি ঝুঁকি কমিয়ে Deploy? লিখিত নীতি Git-এ রাখুন।

৪.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	কেন সমস্যা	সঠিক পথ
SLO 100% ঠিক করা	Budget শূন্য; কোনো পরিবর্তন সম্ভব নয়	99.9% বা 99.95% বাছুন
CPU/Memory-কে SLI বানানো	ব্যবহারকারীর অভিজ্ঞতা মাপে না	সফল Request ও Latency মাপুন
SLI-তে "সফল" সংজ্ঞায়িত না করা	5xx নাকি 4xx-ও? Timeout কী?	সফল/ব্যর্থের সংজ্ঞা লিখে রাখুন
অনেক SLI (১০টির বেশি)	Alert-এর বন্যা, মনোযোগ ছড়িয়ে যায়	প্রতি সার্ভিসে ২–৩টি SLI
SLA ও SLO এক করে ফেলা	অভ্যন্তরীণ লক্ষ্য ভঙ্গ হলেই চুক্তি ভঙ্গ মনে হয়	SLA SLO-র চেয়ে শিথিল রাখুন
Error Budget না দেখে Deploy	Budget শেষ, তবু ঝুঁকি নেওয়া	Deploy-এর আগে Budget যাচাই করুন

৪.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

SLI/SLO ডেটায় সাধারণত সংবেদনশীল তথ্য থাকে না — কারণ এটি Aggregate সংখ্যা। কিন্তু সতর্ক থাকুন: SLI-র ভাঙন (Breakdown) যদি ব্যবহারকারী-স্তরে যায় (যেমন "অমুক ব্যবহারকারীর সফলতার হার"), তাহলে তা ব্যক্তিগত তথ্য হয়ে যায়। SLI সবসময় সামগ্রিক (Aggregate) স্তরে রাখুন। SLO Dashboard শেয়ার করার সময় খেয়াল রাখুন — Dashboard-এ কোনো সংবেদনশীল Label বা ব্যবহারকারীর তথ্য না থাকে। Alert Notification-এ (যেমন Email বা Chat) কখনো ব্যবহারকারীর ব্যক্তিগত তথ্য পাঠাবেন না — শুধু সার্ভিস, SLI ও সংখ্যা পাঠান।

৪.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

উচ্চ SLO (যেমন ৯৯.৯৯৯%)-এর খরচ শুধু টাকার নয় — Multi-region স্থাপত্য, Automated Failover, ২৪/৭ On-call, ব্যাপক পরীক্ষা — সব মিলিয়ে জটিলতা বহু গুণ বাড়ে। TechNova-র মতো ছোট দলের জন্য ৯৯.৯% থেকে ৯৯.৯৫%-এ যাওয়া যুক্তিসঙ্গত হতে পারে, কিন্তু ৯৯.৯৯৯% চাওয়া অপ্রয়োজনীয় ব্যয়। বিপরীত দিকে, খুব শিথিল SLO (যেমন ৯৫%) মানে ব্যবহারকারী নিয়মিত কষ্ট পায় — ব্যবসায়িক ক্ষতি হয়। সঠিক SLO হলো ব্যবসায়িক প্রয়োজন ও খরচের ভারসাম্য — একটি ব্যবসায়িক সিদ্ধান্ত, শুধু প্রযুক্তিগত নয়।

SLI হিসাবের খরচও মনে রাখুন: ৩০ দিনের Window-এ PromQL Query চালানো ব্যয়বহুল হতে পারে। তাই SLI-কে Recording Rule দিয়ে আগে থেকে হিসাব করে রাখুন (প্রতি মিনিটে একবার), আর Dashboard সেই প্রস্তুত মান দেখাক — প্রতিবার কাঁচা ডেটা থেকে হিসাব নয়।

৪.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরবর্তী ধাপ
SLO হঠাৎ ভঙ্গ হচ্ছে	একটি বড় ঘটনা, বা SLI-র সংজ্ঞা বদলেছে	কোন সময়ে ভঙ্গ হয়েছে, তা চিহ্নিত করুন; Deploy History মেলান

SLI ও ব্যবহারকারীর অভিযোগ মিলছে না	SLI ভুল জিনিস মাপছে (যেমন শুধু 5xx, Timeout নয়)	"সফল"-এর সংজ্ঞা পর্যালোচনা করুন; Latency SLI যোগ করুন
Burn Rate Alert খুব ঘন ঘন	Window খুব ছোট, বা Threshold খুব কম	Window ও Threshold সমন্বয় করুন; Multi-window Alert ভাবুন
Error Budget হিসাব মিলছে না	Time Window বা Timezone ভিন্ন	Window (30 দিন, Rolling) ও UTC নিশ্চিত করুন
দল SLO মানে না	SLO ব্যবসা থেকে বিচ্ছিন্ন, বা অবাস্তব	Product দলের সঙ্গে SLO পুনঃনির্ধারণ করুন

TROUBLESHOOTING: SLI ও বাস্তব অভিজ্ঞতা না মিললে প্রথমে SLI-র সংজ্ঞা সন্দেহ করুন, সিস্টেমকে নয়। অনেক সময় SLI "সফল" বলছে, কিন্তু ব্যবহারকারী Timeout পাচ্ছে — কারণ Timeout-কে SLI-তে ব্যর্থ হিসেবে গোনা হয়নি।

৪.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ব্যবহারকারীর যাত্রা থেকে SLI; প্রতি সার্ভিসে ২–৩টি SLI; SLO ৯৯.৯% দিয়ে শুরু; Burn Rate Alert; Error Budget Policy লিখিতভাবে; SLO Dashboard সবার কাছে দৃশ্যমান।

অ্যান্টি-প্যাটার্ন: ১০০% SLO; Infrastructure Metric-কে SLI বানানো; SLA ও SLO এক করা; Budget না দেখে Deploy; SLI-র সংজ্ঞা কোথাও না লেখা — ফলে প্রত্যেকে ভিন্নভাবে বোঝে।

PRO TIP: SLO-র সংখ্যাটি দলের সবাই মুখস্থ রাখতে পারে এমন সহজ রাখুন — "Checkout ৯৯.৯%, মাসে ৪৩ মিনিট"। জটিল SLO কেউ মনে রাখে না, আর যা মনে রাখা যায় না তা মানা হয় না।

৪.১৬ অধ্যায়ের সারাংশ

SLI পরিমাপ, SLO লক্ষ্য, SLA চুক্তি, Error Budget অনুমোদিত ভুল — চারটি একসঙ্গে নির্ভরযোগ্যতার ভাষা তৈরি করে। ৯৯.৯% মানে মাসে ৪৩.২ মিনিট, ৯৯.৯৯% মানে ৪.৩২ মিনিট — প্রতিটি অতিরিক্ত "৯" মানে Downtime দশ ভাগের এক ভাগ, কিন্তু খরচ বহু গুণ। Golden Signals (Latency, Traffic, Errors, Saturation) ব্যবহারকারীর অভিজ্ঞতা দেখায়; RED Method সার্ভিসের জন্য, USE Method Resource-এর জন্য। ভালো SLI ব্যবহারকারীর কাছাকাছি, কম সংখ্যক, ও পরিমাপযোগ্য হয়। Error Budget শেষ হলে Feature থামিয়ে স্থিতিশীলতায় মন দেওয়াই নীতি — সংখ্যা বিতর্ককে সিদ্ধান্তে রূপান্তর করে।

৪.১৭ মূল প্রযুক্তিগত পরিভাষা

SLI (Service Level Indicator): সেবার মানের পরিমাপ — যেমন সফল Request-এর অনুপাত বা p95 Latency।

SLO (Service Level Objective): SLI-এর লক্ষ্যমান ও সময়কাল — যেমন ৩০ দিনে ৯৯.৯% সফল।

SLA (Service Level Agreement): গ্রাহকের সঙ্গে লিখিত চুক্তি; লক্ষ্য পূরণ না হলে ক্ষতিপূরণের বিধান থাকে।

Error Budget: ১০০% থেকে SLO বিয়োগ করে পাওয়া অনুমোদিত ভুলের পরিমাণ — যেমন ৯৯.৯% SLO-তে ০.১%।

Availability: সেবা ব্যবহারযোগ্য থাকার অনুপাত — সাধারণত শতাংশে প্রকাশ করা হয়।

Golden Signals: Latency, Traffic, Errors ও Saturation — ব্যবহারকারীর অভিজ্ঞতা প্রতিফলিত করে এমন চারটি সংকেত।

RED Method: Rate, Errors, Duration — Request-কেন্দ্রিক সার্ভিস পর্যবেক্ষণের পদ্ধতি।

USE Method: Utilization, Saturation, Errors — Resource-কেন্দ্রিক উপাদান পর্যবেক্ষণের পদ্ধতি।

Burn Rate: Error Budget কত দ্রুত শেষ হচ্ছে তার হার — ১ মানে ঠিক অনুমোদিত হারে, ১০ মানে দশ গুণ দ্রুত।

Latency: একটি Request সম্পন্ন হতে কত সময় লাগে; সাধারণত Percentile (p95/p99) দিয়ে মাপা হয়।

Saturation: সিস্টেম বা Resource কতটা পূর্ণ — যেমন CPU ব্যবহার, Connection Pool-এর পূর্ণতা, Queue-র দৈর্ঘ্য।

Recording Rule: Prometheus-এ আগে থেকে হিসাব করে রাখা Query-র ফলাফল; ব্যয়বহুল SLI হিসাবের জন্য ব্যবহৃত হয়।

৪.১৮ জ্ঞান যাচাই — প্রশ্ন

1. SLI, SLO, SLA ও Error Budget-এর পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন।
2. ৯৯.৯% Availability মানে মাসে কত মিনিট Downtime? হিসাবটি দেখান। ৯৯.৯৯%-এর ক্ষেত্রে কত?
3. TechNova-র Checkout-এ গত ৩০ দিনে ১,০০০,০০০ Request-এর মধ্যে ৯৯৮,৫০০টি সফল হলে SLI কত? SLO ৯৯.৯% পূরণ হয়েছে কি?
4. Golden Signals চারটি কী? প্রতিটি TechNova-র কোন দিকটি দেখায়?
5. RED ও USE Method-এর পার্থক্য কী? প্রতিটি কখন ব্যবহার করবেন?
6. Error Budget শেষ হলে দলের কী করা উচিত? কেন?
7. SLA কেন সাধারণত SLO-র চেয়ে শিথিল রাখা হয়?

8. Burn Rate ১০ মানে কী? এটি কেন শুধু "Error বেশি" বলার চেয়ে দরকারি?

৪.১৯ ব্যবহারিক অনুশীলন

1. TechNova-র Checkout, Login ও Product Search — তিনটি যাত্রার জন্য একটি করে SLI লিখুন (কী মাপবেন, "সফল" মানে কী, কোন Window)।
2. ৯৯%, ৯৯.৯%, ৯৯.৯৫% ও ৯৯.৯৯% — প্রতিটির মাসিক ও বার্ষিক অনুমোদিত Downtime হিসাব করে একটি টেবিল বানান।
3. আপনার নিজের একটি সার্ভিসের জন্য SLO ঠিক করুন (৯৯.৯%, ৩০ দিন) ও Error Budget Policy লিখুন: Budget ৫০% শেষ হলে কী করবেন, ১০০% শেষ হলে কী করবেন?
4. ৪.৮-এর Alert Rule-টি আপনার Prometheus-এ বসিয়ে ইচ্ছাকৃতভাবে Error তৈরি করে দেখুন Alert আসে কি না।
5. একটি কাল্পনিক ঘটনার Burn Rate হিসাব করুন: ১ ঘণ্টায় Checkout Error ২% হলে Budget কত গুণ দ্রুত শেষ হচ্ছে? (ইঙ্গিত: অনুমোদিত ০.১%-এর সঙ্গে তুলনা করুন।)

অধ্যায় ৫ • PART I — Observability Foundations

Chapter 5 — Monitoring Strategy, Instrumentation, and Telemetry Collection

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি TechNova-র জন্য একটি Monitoring Strategy লিখতে পারবেন: কী Instrument করবেন, কী ক্রমে, কীভাবে। আপনি Manual বনাম Auto-instrumentation-এর পার্থক্য, Cardinality নিয়ন্ত্রণের নিয়ম, Sampling-এর নকশা, Label-এর নামকরণ প্রথা (Convention), এবং Telemetry Collection পরিকল্পনা ও খরচ/Retention Trade-off ব্যাখ্যা করতে পারবেন।

- কী Instrument করবেন প্রথমে — অগ্রাধিকার ঠিক করা।
- Manual ও Auto-instrumentation-এর সুবিধা-অসুবিধা তুলনা করা।
- Cardinality কী, কেন তা বিস্ফোরিত হয়, কীভাবে নিয়ন্ত্রণ করবেন — তা ব্যাখ্যা করা।
- Label Convention ও Sampling Strategy লেখা।
- খরচ ও Retention-এর ভারসাম্য রেখে Collection পরিকল্পনা করা।

৫.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Instrumentation ছাড়া Observability শুধু ইচ্ছা। কিন্তু সবকিছু Instrument করা যায় না — সময়, খরচ ও জটিলতা সীমিত। যে দল পরিকল্পনা ছাড়া Instrument করে, তারা দুটি ভুলের একটিতে পড়ে: হয় খুব কম Instrument করে — ঘটনার সময় অন্ধ থাকে; নয়তো সবকিছু Instrument করে — Cardinality বিস্ফোরিত হয়, খরচ বাড়ে, Prometheus ধীর হয়ে যায়, আর কেউ Dashboard বোঝে না। Strategy হলো এই দুই প্রান্তের মাঝখানে সচেতন পথ বেছে নেওয়া।

TechNova-র জন্য প্রশ্নটি বাস্তব: তিনজনের দল, সীমিত সময়, সীমিত বাজেট। প্রথম সপ্তাহে কী করবেন? প্রথম মাসে কী? কোন Metric, কোন Log, কোন Trace — আর কোনটি পরে? এই অধ্যায় সেই অগ্রাধিকারের ক্রম দেয়, যাতে সবচেয়ে কম পরিশ্রমে সবচেয়ে বেশি দৃশ্যমানতা (Visibility) পাওয়া যায়।

৫.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১–৪-এর ধারণা (তিনটি স্তম্ভ, Pipeline, SLI/SLO, Golden Signals), Node.js/Express-এর মৌলিক ধারণা, এবং Prometheus Metric প্রকার (Counter, Histogram) জানা থাকলে এই অধ্যায় সহজ হবে। OpenTelemetry-র নাম শুনে থাকলেই চলবে — ধারণাটি এখানে ব্যাখ্যা করা হবে।

৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Instrumentation হলো সিস্টেমকে নিজের সম্পর্কে বলতে শেখানো। কোড যখন কাজ করে, তখন সে নীরবে কাজ করে — Instrumentation সেই নীরবতায় কণ্ঠ দেয়। কিন্তু প্রতিটি কণ্ঠের খরচ আছে: CPU, Memory, Network, Storage। তাই প্রশ্নটি "কী বলাব?" নয়, "কোন কথা শোনা সবচেয়ে দরকারি?" — উত্তরটি ব্যবহারকারীর অভিজ্ঞতা থেকে আসে।

দ্বিতীয় নীতি: Telemetry-র মান তার ব্যবহারের উপর নির্ভর করে। যে Metric কেউ দেখে না, যে Log কেউ খোঁজে না, যে Trace কেউ খোলে না — তা শুধু খরচ। ভালো Strategy প্রতিটি Telemetry-র পেছনে একটি প্রশ্ন রাখে: "এই Metric দিয়ে কোন সিদ্ধান্ত নেব?" উত্তর না থাকলে সেটি Instrument করবেন না।

Manual বনাম Auto-instrumentation

Auto-instrumentation হলো Library বা Agent-এর মাধ্যমে স্বয়ংক্রিয়ভাবে Telemetry তৈরি করা — যেমন Node.js-এ OpenTelemetry Auto-instrumentation HTTP Request, Database Query ও Downstream Call-এর Span স্বয়ংক্রিয়ভাবে তৈরি করে, কোডে হাত না দিয়েই। সুবিধা: দ্রুত শুরু, কম পরিশ্রম, ভুলে যাওয়া কম। অসুবিধা: ব্যবসায়িক প্রসঙ্গ (যেমন Order ID, Tenant) স্বয়ংক্রিয়ভাবে আসে না; অপ্রয়োজনীয় Span-ও তৈরি হতে পারে।

Manual Instrumentation হলো কোডে নিজে হাতে Metric, Log ও Span লেখা — যেমন Checkout সফল হলে একটি Counter বাড়ানো, বা Payment ধাপে একটি Span তৈরি করা। সুবিধা: ঠিক যা দরকার তা-ই, ব্যবসায়িক প্রসঙ্গসহ। অসুবিধা: সময় লাগে, ভুলে যাওয়া যায়, প্রতিটি নতুন Feature-এ মনে করে যোগ করতে হয়। বাস্তবসম্মত পথ: Auto-instrumentation দিয়ে ভিত্তি তৈরি করুন, তারপর ব্যবসায়িকভাবে গুরুত্বপূর্ণ জায়গায় Manual Instrumentation যোগ করুন। TechNova-র জন্য এটাই প্রস্তাবিত ক্রম।

Cardinality — নীরব ঘাতক

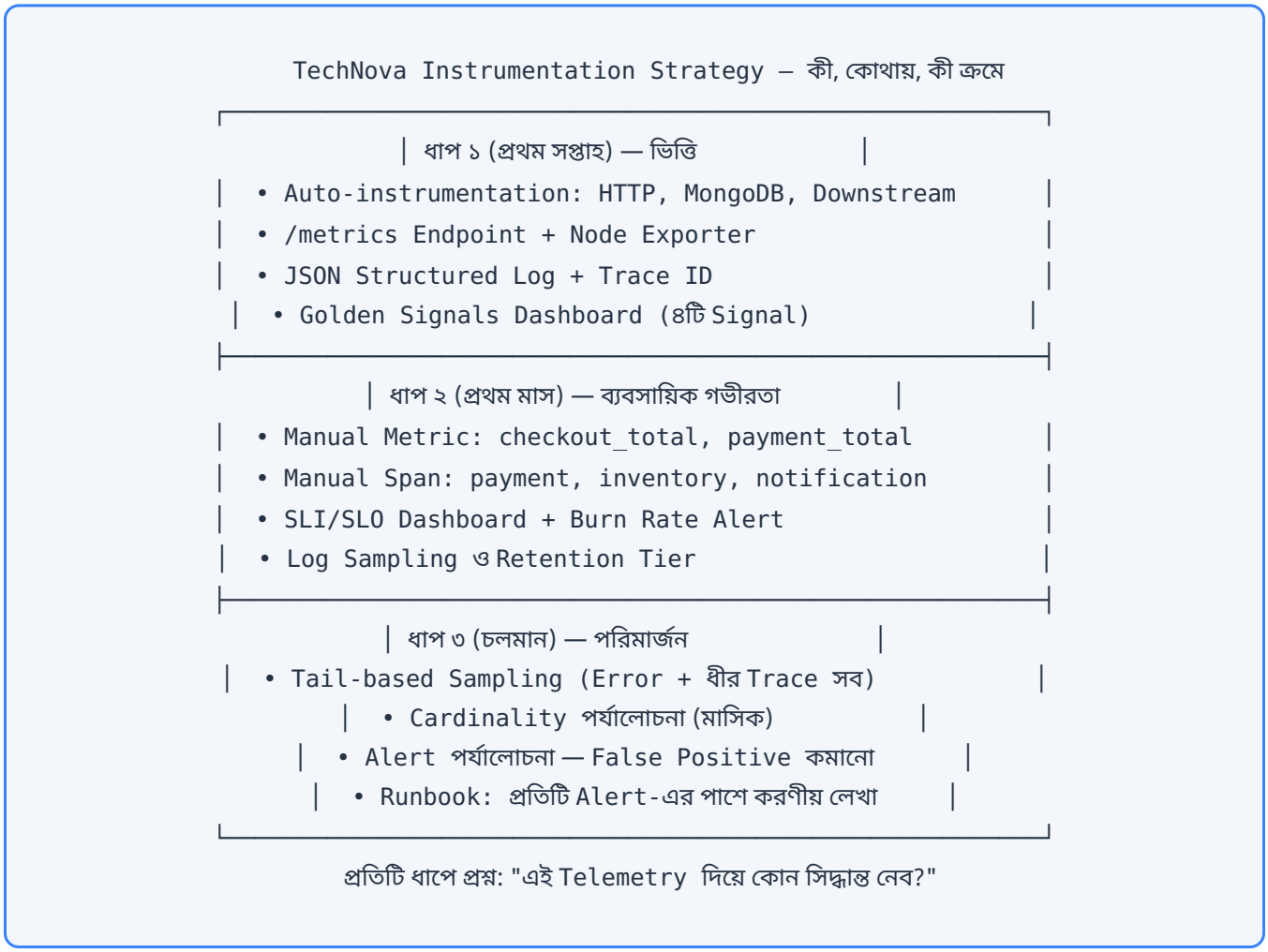
Cardinality মানে একটি Metric-এর সম্ভাব্য Label মানের সংমিশ্রণের সংখ্যা। যেমন `http_requests_total{method, route, status_code}`-এ Method ৪টি, Route ২০টি, Status ৫টি হলে Cardinality সর্বোচ্চ ৪ × ২০ × ৫ = ৪০০টি Time Series — সামলানো যায়। কিন্তু Label-এ User ID যোগ করলে (১০,০০০ ব্যবহারকারী) Cardinality হয় ৪ × ২০ × ৫ × ১০,০০০ = ৪,০০০,০০০ — Prometheus-এর Memory ও Query ধীর হয়ে যায়, Storage ফুলে যায়। এটাই Cardinality বিস্ফোরণ। নিয়ম: Metric Label-এ শুধু কম মানের (Low-cardinality) জিনিস রাখুন — Method, Route, Status, Service, Environment। High-cardinality মান (User ID, Order ID, Email, IP) Log ও Trace-এ রাখুন, Metric-এ নয়।

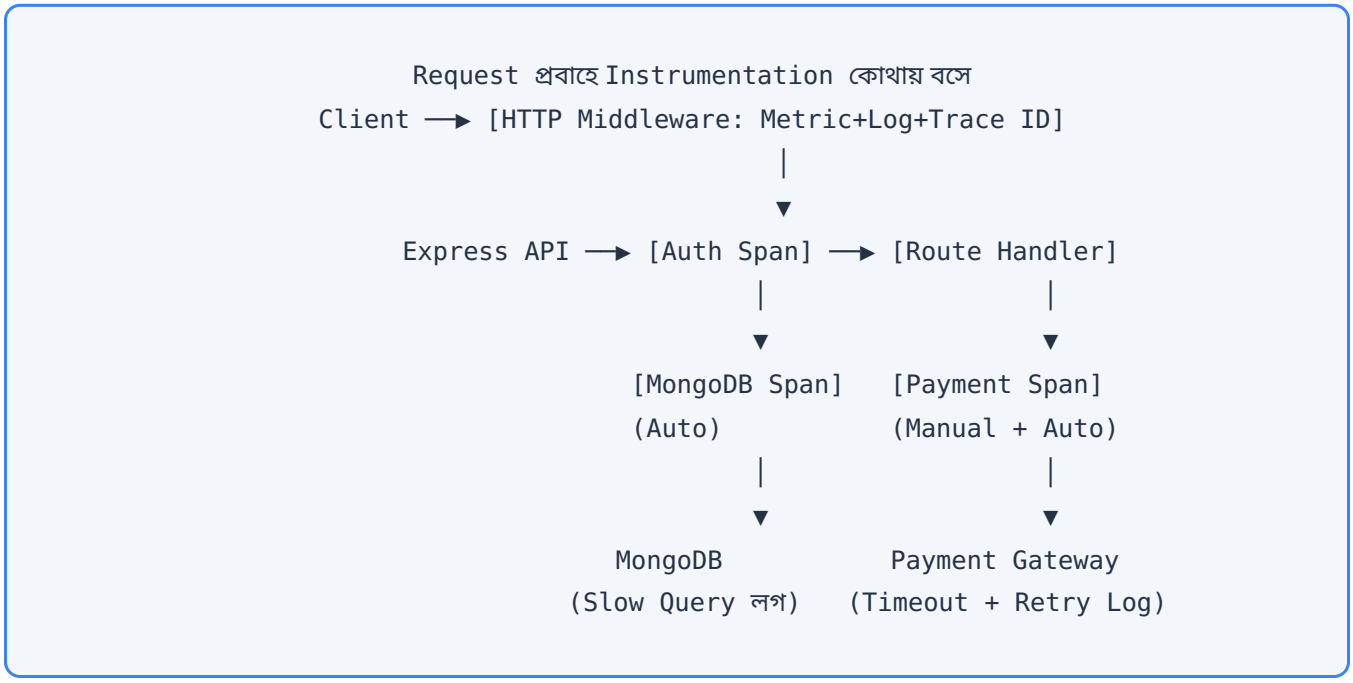
> **CONCEPT:** Metric হলো Aggregate-এর জগৎ — সেখানে ব্যক্তিগত পরিচয়ের জায়গা নেই। User ID, Order ID, Email — এগুলো Log ও Trace-এর জগতের জিনিস। দুটো জগৎ গুলিয়ে ফেললেই Cardinality বিস্ফোরণ হয়।

Sampling — সবকিছু রাখা যায় না

Sampling মানে সব Telemetry না রেখে একটি প্রতিনিধিত্বমূলক অংশ রাখা। দুটি প্রধান ধরন: **Head-based Sampling** — Trace শুরুর সময়ই সিদ্ধান্ত (যেমন ১০% Request Trace করা হবে); সহজ, কিন্তু বিরল Error Trace হারিয়ে যেতে পারে। **Tail-based Sampling** — Trace শেষ হওয়ার পর সিদ্ধান্ত (যেমন সব Error Trace ও ধীর Trace রাখা হবে, সফল Trace-এর ১০% রাখা হবে); বেশি কার্যকর, কিন্তু বেশি জটিল — Collector-কে পুরো Trace ধরে রাখতে হয়। TechNova-র জন্য শুরুতে Head-based (১০%) + সব Error রাখা — এই মিশ্রণই যথেষ্ট।

৫.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র





৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

কী Instrument করবেন প্রথমে — অগ্রাধিকার

অগ্রাধিকারের ক্রমটি ব্যবহারকারীর অভিজ্ঞতা থেকে আসে: (১) সবচেয়ে গুরুত্বপূর্ণ ব্যবহারকারী-যাত্রা — TechNova-তে Checkout, Login, Product Search। (২) সেই যাত্রার Golden Signals — Rate, Errors, Duration। (৩) সেই যাত্রার নির্ভরতা — MongoDB Query, Payment Gateway Call। (৪) Infrastructure — CPU, Memory, Disk, Network। লক্ষ করুন Infrastructure সবার শেষে — কারণ ব্যবহারকারী CPU দেখে না, সে Checkout দেখে। অনেক দল উল্টো ক্রমে শুরু করে — প্রথমে Infrastructure Dashboard বানায়, আর ব্যবহারকারীর যাত্রা অদৃশ্য থাকে।

Label Convention — নামকরণের শৃঙ্খলা

Label-এর নামকরণে ধারাবাহিকতা না থাকলে Correlation ভেঙে যায়। TechNova-র জন্য প্রস্তাবিত প্রথা: সব Telemetry-তে একই নাম ব্যবহার করুন — `service` (যেমন `technova-api`), `env` (`production` / `staging`), `route` (যেমন `/api/orders` — Path Template, নির্দিষ্ট ID নয়), `method` (`GET` / `POST`), `status_code` (`200` / `500`)। নামে Snake Case ব্যবহার করুন, একই অর্থে দুটি নাম ব্যবহার করবেন না (যেমন কোথাও `service`, কোথাও `svc` নয়)। এই প্রথাটি একটি নথিতে লিখে Git-এ রাখুন — নতুন সদস্য যাতে প্রথম দিনেই তা জানতে পারেন।

Route বনাম Path — একটি গুরুত্বপূর্ণ পার্থক্য

Express-এ `req.path` হলো বাস্তব Path — `/orders/8841`। `req.route.path` হলো Path Template — `/orders/:id`। Metric Label-এ সবসময় Template ব্যবহার করুন। Path ব্যবহার করলে প্রতিটি Order ID একটি নতুন Time Series তৈরি করে — Cardinality বিস্ফোরণ। এটি সবচেয়ে সাধারণ ও ব্যয়বহুল ভুলগুলির একটি। অধ্যায় ২-এর Middleware-এ এই কারণেই `route` ব্যবহার করা হয়েছে।

Log Level ও Volume নিয়ন্ত্রণ

Log Level-এর স্তর: `error` (ব্যর্থতা), `warn` (সতর্কতা), `info` (মূল ঘটনা), `debug` (বিস্তারিত)। Production-এ Info স্তর চালু রাখুন, Debug বন্ধ রাখুন — Debug শুধু নির্দিষ্ট সময়ের জন্য, নির্দিষ্ট সার্ভিসে চালু করুন। প্রতিটি Request-এ Info Log একটি লাইন — এর বেশি নয়। Loop-এর ভেতরে Log লিখবেন না; প্রতি ১০০০ বার লিখলেও Volume বিস্ফোরিত হয়। একটি নিয়ম: Log লেখার আগে জিজ্ঞেস করুন — "এই লাইনটি ঘটনার সময় কেউ খুঁজবে কি?" উত্তর না হলে লিখবেন না।

Collection পরিকল্পনা — কী, কত ঘন ঘন, কতদিন

Telemetry	সংগ্রহ	Retention (প্রস্তাব)	মন্তব্য
Metrics (Prometheus)	Scrape ১৫ সেকেন্ড	৩০ দিন (বিস্তারিত), তারপর Downsample	সস্তা, দীর্ঘ Retention সম্ভব
Logs (Structured)	Agent Push	১৪ দিন (Hot), ৯০ দিন (Cold Archive)	Volume নিয়ন্ত্রণে Level ও Sampling
Traces	OTLP Push, ১০% Sampling + সব Error	৭ দিন	সবচেয়ে ব্যয়বহুল; Sampling অপরিহার্য
Uptime Check (Black-box)	প্রতি ১ মিনিটে বাইরে থেকে	৯০ দিন	Monitoring Stack-এর বাইরে রাখুন

খরচ/Retention Trade-off

Retention বাড়ালে খরচ বাড়ে, কিন্তু পুরোনো ঘটনা বিশ্লেষণ করা যায়। Retention কমালে খরচ কমে, কিন্তু গত মাসের ঘটনার কারণ খোঁজা যায় না। ভারসাম্য: Metrics দীর্ঘদিন (সস্তা), Logs মাঝারি (মাঝারি খরচ), Traces সংক্ষিপ্ত (ব্যয়বহুল)। আরেকটি Trade-off: Scrape Interval। ১৫ সেকেন্ড থেকে ৫ সেকেন্ডে নামালে ঘটনা ধরার গতি সামান্য বাড়ে, কিন্তু Storage তিন গুণ বাড়ে — বেশিরভাগ ক্ষেত্রে তা মূল্যহীন। সিদ্ধান্ত নেওয়ার আগে হিসাব করুন: প্রতিদিন কত Series, কত Sample, কত GB — তারপর Retention ঠিক করুন।

> **WARNING:** Metric Label-এ কখনো User ID, Order ID, Email, IP Address বা পূর্ণ Path দেবেন না। একটি ভুল Label পুরো Prometheus-কে ধীর করে দিতে পারে — আর তা সারাতে ঘণ্টা লাগে। নতুন Label যোগ করার আগে Cardinality হিসাব করুন।

৫.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

নিচে TechNova-র API-তে Manual Instrumentation-এর উদাহরণ — Checkout যাত্রায় Business Metric ও Span যোগ করা, এবং OpenTelemetry Auto-instrumentation চালু করার ধারণা।

```
// src/metrics/business.js — ব্যবসায়িক Metric (Node.js sketch)
const client = require('prom-client');

// Counter: Checkout প্রচেষ্টা ও সাফল্য (Low-cardinality Label)
const checkoutTotal = new client.Counter({
  name: 'technova_checkout_total',
  help: 'Total checkout attempts by outcome',
  labelNames: ['outcome'], // 'success' | 'payment_failed' | 'stock_failed'
});

const paymentDuration = new client.Histogram({
  name: 'technova_payment_duration_seconds',
  help: 'Payment gateway call duration',
  labelNames: ['gateway'], // 'sslcommerz' | 'stripe' — কম মান
  buckets: [0.1, 0.3, 1, 2, 5],
});

module.exports = { checkoutTotal, paymentDuration };
```

```
// src/routes/checkout.js — Manual Instrumentation ব্যবহার (sketch)
const { checkoutTotal, paymentDuration } = require('../metrics/business');

router.post('/api/checkout', async (req, res) => {
  const endTimer = paymentDuration.startTimer({ gateway: 'sslcommerz' });
  try {
    const result = await paymentService.charge(req.body);
    endTimer(); // Histogram-এ সময় রেকর্ড
    checkoutTotal.inc({ outcome: 'success' });
    res.json({ order_id: result.orderId });
  } catch (err) {
    endTimer();
    // ব্যর্থতার কারণ অনুযায়ী Outcome — কিন্তু সংবেদনশীল বিবরণ নয়
    checkoutTotal.inc({ outcome: 'payment_failed' });
    req.log.error({ event: 'checkout_failed', trace_id: req.traceId,
```

```
      reason: err.code }); // Stack Trace নয়, শুধু Code
    res.status(502).json({ error: 'payment_failed' });
  }
});
```

```
# Cardinality যাচাই — Prometheus-এ কত Series আছে (নমুনা — চিত্রণমূলক)
curl -s http://203.0.113.20:9090/api/v1/status/tsdb | head -c 600
# "numSeries" দেখুন — হঠাৎ বাড়লে নতুন Label সন্দেহ করুন

# কোন Metric-এর Cardinality বেশি, তা খুঁজতে (PromQL, Prometheus UI-তে)
# topk(10, count by (__name__)({__name__=~".+"}))
# ফলাফলে সবচেয়ে বেশি Series-ওয়ালা Metric-গুলো দেখা যাবে
```

৫.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র জন্য একটি সম্পূর্ণ Instrumentation Checklist (নথি হিসেবে) ও একটি OpenTelemetry Auto-instrumentation চালুর ধারণা — একসঙ্গে।

```
# instrumentation-checklist.md — প্রতিটি নতুন সার্ভিসের জন্য (নথি)
# ১) Auto-instrumentation চালু আছে কি? (HTTP, MongoDB, Downstream)
# ২) /metrics Endpoint আছে কি? Prometheus Scrape করছে কি?
# ৩) Structured Log (JSON) + trace_id প্রতিটি লাইনে আছে কি?
# ৪) Golden Signals Dashboard-এ ৪টি Signal আছে কি?
# ৫) Label Convention মানা হয়েছে কি? (service, env, route, method, status_code)
# ৬) Label-এ User ID / Order ID / পূর্ণ Path নেই তো?
# ৭) SLI/SLO সংজ্ঞায়িত হয়েছে কি? (অধ্যায় ৪)
# ৮) Alert-এর পাশে Runbook লেখা আছে কি?
# ৯) Log Level Production-এ Info কি? Debug বন্ধ কি?
# ১০) Sampling ঠিক আছে কি? (Trace 10% + সব Error)
# ১১) Retention ঠিক আছে কি? (Metrics 30d, Logs 14d, Traces 7d)
# ১২) Secret / PII Log বা Trace-এ যাচ্ছে না তো?
```

```
# OpenTelemetry Auto-instrumentation — ধারণাগত চালু (Node.js 20)
# npm install @opentelemetry/auto-instrumentations-node
# পরিবেশ চলক (Environment Variable) দিয়ে নিয়ন্ত্রণ:
# OTEL_SERVICE_NAME=technova-api
# OTEL_EXPORTER_OTLP_ENDPOINT=http://203.0.113.20:4318
# OTEL_TRACES_SAMPLER=parentbased_traceidratio
# OTEL_TRACES_SAMPLER_ARG=0.1   # 10% Sampling
# node --require @opentelemetry/auto-instrumentations-node/register src/index.js
# (আসল Secret বা Token এখানে লিখবেন না — Endpoint অভ্যন্তরীণ নেটওয়ার্কে রাখুন)
```

এই Checklist-এর শক্তি তার সরলতায়: নতুন সার্ভিস চালু করার সময় বারোটি প্রশ্নের উত্তর "হ্যাঁ" না হলে সার্ভিস Production-এ যাবে না। এটি একটি Definition of Done — Observability-র জন্য।

৫.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি: TechNova-তে নতুন একজন ডেভেলপার Product Search Endpoint-এ Metric যোগ করেন — Label-এ `query` (ব্যবহারকারীর Search শব্দ) ব্যবহার করে। প্রথম দিন সব ঠিক থাকে। দ্বিতীয় দিন Prometheus-এর Memory

দ্বিগুণ হয়, Query ধীর হয়, Grafana Timeout দেয়। কারণ: প্রতিটি ভিন্ন Search শব্দ একটি নতুন Time Series তৈরি করেছে — হাজার হাজার Series। Cardinality বিস্ফোরণ।

সমাধান: Label থেকে `query` সরিয়ে ফেলা হয়; Search শব্দ Log-এ রাখা হয় (সেখানে High-cardinality চলে), আর Metric-এ শুধু `route`, `status_code` রাখা হয়। Prometheus পুনরায় চালু করে পুরোনো Series মুছে ফেলা হয়। শিক্ষা: নতুন Label যোগ করার আগে Cardinality হিসাব করা একটি Code Review-এর অংশ হওয়া উচিত — ঠিক যেমন Security Review। একটি ভুল Label-এর খরচ ঘণ্টার পর ঘণ্টা Downtime হতে পারে।

৫.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **যাত্রা বাছুন:** TechNova-র সবচেয়ে গুরুত্বপূর্ণ যাত্রা (Checkout) বাছুন; তার Golden Signals লিখুন।
2. **Auto-instrumentation বসান:** OpenTelemetry Auto-instrumentation দিয়ে HTTP, MongoDB ও Downstream Call-এর Trace চালু করুন (৫.৮ দেখুন)।
3. **ভিত্তি Metric বসান:** Request Counter, Latency Histogram, `/metrics` Endpoint, Node Exporter — অধ্যায় ২ ও ৩ অনুযায়ী।
4. **Structured Log চালু করুন:** JSON Log + Trace ID — প্রতিটি Request-এ একটি Info লাইন।
5. **Manual Instrumentation যোগ করুন:** Checkout-এ Business Counter (৫.৭ দেখুন) — সাফল্য/ব্যর্থতার Outcome-সহ।
6. **Label পর্যালোচনা করুন:** ৫.৭-এর Cardinality যাচাই চালিয়ে দেখুন কোনো Label বিস্ফোরিত হচ্ছে কি না।
7. **Sampling ও Retention ঠিক করুন:** Trace ১০% + সব Error; Metrics ৩০ দিন, Logs ১৪ দিন, Traces ৭ দিন।
8. **Checklist Git-এ রাখুন:** ৫.৮-এর Checklist প্রতিটি নতুন সার্ভিসের Definition of Done-এ যোগ করুন।

৫.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	পরিণতি	সঠিক পথ
Label-এ User ID / Query / পূর্ণ Path	Cardinality বিস্ফোরণ	High-cardinality Log/Trace-এ রাখুন
সবকিছু Instrument করা	খরচ ও জটিলতা বাড়ে, কেউ বোঝে না	প্রশ্ন-ভিত্তিক Instrument: কোন সিদ্ধান্তের জন্য?
শুধু Auto-instrumentation	ব্যবসায়িক প্রসঙ্গ অনুপস্থিত	Auto + Manual মিশ্রণ
শুধু Manual Instrumentation	ভুলে যাওয়া, অসম্পূর্ণ Coverage	Auto দিয়ে ভিত্তি, Manual দিয়ে গভীরতা
Debug Log Production-এ চালু	Volume ও খরচ বিস্ফোরণ	Production-এ Info; Debug সাময়িকভাবে
Sampling ছাড়া সব Trace রাখা	Storage খরচ বিস্ফোরণ	10% + সব Error + ধীর Trace

৫.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Instrumentation-এর সময় নিরাপত্তা সবচেয়ে বেশি ঝুঁকিতে পড়ে — কারণ কোডের ভেতরে Telemetry লেখা হয়, আর ভুল Field সহজেই সংবেদনশীল তথ্য প্রকাশ করে। নিয়ম: (১) Log, Trace Attribute ও Metric Label — তিন জায়গাতেই Password, Token, Authorization Header, Credit Card, OTP কখনো লিখবেন না। (২) Request Body ও Query Parameter পুরো Log করবেন না — তাতে Email, ঠিকানা, Token থাকতে পারে। (৩) Error Log-এ Stack Trace রাখা যায়, কিন্তু Stack Trace-এ সংবেদনশীল Variable-এর মান থাকলে তা Mask করুন। (৪) Telemetry Endpoint-এ (OTLP,

Pushgateway) Authentication চালু করুন — নইলে বাইরে থেকে ভুয়া ডেটা পাঠানো যায়। (৫) নমুনা কনফিগারেশনে আসল Secret নয়, `CHANGE-ME-...` প্লেসহোল্ডার ব্যবহার করুন।

৫.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Instrumentation-এর নিজস্ব খরচ সাধারণত ১–৩% CPU — কিন্তু ভুল Instrumentation-এর খরচ অনেক বেশি। Synchronous Log (প্রতিটি Log-এর জন্য Disk অপেক্ষা) Request-কে ধীর করে; Asynchronous/Buffered Log ব্যবহার করুন। High-cardinality Metric Prometheus-এর Memory ও Query-কে ধীর করে — Cardinality নিয়ন্ত্রণই সবচেয়ে বড় খরচ-নিয়ন্ত্রণ। Trace-এর ক্ষেত্রে Sampling-ই খরচ নিয়ন্ত্রণ করে — ১০০% থেকে ১০%-এ নামালে খরচ প্রায় দশ ভাগের এক ভাগ হয়, অথচ বেশিরভাগ অনুসন্ধানক্ষমতা থাকে — কারণ Error ও ধীর Trace সব রাখা হয়।

খরচের হিসাব করার একটি সহজ উপায়: প্রতিদিন কত Log লাইন, কত Trace Span, কত Metric Sample তৈরি হয় — তা এক সপ্তাহ মেপে দেখুন, তারপর Retention দিয়ে গুণ করুন। এই হিসাব ছাড়া Retention ঠিক করা অনুমান — আর অনুমান থেকে Storage বিলে চমক আসে।

৫.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	পরবর্তী ধাপ
Prometheus Memory হঠাৎ বাড়ছে	নতুন High-cardinality Label	Topk Query দিয়ে কোন Metric বাড়ছে দেখুন; Label সরান
Trace অনুপস্থিত (কিছু Request-এর নেই)	Sampling (10%) — স্বাভাবিক	Error Trace সব রাখা হচ্ছে কি না যাচাই করুন
Log Volume হঠাৎ বাড়ছে	Debug Level, বা Loop-এ Log	Log Level ও উৎস খুঁজুন
Auto-instrumentation কাজ করছে না	Library সংস্করণ, বা Require ক্রম ভুল	Startup Log দেখুন; Require সবার আগে কি না যাচাই করুন
Metric-এ Route "unknown" দেখাচ্ছে	Route Template পাওয়া যায়নি (404 বা Middleware ক্রম)	Middleware ক্রম ও 404 Handler যাচাই করুন

TROUBLESHOOTING: Cardinality সমস্যায় প্রথমে `topk` Query চালান — কোন Metric সবচেয়ে বেশি Series তৈরি করছে তা দেখুন। তারপর সেই Metric-এর Label তালিকা দেখুন। বেশিরভাগ ক্ষেত্রে একটি Label-ই দায়ী হয় — User ID, Query, বা পূর্ণ Path।

৫.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ব্যবহারকারীর যাত্রা থেকে শুরু; Auto + Manual মিশ্রণ; Low-cardinality Label; Route Template ব্যবহার; Structured Log + Trace ID; Sampling (১০% + সব Error); Checklist Git-এ; Cardinality মাসিক পর্যালোচনা; প্রতিটি Alert-এর পাশে Runbook।

অ্যান্টি-প্যাটার্ন: Label-এ High-cardinality মান; সবকিছু Instrument করা; Debug Log Production-এ; Sampling ছাড়া সব Trace; Instrumentation-কে "পরে করব" বলে ফেলে রাখা; Telemetry-র পেছনে কোনো প্রশ্ন না থাকা।

PRO TIP: প্রতিটি Pull Request-এ জিজ্ঞেস করুন: "এই পরিবর্তন ভাঙলে আমরা কীভাবে জানব?" উত্তরটি যদি Telemetry না হয়, তাহলে Instrumentation-সহ Pull Request পাঠান। Observability কোডের অংশ — পরের কাজ নয়।

৫.১৬ অধ্যায়ের সারাংশ

Monitoring Strategy শুরু হয় ব্যবহারকারীর যাত্রা থেকে — Checkout, Login, Product Search — তারপর Golden Signals, তারপর নির্ভরতা, তারপর Infrastructure। Auto-instrumentation ভিত্তি দেয়, Manual Instrumentation ব্যবসায়িক গভীরতা দেয় — দুটোর মিশ্রণই সঠিক। Cardinality হলো Metric-এর নীরব ঘাতক: Label-এ শুধু Low-cardinality মান রাখুন, High-cardinality মান Log ও Trace-এ রাখুন, আর Route-এর ক্ষেত্রে সবসময় Template ব্যবহার করুন। Sampling (১০% + সব Error + ধীর Trace) ও Retention Tier (Metrics ৩০ দিন, Logs ১৪ দিন, Traces ৭ দিন) খরচ নিয়ন্ত্রণ করে। আর সবকিছুর পেছনে একটি প্রশ্ন থাকা উচিত: "এই Telemetry দিয়ে কোন সিদ্ধান্ত নেব?"

৫.১৭ মূল প্রযুক্তিগত পরিভাষা

Instrumentation: কোডে Telemetry তৈরি করার ব্যবস্থা যোগ করা — Metric, Log ও Trace নির্গমন।

Auto-instrumentation: Library বা Agent-এর মাধ্যমে কোডে হাত না দিয়ে স্বয়ংক্রিয়ভাবে Telemetry তৈরি করা।

Manual Instrumentation: কোডে নিজে হাতে Metric, Log ও Span লেখা — ব্যবসায়িক প্রসঙ্গসহ।

Cardinality: একটি Metric-এর সম্ভাব্য Label মানের সংমিশ্রণের সংখ্যা; বেশি হলে Prometheus ধীর হয়।

Label: Metric-এর সঙ্গে জুড়ে দেওয়া Key-Value চিহ্ন — যেমন service, route, status_code।

Sampling: সব Telemetry না রেখে একটি প্রতিনিধিত্বমূলক অংশ রাখা — খরচ নিয়ন্ত্রণের উপায়।

Head-based Sampling: Trace শুরুর সময়ই রাখা হবে কি না সিদ্ধান্ত নেওয়া; সহজ কিন্তু বিরল Error হারাতে পারে।

Tail-based Sampling: Trace শেষ হওয়ার পর সিদ্ধান্ত — Error ও ধীর Trace সব রাখা; বেশি কার্যকর।

Label Convention: Label-এর নামকরণের ধারাবাহিক প্রথা — Correlation-এর ভিত্তি।

Route Template: Path-এর সাধারণ রূপ — যেমন /orders/:id; Metric Label-এ এটিই ব্যবহার করতে হয়।

Runbook: একটি Alert-এর ক্ষেত্রে ধাপে ধাপে করণীয় লেখা নথি — ঘটনার সময় দ্রুত সাড়ার জন্য।

Definition of Done: একটি কাজ সম্পূর্ণ মানার শর্ত — এখানে Instrumentation Checklist-সহ।

৫.১৮ জ্ঞান যাচাই — প্রশ্ন

1. কী Instrument করবেন প্রথমে — অগ্রাধিকারের ক্রমটি ব্যাখ্যা করুন। Infrastructure কেন সবার শেষে?
2. Manual ও Auto-instrumentation-এর সুবিধা-অসুবিধা কী? TechNova-র জন্য কোন মিশ্রণটি সঠিক?
3. Cardinality কী? Label-এ User ID যোগ করলে কী হয় — সংখ্যা দিয়ে দেখান।
4. Route Template বনাম পূর্ণ Path — Metric Label-এ কোনটি ব্যবহার করবেন, কেন?
5. Head-based ও Tail-based Sampling-এর পার্থক্য কী?
6. Production-এ Log Level কেন Info রাখা উচিত, Debug নয়?
7. Metrics, Logs ও Traces-এর Retention কেন আলাদা? আপনার প্রস্তাবিত Retention লিখুন।
8. "এই Telemetry দিয়ে কোন সিদ্ধান্ত নেব?" — এই প্রশ্নটি কেন প্রতিটি Instrumentation-এর আগে করা উচিত?

৫.১৯ ব্যবহারিক অনুশীলন

1. TechNova-র Checkout যাত্রার জন্য একটি Instrumentation পরিকল্পনা লিখুন: কোন Metric, কোন Log Event, কোন Span — টেবিল আকারে।
2. একটি Express অ্যাপে OpenTelemetry Auto-instrumentation চালু করুন; একটি Request-এর Trace দেখুন ও Span-গুলোর নাম লিখুন।
3. ৫.৭-এর Business Counter আপনার অ্যাপে যোগ করুন; Checkout সফল ও ব্যর্থ করে Metric-এর মান দেখুন।
4. ইচ্ছাকৃতভাবে একটি High-cardinality Label (যেমন Query) যোগ করে Prometheus-এর Series সংখ্যা কীভাবে বাড়ে, তা পর্যবেক্ষণ করুন; তারপর Label সরিয়ে ফেলুন।
5. ৫.৮-এর Checklist ব্যবহার করে আপনার একটি সার্ভিস পর্যালোচনা করুন; কোন প্রশ্নের উত্তর "না" এল, তার তালিকা ও সংশোধনের পরিকল্পনা লিখুন।
6. আপনার Telemetry-র এক সপ্তাহের Volume মেপে (Log লাইন, Trace Span, Metric Sample) একটি খরচ/Retention পরিকল্পনা লিখুন।

PART II • DEVOPS ENGINEERING MASTERY

Metrics and Prometheus Engineering

Metrics-এর ধরন, Prometheus আর্কিটেকচার, PromQL, Exporter ও Instrumentation, Alertmanager ও Kubernetes মনিটরিং।

অধ্যায় ৬ • PART II — Metrics and Prometheus Engineering

Chapter 6 — Metrics Fundamentals — Counters, Gauges, Histograms, and Summaries

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Prometheus-এর চারটি মৌলিক `Metric` ধরন — `Counter`, `Gauge`, `Histogram` এবং `Summary` — প্রত্যেকটির সংজ্ঞা, ব্যবহারের ক্ষেত্র, সীমাবদ্ধতা ও অভ্যন্তরীণ গঠন ব্যাখ্যা করতে পারবেন। আপনি বুঝতে পারবেন কোন পরিমাপের জন্য কোন ধরনটি সঠিক, কেন `Counter`-কে কখনো সরাসরি গ্রাফ করা যায় না, কেন `Gauge` কমে-বাড়ে, কেন লেটেন্সি মাপতে `Histogram` প্রয়োজন এবং কেন `Summary` এখন প্রায়ই এড়িয়ে চলা হয়। আপনি Prometheus-এর `exposition format` পড়তে পারবেন, মেট্রিকের নামকরণের নিয়ম — `_total`, `_bucket`, `_sum`, `_count` প্রত্যয় — প্রয়োগ করতে পারবেন, `cardinality` বা সিরিজ-বিস্ফোরণের ঝুঁকি হিসাব করতে পারবেন এবং TechNova-র MERN অ্যাপ্লিকেশনের জন্য একটি সঠিক মেট্রিক পরিকল্পনা লিখতে পারবেন।

- চারটি মেট্রিক ধরনের মধ্যে পার্থক্য ও নির্বাচনের যুক্তি ব্যাখ্যা করা।
- `Counter reset`, `rate()` ও `increase()`-এর সম্পর্ক বোঝা।
- `Histogram` বাকেট, `histogram_quantile()` ও `quantile` গণনার পদ্ধতি শেখা।
- `Summary`-এর ক্লায়েন্ট-সাইড `quantile` ও তার অসুবিধা মূল্যায়ন করা।
- নামকরণ, একক (`unit`) ও লেবেল ব্যবহারের নিয়ম প্রয়োগ করা।
- `Cardinality` হিসাব করে সিরিজ-বিস্ফোরণ এড়ানো।

৬.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

প্রোডাকশন সিস্টেম সম্পর্কে সিদ্ধান্ত নেওয়া হয় সংখ্যার ভিত্তিতে — "সার্ভার কি ধীর?" নয়, "৯৫ শতাংশ অনুরোধ কত মিলিসেকেন্ডে শেষ হচ্ছে?" এই প্রশ্নের উত্তরে। সেই সংখ্যা যদি ভুলভাবে মাপা হয়, তাহলে সিদ্ধান্তও ভুল হবে। TechNova-র কথাই ধরুন: তাদের MERN অ্যাপে প্রতিদিন হাজারো ব্যবহারকারী লগইন করেন, অর্ডার দেন, পেমেন্ট করেন। একদিন সকালে গ্রাহক অভিযোগ করলেন — "অ্যাপ ধীর"। লগে শুধু কিছু ত্রুটির বার্তা, কিন্তু কতজন আক্রান্ত, কতক্ষণ ধরে, কোন API ধীর — তা জানা গেল না। কারণ সেখানে কোনো সঠিক মেট্রিক ছিল না। `Counter` দিয়ে অনুরোধ গোনা হয়নি, `Histogram` দিয়ে লেটেন্সি মাপা হয়নি, `Gauge` দিয়ে চলমান সংযোগ দেখা হয়নি। ফলে দলটি অন্ধকারে হাতড়েছে।

মেট্রিক হলো observability-র সংখ্যাগত ভাষা। সঠিক মেট্রিক থাকলে আপনি SLI ও SLO সংজ্ঞায়িত করতে পারেন, অ্যালার্ট লিখতে পারেন, ক্ষমতা পরিকল্পনা করতে পারেন, রিগ্রেশন ধরতে পারেন। ভুল মেট্রিক থাকলে আপনি ভুল অ্যালার্ট পাবেন, ভুল স্কেলিং করবেন, ভুল জায়গায় অপ্টিমাইজ করবেন। যেমন — গড় লেটেন্সি (`average latency`) দেখে মনে হতে পারে সব ঠিক আছে, অথচ ৫% ব্যবহারকারী ১০ সেকেন্ড অপেক্ষা করছেন। গড় লুকিয়ে ফেলে সেই যন্ত্রণা; `Histogram`-এর p95 ও p99 তা প্রকাশ করে। এই অধ্যায় সেই পার্থক্যটাই শেখায় — কী মাপবেন, কীভাবে মাপবেন, আর কী মাপবেন না।

> **REAL WORLD:** TechNova-র `/api/orders` এন্ডপয়েন্ট একবার ধীর হয়ে গিয়েছিল কারণ MongoDB-র একটি কুয়েরি ইনডেক্স ছাড়া চলছিল। গড় লেটেন্সি ছিল মাত্র ২১০ মিলিসেকেন্ড — দেখতে স্বাভাবিক। কিন্তু p99 ছিল ৮.৪ সেকেন্ড। যারা বড় অর্ডার দিচ্ছিলেন, তারাই আটকে যাচ্ছিলেন। `Histogram` না থাকলে এই সমস্যা সপ্তাহখানেক ধরা পড়ত না।

৬.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায় পড়ার আগে আপনার Part I-এর ধারণাগুলো জানা থাকা ভালো — `monitoring` বনাম `observability`, তিনটি স্তম্ভ (Metrics, Logs, Traces), SLI/SLO ও `golden signals` (লেটেন্সি, ট্র্যাফিক, ত্রুটি, স্যাচুরেশন)। এ ছাড়া প্রয়োজন: HTTP-র মৌলিক ধারণা (স্ট্যাটাস কোড, অনুরোধ-উত্তর), Node.js ও Express-এর সাধারণ পরিচিতি, Linux টার্মিনালে কাজ করার অভিজ্ঞতা এবং গড়, মধ্যমা (median) ও শতকরা (percentile) — এই তিনটি পরিসংখ্যানগত ধারণা। গাণিতিক গভীরতা লাগবে না; তবে `quantile` বলতে কী বোঝায় তা বুঝলে এই অধ্যায় অনেক সহজ হবে। Prometheus বা PromQL আগে না জানলেও চলবে — সেগুলো পরের অধ্যায়ে আসবে।

আপনি যদি `time series` শব্দটি আগে না শুনে থাকেন, তাহলে মনে রাখুন: `time series` হলো সময়ের সঙ্গে সঙ্গে বদলাতে থাকা একটি সংখ্যার ধারা — যেমন প্রতি ১৫ সেকেন্ডে রেকর্ড করা CPU ব্যবহারের হার। Prometheus সেই ধারাগুলোই সংরক্ষণ করে। এই অধ্যায়ের প্রতিটি মেট্রিক ধরন আসলে একটি করে `time series` তৈরির নিয়ম।

৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: একটি চলমান প্রোগ্রাম সম্পর্কে আপনি কী কী জানতে চান? তিনটি মৌলিক প্রশ্ন — (১) কতবার কিছু ঘটেছে? (২) এই মুহূর্তে কোনো কিছুর মান কত? (৩) ঘটনাগুলো কতক্ষণ সময় নিয়েছে, এবং তার বণ্টন কেমন? এই তিন প্রশ্নের উত্তরই চারটি মেট্রিক ধরন। প্রথম প্রশ্নের উত্তর `Counter` — শুধু বাড়ে, কখনো কমে না। দ্বিতীয় প্রশ্নের উত্তর `Gauge` — বাড়তেও পারে, কমতেও পারে। তৃতীয় প্রশ্নের উত্তর `Histogram` অথবা `Summary` — সময়ের বণ্টন মাপে।

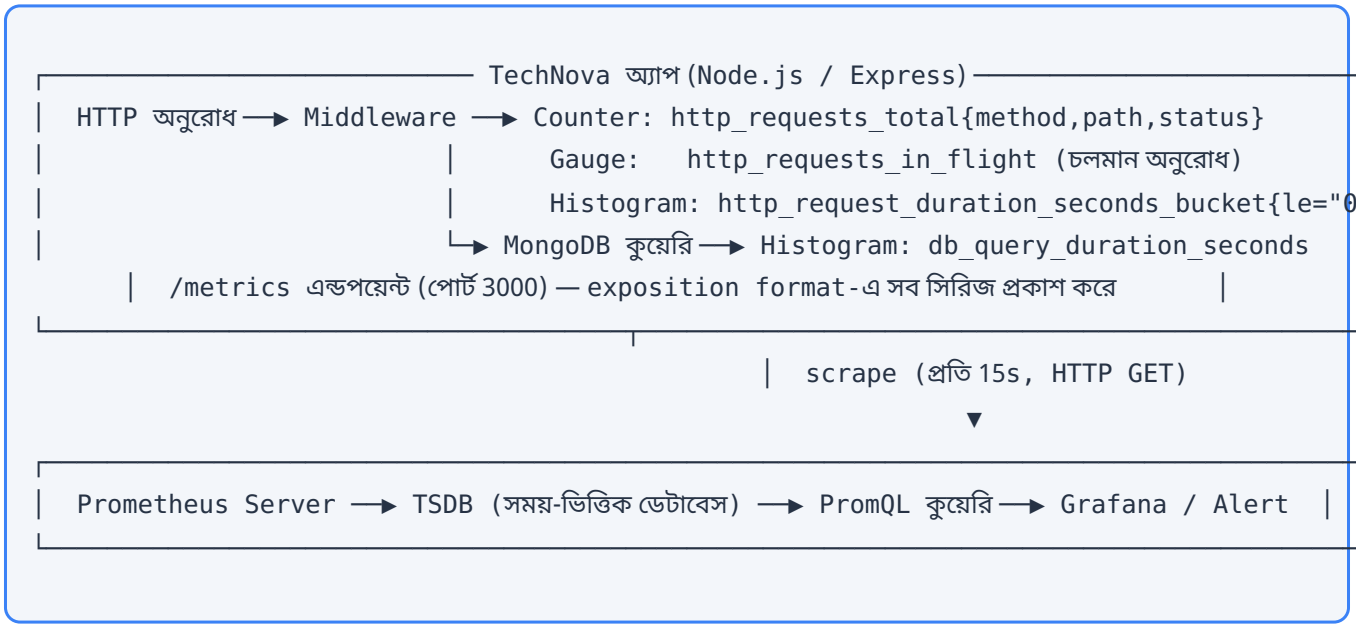
কেন চারটি ধরন, একটি নয়? কারণ প্রতিটি ধরনের পেছনে আলাদা গাণিতিক ধারণা ও আলাদা খরচ আছে। `Counter` সবচেয়ে সস্তা ও নিরাপদ — একটি সংখ্যা মনে রাখলেই চলে। `Histogram` বেশি তথ্য দেয়, কিন্তু বেশি সিরিজ তৈরি করে। `Summary` ক্লায়েন্টের ভেতরেই শতকরা হিসাব করে ফেলে, ফলে সার্ভারে গণনা কমে, কিন্তু পরে আর একত্র (aggregate) করা যায় না। এই ট্রেড-অফ বোঝাই এই অধ্যায়ের মূল কাজ। একটি মেট্রিক বেছে নেওয়া মানে একটি প্রশ্ন বেছে নেওয়া — "আমি ভবিষ্যতে এই সংখ্যা দিয়ে কী জিজ্ঞাসা করব?" সেই প্রশ্ন আগে ঠিক করুন, তারপর ধরন বাছুন।

> **CONCEPT:** মেট্রিক একটি পরিমাপের চুক্তি (contract)। `Counter` চুক্তি করে — "আমি কেবল বাড়ব।" Prometheus সেই চুক্তির ওপর ভরসা করে `rate()` হিসাব করে; প্রক্রিয়া পুনরায় চালু হলে কাউন্টার শূন্যে ফিরে যায়, আর Prometheus সেই পতনকে `reset` হিসেবে চিনে নেয়। চুক্তি ভাঙলে — অর্থাৎ কাউন্টার কমিয়ে দিলে — `rate()`-এর ফল ভুল হয়ে যায়।

মেট্রিক, সিরিজ ও লেবেল

Prometheus-এ প্রতিটি মেট্রিক একটি নাম ও একগুচ্ছ `label` (কী-মান জোড়া) দিয়ে চিহ্নিত হয়। নাম ও লেবেলের প্রতিটি অনন্য সমন্বয় একটি `time series`। যেমন `http_requests_total{method="GET", status="200"}` একটি সিরিজ, আর `{method="POST", status="500"}` আরেকটি। লেবেল যত বেশি মান নেবে, সিরিজ তত বেশি হবে — এটাই `cardinality`। লেবেল শক্তিশালী, কিন্তু তার দাম আছে; এই অধ্যায়ের শেষভাগে তার হিসাব আছে।

৬.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



লক্ষ্য করুন: মেট্রিকের মান অ্যাপ্লিকেশনের মেমরিতে থাকে; Prometheus সেগুলো টেনে নেয় (`pull` মডেল)। অ্যাপ নিজে কোথাও পাঠায় না। এই `pull` মডেলের কারণে কোনো অ্যাপ বন্ধ থাকলে Prometheus তা জানতে পারে (`up` মেট্রিক শূন্য হয়), আর কোনো কেন্দ্রীয় সার্ভারের ওপর নির্ভর করতে হয় না। `Counter` ও `Gauge` একটি করে সিরিজ; `Histogram` প্রতিটি বাকেটের জন্য একটি করে সিরিজ তৈরি করে — একটি `Histogram` মানে প্রায় এক ডজন সিরিজ। এই খরচের হিসাব মাথায় রেখে বাকেট বাছতে হয়।

৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Counter — শুধু বাড়ে এমন সংখ্যা

`Counter` হলো একটি একমুখী সংখ্যা: শূন্য থেকে শুরু হয়, কেবল বাড়ে, প্রক্রিয়া পুনরায় চালু হলে শূন্যে ফিরে যায়। ব্যবহার: মোট অনুরোধের সংখ্যা, মোট ত্রুটি, মোট সম্পন্ন কাজ, মোট বাইট পাঠানো হয়েছে। নামের শেষে `_total` প্রত্যয় দেওয়া নিয়ম। `Counter`-কে কখনো সরাসরি গ্রাফ করবেন না — তার মান কেবলই বাড়তে থাকে, গ্রাফটি একটি ঢালু রেখা হবে যার কোনো অর্থ নেই। এর বদলে `rate()` বা `increase()` ব্যবহার করুন: `rate(http_requests_total[5m])` বলে প্রতি সেকেন্ডে গড়ে কত অনুরোধ আসছে। `Counter reset` হলে Prometheus তা শনাক্ত করে — মান হঠাৎ কমে গেলে সে ধরে নেয় কাউন্টার নতুন করে শুরু হয়েছে, এবং `rate()` গণনায় সেটি সামলে নেয়। তাই `Counter` কখনো নিজে থেকে কমাবেন না; ভুল করে কমালে `rate()` ভুল হবে।

Gauge — এই মুহূর্তের মান

`Gauge` হলো একটি বর্তমান মান: বাড়তে পারে, কমতে পারে, যেকোনো সংখ্যা হতে পারে। ব্যবহার: চলমান অনুরোধের সংখ্যা, মেমরি ব্যবহার, CPU ব্যবহারের শতাংশ, কিউ-তে অপেক্ষমাণ কাজ, তাপমাত্রা, বর্তমানে সংযুক্ত ব্যবহারকারী। `Gauge`-কে সরাসরি গ্রাফ করা যায় — তার মানই অর্থবহ। `Gauge`-কে `rate()` দিয়ে মাপবেন না; তার বদলে সরাসরি মান দেখুন, অথবা পরিবর্তনের জন্য `delta()` ব্যবহার করুন। একটি সাধারণ ভুল: "সক্রিয় ব্যবহারকারী" গুনতে গিয়ে `Counter` ব্যবহার করা — ব্যবহারকারী লগআউট করলে সংখ্যা কমে, তাই সেটি `Gauge`-এর কাজ। আরেকটি ভুল: `Gauge` দিয়ে মোট অনুরোধ গোনা — প্রক্রিয়া পুনরায় চালু হলে আগের মোট হারিয়ে যায়, অথচ `Counter`-এর ক্ষেত্রে Prometheus `reset` সামলে নেয়।

Histogram — বণ্টন মাপার যন্ত্র

`Histogram` একটি মানের বণ্টন (distribution) মাপে — সাধারণত লেটেন্সি বা আকার। আপনি আগে থেকে কয়েকটি সীমা (`bucket`) ঠিক করে দেন, যেমন ০.০০৫, ০.০১, ০.০২৫, ০.০৫, ০.১, ০.২৫, ০.৫, ১, ২.৫, ৫, ১০ সেকেন্ড। প্রতিটি পর্যবেক্ষণ সেই সীমাগুলোর মধ্যে পড়ে, এবং Prometheus গুনে রাখে প্রতিটি বাকেটে কতগুলো পর্যবেক্ষণ পড়েছে। বাকেটগুলো ক্রমবর্ধমান (cumulative): `le="0.1"` বাকেটে আছে ০.১ সেকেন্ড বা তার কম সময়ে শেষ হওয়া সব অনুরোধের সংখ্যা। এ ছাড়া `_sum` (সব পর্যবেক্ষণের যোগফল) ও `_count` (মোট পর্যবেক্ষণের সংখ্যা) নামে দুটি সিরিজ থাকে। এই তিনটি তথ্য দিয়ে `histogram_quantile(0.95, ...)` ফাংশন p95 বের করে, `_sum / _count` দিয়ে গড় বের হয়, এবং বাকেট থেকে "কত শতাংশ অনুরোধ ২০০ মিলিসেকেন্ডের মধ্যে শেষ হলো" — এই ধরনের প্রশ্নের উত্তর মেলে। `Histogram`-এর সবচেয়ে বড় সুবিধা: পরে যেকোনো শতকরা (p50, p90, p95, p99) হিসাব করা যায়, এবং একাধিক সার্ভারের `Histogram` একত্র করা যায় — কারণ বাকেটের গণনা যোগ করা যায়।

Summary — ক্লায়েন্ট-সাইড শতকরা

`Summary` দেখতে `Histogram`-এর মতো, কিন্তু ভেতরে আলাদা। `Summary` ক্লায়েন্ট লাইব্রেরির ভেতরেই সরাসরি `quantile` হিসাব করে — যেমন ০.৫, ০.৯, ০.৯৯ — এবং সেগুলোকে `{quantile="0.95"}` লেবেলসহ আলাদা সিরিজ হিসেবে প্রকাশ করে। সঙ্গে `_sum` ও `_count` থাকে, কিন্তু কোনো বাকেট থাকে না। সুবিধা: সার্ভারে গণনা লাগে না, সরাসরি শতকরা দেখা যায়। অসুবিধা দুটি ও বড়: প্রথমত, একাধিক সার্ভারের `quantile` একত্র করা যায় না — দুটি সার্ভারের p95-এর গড় প্রকৃত p95 নয়। দ্বিতীয়ত, `quantile` আগে থেকে ঠিক করে দিতে হয়; পরে p99.9 জানতে চাইলে আর উপায় নেই। এই কারণে আধুনিক Prometheus ব্যবহারে `Histogram`-ই প্রায় সবসময় পছন্দের পথ, আর `Summary` কেবল তখনই ব্যবহার করা হয় যখন ক্লায়েন্ট-সাইড গণনাই একমাত্র বিকল্প। Prometheus 3.x-এ `native histogram` নামে আরও উন্নত একটি ব্যবস্থা আছে, যেখানে বাকেট স্বয়ংক্রিয়ভাবে তৈরি হয় — সেটি ভবিষ্যতের পথ, তবে এই বইয়ে প্রথমে ক্লাসিক `Histogram` ভালো করে শিখুন।

ধরন	বাড়ে / কমে	সরাসরি গ্রাফ	মূল ব্যবহার	একত্র করা যায়?
Counter	শুধু বাড়ে	না — rate() ব্যবহার করুন	মোট অনুরোধ, মোট ত্রুটি	হ্যাঁ (sum)
Gauge	বাড়ে ও কমে	হ্যাঁ	মেমরি, চলমান অনুরোধ, কিউ	হ্যাঁ (sum/avg)
Histogram	বাকেট গণনা বাড়ে	না — quantile হিসাব করুন	লেটেন্সি, আকারের বণ্টন	হ্যাঁ (বাকেট যোগ)
Summary	quantile মান বদলায়	হ্যাঁ (quantile সিরিজ)	লেটেন্সি (একক সার্ভার)	না (quantile যোগ হয় না)

৬.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

মেট্রিক নিজে কোনো কনফিগারেশন ফাইল নয় — এটি কোডে সংজ্ঞায়িত হয়, আর Prometheus-এর `scrape` কনফিগারেশনে সেই কোডের এন্ডপয়েন্টটি যুক্ত হয়। নিচের উদাহরণটি লাইন ধরে ধরে পড়ুন।

```
# Node.js-এ prom-client দিয়ে একটি Counter সংজ্ঞায়িত করা
const client = require('prom-client');          # prom-client লাইব্রেরি আনা হচ্ছে
const httpRequests = new client.Counter({       # নতুন Counter তৈরি
  name: 'http_requests_total',                  # নাম — _total প্রত্যয় বাধ্যতামূলক নিয়ম
  help: 'Total number of HTTP requests',        # HELP: মানুষের পড়ার মতো বিবরণ
  labelNames: ['method', 'path', 'status'],     # লেবেল — প্রতিটি অনন্য সমন্বয় একটি সিরিজ
});
httpRequests.inc({ method: 'GET', path: '/api/orders', status: '200' });  # ১ বাড়ানো
```

```
# Prometheus-এর scrape কনফিগারেশন (prometheus.yml-এর অংশ)
scrape_configs:
  - job_name: 'technova-api'       # এই টার্গেট গোষ্ঠীর নাম
    scrape_interval: 15s           # প্রতি ১৫ সেকেন্ডে একবার /metrics টানা হবে
    static_configs:
      - targets: ['203.0.113.10:3000']  # TechNova API সার্ভারের ঠিকানা (কাল্পনিক IP)
```

প্রথম অংশে `help` লেখাটি ছোট মনে হতে পারে, কিন্তু এটি খুব গুরুত্বপূর্ণ — ছয় মাস পরে অন্য কেউ যখন মেট্রিকটি দেখবে, তখন এই লেখাটিই বলবে সংখ্যাটির অর্থ কী। `labelNames`-এ শুধু সেই লেবেলগুলোই রাখুন যেগুলো দিয়ে আপনি সত্যিই ভাগ করে দেখতে চান। `path` লেবেলে পুরো URL বসাবেন না — `/api/orders/12345` আর `/api/orders/67890` দুটি আলাদা সিরিজ হয়ে যাবে। এর বদলে রুটের প্যাটার্ন বসান: `/api/orders/:id`। দ্বিতীয় অংশে `scrape_interval` ১৫ সেকেন্ড মানে Prometheus প্রতি ১৫ সেকেন্ডে একবার মান সংগ্রহ করবে; এর চেয়ে ঘন ঘন করলে স্টোরেজ ও নেটওয়ার্ক খরচ বাড়ে, আর বেশি বিরতি দিলে স্বল্পস্থায়ী সমস্যা ধরা পড়ে না।

৬.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র API-র জন্য চারটি ধরনের মেট্রিক একসঙ্গে ব্যবহারের সম্পূর্ণ উদাহরণ:

```
# exposition format — /metrics এন্ডপয়েন্টে Prometheus যা দেখে (নমুনা আউটপুট — চিত্রণমূলক)
# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",path="/api/orders",status="200"} 4821
http_requests_total{method="GET",path="/api/orders",status="500"} 17

# HELP http_requests_in_flight Current in-flight HTTP requests
# TYPE http_requests_in_flight gauge
http_requests_in_flight 12

# HELP http_request_duration_seconds HTTP request latency in seconds
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{path="/api/orders",le="0.005"} 120
http_request_duration_seconds_bucket{path="/api/orders",le="0.01"} 410
http_request_duration_seconds_bucket{path="/api/orders",le="0.05"} 1950
http_request_duration_seconds_bucket{path="/api/orders",le="0.1"} 3800
http_request_duration_seconds_bucket{path="/api/orders",le="0.5"} 4720
http_request_duration_seconds_bucket{path="/api/orders",le="1"} 4790
http_request_duration_seconds_bucket{path="/api/orders",le="+Inf"} 4838
http_request_duration_seconds_sum{path="/api/orders"} 214.62
http_request_duration_seconds_count{path="/api/orders"} 4838

# HELP db_query_duration_seconds Database query latency (summary — তুলনার জন্য)
# TYPE db_query_duration_seconds summary
db_query_duration_seconds{quantile="0.5"} 0.012
db_query_duration_seconds{quantile="0.9"} 0.041
db_query_duration_seconds{quantile="0.99"} 0.18
db_query_duration_seconds_sum 96.4
db_query_duration_seconds_count 4838
```

এই উদাহরণ থেকে তিনটি হিসাব হাতে-কলমে করা যায়। প্রথমত, গড় লেটেন্সি: `214.62 / 4838 ≈ 0.044` সেকেন্ড, অর্থাৎ ৪৪ মিলিসেকেন্ড। দ্বিতীয়ত, ০.১ সেকেন্ডের মধ্যে শেষ হওয়া অনুরোধের হার: `3800 / 4838 ≈ 78.5%`। তৃতীয়ত, `le="+Inf"` বাকেটের মান `_count`-এর সমান — এটি একটি ভালো স্বাস্থ্য-পরীক্ষা: দুটি না মিললে কোথাও গণ্ডগোল

আছে। আরও লক্ষ্য করুন, `Histogram`-এর বাকেটগুলো ক্রমবর্ধমান — `le="0.05"`-এর ১৯৫০-এর মধ্যে `le="0.01"`-এর ৪১০-ও অন্তর্ভুক্ত। এই ক্রমবর্ধমান ধর্মের কারণেই `histogram_quantile()` কাজ করে।

৬.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র ব্ল্যাক ফ্রাইডে বিক্রির দিন সকাল ১০টায় ট্র্যাফিক স্বাভাবিকের পাঁচ গুণ বেড়ে গেল। অন-কল প্রকৌশলী রায়হান Grafana খুলে দেখলেন `http_requests_total`-এর `rate()` পাঁচ গুণ বেড়েছে, কিন্তু `http_request_duration_seconds`-এর p95 তখনও ১৮০ মিলিসেকেন্ডের নিচে — সিস্টেম সামলাচ্ছে। দুপুরে হঠাৎ p99 লাফিয়ে ৬ সেকেন্ডে উঠল, অথচ p50 স্থির। মানে অধিকাংশ ব্যবহারকারী ঠিক আছেন, কিন্তু একটি ছোট অংশ ভীষণ ধীরগতির শিকার। `path` লেবেল দিয়ে ভাগ করে দেখা গেল — ধীরগতি শুধু `/api/payments`-এ। পেমেন্ট গেটওয়ের একটি ডাউনস্ট্রিম সার্ভিস ধীর হয়ে গিয়েছিল। দলটি পেমেন্টের টাইমআউট কমিয়ে, সার্কিট ব্রেকার চালু করে সমস্যা সীমিত করল। এই ঘটনায় `Histogram` না থাকলে শুধু গড় দেখে মনে হতো সব ঠিক আছে, আর সমস্যা ধরা পড়ত গ্রাহকের অভিযোগে — অনেক দেরিতে।

আরেকটি পরিস্থিতি: মেমরি লিক। TechNova-র একটি সার্ভিসে `process_resident_memory_bytes` নামে একটি `Gauge` ধীরে ধীরে বাড়ছিল — প্রতি ঘণ্টায় ৪০ মেগাবাইট। দুই দিন পর সার্ভার ক্র্যাশ করত। `Gauge`-এর ধারাবাহিক বৃদ্ধি দেখে দলটি আগেই রিস্টার্টের পরিকল্পনা করল ও লিকের উৎস খুঁজে বের করল। `Counter` দিয়ে এই সমস্যা ধরা যেত না — মেমরি তো "মোট" কিছু নয়, এটি একটি বর্তমান মান।

> **REAL WORLD:** একটি জনপ্রিয় ভুল হলো লেটেন্সি মাপতে `Gauge` ব্যবহার করা — "শেষ অনুরোধটি কত সময় নিয়েছে"। সেটি কেবল একটি অনুরোধের গল্প বলে; আগের হাজারো অনুরোধের কিছুই বলে না। লেটেন্সি একটি বণ্টন, তাই তার জন্য `Histogram` প্রয়োজন। শেষ মানটি জানতে চাইলে লগ দেখুন; বণ্টন জানতে চাইলে মেট্রিক দেখুন।

৬.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. আপনার Node.js প্রজেক্টে `prom-client` ইনস্টল করুন: `npm install prom-client`।
2. `src/metrics.js` নামে একটি ফাইল তৈরি করুন; সেখানে চারটি মেট্রিক সংজ্ঞায়িত করুন — অনুরোধের `Counter`, চলমান অনুরোধের `Gauge`, লেটেন্সি `Histogram` (বাকেট: ০.০০৫, ০.০১, ০.০২৫, ০.০৫, ০.১, ০.২৫, ০.৫, ১, ২.৫, ৫, ১০), এবং MongoDB কুয়েরির জন্য আরেকটি `Histogram`।
3. Express `middleware` লিখুন যা প্রতিটি অনুরোধের শুরুতে `Gauge` বাড়ায় ও `Histogram` টাইমার চালু করে, আর শেষে `Gauge` কমায়, `Counter` বাড়ায় ও সময় রেকর্ড করে। `path` লেবেলে রুটের প্যাটার্ন ব্যবহার করুন, আসল ID নয়।
4. `GET /metrics` এন্ডপয়েন্ট যোগ করুন যা `prom-client`-এর `register.metrics()` ফেরত দেয়, `Content-Type` হিসেবে `register.contentType` ব্যবহার করে।
5. Docker Compose দিয়ে Prometheus চালু করুন (অধ্যায় ৭-এ বিস্তারিত আছে), `prometheus.yml`-এ আপনার অ্যাপকে টার্গেট হিসেবে যোগ করুন।
6. ব্রাউজারে `http://203.0.113.10:9090` খুলে `http_requests_total` কুয়েরি করুন; তারপর `rate(http_requests_total[5m])` ও `histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))` চালিয়ে দেখুন।
7. কৃত্রিম লোড তৈরি করুন — উদাহরণস্বরূপ কয়েকশো অনুরোধ পাঠান, কিছু ধীর অনুরোধ যোগ করুন — এবং দেখুন p50 বনাম p95 কীভাবে আলাদা আচরণ করে।

8. একটি ভুল লেবেল ইচ্ছা করে যোগ করুন (যেমন `user_id`), সিরিজ সংখ্যা কীভাবে বাড়ে তা Prometheus-এর `prometheus_tsdb_head_series` মেট্রিক দিয়ে দেখুন, তারপর সেটি সরিয়ে ফেলুন।

৬.১১ সাধারণ ভুল ও ভুল ধারণা

- **Counter-কে সরাসরি গ্রাফ করা:** মান কেবল বাড়ে; গ্রাফ অর্থহীন। সবসময় `rate()` বা `increase()` ব্যবহার করুন।
- **Gauge দিয়ে মোট গোনা:** পুনরায় চালু হলে ইতিহাস হারায়; মোট গুনতে `Counter` ব্যবহার করুন।
- **গড় লেটেন্সিকেই সত্য ধরা:** গড় লুকিয়ে ফেলে খারাপ অভিজ্ঞতা; p95/p99 দেখুন।
- **Summary-কে একাধিক সার্ভারে একত্র করা:** `quantile`-এর গড় প্রকৃত `quantile` নয়; একত্র করতে হলে `Histogram` ব্যবহার করুন।
- **লেবেলে উচ্চ-কার্ডিনালিটি মান বসানো:** `user_id`, `request_id`, ইমেইল, পুরো URL — এগুলো সিরিজ-বিস্ফোরণ ঘটায়। নিচের ৬.১৩ অংশে হিসাব দেখুন।
- **বাকেট ভুল বাছা:** সব বাকেট ১ সেকেন্ডের বেশি হলে ৫০ মিলিসেকেন্ডের অনুরোধগুলোর পার্থক্য ধরা পড়বে না; আবার অপ্রয়োজনীয় সূক্ষ্ম বাকেট সিরিজ বাড়ায়।
- **একক না লেখা:** `http_request_duration` না লিখে `http_request_duration_seconds` লিখুন — একক নামেই থাকা উচিত, আর মান সবসময় সেকেন্ড বা বাইটের মতো ভিত্তি এককে রাখুন।
- **HELP লেখা বাদ দেওয়া:** ছয় মাস পরে কেউ বুঝবে না সংখ্যাটির অর্থ কী।

> **WARNING:** লেবেলে কখনো গোপন তথ্য বসাবেন না — পাসওয়ার্ড, টোকেন, ইমেইল ঠিকানা, ব্যক্তিগত শনাক্তকারী। মেট্রিক লেবেল লগের মতো সংরক্ষিত হয়, সহজে মুছে ফেলা যায় না, এবং অনেক ড্যাশবোর্ডে দৃশ্যমান থাকে। গোপন তথ্য মেট্রিকে এলে তা ডেটা ফাঁসের ঘটনা হয়ে যায়।

৬.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

মেট্রিক নিরীহ মনে হলেও এতে সংবেদনশীল তথ্য ফাঁস হতে পারে। প্রথমত, লেবেল: `path` লেবেলে যদি ব্যবহারকারীর ইমেইল বা টোকেনসহ URL চলে আসে, সেটি Prometheus-এ সংরক্ষিত হবে। তাই `path` লেবেল সবসময় প্যাটার্ন আকারে রাখুন, আর `relabeling` দিয়ে সন্দেহজনক লেবেল ফেলে দিন (বিস্তারিত অধ্যায় ৭-এ)। দ্বিতীয়ত, `/metrics` এন্ডপয়েন্ট: এটি খোলা রাখলে যে কেউ আপনার সিস্টেমের অভ্যন্তরীণ অবস্থা দেখতে পারে — অনুরোধের হার, ত্রুটির হার, সংস্করণের তথ্য। প্রোডাকশনে `/metrics` শুধু Prometheus সার্ভারের IP থেকে পৌঁছানো যায় এমন নেটওয়ার্কে রাখুন, অথবা বিপরীত প্রক্সির পেছনে প্রমাণীকরণ (authentication) বসান। তৃতীয়ত, `Histogram`-এর বাকেট থেকে আক্রমণকারী অনুমান করতে পারে আপনার সিস্টেমের আচরণ — যেমন পেমেন্ট প্রক্রিয়াকরণের সময়। এটি সাধারণত কম ঝুঁকির, তবে মেট্রিক এন্ডপয়েন্ট জনসাধারণের জন্য খোলা রাখা কখনোই ভালো অভ্যাস নয়। চতুর্থত, `Summary` ও `Histogram`-এ ব্যক্তিগত তথ্য — যেমন একজন ব্যবহারকারীর লেনদেনের পরিমাণ — সরাসরি মাপবেন না; পরিমাণের বণ্টন মাপলে তা সমষ্টিগত রাখুন, ব্যক্তি-নির্দিষ্ট নয়।

৬.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

প্রতিটি মেট্রিকের খরচ আছে — মেমরি, ডিস্ক, CPU ও নেটওয়ার্ক। খরচের মূল চালক `cardinality`: মোট সিরিজের সংখ্যা। একটি হিসাব করুন। ধরুন TechNova-র `http_requests_total`-এ লেবেল আছে তিনটি: `method` (৪টি মান: GET, POST, PUT, DELETE), `path` (২০টি রুট), `status` (৬টি মান)। সম্ভাব্য সিরিজ: `৪ × ২০ × ৬ = ৪৮০`। এটি

সামলানো যায়। এবার ভুলটি করুন — `user_id` লেবেল যোগ করুন, যেখানে ব্যবহারকারী ৫০,০০০ জন। সম্ভাব্য সিরিজ: `৪৮০ × ৫০,০০০ = ২৪,০০০,০০০` — দুই কোটি চল্লিশ লাখ সিরিজ! প্রতিটি সিরিজ মেমরিতে জায়গা নেয়, প্রতিটি `scrape`-এ নেটওয়ার্কে যায়, ডিস্কে লেখা হয়। Prometheus কয়েক লাখ সিরিজ সামলাতে পারে, কিন্তু দুই কোটি সিরিজে সার্ভার ধীর হয়ে যাবে, কুয়েরি মিনিটের পর মিনিট চলবে, ডিস্ক ভরে যাবে। এটাই সিরিজ-বিস্ফোরণ।

`Histogram`-এর খরচ আরও বেশি: প্রতি বাকেট একটি সিরিজ। ১১টি বাকেট ও ২০টি রুট মানে একটি `Histogram` থেকেই `১১ × ২০ = ২২০` সিরিজ, সঙ্গে `_sum` ও `_count`। তাই বাকেটের সংখ্যা ১০–১২-এর মধ্যে রাখুন, আর `Histogram`-এ লেবেল কম রাখুন। `scrape_interval` কমালেও খরচ বাড়ে — ১৫ সেকেন্ডের বদলে ৫ সেকেন্ড করলে ডেটা তিন গুণ হয়। আর `retention` (কতদিন ডেটা রাখা হবে) সরাসরি ডিস্কের খরচ নির্ধারণ করে। নিয়মটি মনে রাখুন: লেবেল যোগ করার আগে জিজ্ঞাসা করুন — "আমি কি সত্যিই এই ভাগে ডেটা দেখব?" উত্তর না হলে লেবেলটি বাদ দিন।

পরিস্থিতি	সিরিজের সংখ্যা (প্রায়)	মন্তব্য
Counter, ৩ লেবেল, সীমিত মান	৪৮০	নিরাপদ
Histogram, ১১ বাকেট, ২০ রুট	২৬০	গ্রহণযোগ্য
উপরের Counter + user_id (৫০,০০০)	২৪,০০০,০০০	বিস্ফোরণ — কখনো নয়
Histogram + user_id	২৬ কোটির বেশি	সার্ভার অচল হয়ে যাবে

৬.১৪ সমস্যা সমাধান (Troubleshooting)

সমস্যা: `rate()` শূন্য দেখাচ্ছে, অথচ অনুরোধ আসছে। কারণ হতে পারে `Counter`-টি আসলে বাড়ছে না — কোডে `inc()` কল হচ্ছে কি না দেখুন — অথবা `scrape` ব্যর্থ হচ্ছে। Prometheus-এর `Targets` পৃষ্ঠায় টার্গেটের অবস্থা দেখুন; `up` মেট্রিক শূন্য হলে `scrape` হচ্ছে না। আরেকটি কারণ: `rate()`-এর সময়-পরিসর খুব ছোট — `[1m]`-এর বদলে `[5m]` ব্যবহার করুন, যাতে অন্তত দুটি `scrape` পড়ে।

সমস্যা: `histogram_quantile()` অদ্ভুত মান দিচ্ছে বা ফাঁকা। প্রথমে দেখুন `_bucket` সিরিজগুলো আসছে কি না; `le` লেবেল ছাড়া `Histogram` কাজ করে না। দ্বিতীয়ত, `rate()` ছাড়া সরাসরি `_bucket`-এ `histogram_quantile()` চালালে ভুল হবে — সবসময় `histogram_quantile(0.95, sum by (le) (rate(..._bucket[5m])))` আকারে লিখুন। তৃতীয়ত, বাকেটের সীমা যদি আপনার লেটেন্সির চেয়ে অনেক বড় হয়, তাহলে সব পর্যবেক্ষণ প্রথম বাকেটেই পড়বে, আর `quantile` অনুমান ভুল হবে।

সমস্যা: মেট্রিক হঠাৎ উধাও। `Counter` ও `Histogram` তখনই প্রকাশিত হয় যখন অন্তত একবার মান রেকর্ড করা হয়েছে। কোনো এন্ডপয়েন্টে এখনো অনুরোধ না এলে তার সিরিজ থাকবে না — এটি স্বাভাবিক। আর প্রক্রিয়া পুনরায় চালু হলে `Counter` শূন্য থেকে শুরু হয়; `rate()` তা সামলে নেয়, কিন্তু সরাসরি মান দেখলে আতঙ্কিত হবেন না।

সমস্যা: সিরিজ সংখ্যা অস্বাভাবিকভাবে বাড়ছে। `prometheus_tsdb_head_series` মেট্রিকটি দেখুন। বাড়তে থাকলে `topk(10, count by (__name__)({__name__=~".+"}))` কুয়েরি দিয়ে দেখুন কোন মেট্রিক সবচেয়ে বেশি সিরিজ তৈরি করছে, তারপর তার লেবেলগুলো পরীক্ষা করুন — প্রায়ই দেখা যায় `path` লেবেলে আসল ID ঢুকে গেছে।

> **TROUBLESHOOTING:** `rate()` যদি `NaN` বা ফাঁকা দেয়, প্রথমে `increase()` দিয়ে পরীক্ষা করুন — সমস্যা ডেটায় না কুয়েরিতে, তা বোঝা যাবে। আর মনে রাখুন: `rate()` প্রথম দুটি নমুনার আগে কিছু দেয় না, তাই নতুন মেট্রিকে কয়েক মিনিট অপেক্ষা করুন।

৬.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: মেট্রিকের নাম ছোট অক্ষরে, আন্ডারস্কোর দিয়ে, এককসহ লিখুন (`http_request_duration_seconds`)। প্রতিটি মেট্রিকে অর্থবহ `HELP` লিখুন। `Counter`-এর নামে `_total` রাখুন। লেবেলের মান সীমিত রাখুন — `status` লেবেলে পুরো স্ট্যাটাস বার্তা নয়, শুধু কোড (`200`, `500`)। `Histogram`-এর বাকেট আপনার সিস্টেমের আসল লেটেন্সি অনুযায়ী বাছুন — প্রথমে পর্যবেক্ষণ করুন, তারপর বাকেট ঠিক করুন। `RED` পদ্ধতি অনুসরণ করুন: প্রতিটি সার্ভিসের জন্য `Rate` (অনুরোধের হার), `Errors` (ত্রুটির হার), `Duration` (লেটেন্সির বণ্টন) — এই তিনটি মেট্রিক থাকা উচিত। `USE` পদ্ধতিও মনে রাখুন: প্রতিটি সম্পদের জন্য `Utilization`, `Saturation`, `Errors`।

অ্যান্টি-প্যাটার্ন: প্রতিটি ফাংশন কলে মেট্রিক বসানো; লগের পরিবর্তে মেট্রিক দিয়ে ডিবাগ করার চেষ্টা করা; `user_id` বা `request_id` লেবেলে বসানো; একই জিনিস দুটি নামে মাপা; `Gauge`-এ টাইমস্ট্যাম্প বসানো (সময়ের জন্য `Counter` বা লগ ব্যবহার করুন); বাকেট ছাড়া `Histogram` সংজ্ঞায়িত করা। আর সবচেয়ে বড় অ্যান্টি-প্যাটার্ন: মেট্রিক সংগ্রহ করা কিন্তু কখনো দেখা না — মেট্রিক তখনই মূল্যবান যখন কেউ নিয়মিত তার দিকে তাকায় ও তার ভিত্তিতে অ্যালার্ট লেখে।

> **PRO TIP:** নতুন সার্ভিস শুরু করার সময় প্রথমে মাত্র চারটি মেট্রিক লিখুন — অনুরোধের `Counter`, ত্রুটির `Counter`, লেটেন্সির `Histogram` ও চলমান অনুরোধের `Gauge`। এই চারটি দিয়েই বেশিরভাগ ঘটনা ধরা যায়। পরে প্রয়োজনমতো যোগ করুন; শুরুতে বিশটি মেট্রিক লিখলে কোনোটিই ঠিকমতো দেখা হয় না।

৬.১৬ অধ্যায়ের সারাংশ

এই অধ্যায়ে আমরা শিখেছি যে Prometheus-এর চারটি মেট্রিক ধরন চারটি ভিন্ন প্রশ্নের উত্তর দেয়: `Counter` গোনে কতবার কিছু ঘটেছে, `Gauge` বলে এই মুহূর্তে মান কত, `Histogram` ও `Summary` মাপে মানের বণ্টন — বিশেষ করে লেটেন্সি। `Counter`-কে সবসময় `rate()` বা `increase()` দিয়ে পড়তে হয়, কারণ তার কাঁচা মান কেবল বাড়তেই থাকে; প্রক্রিয়া পুনরায় চালু হলে সে শূন্যে ফিরে যায়, আর Prometheus সেই `reset` সামলে নেয়। `Histogram`-এর ক্রমবর্ধমান বাকেট, `_sum` ও `_count` দিয়ে যেকোনো শতকরা হিসাব করা যায় এবং একাধিক সার্ভারের তথ্য একত্র করা যায় — তাই এটি লেটেন্সি মাপার পছন্দের পথ। `Summary` ক্লায়েন্টের ভেতরে শতকরা হিসাব করে, কিন্তু একত্র করা যায় না বলে তার ব্যবহার সীমিত। নামকরণের নিয়ম — `_total`, `_bucket`, `_sum`, `_count`, এককসহ নাম — মেট্রিককে বোধগম্য রাখে। সবচেয়ে গুরুত্বপূর্ণ শিক্ষা: লেবেলের `cardinality` নিয়ন্ত্রণে রাখুন; `user_id` বা `request_id`-এর মতো সীমাহীন মান লেবেলে বসালে সিরিজ-বিস্ফোরণ ঘটে, যা Prometheus সার্ভারকে অচল করে দিতে পারে। TechNova-র উদাহরণে দেখেছি, গড় লেটেন্সি বিভ্রান্ত করতে পারে, কিন্তু p95 ও p99 সত্য প্রকাশ করে।

৬.১৭ মূল প্রযুক্তিগত পরিভাষা

Counter: কেবল বাড়ে এমন মেট্রিক; মোট ঘটনার সংখ্যা গোনে, প্রক্রিয়া পুনরায় চালু হলে শূন্যে ফিরে যায়।

Gauge: বাড়তে ও কমতে পারে এমন বর্তমান মানের মেট্রিক; যেমন মেমরি ব্যবহার বা চলমান অনুরোধ।

Histogram: পর্যবেক্ষণের বণ্টন মাপার মেট্রিক; আগে থেকে ঠিক করা বাকেটে গণনা জমা রাখে।

Summary: ক্লায়েন্টের ভেতরে সরাসরি quantile হিসাব করে প্রকাশ করা মেট্রিক; একত্র করা যায় না।

Bucket: Histogram-এর একটি সীমা; le (less than or equal) মানের নিচে পড়া পর্যবেক্ষণের সংখ্যা গোনা হয়।

Quantile: বণ্টনের একটি শতকরা বিন্দু; p95 মানে ৯৫% পর্যবেক্ষণ এই মানের নিচে।

Cardinality: একটি মেট্রিকের মোট time series-এর সংখ্যা; লেবেলের মানের সমন্বয়ে নির্ধারিত হয়।

Time Series: একটি মেট্রিক নাম ও লেবেল সমন্বয়ের সময়ের সঙ্গে বদলাতে থাকা মানের ধারা।

Label: মেট্রিকের সঙ্গে যুক্ত কী-মান জোড়া, যা দিয়ে সিরিজকে ভাগ করে দেখা যায়।

Exposition Format: /metrics এন্ডপয়েন্টে Prometheus-এর পড়ার উপযোগী লেখ্য রূপ; HELP, TYPE ও নমুনা লাইন নিয়ে গঠিত।

Counter Reset: প্রক্রিয়া পুনরায় চালু হলে Counter শূন্যে ফিরে যাওয়া; Prometheus rate() গণনায় এটি সামলে নেয়।

Scrape: Prometheus-এর HTTP GET করে টার্গেট থেকে মেট্রিক সংগ্রহ করার কাজ।

৬.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Counter ও Gauge-এর মূল পার্থক্য কী? মোট অনুরোধ গুনতে কোনটি ব্যবহার করবেন, আর কেন?
2. Counter-কে সরাসরি গ্রাফ করলে কী সমস্যা হয়? এর বদলে কোন ফাংশন ব্যবহার করা উচিত?
3. Histogram-এর বাকেট কেন ক্রমবর্ধমান (cumulative) হয়? le="+Inf" বাকেটের মান _count-এর সমান হওয়া উচিত কেন?
4. গড় লেটেন্সি ৪৪ মিলিসেকেন্ড হলেও p99 ৮ সেকেন্ড হতে পারে কীভাবে? গড় কেন বিভ্রান্তিকর হতে পারে?
5. Summary-এর quantile একাধিক সার্ভারে একত্র করা যায় না কেন? Histogram-এ এই সমস্যা নেই কেন?
6. একটি Counter-এ method (৪টি), path (২০টি), status (৬টি) ও user_id (৫০,০০০ জন) লেবেল থাকলে সম্ভাব্য সিরিজ কত? এই হিসাব থেকে কী শিক্ষা মেলে?
7. মেট্রিকের নামে একক (যেমন _seconds) রাখা কেন গুরুত্বপূর্ণ? ভিত্তি একক ব্যবহারের সুবিধা কী?
8. path লেবেলে /api/orders/12345-এর মতো আসল ID বসালে কী ক্ষতি হয়? এর সঠিক বিকল্প কী?
9. Gauge দিয়ে লেটেন্সি মাপলে কী তথ্য হারিয়ে যায়? লেটেন্সি মাপার সঠিক ধরন কোনটি?
10. Histogram-এর বাকেট ভুল বাছলে histogram_quantile()-এর ফল কীভাবে ভুল হতে পারে?

৬.১৯ ব্যবহারিক অনুশীলন

1. একটি ছোট Express অ্যাপ তৈরি করুন যাতে prom-client দিয়ে http_requests_total (Counter), http_requests_in_flight (Gauge) ও http_request_duration_seconds (Histogram) সংজ্ঞায়িত আছে।
2. /metrics এন্ডপয়েন্ট যোগ করুন এবং ব্রাউজারে খুলে exposition format পড়ুন; প্রতিটি HELP ও TYPE লাইন নিজের ভাষায় ব্যাখ্যা লিখুন।
3. একটি স্ক্রিপ্ট দিয়ে ২০০টি অনুরোধ পাঠান, যার ১০% ইচ্ছা করে ২ সেকেন্ড দেরি করে; তারপর _sum / _count দিয়ে গড় ও বাকেট থেকে ৯০% অনুরোধ কত সময়ের মধ্যে শেষ হয়েছে তা হাতে হিসাব করুন।
4. ইচ্ছা করে path লেবেলে আসল ID বসিয়ে দেখুন সিরিজ সংখ্যা কীভাবে বাড়ে; তারপর রুটের প্যাটার্ন ব্যবহার করে সমস্যাটি ঠিক করুন।
5. একটি Summary মেট্রিকও যোগ করুন; দুটি সার্ভারের quantile গড় করে দেখান কেন সেটি প্রকৃত quantile নয়, এবং Histogram দিয়ে সঠিক উত্তর বের করুন।
6. আপনার মেট্রিক পরিকল্পনাটি একটি নথিতে লিখুন: প্রতিটি মেট্রিকের নাম, ধরন, লেবেল, HELP লেখা ও কেন এই ধরনটি বেছে নেওয়া হয়েছে।

অধ্যায় ৭ • PART II — Metrics and Prometheus Engineering

Chapter 7 — Prometheus Architecture and Data Collection

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Prometheus সার্ভারের ভেতরের গঠন ব্যাখ্যা করতে পারবেন — `retrieval` (scrape), `TSDB`, `WAL`, `PromQL` ইঞ্জিন, `rule evaluation` ও `service discovery` কীভাবে একসঙ্গে কাজ করে। আপনি `prometheus.yml` ফাইলের প্রতিটি ক্ষেত্রের অর্থ বুঝবেন, `scrape_interval`, `retention` ও স্টোরেজের হিসাব করতে পারবেন, `relabeling` দিয়ে টার্গেট ও লেবেল নিয়ন্ত্রণ করতে পারবেন এবং `remote write/read`, `federation` ও উচ্চ-উপলভ্যতা (high availability) ধারণার সীমা ব্যাখ্যা করতে পারবেন।

- Prometheus-এর pull মডেল ও তার সুবিধা-অসুবিধা ব্যাখ্যা করা।
- TSDB ও WAL-এর ভূমিকা এবং ডেটা লেখার পথ বোঝা।
- prometheus.yml ক্ষেত্র ধরে ধরে পড়া ও লেখা।
- Service discovery ও relabeling প্রয়োগ করা।
- Retention, স্টোরেজ হিসাব ও HA ধারণা মূল্যায়ন করা।

৭.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Prometheus ব্যবহার করা সহজ — একটি বাইনারি চালু করলেই চলে। কিন্তু প্রোডাকশনে সেটি ভেঙে পড়ে তখনই যখন কেউ তার ভেতরের গঠন বোঝে না। TechNova-র প্রথম Prometheus ইনস্টলেশনে ডিস্ক এক সপ্তাহে ভরে গিয়েছিল, কারণ কেউ `retention` নির্ধারণ করেনি আর একটি ভুল লেবেল লাখ লাখ সিরিজ তৈরি করছিল। আরেকবার একটি সার্ভিসের মেট্রিক হঠাৎ উধাও হয়েছিল — কারণ টার্গেটের IP বদলে গিয়েছিল, আর `static_configs`-এ পুরোনো IP লেখা ছিল। `Service discovery` জানা থাকলে এই সমস্যা হতো না। আর্কিটেকচার বোঝা মানে এই ধরনের ব্যর্থতা আগেই অনুমান করা। Prometheus কোনো যাদুর বাক্স নয় — এটি একটি ডেটাবেস, একটি স্ক্র্যাপার ও একটি কুয়েরি ইঞ্জিনের সমন্বয়, যার প্রতিটি অংশের নিজস্ব সীমা আছে।

৭.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর চারটি মেট্রিক ধরন, `time series` ও `label`-এর ধারণা এবং `exposition format` জানা থাকতে হবে। এ ছাড়া প্রয়োজন: Linux প্রক্রিয়া ও ফাইল সিস্টেমের সাধারণ ধারণা, YAML-এর মৌলিক গঠন (ইনডেন্টেশন, কী-মান, তালিকা), HTTP-র ধারণা এবং Docker Compose-এ একটি সার্ভিস চালানোর অভিজ্ঞতা। Kubernetes-এর ধারণা থাকলে `service discovery` অংশটি সহজ হবে, তবে আবশ্যক নয় — সেটি অধ্যায় ১১-এ বিস্তারিত আসবে।

৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

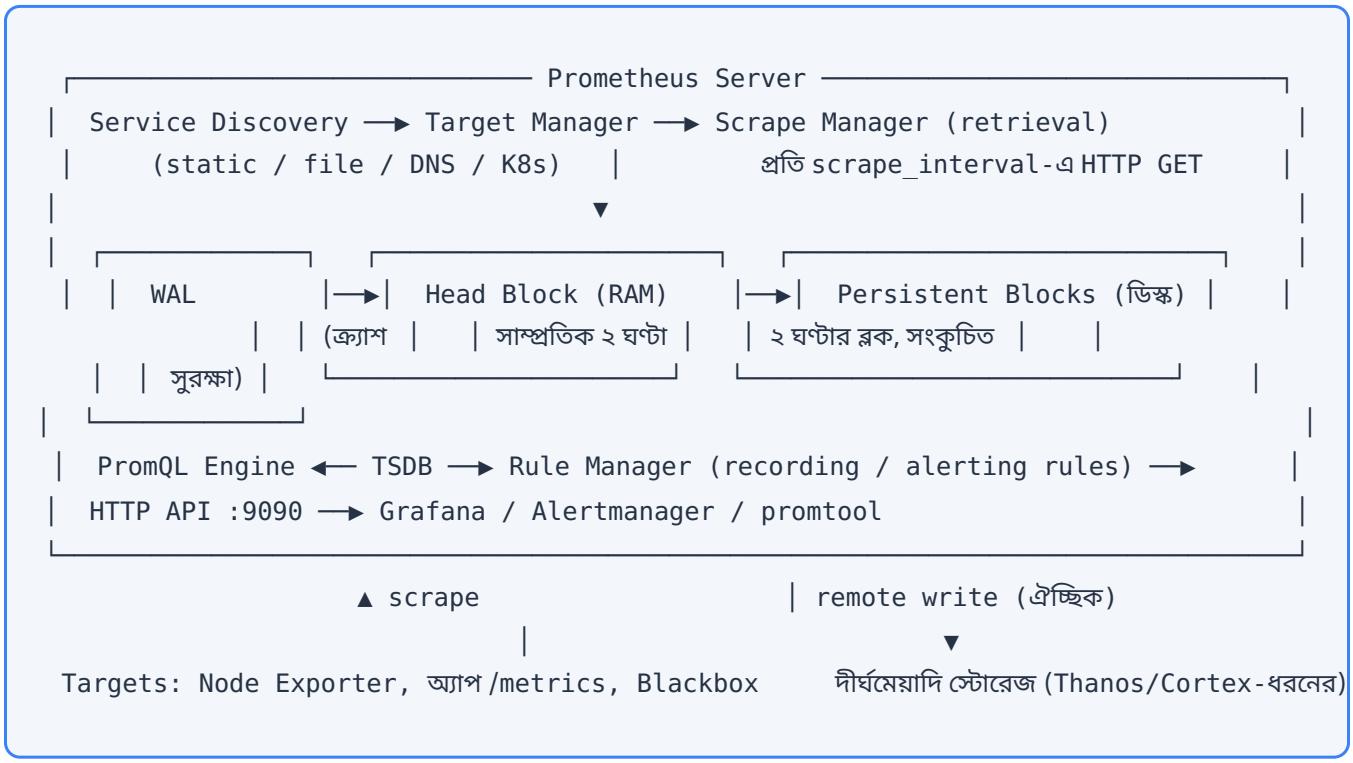
প্রথম নীতি থেকে ভাবুন: একটি মনিটরিং সিস্টেমের কাজ তিনটি — সংগ্রহ করা, সংরক্ষণ করা, জিজ্ঞাসা করা। Prometheus এই তিনটি কাজকে আলাদা অংশে ভাগ করে। সংগ্রহের জন্য `retrieval` — এটি নিয়মিত বিরতিতে টার্গেটগুলোতে HTTP অনুরোধ পাঠায়। সংরক্ষণের জন্য `TSDB` — সময়-ভিত্তিক ডেটাবেস, যা সংখ্যাকে সময়ের ক্রমে সাজিয়ে রাখে। জিজ্ঞাসার জন্য `PromQL` ইঞ্জিন — যা TSDB থেকে ডেটা তুলে গণনা করে। এই ভাগের কারণে প্রতিটি অংশ আলাদাভাবে বোঝা ও

সমস্যা খুঁজে বের করা যায়: ডেটা না এলে সমস্যা সংগ্রহে, ডিস্ক ভরে গেলে সমস্যা সংরক্ষণে, কুয়েরি ধীর হলে সমস্যা জিজ্ঞাসায়।

কেন `pull` মডেল, `push` নয়? কারণ `pull`-এ কেন্দ্রীয় সার্ভার নিয়ন্ত্রণে থাকে — সে ঠিক করে কখন সংগ্রহ করবে, কোন টার্গেট থেকে করবে, ব্যর্থ হলে কী করবে। টার্গেট বন্ধ থাকলে Prometheus তা জানতে পারে (`up == 0`), আর টার্গেটের দিকে তাকিয়ে তার স্বাস্থ্য বোঝা যায়। `Push` মডেলে টার্গেট নিজে ডেটা পাঠায়; টার্গেট বন্ধ থাকলে কেন্দ্রীয় সার্ভার বুঝতে পারে না সে বন্ধ, না কেবল নীরব। তবে স্বল্পস্থায়ী কাজের (যেমন ব্যাচ জব) জন্য `push`-ও দরকার হয় — সেজন্য আছে `Pushgateway`, অধ্যায় ৯-এ তার সঠিক ব্যবহার আছে।

> **CONCEPT:** Prometheus একটি সময়-ভিত্তিক ডেটাবেস, সম্পর্কভিত্তিক (relational) ডেটাবেস নয়। এখানে কোনো টেবিল বা JOIN নেই; আছে সিরিজ ও নমুনা। একটি নমুনা হলো (সময়, মান) জোড়া। TSDB এই নমুনাগুলোকে চাপ দিয়ে (compress) সংরক্ষণ করে, যাতে কোটি কোটি নমুনা অল্প ডিস্কে ধরে। এই নকশাই Prometheus-কে দ্রুত করে, আবার এর সীমাও নির্ধারণ করে — উচ্চ cardinality-র সিরিজ এই চাপ-ব্যবস্থার জন্য সবচেয়ে ব্যয়বহুল।

৭.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



ডেটার পথটি লক্ষ্য করুন। `Scrape` থেকে আসা নমুনা প্রথমে `WAL` (Write-Ahead Log)-এ লেখা হয় — ডিস্কে একটি লগ, যাতে হঠাৎ বিদ্যুৎ চলে গেলেও সাম্প্রতিক ডেটা হারিয়ে না যায়। তারপর নমুনা যায় `head block`-এ, যা মেমরিতে থাকে এবং সাম্প্রতিক প্রায় দুই ঘণ্টার ডেটা ধারণ করে। দুই ঘণ্টা পর `head block` ডিস্কে একটি স্থায়ী ব্লক হিসেবে লেখা হয় ও সংকুচিত হয়, আর পুরোনো ব্লকগুলো নিয়মিত `compaction`-এর মাধ্যমে একত্র করা হয়। `PromQL` কুয়েরি `head block` ও ডিস্কের ব্লক — দুই জায়গা থেকেই ডেটা পড়ে। `Rule Manager` নির্দিষ্ট বিরতিতে `recording rule` ও `alerting rule` মূল্যায়ন করে; ফলাফল আবার TSDB-তে লেখা হয় অথবা Alertmanager-এ পাঠানো হয়।

৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Scrape লুপ ও টার্গেটের জীবনচক্র

প্রতিটি `scrape job`-এর একটি `scrape_interval` (সাধারণত ১৫ সেকেন্ড) থাকে। `Scrape Manager` প্রতিটি টার্গেটে HTTP GET পাঠায়, `exposition format` পড়ে, নমুনাগুলোকে TSDB-তে লেখে। প্রতিটি `scrape`-এর জন্য Prometheus স্বয়ংক্রিয়ভাবে তিনটি মেট্রিক তৈরি করে: `up` (১ মানে সফল, ০ মানে ব্যর্থ), `scrape_duration_seconds` (কত সময় লেগেছে) ও `scrape_samples_scraped` (কতগুলো নমুনা পাওয়া গেছে)। এই তিনটি মেট্রিকই Prometheus-এর নিজের স্বাস্থ্য পরীক্ষার প্রধান যন্ত্র। `scrape_timeout` সাধারণত `scrape_interval`-এর চেয়ে কম রাখা হয় — যেমন ১৫ সেকেন্ড বিরতিতে ১০ সেকেন্ড সময়সীমা — যাতে একটি ধীর টার্গেট পরের `scrape`-কে আটকে না রাখে।

TSDB, WAL ও ব্লক

TSDB-র প্রতিটি ব্লক দুই ঘণ্টার ডেটা ধারণ করে। ব্লকে দুটি প্রধান অংশ থাকে: `chunks` (সংকুচিত নমুনা) ও `index` (কোন সিরিজ কোথায় আছে তার সূচি)। এই নকশার কারণে পুরোনো ডেটা পড়া দ্রুত, কিন্তু নতুন সিরিজ তৈরি করা ব্যয়বহুল — কারণ প্রতিটি নতুন সিরিজের জন্য সূচিতে নতুন প্রবেশ লাগে। তাই উচ্চ `cardinality` মেমরি ও CPU — দুটোই খায়। `WAL` ডিস্কে থাকে; Prometheus পুনরায় চালু হলে সে WAL পড়ে `head block` পুনর্গঠন করে, ফলে সাম্প্রতিক ডেটা হারায় না। তবে WAL পুনরুদ্ধারে সময় লাগে — WAL বড় হলে চালু হতে মিনিটখানেক লাগতে পারে।

Service Discovery

সার্ভারের IP বদলায়, কনটেইনার ওঠে-নামে, Kubernetes Pod তৈরি-ধ্বংস হয়। প্রতিবার হাতে `prometheus.yml` বদলানো অসম্ভব। `Service discovery` এই সমস্যার সমাধান: Prometheus একটি উৎস (যেমন Kubernetes API, Consul, DNS, ফাইল) থেকে টার্গেটের তালিকা স্বয়ংক্রিয়ভাবে পায়। প্রতিটি আবিষ্কৃত টার্গেটের সঙ্গে কিছু `meta label` আসে — যেমন Kubernetes-এ `__meta_kubernetes_pod_name`। `Relabeling` দিয়ে এই `meta label`-গুলো থেকে আসল লেবেল তৈরি করা হয়, অপ্রয়োজনীয় টার্গেট বাদ দেওয়া হয়, আর `__address__` ঠিক করা হয়। নিয়মটি মনে রাখুন: `__` দিয়ে শুরু হওয়া লেবেল অভ্যন্তরীণ; `scrape`-এর আগে সেগুলো ফেলে দেওয়া হয়, যদি না `relabeling` দিয়ে সেগুলোকে সাধারণ লেবেলে রূপান্তর করা হয়।

Retention ও Remote Write/Read

Prometheus নিজে স্বল্প-মেয়াদি স্টোরেজের জন্য নকশা করা — সাধারণত ১৫ দিনের `retention`। দীর্ঘমেয়াদি সংরক্ষণের জন্য `remote write` দিয়ে নমুনা বাইরের স্টোরেজে পাঠানো হয়, আর `remote read` দিয়ে সেখান থেকে কুয়েরি করা যায়। Thanos বা Cortex-ধরনের ব্যবস্থা এই ধারণার ওপর গড়ে উঠেছে — একাধিক Prometheus-এর ডেটা একত্র করে দীর্ঘদিন রাখা ও এক জায়গা থেকে কুয়েরি করা। উচ্চ-উপলভ্যতার জন্য সাধারণ পদ্ধতি: একই কনফিগারেশন দিয়ে দুটি অভিন্ন Prometheus চালানো — দুজনেই একই টার্গেট `scrape` করে; একজন বন্ধ থাকলে অন্যজন ডেটা দেয়। এটি সত্যিকারের ক্লাস্টার নয় — দুজনের মধ্যে ডেটা সমন্বয় হয় না — তবে বেশিরভাগ দলের জন্য যথেষ্ট। `Federation` হলো আরেকটি পুরোনো পদ্ধতি: একটি কেন্দ্রীয় Prometheus অন্য Prometheus-দের কাছ থেকে নির্বাচিত মেট্রিক টেনে নেয়। এটি এখন কম ব্যবহৃত হয়, কারণ `remote write` আরও নমনীয়।

উপাদান	কাজ	ব্যর্থ হলে কী হয়
Scrape Manager	টার্গেট থেকে মেট্রিক সংগ্রহ	up == 0, ডেটায় ফাঁক
WAL	ক্র্যাশ-সুরক্ষা লগ	পুনরায় চালু হলে সাম্প্রতিক ডেটা হারাতে পারে
Head Block	সাম্প্রতিক ২ ঘণ্টার ডেটা (RAM)	মেমরি চাপ বাড়ে

TSDB ব্লক	সংকুচিত স্থায়ী সংরক্ষণ	ডিস্ক ভরে গেলে লেখা বন্ধ
Rule Manager	Recording ও alerting rule মূল্যায়ন	অ্যালার্ট দেরিতে বা না আসা
PromQL Engine	কুয়েরি গণনা	ভারী কুয়েরিতে CPU ও মেমরি চাপ

৭.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
global:                                  # সব job-এর জন্য প্রযোজ্য ডিফল্ট
  scrape_interval: 15s                   # ডিফল্ট সংগ্রহ বিরতি
  scrape_timeout: 10s                    # একক scrape-এর সময়সীমা
  evaluation_interval: 15s               # rule মূল্যায়নের বিরতি
  external_labels:                       # remote write / federation-এ পরিচয়
    cluster: 'technova-prod'
    replica: 'prom-01'

rule_files:                              # rule ফাইলের তালিকা
  - 'rules/*.yml'

alerting:                                # Alertmanager-এর ঠিকানা
  alertmanagers:
    - static_configs:
        - targets: ['203.0.113.20:9093']

scrape_configs:
  - job_name: 'prometheus'               # Prometheus নিজেকে scrape করে
    static_configs:
      - targets: ['localhost:9090']
  - job_name: 'technova-api'
    scrape_interval: 15s
    metrics_path: /metrics               # ডিফল্ট পথ; বদলাতে চাইলে এখানে
    static_configs:
      - targets: ['203.0.113.10:3000']
        labels: { env: 'prod' }          # এই job-এর সব সিরিজে env লেবেল যোগ হবে
  - job_name: 'node'
    file_sd_configs:                     # ফাইল-ভিত্তিক service discovery
      - files: ['/etc/prometheus/targets/nodes.json']
        refresh_interval: 30s
```

`global` অংশের মানগুলো প্রতিটি `job` উত্তরাধিকার পায়; কোনো `job`-এ আলাদা মান দিলে সেটি প্রাধান্য পায়। `external_labels` খুব গুরুত্বপূর্ণ — দুটি অভিন্ন Prometheus চালালে এই লেবেল দিয়ে তাদের আলাদা করা যায়, নইলে `remote write`-এ ডেটা মিশে যাবে। `evaluation_interval` নির্ধারণ করে `alerting rule` কত ঘন ঘন পরীক্ষা করা হবে; ১৫ সেকেন্ড সাধারণ মান। `file_sd_configs` একটি JSON ফাইল থেকে টার্গেট পড়ে — Ansible বা অন্য টুল দিয়ে সেই ফাইলটি আপডেট করা যায়, Prometheus-কে পুনরায় চালু না করেই। কনফিগারেশন পরীক্ষা করতে `promtool check config prometheus.yml` চালান; ভুল YAML-এ Prometheus চালুই হবে না, তাই চালুর আগে পরীক্ষা করা অভ্যাস করুন।

```
promtool check config /etc/prometheus/prometheus.yml   # কনফিগারেশন যাচাই
# (নমুনা আউটপুট — চিত্রণমূলক)
# Checking /etc/prometheus/prometheus.yml
#   SUCCESS: 2 rules files found, 12 rules loaded
```

```
sudo systemctl reload prometheus   # কনফিগারেশন পুনরায় লোড (SIGHUP-এর মতো)
curl -s http://203.0.113.10:9090/-/healthy   # স্বাস্থ্য পরীক্ষা
```

৭.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র প্রোডাকশন `prometheus.yml` — API, Node Exporter, Blackbox ও নিজের স্বাস্থ্যসহ, সঙ্গে একটি `relabeling` উদাহরণ:

```
global:
  scrape_interval: 15s
  scrape_timeout: 10s
  evaluation_interval: 15s
  external_labels: { cluster: 'technova-prod', replica: 'prom-01' }
rule_files: ['rules/recording.yml', 'rules/alerts.yml']
alerting:
  alertmanagers:
    - static_configs:
        - targets: ['203.0.113.20:9093']
scrape_configs:
  - job_name: 'prometheus'
    static_configs: [{ targets: ['localhost:9090'] }]
  - job_name: 'technova-api'
    scrape_interval: 15s
    static_configs:
      - targets: ['203.0.113.10:3000', '203.0.113.11:3000']
        labels: { env: 'prod', team: 'platform' }
  - job_name: 'node'
    static_configs:
      - targets: ['203.0.113.10:9100', '203.0.113.11:9100']
  - job_name: 'blackbox-http'
    metrics_path: /probe
    params: { module: ['http_2xx'] }
    static_configs:
      - targets: ['https://technova.example/api/health']
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
      - target_label: __address__
        replacement: '203.0.113.30:9115'   # Blackbox Exporter-এর ঠিকানা
storage:
  tsdb:
    retention.time: 15d
    retention.size: 50GB
```

Blackbox অংশটি মনোযোগ দিয়ে পড়ুন: `relabeling` এখানে তিনটি কাজ করে — প্রথমে আসল URL-টি `__param_target` প্যারামিটারে বসায়, তারপর সেটিকে `instance` লেবেল করে, তারপর `__address__`-কে Blackbox Exporter-এর ঠিকানায় বদলে দেয়। ফলে Prometheus Blackbox-কে জিজ্ঞাসা করে, "এই URL-টি পরীক্ষা করো", আর ফলাফলে `instance` হিসেবে আসল URL-টিই থাকে। `retention.size` ও `retention.time` — দুটির মধ্যে যেটি আগে পূর্ণ হয়, সেটি কার্যকর হয়; দুটি একসঙ্গে দিলে ডিস্ক ভরে যাওয়ার ঝুঁকি কমে।

৭.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র একটি রাতে Prometheus-এর ডিস্ক ৯০% পূর্ণ হয়ে গেল। কারণ অনুসন্ধানে দেখা গেল, একটি নতুন ডিপ্লয়ে `path` লেবেলে ব্যবহারকারীর ID ঢুকিয়ে দিয়েছিল — সিরিজ সংখ্যা এক লাফে ৪০,০০০ থেকে ৯ লাখে উঠেছিল। দলটি প্রথমে ভুল লেবেলটি সরিয়ে কোড ঠিক করল, তারপর `retention.size` কমিয়ে পুরোনো ব্লক মুছে ফেলার ব্যবস্থা করল, আর Prometheus-এর নিজের মেট্রিক `prometheus_tsdb_head_series`-এর ওপর অ্যালার্ট বসাল যাতে ভবিষ্যতে সিরিজ বাড়লেই খবর পাওয়া যায়। এই ঘটনার শিক্ষা: Prometheus-এর নিজের স্বাস্থ্য পর্যবেক্ষণ করা প্রোডাকশনের প্রথম কাজ — যে যন্ত্র দিয়ে সবকিছু দেখা হয়, তার নিজের দৃষ্টি যেন অন্ধ না হয়। আরেকটি পরিস্থিতি: একটি সার্ভারের IP বদলে যাওয়ার পর `static_configs`-এর টার্গেটটি মৃত হয়ে গেল; `up` মেট্রিক শূন্য দেখে দলটি বুঝল সমস্যা অ্যাপে নয়, কনফিগারেশনে। পরে তারা `file_sd_configs`-এ চলে গেল, যাতে Ansible টার্গেট ফাইলটি স্বয়ংক্রিয়ভাবে আপডেট করে।

৭.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Docker Compose-এ Prometheus 3.x চালু করুন: `prometheus` ইমেজ, পোর্ট ৯০৯০, ভলিউমে `prometheus.yml` ও ডেটা ডিরেক্টরি মাউন্ট করুন।
2. `promtool check config` দিয়ে কনফিগারেশন যাচাই করুন, তারপর `http://203.0.113.10:9090/-/healthy` খুলে দেখুন।
3. `Targets` পৃষ্ঠায় প্রতিটি টার্গেটের অবস্থা দেখুন; `up` কুয়েরি করে ১ পাচ্ছেন কি না যাচাই করুন।
4. একটি টার্গেট ইচ্ছা করে বন্ধ করুন; `up` কীভাবে ০ হয় ও `scrape_duration_seconds` কী দেখায় তা লক্ষ্য করুন।
5. `file_sd_configs` ব্যবহার করুন: একটি JSON ফাইলে টার্গেট লিখুন, ফাইলটি বদলে দেখুন Prometheus পুনরায় চালু না করেই নতুন টার্গেট তুলে নেয় কি না।
6. `retention.time: 15d` ও `retention.size: 50GB` সেট করুন; ডিস্ক ব্যবহার `prometheus_tsdb_storage_blocks_bytes` দিয়ে পর্যবেক্ষণ করুন।
7. দুটি অভিন্ন Prometheus চালিয়ে দেখুন `external_labels`-এর `replica` লেবেল কীভাবে তাদের আলাদা করে।

৭.১১ সাধারণ ভুল ও ভুল ধারণা

- **Prometheus-কে দীর্ঘমেয়াদি স্টোরেজ ভাবা:** এটি স্বল্প-মেয়াদির জন্য; বছরখানেকের ডেটা চাইলে `remote write` ব্যবহার করুন।
- **scrape_interval খুব ছোট করা:** ১ সেকেন্ড বিরতি মানে ১৫ গুণ বেশি ডেটা ও লোড; ১৫ সেকেন্ডই সাধারণ মান।
- **retention না সেট করা:** ডিফল্টে ডিস্ক ভরে যাওয়া পর্যন্ত ডেটা জমে; সবসময় সময় ও আকার — দুটো সীমাই দিন।
- **static_configs-এ হাতে IP লেখা ও ভুলে যাওয়া:** সার্ভার বাড়লে service discovery ব্যবহার করুন।
- **relabeling-কে জটিল ভাবা:** এটি কেবল লেবেল বদলের নিয়ম; একবারে একটি নিয়ম লিখে পরীক্ষা করুন।
- **দুটি Prometheus-কে ক্লাস্টার ভাবা:** তারা একে অপরের ডেটা জানে না; HA মানে দুটি স্বাধীন কপি।
- **promtool ছাড়া কনফিগারেশন চালু করা:** একটি ভুল ইনডেন্টেশন Prometheus-কে চালু হতে দেবে না।

৭.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Prometheus-এর ওয়েব UI-তে প্রমাণীকরণ (authentication) ডিফল্টে থাকে না — যে কেউ পোর্ট ৯০৯০-এ পৌঁছাতে পারলে সব মেট্রিক ও কুয়েরি দেখতে পারে। প্রোডাকশনে Prometheus-কে ব্যক্তিগত নেটওয়ার্কে রাখুন, ফায়ারওয়াল দিয়ে পোর্ট সীমিত করুন, আর প্রয়োজনে বিপরীত প্রক্সির (যেমন Nginx) পেছনে প্রমাণীকরণ বসান। `/metrics` এন্ডপয়েন্টেও

একই কথা প্রযোজ্য। `remote write`-এর ক্ষেত্রে প্রাপকের ঠিকানায় প্রমাণীকরণ (বেয়ারার টোকেন বা TLS) ব্যবহার করুন — টোকেনটি কনফিগারেশন ফাইলে `CHANGE-ME-...` আকারে প্লেসহোল্ডার হিসেবে রাখুন, আসল মান পরিবেশ-ভিত্তিক গোপন ব্যবস্থায় (secret management) রাখুন। `relabeling` দিয়ে সংবেদনশীল লেবেল — যেমন ইমেইল বা টোকেন — `scrape`-এর সময়েই ফেলে দিন, যাতে সেগুলো কখনো TSDB-তে না পৌঁছায়। একবার TSDB-তে ঢুকে গেলে লেবেল মুছে ফেলা কঠিন; প্রতিরোধই একমাত্র নির্ভরযোগ্য পথ।

> **WARNING:** Prometheus-এর ডেটা ডিরেক্টরি মুছে ফেললে সব ইতিহাস হারিয়ে যাবে। retention বা ডিস্ক পরিষ্কারের আগে ব্যাকআপের কথা ভাবুন, আর প্রোডাকশন Prometheus-এ কখনো হাতে ব্লক মুছবেন না — retention সেটিংসকেই কাজটি করতে দিন।

৭.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Prometheus-এর খরচের তিনটি চালক: সিরিজের সংখ্যা, `scrape_interval` ও `retention`। একটি মোটামুটি হিসাব: প্রতি সিরিজ প্রতি `scrape`-এ একটি নমুনা; ১ লাখ সিরিজ ও ১৫ সেকেন্ড বিরতিতে প্রতি সেকেন্ডে প্রায় ৬,৭০০ নমুনা লেখা হয়। সংকুচিত অবস্থায় প্রতিটি নমুনা গড়ে ১–২ বাইট জায়গা নেয়। তাই ১ লাখ সিরিজ, ১৫ দিনের `retention`-এ ডিস্ক লাগে প্রায় ১৫–৩০ গিগাবাইট। সিরিজ ১০ লাখ হলে ডিস্ক ও মেমরি — দুটোই দশ গুণ। মেমরির নিয়ম: `head block`-এর প্রতিটি সিরিজ প্রায় ১–২ কিলোবাইট মেমরি নেয়; ১০ লাখ সিরিজ মানে ১–২ গিগাবাইট শুধু মেমরিতে। ভারী `PromQL` কুয়েরিও খরচ বাড়ায় — দীর্ঘ সময়-পরিসরে অনেক সিরিজের ওপর `rate()` চালালে CPU ও মেমরি চাপ বাড়ে; সেজন্য ঘন ঘন ব্যবহৃত ভারী কুয়েরিকে `recording rule` হিসেবে আগে থেকে হিসাব করে রাখা হয় (অধ্যায় ৮ ও ১০-এ বিস্তারিত)। `scrape` ব্যর্থতা ও পুনঃচেষ্টাও খরচ বাড়ায় — ধীর টার্গেটের `scrape_timeout` কম রাখুন।

৭.১৪ সমস্যা সমাধান (Troubleshooting)

টার্গেট Down দেখাচ্ছে: প্রথমে `Targets` পৃষ্ঠায় ত্রুটির বার্তা পড়ুন — সংযোগ প্রত্যাখ্যাত হলে টার্গেট চলছে না বা ফায়ারওয়াল আটকাচ্ছে; সময়সীমা পেরিয়ে গেলে টার্গেট ধীর বা `scrape_timeout` খুব কম। টার্গেট সার্ভার থেকে `curl` দিয়ে `/metrics` পরীক্ষা করুন।

ডিস্ক ভরে যাচ্ছে: `prometheus_tsdb_storage_blocks_bytes` ও `prometheus_tsdb_head_series` দেখুন। সিরিজ অস্বাভাবিক হলে অধ্যায় ৬-এর `cardinality` পরীক্ষা করুন; `retention.size` কমান; পুরোনো ব্লক মুছে ফেলার জন্য Prometheus পুনরায় চালু করে `--storage.tsdb.retention.size` কার্যকর করুন।

Prometheus চালু হতে দেরি: WAL বড় হলে পুনরুদ্ধারে সময় লাগে; লগে `WAL replay` দেখুন। ঘন ঘন পুনরায় চালু এড়িয়ে চলুন।

কুয়েরি ধীর: সময়-পরিসর কমান, লেবেল দিয়ে সিরিজ সীমিত করুন, ভারী কুয়েরিকে `recording rule`-এ রূপান্তর করুন।

Rule লোড হচ্ছে না: `promtool check rules` চালান; YAML ইনডেন্টেশন ও `expr`-এর বাক্যগঠন পরীক্ষা করুন।

> **TROUBLESHOOTING:** Prometheus-এর লগই প্রথম সাক্ষী। `journalctl -u prometheus -f` চালিয়ে দেখুন — "out of bounds" বা "duplicate series" বার্তা থাকলে প্রায়ই দুটি `scrape job` একই টার্গেটকে একই লেবেলে সংগ্রহ করছে।

৭.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি পরিবর্তনের আগে `promtool check config` চালান; কনফিগারেশন Git-এ রাখুন; Prometheus-এর নিজের মেট্রিক (`up`, `prometheus_tsdb_head_series`, ডিস্ক ব্যবহার) পর্যবেক্ষণ করুন; `retention`-এ সময় ও আকার — দুটো সীমাই দিন; `external_labels`-এ `cluster` ও `replica` রাখুন; `scrape_interval` ১৫ সেকেন্ড থেকে শুরু করুন; ভারী কুয়েরির জন্য `recording rule` ব্যবহার করুন।

অ্যান্টি-প্যাটার্ন: Prometheus-কে লগ স্টোরেজ হিসেবে ব্যবহার করা; প্রতিটি অ্যাপে আলাদা Prometheus চালিয়ে কেন্দ্রীয় দৃষ্টি হারানো; `scrape_interval` ১ সেকেন্ড করে ডিস্ক ভরিয়ে ফেলা; `relabeling` ছাড়াই উচ্চ-কার্ডিনালিটি লেবেল ঢুকতে দেওয়া; HA না ভেবে একটিমাত্র Prometheus-এর ওপর নির্ভর করা।

৭.১৬ অধ্যায়ের সারাংশ

Prometheus তিনটি অংশের সমন্বয়: `retrieval` টার্গেট থেকে মেট্রিক টেনে আনে, `TSDB` তা WAL-এর সুরক্ষায় `head block` ও সংকুচিত ব্লকে সংরক্ষণ করে, আর `PromQL` ইঞ্জিন তা থেকে প্রশ্নের উত্তর দেয়। `Pull` মডেলের কারণে টার্গেটের স্বাস্থ্য সরাসরি জানা যায়, আর `service discovery` ও `relabeling` গতিশীল পরিবেশে টার্গেট ব্যবস্থাপনাকে স্বয়ংক্রিয় করে। `prometheus.yml`-এর `global`, `scrape_configs`, `rule_files` ও `alerting` অংশগুলো পড়তে ও লিখতে পারা প্রোডাকশন কাজের ভিত্তি। `Retention` ও সিরিজের সংখ্যা ডিস্ক ও মেমরির খরচ নির্ধারণ করে; দীর্ঘমেয়াদি সংরক্ষণের জন্য `remote write` আর উচ্চ-উপলভ্যতার জন্য অভিন্ন কপি চালানোই সাধারণ পদ্ধতি। Prometheus-এর নিজের স্বাস্থ্য পর্যবেক্ষণ করা — `up`, সিরিজ সংখ্যা, ডিস্ক ব্যবহার — প্রোডাকশনের প্রথম দায়িত্ব।

৭.১৭ মূল প্রযুক্তিগত পরিভাষা

TSDB: Time Series Database — সময়ের ক্রমে নমুনা সংরক্ষণকারী ডেটাবেস; Prometheus-এর সংরক্ষণ স্তর।

WAL: Write-Ahead Log — ডিস্কে লেখা লগ; ক্র্যাশের পর সাম্প্রতিক ডেটা পুনরুদ্ধারে ব্যবহৃত হয়।

Head Block: মেমরিতে থাকা সাম্প্রতিক প্রায় দুই ঘণ্টার ডেটার ব্লক।

Scrape: Prometheus-এর HTTP GET করে টার্গেট থেকে মেট্রিক সংগ্রহ করা।

Target: একটি scrape করা যায় এমন এন্ডপয়েন্ট; যেমন একটি অ্যাপের /metrics ঠিকানা।

Job: একই উদ্দেশ্যের টার্গেটের গোষ্ঠী; scrape_configs-এ job_name দিয়ে চিহ্নিত হয়।

Service Discovery: টার্গেটের তালিকা স্বয়ংক্রিয়ভাবে খুঁজে পাওয়ার ব্যবস্থা; যেমন ফাইল, DNS বা Kubernetes API থেকে।

Relabeling: scrape-এর আগে লেবেল বদলানো, যোগ করা বা টার্গেট বাদ দেওয়ার নিয়ম।

Retention: ডেটা কতদিন বা কত আকার পর্যন্ত রাখা হবে তার সীমা।

Remote Write / Read: নমুনা বাইরের দীর্ঘমেয়াদি স্টোরেজে পাঠানো ও সেখান থেকে পড়া।

Federation: একটি কেন্দ্রীয় Prometheus-এর অন্য Prometheus থেকে নির্বাচিত মেট্রিক সংগ্রহ করা।

Compaction: ছোট ব্লকগুলোকে একত্র করে বড়, আরও সংকুচিত ব্লক বানানো।

৭.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Prometheus কেন pull মডেল ব্যবহার করে? টার্গেট বন্ধ থাকলে pull মডেল কীভাবে তা জানায়?
2. WAL-এর কাজ কী? WAL না থাকলে ক্র্যাশের পর কী হারাত?
3. Head block ও স্থায়ী ব্লকের পার্থক্য কী? ব্লক কত ঘণ্টার ডেটা ধারণ করে?
4. up, scrape_duration_seconds ও scrape_samples_scraped — এই তিনটি মেট্রিক দিয়ে কী বোঝা যায়?
5. Service discovery ছাড়া static_configs-এ হাতে IP লিখলে কী সমস্যা হয়?
6. Relabeling-এ __meta_kubernetes_pod_name-এর মতো লেবেলের ভূমিকা কী? __ দিয়ে শুরু হওয়া লেবেলের বিশেষত্ব কী?
7. Retention-এ সময় ও আকার — দুটি সীমা একসঙ্গে দেওয়া কেন ভালো?
8. দুটি অভিন্ন Prometheus চালানোকে কেন সত্যিকারের ক্লাস্টার বলা যায় না? external_labels এখানে কী কাজ করে?
9. Remote write ও federation-এর মধ্যে পার্থক্য কী? কোনটি আধুনিক পছন্দ?
10. promtool check config কেন চালু করার আগে চালানো উচিত?

৭.১৯ ব্যবহারিক অনুশীলন

1. Docker Compose-এ Prometheus চালু করুন; prometheus.yml-এ নিজের নামে একটি job যোগ করে promtool দিয়ে যাচাই করুন।
2. Targets পৃষ্ঠা থেকে একটি টার্গেটের scrape_duration_seconds ও scrape_samples_scraped পড়ে তার অর্থ লিখুন।
3. file_sd_configs ব্যবহার করে JSON ফাইল থেকে টার্গেট যোগ করুন; ফাইল বদলে Prometheus পুনরায় চালু না করে টার্গেট বদলান।
4. Blackbox Exporter-এর relabeling উদাহরণটি নিজের ল্যাবে চালিয়ে একটি ওয়েবসাইটের স্বাস্থ্য পরীক্ষা করুন।
5. retention.size ছোট করে সেট করুন ও ডিস্ক ব্যবহার পর্যবেক্ষণ করে হিসাব মিলিয়ে দেখুন।
6. Prometheus-এর নিজের স্বাস্থ্যের জন্য তিনটি অ্যালার্টের খসড়া লিখুন: up == 0, সিরিজ সংখ্যা অস্বাভাবিক বৃদ্ধি, ডিস্ক ৮০% পূর্ণ।

অধ্যায় ৮ • PART II — Metrics and Prometheus Engineering

Chapter 8 — PromQL — Query Language and Metrics Analysis

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি `PromQL` দিয়ে Prometheus-কে প্রশ্ন করতে পারবেন — `instant vector` ও `range vector`-এর পার্থক্য, লেবেল ম্যাচার, `rate()`, `increase()`, `irate()`, একত্রীকরণ অপারেটর, `histogram_quantile()`, সাধারণ ফাংশন, সাবকুয়েরি ও `recording rule`-ধরনের কুয়েরি লিখতে পারবেন। আপনি SLO `burn-rate` কুয়েরি পড়তে পারবেন, সাধারণ ভুলগুলো চিনতে পারবেন এবং TechNova-র API-র লেটেন্সি, ত্রুটির হার ও স্যাচুরেশন বিশ্লেষণ করতে পারবেন।

- ভেক্টরের ধরন ও সিলেক্টর লেখা।
- Counter-এর জন্য rate/increase/irate-এর সঠিক ব্যবহার।
- by/without দিয়ে একত্রীকরণ ও লেবেল নিয়ন্ত্রণ।
- Histogram quantile ও SLO burn-rate কুয়েরি লেখা।
- সাধারণ কুয়েরি ভুল শনাক্ত ও সংশোধন করা।

৮.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

মেট্রিক সংগ্রহ করা কেবল অর্ধেক কাজ; বাকি অর্ধেক হলো সঠিক প্রশ্ন করা। একই ডেটা থেকে একজন প্রকৌশলী সত্য বের করেন, আরেকজন বিভ্রান্ত হন — পার্থক্যটা কুয়েরিতে। TechNova-র একবার এমন হয়েছিল: একজন প্রকৌশলী `http_requests_total`-কে সরাসরি গ্রাফ করে বললেন "ট্র্যাফিক কমছে", অথচ আসলে `rate()` স্থির ছিল — গ্রাফটি ছিল ক্রমবর্ধমান মোটের, হারের নয়। আরেকবার `avg()` দিয়ে গড় লেটেন্সি দেখে দলটি ভেবেছিল সব ঠিক আছে, অথচ p99 ভয়াবহ ছিল। `PromQL` শেখা মানে এই ভুলগুলো থেকে বাঁচা — সংখ্যাকে সঠিকভাবে জিজ্ঞাসা করা। অ্যালার্ট, ড্যাশবোর্ড, SLO — সবকিছুই `PromQL`-এর ওপর দাঁড়িয়ে আছে; কুয়েরি ভুল হলে অ্যালার্টও ভুল হবে।

৮.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬ ও ৭ জানা থাকতে হবে — চারটি মেট্রিক ধরন, `time series`, লেবেল, `scrape` ও TSDB। গাণিতিক পূর্বশর্ত সামান্য: হার (প্রতি সেকেন্ডে কত), শতকরা ও গড়ের ধারণা। প্রোগ্রামিং অভিজ্ঞতা থাকলে ভালো, তবে আবশ্যক নয় — `PromQL` একটি ঘোষণামূলক (declarative) ভাষা, অর্থাৎ আপনি "কী চাই" বলেন, "কীভাবে হিসাব করবে" বলেন না। Prometheus-এর ওয়েব UI-তে কুয়েরি চালানোর অভিজ্ঞতা থাকলে এই অধ্যায়ের উদাহরণগুলো সরাসরি পরীক্ষা করতে পারবেন।

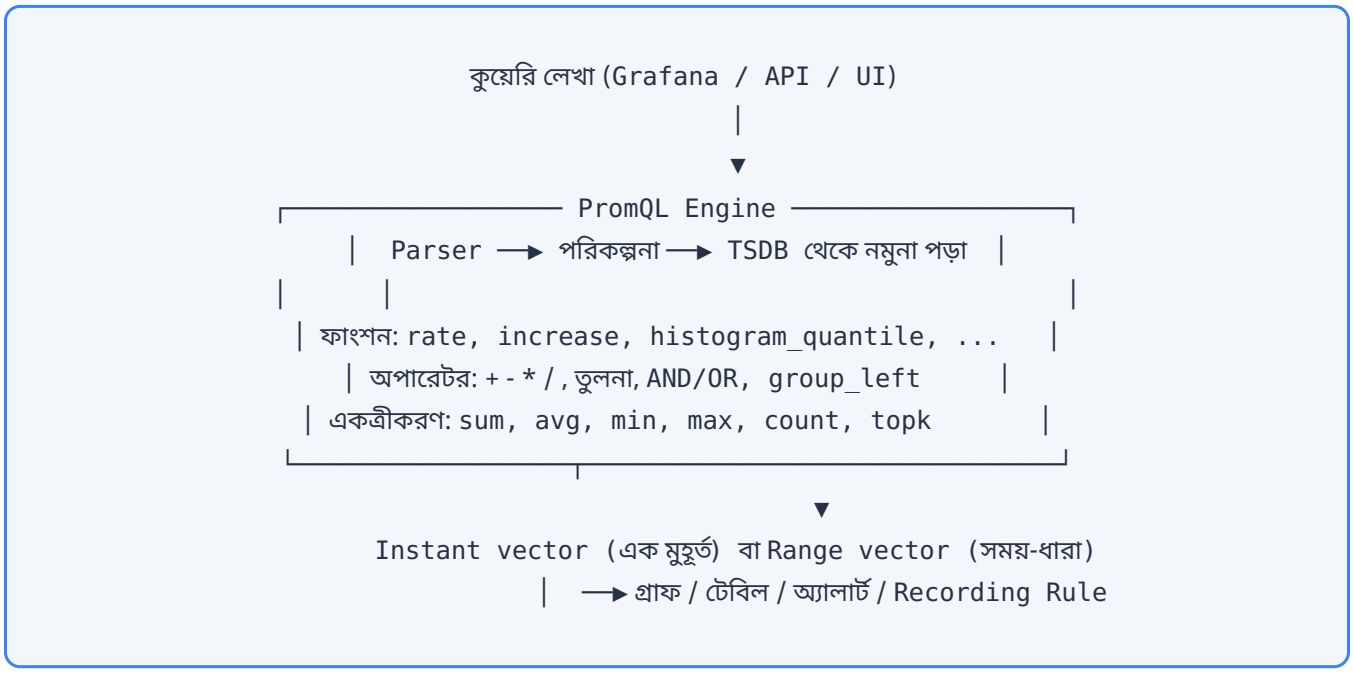
৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: আপনার কাছে সময়ের সঙ্গে বদলাতে থাকা অনেকগুলো সংখ্যার ধারা আছে। আপনি জানতে চান — এই মুহূর্তে মান কত? গত পাঁচ মিনিটে হার কত ছিল? সব সার্ভার মিলিয়ে মোট কত? সবচেয়ে খারাপ ৫% কেমন ছিল? `PromQL` এই প্রশ্নগুলোর ব্যাকরণ। তার মূল একক `vector`। `Instant vector` হলো একটি নির্দিষ্ট মুহূর্তে প্রতিটি সিরিজের মানের সেট; `range vector` হলো একটি সময়-পরিসরে প্রতিটি সিরিজের মানের ধারা। বেশিরভাগ ফাংশন — যেমন `rate()` — `range vector` নেয় ও `instant vector` দেয়। এই পার্থক্য না বুঝলে `PromQL`-এর অর্ধেক ত্রুটি বোঝা যাবে

না। আরেকটি মূল ধারণা: `PromQL`-এর প্রতিটি অপারেশন লেবেলের ওপর কাজ করে। দুটি ভেক্টর যোগ করতে গেলে তাদের লেবেল মিলতে হয়; না মিললে ফল ফাঁকা হয়। এই "লেবেল মেলানো" নিয়মটি প্রথমে অদ্ভুত লাগে, কিন্তু এটিই `PromQL`-কে শক্তিশালী করে — আপনি লেবেল দিয়ে যেকোনো ভাগে ডেটা কাটতে পারেন।

> **CONCEPT:** `PromQL`-এ একটি কুয়েরি একটি পাইপলাইন: সিলেক্টর সিরিজ বেছে নেয়, ফাংশন সেগুলোকে রূপান্তর করে, অপারেটর সেগুলোকে একত্র করে। যেমন `sum by (path) (rate(http_requests_total{status="500"}[5m]))`-এ প্রথমে `status="500"` সিরিজগুলো বাছা হয়, তারপর `rate()` প্রতি সেকেন্ডের হার বের করে, তারপর `sum by (path)` প্রতিটি পথের হার যোগ করে। ডান থেকে বামে নয়, ভেতর থেকে বাইরে পড়ুন।

৮.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



ইঞ্জিন প্রথমে কুয়েরিটি বিশ্লেষণ (parse) করে, তারপর TSDB থেকে প্রয়োজনীয় সিরিজ ও সময়-পরিসরের নমুনা পড়ে, তারপর ফাংশন ও অপারেটর প্রয়োগ করে। গুরুত্বপূর্ণ বিষয়: `PromQL` TSDB-তে লেখে না — এটি কেবল পড়ে ও গণনা করে। `Recording rule` ব্যতিক্রম: তার ফলাফল আবার TSDB-তে লেখা হয়, যাতে ভারী কুয়েরি বারবার না চালাতে হয়। আরেকটি বিষয়: কুয়েরি যত বেশি সিরিজ ও যত দীর্ঘ সময়-পরিসর ছোঁয়, তত বেশি CPU ও মেমরি লাগে। তাই ভালো কুয়েরি লেখা মানে কম সিরিজ ছোঁয়া — লেবেল ম্যাচার দিয়ে আগেই সিরিজ সীমিত করা।

৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

সিলেক্টর ও লেবেল ম্যাচার

সবচেয়ে সহজ কুয়েরি একটি মেট্রিকের নাম: `http_requests_total`। লেবেল দিয়ে সীমিত করুন: `http_requests_total{method="GET", status="200"}`। ম্যাচার চার ধরনের: `=` (সমান), `!=` (সমান নয়), `=~` (রেগেক্স মেলে), `!~` (রেগেক্স মেলে না)। যেমন `status=~"5.."` মানে ৫ দিয়ে শুরু হওয়া সব স্ট্যাটাস। `__name__` লেবেল দিয়ে নামের ওপর রেগেক্সও চালানো যায়: `{__name__=~"http_.*"}`। মনে রাখুন: রেগেক্স ম্যাচার শক্তিশালী কিন্তু ব্যয়বহুল — খুব সাধারণ রেগেক্স অনেক সিরিজ ছুঁয়ে ফেলতে পারে।

Instant বনাম Range Vector

`http_requests_total` একটি `instant vector` — এই মুহূর্তের মান। `http_requests_total[5m]` একটি `range vector` — গত পাঁচ মিনিটের সব নমুনা। `Range vector` সরাসরি গ্রাফ করা যায় না; তাকে প্রথমে কোনো ফাংশন দিয়ে `instant vector`-এ রূপান্তর করতে হয়। Grafana-র গ্রাফ প্যানেলে আপনি সময়-পরিসর বদলালে Prometheus আসলে অনেকগুলো `instant vector` হিসাব করে সেগুলোকে জোড়া দেয় — প্রতিটি বিন্দু একটি আলাদা মূল্যায়ন।

rate, increase ও irate

`rate(http_requests_total[5m])` — গত পাঁচ মিনিটে প্রতি সেকেন্ডে গড়ে কত অনুরোধ। `increase(http_requests_total[1h])` — গত এক ঘণ্টায় মোট কত বেড়েছে। `irate()` — শেষ দুটি নমুনার ভিত্তিতে তাৎক্ষণিক হার; দ্রুত বদল ধরে, কিন্তু অস্থির (noisy)। নিয়ম: অ্যালার্ট ও ড্যাশবোর্ডে `rate()` ব্যবহার করুন; `irate()` কেবল দ্রুত পরিবর্তনশীল জিনিস দেখতে। `rate()`-এর সময়-পরিসর `scrape_interval`-এর অন্তত চার গুণ রাখুন — ১৫ সেকেন্ড বিরতিতে `[5m]` নিরাপদ, `[1m]` ঝুঁকিপূর্ণ। `Counter reset` হলে `rate()` তা সামলে নেয়; কিন্তু `rate()`-কে `Gauge`-এর ওপর চালাবেন না — `Gauge`-এর জন্য সরাসরি মান দেখুন।

একত্রীকরণ: sum, avg, by ও without

`sum(http_requests_total)` সব সিরিজ যোগ করে। `sum by (path) (...)` প্রতিটি পথ আলাদাভাবে যোগ করে; `sum without (instance) (...)` `instance` লেবেল বাদ দিয়ে বাকিগুলো দিয়ে ভাগ করে। পার্থক্যটি সূক্ষ্ম কিন্তু গুরুত্বপূর্ণ: `by` বলে "কেবল এই লেবেলগুলো রাখো", `without` বলে "এই লেবেলগুলো বাদ দাও, বাকি সব রাখো"। নতুন লেবেল যোগ হলে `without` অপ্রত্যাশিতভাবে ভাগ বদলে দিতে পারে; তাই `by` সাধারণত নিরাপদ। অন্যান্য অপারেটর: `avg`, `min`, `max`, `count`, `stddev`, `topk(5, ...)` (সবচেয়ে বড় পাঁচটি), `quantile(0.95, ...)` — তবে মনে রাখুন, `quantile()` এগ্রিগেটরটি `Histogram`-এর `histogram_quantile()` থেকে আলাদা; এটি কাঁচা মানের ওপর কাজ করে।

Histogram Quantile ও ফাংশন

লেটেন্সির p95: `histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))`। এখানে `sum by (le)` অপরিহার্য — একাধিক সার্ভারের বাকেট যোগ না করলে ফল ভুল হবে। অন্যান্য দরকারি ফাংশন: `predict_linear(node_filesystem_avail_bytes[1h], 3600 * 24)` — বর্তমান হারে ডিস্ক কখন ভরবে তার পূর্বাভাস; `changes(up[1h])` — গত এক ঘণ্টায় `up` কতবার বদলেছে; `absent(http_requests_total)` — সিরিজ না থাকলে ১ দেয়, অর্থাৎ মেট্রিক উধাও হলে অ্যালার্ট দেওয়া যায়; `delta()`, `deriv()` — `Gauge`-এর পরিবর্তন ও ঢাল। `label_replace()` ও `label_join()` দিয়ে লেবেলের মান বদলানো যায় — যেমন দীর্ঘ `instance` থেকে শুধু হোস্টনাম বের করা।

সাবকুয়েরি ও Recording-Rule ধরনের কুয়েরি

সাবকুয়েরি হলো কুয়েরির ভেতরে কুয়েরি: `avg_over_time(rate(http_requests_total[5m])[1h:5m])` — গত এক ঘণ্টায় প্রতি পাঁচ মিনিটের `rate()`-এর গড়। বাক্যগঠন `[পরিসর:ধাপ]`। সাবকুয়েরি শক্তিশালী, কিন্তু ব্যয়বহুল — ঘন ঘন ব্যবহার করলে `recording rule`-এ রূপান্তর করুন: ভারী কুয়েরিটি নিয়মিত হিসাব করে ফলাফলকে নতুন মেট্রিক হিসেবে সংরক্ষণ করুন, তারপর ড্যাশবোর্ডে সেই মেট্রিকটি ব্যবহার করুন।

SLO Burn-Rate উদাহরণ

ধরুন TechNova-র SLO: ৯৯.৯% অনুরোধ সফল হতে হবে। ত্রুটির বাজেট পুড়ে যাওয়ার হার (burn rate) মাপতে: `sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))` — বর্তমান ত্রুটির হার। SLO-র অনুমোদিত ত্রুটির হার ০.১% হলে, `burn rate = বর্তমান হার / 0.001`। Burn rate ১ মানে বাজেট ঠিক হারে পুড়ছে; ১৪ মানে ১৪ গুণ দ্রুত পুড়ছে — জরুরি অ্যালার্টের যোগ্য। বহু-উইন্ডো পদ্ধতিতে ১ ঘণ্টা ও ৬ ঘণ্টা — দুটি উইন্ডো একসঙ্গে দেখা হয়, যাতে ক্ষণস্থায়ী স্পাইকে ভুল অ্যালার্ট না যায় (বিস্তারিত অধ্যায় ১০-এ)।

কুয়েরি	অর্থ	কখন ব্যবহার
rate(http_requests_total[5m])	প্রতি সেকেন্ডে অনুরোধের হার	ট্র্যাফিক দেখতে
sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))	ত্রুটির অনুপাত	SLO ও অ্যালার্ট
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))	p95 লেটেন্সি	ব্যবহারকারীর অভিজ্ঞতা
avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m]))	অলস CPU-র হার	স্যাচুরেশন
predict_linear(node_filesystem_avail_bytes[6h], 86400)	২৪ ঘণ্টা পরে ফাঁকা জায়গা	ক্ষমতা পরিকল্পনা

৮.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

`PromQL`-এর কোনো কনফিগারেশন ফাইল নেই, তবে কুয়েরি লেখার সময় কয়েকটি নিয়ম মনে রাখুন। নিচের উদাহরণগুলো লাইন ধরে ধরে পড়ুন।

```
# ত্রুটির হার — ধাপে ধাপে গঠন
sum(rate(http_requests_total{status=~"5.."}[5m]))   # ৫xx অনুরোধের মোট হার (প্রতি সেকেন্ড)
  / sum(rate(http_requests_total[5m]))              # ভাগ: সব অনুরোধের মোট হার
# ফল: 0.0042 মানে 0.42% অনুরোধ ব্যর্থ হচ্ছে

# CPU ব্যবহার — idle বাদ দিয়ে হিসাব
100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))
# node_cpu_seconds_total একটি Counter; mode="idle" বাদ দিলে ব্যস্ত CPU-র হার মেলে

# Recording rule (rules/recording.yml) — ভারী কুয়েরি আগে থেকে হিসাব করা
groups:
  - name: technova-api
    interval: 30s
    rules:
      - record: job:http_requests_total:rate5m
        expr: sum by (job) (rate(http_requests_total[5m]))
# এরপর ড্যাশবোর্ডে job:http_requests_total:rate5m ব্যবহার করুন — দ্রুত চলবে
```

প্রথম কুয়েরিতে ভাগের দুই পাশে `sum()` আছে — কারণ `rate()`-এর ফল প্রতিটি সিরিজে আলাদা; ভাগ করার আগে মোট নিতে হয়, নইলে লেবেল না মেলায় ফল ফাঁকা হতে পারে। দ্বিতীয় কুয়েরিতে `avg()` আছে — একাধিক CPU কোরের গড়। তৃতীয় উদাহরণটি `recording rule`: `record` নামটি একটি নতুন মেট্রিকের নাম হয়, আর `expr` তার হিসাব। নামকরণের

প্রচলিত ধরন `level:metric:operation` — যেমন `job:http_requests_total:rate5m`। `interval` ৩০ সেকেন্ড মানে এই হিসাব প্রতি ৩০ সেকেন্ডে একবার চলবে।

৮.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র দৈনন্দিন বিশ্লেষণের জন্য একটি কুয়েরি সংগ্রহ — প্রতিটি কুয়েরির পাশে তার উদ্দেশ্য লেখা আছে:

```
# ১. প্রতি এন্ডপয়েন্টের p95 লেটেন্সি
histogram_quantile(0.95,
  sum by (le, path) (rate(http_request_duration_seconds_bucket[5m])))

# ২. ত্রুটির হার — শতাংশে
100 * sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m]))

# ৩. সবচেয়ে ধীর ৫টি এন্ডপয়েন্ট
topk(5,
  histogram_quantile(0.95,
    sum by (le, path) (rate(http_request_duration_seconds_bucket[5m]))))

# ৪. মেমরি ব্যবহারের হার — ১ ঘণ্টায় কত বেড়েছে
node_memory_MemTotal_bytes - node_memory_MemAvailable_bytes

# ৫. ডিস্ক ৪ ঘণ্টার মধ্যে ভরে যাবে কি না (সতর্কতা)
predict_linear(node_filesystem_avail_bytes{mountpoint="/"}[1h], 4 * 3600) < 0

# ৬. কোনো সার্ভিসের মেট্রিক উধাও হয়েছে কি না
absent(up{job="technova-api"} == 1)

# ৭. গত ১ ঘণ্টায় কতবার সার্ভিস পুনরায় চালু হয়েছে
changes(process_start_time_seconds{job="technova-api"}[1h])
```

পাঁচ নম্বর কুয়েরিটি লক্ষ্য করুন: `predict_linear()` ভবিষ্যতের মান অনুমান করে; `< 0` তুলনা করে বুলিয়ান ফল দেয় — অ্যালার্টের জন্য আদর্শ। সাত নম্বর কুয়েরিটি একটি চতুর কৌশল: `process_start_time_seconds` একটি `Gauge`, যা প্রক্রিয়া চালুর সময়ের টাইমস্ট্যাম্প ধারণ করে; পুনরায় চালু হলে মান বদলায়, আর `changes()` সেই বদলের সংখ্যা গোনে। এভাবে পুনরায় চালুর ঘটনা ধরা যায়।

৮.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র এক সকালে `/api/orders`-এর p95 হঠাৎ ২০০ মিলিসেকেন্ড থেকে ২ সেকেন্ডে উঠল। দলটি প্রথমে `topk()` দিয়ে দেখল কোন এন্ডপয়েন্ট সবচেয়ে খারাপ — শুধু `/api/orders`। তারপর `by (status)` দিয়ে ভাগ করে দেখল ত্রুটির হার বাড়েনি — অনুরোধ সফল হচ্ছে, কিন্তু ধীরে। তারপর MongoDB-র `Histogram` দেখল — ডাটাবেস কুয়েরির p95-ও বেড়েছে। শেষে `node_cpu_seconds_total` ও মেমরি দেখে বোঝা গেল ডাটাবেস সার্ভারের CPU ৯৫% ছুঁয়েছে — একটি ভারী রিপোর্ট কুয়েরি চলছিল। রিপোর্টটি বন্ধ করতেই p95 স্বাভাবিক হলো। এই তদন্তের প্রতিটি ধাপ একটি `PromQL` কুয়েরি ছিল — সিলেক্টর দিয়ে ভাগ করা, `rate()` দিয়ে হার বের করা, `histogram_quantile()` দিয়ে বণ্টন দেখা। কুয়েরি না জানলে এই তদন্ত ঘণ্টার পর ঘণ্টা লগ ঘেঁটে করতে হতো।

৮.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Prometheus UI খুলে `http_requests_total` লিখে দেখুন — instant vector-এর ফল কেমন দেখায়।

2. তারপর `rate(http_requests_total[5m])` লিখে গ্রাফ করুন; `[1m]` ও `[15m]`-এর ফল তুলনা করে দেখুন সময়-পরিসর কীভাবে মসৃণতা বদলায়।
3. ত্রুটির হারের কুয়েরিটি লিখুন ও একটি কৃত্রিম ৫০০ ত্রুটি তৈরি করে দেখুন হার কীভাবে বাড়ে।
4. p50, p95 ও p99 — তিনটি `histogram_quantile()` কুয়েরি একসঙ্গে গ্রাফ করে তাদের পার্থক্য লক্ষ্য করুন।
5. `topk(5, ...)` দিয়ে সবচেয়ে ব্যস্ত এন্ডপয়েন্ট খুঁজুন।
6. একটি `recording rule` লিখুন (`job:http_requests_total:rate5m`), `promtool check rules` দিয়ে যাচাই করুন, Prometheus পুনরায় লোড করুন ও নতুন মেট্রিকটি কুয়েরি করুন।
7. SLO burn-rate কুয়েরিটি লিখুন; ত্রুটির হার ০.১%-এর নিচে থাকলে burn rate ১-এর নিচে দেখায় কি না যাচাই করুন।

৮.১১ সাধারণ ভুল ও ভুল ধারণা

- **Counter-এ সরাসরি rate() ছাড়া গ্রাফ:** কাঁচা Counter-এর গ্রাফ অর্থহীন — সবসময় `rate()` বা `increase()` ব্যবহার করুন।
- **Gauge-এ rate() চালানো:** `rate()` শুধু Counter-এর জন্য; Gauge-এর জন্য সরাসরি মান বা `delta()` ব্যবহার করুন।
- **খুব ছোট range:** `rate(x[1m])` ১৫ সেকেন্ড scrape-এ অস্থির হয়; অন্তত `[5m]` ব্যবহার করুন।
- **histogram_quantile()-এ sum by (le) ভুলে যাওয়া:** একাধিক সার্ভারের বাকেট যোগ না করলে ফল ভুল হবে।
- **by ও without গুলিয়ে ফেলা:** `without` নতুন লেবেল এলে ভাগ বদলে দেয়; `by` নিরাপদ।
- **ভাগের আগে sum না করা:** লেবেল না মিললে ভাগের ফল ফাঁকা হয় — দুই পাশেই একই লেবেল রাখুন।
- **irate() দিয়ে অ্যালার্ট লেখা:** `irate()` অস্থির; অ্যালার্টে `rate()` ব্যবহার করুন।
- **absent() ভুল বোঝা:** `absent()` সিরিজ না থাকলে ১ দেয়, থাকলে ফাঁকা — তার উল্টো নয়।

> **WARNING:** `rate()` প্রথম দুটি নমুনার আগে কিছু দেয় না। নতুন মেট্রিক বা নতুন সার্ভিসে প্রথম কয়েক মিনিট ফাঁকা গ্রাফ দেখে আতঙ্কিত হবেন না — এটি স্বাভাবিক আচরণ।

৮.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

`PromQL` নিজে কোনো গোপন তথ্য তৈরি করে না, তবে কুয়েরির ফলাফলে সংবেদনশীল তথ্য থাকতে পারে — যেমন লেবেলে ব্যবহারকারীর নাম বা ইমেইল। ড্যাশবোর্ড শেয়ার করার আগে লেবেলগুলো পরীক্ষা করুন। Grafana-র ড্যাশবোর্ডে `PromQL` কুয়েরি দৃশ্যমান থাকে; সেখানে কখনো টোকেন বা পাসওয়ার্ড লিখবেন না। আরেকটি বিষয়: ভারী কুয়েরি দিয়ে Prometheus-কে আক্রমণ করা যায় — একটি দীর্ঘ সময়-পরিসরে অনেক সিরিজের ওপর কুয়েরি চালালে সার্ভারের CPU ও মেমরি চাপ বাড়ে। প্রোডাকশনে কুয়েরির সময়-পরিসর ও সিরিজের সংখ্যায় সীমা বসান, আর Grafana-র কুয়েরি টাইমআউট নির্ধারণ করুন। `PromQL`-এর `label_replace()` দিয়ে সংবেদনশীল লেবেলের মান লুকানো যায় না — সেটি কেবল প্রদর্শনের জন্য; আসল সুরক্ষা হলো লেবেলে সংবেদনশীল তথ্য না রাখা।

৮.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

কুয়েরির খরচ নির্ভর করে তিনটি জিনিসের ওপর: সিরিজের সংখ্যা, সময়-পরিসরের দৈর্ঘ্য ও ধাপের সংখ্যা। Grafana-র একটি গ্রাফ ২৪ ঘণ্টার ডেটা ১০০টি বিন্দুতে দেখাতে চাইলে Prometheus ১০০ বার কুয়েরি মূল্যায়ন করে; প্রতিটি মূল্যায়নে `[5m]` পরিসরের নমুনা পড়া হয়। সিরিজ ১০ হাজার হলে এটি ভারী হয়। সমাধান তিনটি: প্রথমত, সিলেক্টরে লেবেল ম্যাচার

দিয়ে সিরিজ কমান। দ্বিতীয়ত, ভারী কুয়েরিকে `recording rule`-এ রূপান্তর করুন — হিসাব একবার করে ফল সংরক্ষণ করুন, ড্যাশবোর্ডে সেই ফল ব্যবহার করুন। তৃতীয়ত, Grafana-র `min step` বাড়িয়ে ধাপের সংখ্যা কমান। `topk()` ও `quantile()` এগ্রিগেটর তুলনামূলক ব্যয়বহুল; `histogram_quantile()` সঠিকভাবে লিখলে দ্রুত চলে, কিন্তু ভুল লেবেল দিয়ে লিখলে ধীর হয়। নিয়মটি মনে রাখুন: কুয়েরি লেখার আগে ভাবুন — "এই কুয়েরি কতগুলো সিরিজ ছোঁবে?" উত্তর হাজার ছাড়ালে `recording rule` বিবেচনা করুন।

৮.১৪ সমস্যা সমাধান (Troubleshooting)

কুয়েরি ফাঁকা ফল দিচ্ছে: প্রথমে সিলেক্টরটি একা চালান — সিরিজ আছে কি না দেখুন। তারপর লেবেল ম্যাচার একে একে সরিয়ে দেখুন কোনটি ফল শূন্য করছে। ভাগের কুয়েরিতে দুই পাশের লেবেল মিলছে কি না পরীক্ষা করুন — `sum()` দিয়ে লেবেল সরিয়ে নিলে প্রায়ই সমস্যা মেটে।

rate() শূন্য বা NaN: `scrape` হচ্ছে কি না (`up` দেখুন), সময়-পরিসরে অন্তত দুটি নমুনা আছে কি না দেখুন, আর `Counter` আসলে বাড়ছে কি না যাচাই করুন।

histogram_quantile() অদ্ভুত মান: `sum by (le)` আছে কি না, বাকেটের সীমা উপযুক্ত কি না, আর `rate()` ব্যবহার করা হয়েছে কি না — এই তিনটি পরীক্ষা করুন।

কুয়েরি ধীর: সময়-পরিসর কমান, লেবেল ম্যাচার যোগ করুন, অথবা `recording rule`-এ রূপান্তর করুন। Prometheus UI-তে কুয়েরির সময় দেখা যায় — সেটি পর্যবেক্ষণ করুন।

> **TROUBLESHOOTING:** কুয়েরি ডিবাগ করার সবচেয়ে ভালো পদ্ধতি হলো ভেতর থেকে বাইরে গড়ে তোলা — প্রথমে সিলেক্টর, তারপর rate(), তারপর sum()। একবারে পুরো কুয়েরি লিখে ভুল খোঁজা কঠিন; ধাপে ধাপে গড়লে ভুলের জায়গাটি স্পষ্ট হয়।

৮.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: `rate()`-এর পরিসর অন্তত `[5m]` রাখুন; `by` ব্যবহার করুন, `without` এড়িয়ে চলুন; ভাগের আগে দুই পাশে `sum()` করুন; ভারী কুয়েরিকে `recording rule`-এ রূপান্তর করুন; কুয়েরির নামকরণে `level:metric:operation` ধরন অনুসরণ করুন; SLO কুয়েরিতে বহু-উইন্ডো পদ্ধতি ব্যবহার করুন।

অ্যান্টি-প্যাটার্ন: কাঁচা Counter গ্রাফ করা; Gauge-এ `rate()` চালানো; `irate()` দিয়ে অ্যালার্ট লেখা; `histogram_quantile()`-এ `sum by (le)` ভুলে যাওয়া; রেগেক্স ম্যাচারে `.*` দিয়ে সব সিরিজ ছোঁয়া; একই ভারী কুয়েরি দশটি ড্যাশবোর্ডে কপি করা — বদলে একটি `recording rule` লিখুন।

> **PRO TIP:** প্রতিটি অ্যালার্ট লেখার আগে তার কুয়েরিটি Prometheus UI-তে গত ২৪ ঘণ্টার গ্রাফে দেখুন — কুয়েরিটি অতীতে কখন সত্য হতো, তা দেখলে ভুল অ্যালার্টের ঝুঁকি বোঝা যায়। যে কুয়েরি অতীতে বারবার সত্য হয়েছে, সেটি প্রোডাকশনে শব্দদূষণ করবে।

৮.১৬ অধ্যায়ের সারাংশ

`PromQL` হলো Prometheus-কে প্রশ্ন করার ভাষা, যার মূল একক `vector` — `instant vector` একটি মুহূর্তের মান দেয়, `range vector` একটি সময়-পরিসরের ধারা দেয়। সিলেক্টর ও লেবেল ম্যাচার দিয়ে সিরিজ বাছা হয়; `rate()`,

`increase()` ও `irate()` Counter থেকে হার বের করে; `sum by` ও অন্যান্য এগ্রিগেটর সিরিজ একত্র করে; `histogram_quantile()` বাকেট থেকে শতকরা বের করে। `predict_linear()`, `changes()` ও `absent()`-এর মতো ফাংশন ক্ষমতা পরিকল্পনা, পুনরায় চালু শনাক্তকরণ ও নিখোঁজ মেট্রিক ধরতে সাহায্য করে। SLO `burn-rate` কুয়েরি ত্রুটির বাজেট কত দ্রুত পুড়ছে তা মাপে। ভারী কুয়েরিকে `recording rule`-এ রূপান্তর করলে ড্যাশবোর্ড ও অ্যালার্ট দ্রুত চলে। সবচেয়ে গুরুত্বপূর্ণ শিক্ষা: কুয়েরি ভেতর থেকে বাইরে গড়ে তুলুন, ভাগের আগে লেবেল মিলিয়ে নিন, আর `rate()`-এর পরিসর যথেষ্ট বড় রাখুন।

৮.১৭ মূল প্রযুক্তিগত পরিভাষা

PromQL: Prometheus Query Language — Prometheus-কে প্রশ্ন করার ঘোষণামূলক ভাষা।

Instant Vector: একটি নির্দিষ্ট মুহূর্তে প্রতিটি সিরিজের মানের সেট।

Range Vector: একটি সময়-পরিসরে প্রতিটি সিরিজের মানের ধারা; যেমন metric[5m]।

Label Matcher: লেবেলের মান দিয়ে সিরিজ বাছাইয়ের শর্ত; =, !=, =~, !~ চার ধরনের।

rate(): Counter থেকে প্রতি সেকেন্ডে গড় বৃদ্ধির হার বের করার ফাংশন।

increase(): একটি সময়-পরিসরে Counter মোট কত বেড়েছে তা বের করার ফাংশন।

irate(): শেষ দুটি নমুনার ভিত্তিতে তাৎক্ষণিক হার; দ্রুত বদল ধরে কিন্তু অস্থির।

Aggregation Operator: সিরিজ একত্র করার অপারেটর; যেমন sum, avg, min, max, topk।

histogram_quantile(): Histogram বাকেট থেকে নির্দিষ্ট quantile (যেমন p95) বের করার ফাংশন।

Recording Rule: ভারী কুয়েরি নিয়মিত হিসাব করে ফলাফলকে নতুন মেট্রিক হিসেবে সংরক্ষণ করার নিয়ম।

Subquery: কুয়েরির ভেতরে আরেকটি কুয়েরি; [পরিসর:ধাপ] বাক্যগঠনে লেখা হয়।

Burn Rate: SLO-র ত্রুটির বাজেট কত দ্রুত পুড়ছে তার অনুপাত; ১ মানে নির্ধারিত হারে।

৮.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Instant vector ও range vector-এর পার্থক্য কী? rate() কোনটি নেয়, কোনটি দেয়?
2. rate(), increase() ও irate()-এর মধ্যে পার্থক্য কী? অ্যালার্টে কোনটি ব্যবহার করবেন?
3. sum by (path) ও sum without (instance)-এর পার্থক্য কী? by কেন সাধারণত নিরাপদ?
4. histogram_quantile()-এ sum by (le) ভুলে গেলে কী ভুল হয়?
5. status=~"5.." ম্যাচারটি কী বোঝায়? =~ ও = এর পার্থক্য কী?
6. ত্রুটির হার বের করতে ভাগের দুই পাশে sum() করা কেন দরকার?
7. predict_linear() দিয়ে ডিস্ক কখন ভরবে তা কীভাবে অনুমান করা যায়?
8. absent() কীভাবে নিখোঁজ মেট্রিক ধরতে সাহায্য করে?
9. Recording rule কখন ব্যবহার করবেন? তার নামকরণের প্রচলিত ধরনটি কী?
10. Burn rate ১৪ মানে কী? বহু-উইন্ডো পদ্ধতি কেন ব্যবহার করা হয়?

৮.১৯ ব্যবহারিক অনুশীলন

1. Prometheus UI-তে rate(), increase() ও irate() — তিনটির ফল একই মেট্রিকে তুলনা করে পার্থক্য লিখুন।
2. p50, p95 ও p99-এর তিনটি কুয়েরি লিখে একসঙ্গে গ্রাফ করুন; একটি ধীর এন্ডপয়েন্টে তাদের পার্থক্য ব্যাখ্যা করুন।
3. ত্রুটির হারের কুয়েরিটি লিখুন ও কৃত্রিম ত্রুটি তৈরি করে burn rate হিসাব করুন।
4. একটি recording rule লিখুন, promtool দিয়ে যাচাই করুন ও নতুন মেট্রিকটি কুয়েরি করুন।
5. topk() দিয়ে সবচেয়ে ব্যস্ত ৫টি এন্ডপয়েন্ট খুঁজুন ও তাদের p95 তুলনা করুন।
6. একটি সাবকুয়েরি লিখুন: গত ১ ঘণ্টায় প্রতি ৫ মিনিটের rate()-এর গড় বের করুন।

অধ্যায় ৯ • PART II — Metrics and Prometheus Engineering

Chapter 9 — Exporters, Node Exporter, and Application Instrumentation

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি `Exporter` কী ও কেন দরকার তা ব্যাখ্যা করতে পারবেন, `Node Exporter` দিয়ে হোস্টের মেট্রিক সংগ্রহ করতে পারবেন, `Blackbox Exporter` দিয়ে বাইরে থেকে সেবার স্বাস্থ্য পরীক্ষা করতে পারবেন, `prom-client` দিয়ে Node.js অ্যাপে নিজের মেট্রিক যোগ করতে পারবেন এবং `Pushgateway` কখন ব্যবহার করা যায় আর কখন যায় না তা বিচার করতে পারবেন।

- Exporter-এর ভূমিকা ও প্রচলিত Exporter-গুলো চেনা।
- Node Exporter-এর collector ও প্রধান মেট্রিক পড়া।
- prom-client দিয়ে Counter, Gauge ও Histogram লেখা।
- /metrics এন্ডপয়েন্ট নিরাপদে প্রকাশ করা।
- Pushgateway-এর সঠিক ও ভুল ব্যবহারের পার্থক্য।

৯.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Prometheus নিজে কিছু মাপে না — সে শুধু সংগ্রহ করে। মাপার কাজটি করে `Exporter` ও অ্যাপের ভেতরের `instrumentation`। TechNova-র সার্ভার একবার ধীর হয়ে গিয়েছিল, কিন্তু Prometheus-এ কোনো সতর্কতা আসেনি — কারণ Node Exporter চালুই ছিল না, CPU ও মেমরির কোনো মেট্রিক ছিল না। আবার আরেকবার অ্যাপের ভেতরের ধীরগতি ধরা পড়েনি, কারণ অ্যাপে কোনো `Histogram` ছিল না — শুধু সার্ভারের CPU দেখে অ্যাপের লেটেন্সি বোঝা যায় না। `Exporter` হলো Prometheus-এর চোখ ও কান; চোখ না থাকলে সার্ভার অন্ধ। আর `instrumentation` হলো অ্যাপের নিজের কথা বলার ক্ষমতা — অ্যাপ নিজে না বললে তার ভেতরের গল্প কেউ জানতে পারে না।

৯.৩ পূর্বশর্ত জ্ঞান

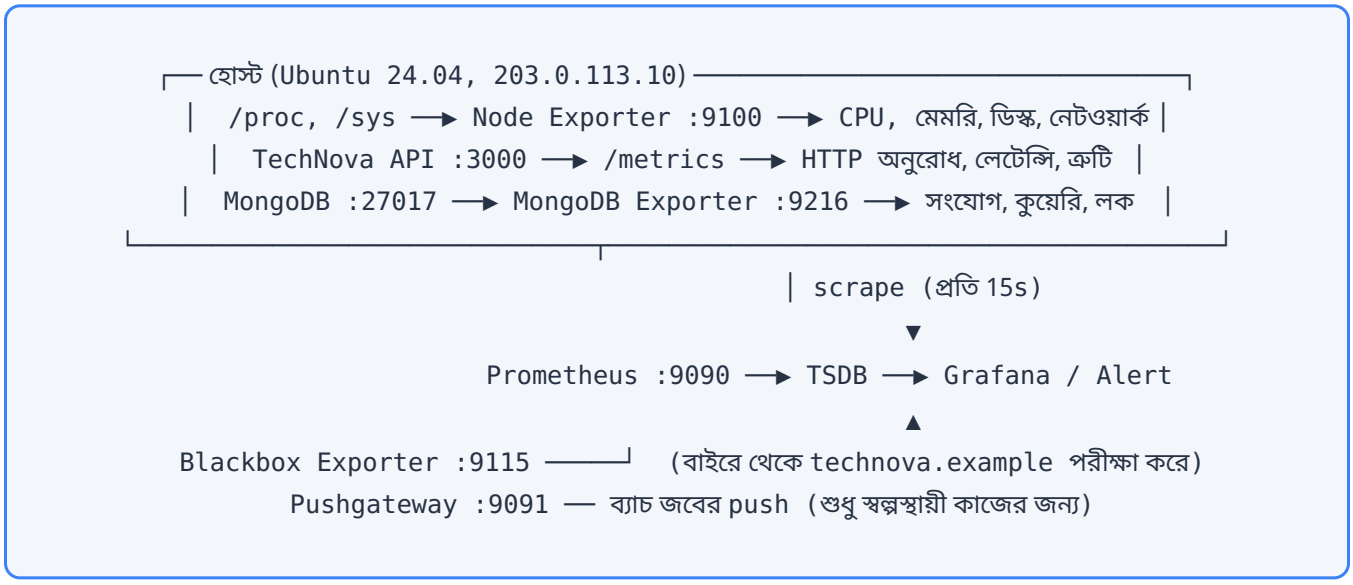
অধ্যায় ৬–৮ জানা থাকতে হবে — চারটি মেট্রিক ধরন, `scrape`, `PromQL` ও `histogram_quantile()`। এ ছাড়া প্রয়োজন: Node.js ও Express-এর মৌলিক জ্ঞান, Linux-এ CPU, মেমরি, ডিস্ক ও নেটওয়ার্কের ধারণা, এবং `npm` দিয়ে প্যাকেজ ইনস্টল করার অভিজ্ঞতা। Docker Compose জানা থাকলে Exporter চালানো সহজ হবে।

৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: Prometheus HTTP দিয়ে মেট্রিক টানে। কিন্তু সব সিস্টেম HTTP বলে না — Linux কার্নেল বলে `/proc` ফাইলের ভাষায়, MongoDB বলে তার নিজের প্রোটোকলে, একটি ব্যাচ জব তো চলেই স্বল্প সময়ের জন্য। `Exporter` হলো অনুবাদক: সে সিস্টেমের নিজের ভাষা পড়ে Prometheus-এর `exposition format`-এ অনুবাদ করে দেয়। Node Exporter `/proc` পড়ে CPU, মেমরি, ডিস্কের মেট্রিক বানায়; Blackbox Exporter বাইরে থেকে HTTP অনুরোধ পাঠিয়ে সাড়া মাপে; MongoDB Exporter ডাটাবেসের ভেতরের পরিসংখ্যান বের করে আনে। আর অ্যাপের নিজের মেট্রিকের জন্য কোনো অনুবাদক লাগে না — অ্যাপ নিজেই `prom-client`-এর মতো লাইব্রেরি দিয়ে `/metrics` এন্ডপয়েন্টে নিজের ভাষায় কথা বলে। নিয়মটি সহজ: যা Prometheus-এর ভাষা বলে না, তার জন্য Exporter লাগে; যা নিজে বলতে পারে, তার জন্য instrumentation লাগে।

> **CONCEPT:** Exporter কোনো এজেন্ট নয়, এটি একটি ছোট HTTP সার্ভার। সে নিজে ডেটা সংরক্ষণ করে না, অ্যালার্ট দেয় না, কোথাও পাঠায় না — সে কেবল জিজ্ঞাসা করলে বর্তমান মান বলে। এই সরলতাই তার শক্তি: Exporter বন্ধ থাকলে Prometheus `up == 0` দেখে তা জানতে পারে, আর Exporter-এর কোনো অবস্থা (state) না থাকায় তাকে পুনরায় চালু করা নিরাপদ।

৯.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



লক্ষ্য করুন, প্রতিটি Exporter আলাদা পোর্টে চলে — Node Exporter ৯১০০, Blackbox ৯১১৫, Pushgateway ৯০৯১। এই পোর্টগুলো প্রচলিত (conventional); বদলানো যায়, তবে প্রচলিত পোর্ট ব্যবহার করলে অন্যরা সহজে চিনতে পারে। অ্যাপের `/metrics` সাধারণত অ্যাপের নিজের পোর্টেই থাকে — TechNova-র ক্ষেত্রে ৩০০০। Prometheus প্রতিটি টার্গেটকে আলাদা `job` হিসেবে `scrape` করে, যাতে `job` লেবেল দিয়ে উৎস আলাদা করা যায়।

৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Node Exporter ও Collector

Node Exporter অনেকগুলো `collector` নিয়ে গঠিত — প্রতিটি collector একটি বিষয় মাপে। প্রধান collector-গুলো: `cpu` (`node_cpu_seconds_total{mode="idle|user|system|iowait"}` — একটি Counter, তাই `rate()` দিয়ে ব্যবহার করুন), `meminfo` (`node_memory_MemTotal_bytes`, `node_memory_MemAvailable_bytes` — Gauge), `filesystem` (`node_filesystem_avail_bytes{mountpoint="/"}`), `diskstats` (ডিস্ক I/O), `netdev` (নেটওয়ার্ক বাইট), `loadavg` (`node_load1` ইত্যাদি), `systemd` (সার্ভিসের অবস্থা)। অপ্রয়োজনীয় collector বন্ধ করা যায় `--no-collector.xxx` ফ্ল্যাগ দিয়ে — যেমন `ethtool` বা `wifi` সার্ভারে দরকার নেই। Node Exporter-এর মেট্রিকের নামে `node_` উপসর্গ থাকে, যাতে অন্য Exporter-এর সঙ্গে নাম না মেলে। একটি সাধারণ ভুল: `node_memory_MemFree_bytes` দিয়ে মেমরি ব্যবহার মাপা — Linux ফাঁকা মেমরি ক্যাশে হিসেবে ব্যবহার করে, তাই `MemAvailable_bytes`-ই সঠিক পরিমাপ।

Blackbox Exporter

Blackbox Exporter ভেতর থেকে নয়, বাইরে থেকে পরীক্ষা করে: একটি URL-এ HTTP অনুরোধ পাঠিয়ে দেখে সাড়া আসে কি না, কত সময় লাগে, TLS সার্টিফিকেটের মেয়াদ কতদিন আছে। এটি সেই সেবাগুলোর জন্য দরকার, যেগুলোতে নিজে Exporter বসানো যায় না — যেমন তৃতীয় পক্ষের API বা জনসাধারণের ওয়েবসাইট। প্রধান মেট্রিক: `probe_success` (১ মানে সফল), `probe_duration_seconds`, `probe_http_status_code`, `probe_ssl_earliest_cert_expiry`। অধ্যায় ৭-এর `relabeling` উদাহরণে দেখেছেন কীভাবে Prometheus Blackbox-কে বলে কোন URL পরীক্ষা করতে হবে।

Application Instrumentation — prom-client

Node.js-এ `prom-client` সবচেয়ে প্রচলিত লাইব্রেরি। তিনটি মেট্রিক দিয়ে শুরু করুন: অনুরোধের `Counter` (`http_requests_total`), লেটেন্সির `Histogram` (`http_request_duration_seconds`), চলমান অনুরোধের `Gauge` (`http_requests_in_flight`)। `Histogram`-এর বাকেট আপনার লেটেন্সি অনুযায়ী বাছুন — API-র জন্য ০.০০৫ থেকে ১০ সেকেন্ডের বাকেট সাধারণ। ডাটাবেস কুয়েরির জন্যও আলাদা `Histogram` রাখুন — অ্যাপ ধীর হলে বোঝা যাবে ধীরগতি অ্যাপে না ডাটাবেসে। `prom-client` ডিফল্ট মেট্রিকও দেয় — `process_resident_memory_bytes`, `nodejs_heap_size_used_bytes`, ইভেন্ট লুপের বিলম্ব (`nodejs_eventloop_lag_seconds`) — Node.js-এর মেমরি লিক ও ইভেন্ট-লুপ ব্লক ধরতে এগুলো অমূল্য।

Pushgateway — কখন, কখন নয়

`Pushgateway` স্বল্পস্থায়ী কাজের জন্য — যেমন প্রতি রাতে চলা একটি ব্যাচ জব, যা শেষ হওয়ার আগেই Prometheus `scrape` করার সুযোগ পায় না। জবটি শেষে তার মেট্রিক Pushgateway-এ ঠেলে দেয়, আর Prometheus সেখান থেকে `scrape` করে। কিন্তু দীর্ঘস্থায়ী সার্ভিসের জন্য Pushgateway ব্যবহার করবেন না — কারণ Pushgateway-তে শেষ পাঠানো মানটি থেকে যায়, এমনকি সার্ভিস বন্ধ হয়ে গেলেও; Prometheus তখন মনে করে সার্ভিস এখনো চলছে ও পুরোনো মানটি সত্য। এ ছাড়া Pushgateway-তে `instance` লেবেলের অর্থ বদলে যায়, আর একাধিক সার্ভিসের মেট্রিক মিশে যেতে পারে। নিয়ম: Pushgateway কেবল ব্যাচ জবের জন্য; সার্ভিসের জন্য সরাসরি `/metrics`।

Exporter / পদ্ধতি	পোর্ট	মাপে	কখন ব্যবহার
Node Exporter	9100	CPU, মেমরি, ডিস্ক, নেটওয়ার্ক	প্রতিটি Linux হোস্টে
Blackbox Exporter	9115	বাইরে থেকে HTTP/DNS/TCP পরীক্ষা	বাইরের সেবা ও TLS মেয়াদ
prom-client (/metrics)	অ্যাপের পোর্ট	অনুরোধ, লেটেন্সি, ত্রুটি	নিজের অ্যাপে
MongoDB Exporter	9216	সংযোগ, কুয়েরি, লক	MongoDB পর্যবেক্ষণে
Pushgateway	9091	ব্যাচ জবের শেষ মান	শুধু স্বল্পস্থায়ী জব

৯.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# Node Exporter চালানো (systemd সার্ভিসের অংশ)
node_exporter --web.listen-address=:9100 \
  --collector.systemd \              # systemd সার্ভিসের অবস্থা সংগ্রহ করবে
  --no-collector.wifi \              # সার্ভারে wifi collector দরকার নেই
  --no-collector.ethtool             # ethtool collector বন্ধ

# Docker Compose-এ Node Exporter
  node-exporter:
```

```
    image: prom/node-exporter:latest
    ports: ['9100:9100']
    volumes:
      - /proc:/host/proc:ro          # শুধু পড়ার জন্য /proc মাউন্ট
      - /sys:/host/sys:ro
    command: ['--path.procfs=/host/proc', '--path.sysfs=/host/sys']

# Prometheus-এ job যোগ করা
  - job_name: 'node'
    static_configs:
      - targets: ['203.0.113.10:9100', '203.0.113.11:9100']
```

`/proc` ও `/sys` শুধু পড়ার জন্য (`ro`) মাউন্ট করা হয় — Exporter-এর লেখার প্রয়োজন নেই, আর শুধু-পড়া মাউন্ট নিরাপত্তার জন্য ভালো। `--path.procfs` ফ্ল্যাগটি Docker-এ অপরিহার্য — নইলে Exporter কনটেইনারের নিজের `/proc` পড়বে, হোস্টের নয়। এই ভুলটি খুব সাধারণ: Docker-এ Node Exporter চালিয়ে হোস্টের বদলে কনটেইনারের CPU দেখা যায়।

৯.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র Express অ্যাপে সম্পূর্ণ instrumentation — `src/metrics.js` ও `src/app.js`-এর অংশ:

```
// src/metrics.js
const client = require('prom-client');
client.collectDefaultMetrics();   // Node.js ডিফল্ট মেট্রিক (মেমরি, ইভেন্ট লুপ)

const httpRequests = new client.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'path', 'status'],
});
const inFlight = new client.Gauge({
  name: 'http_requests_in_flight',
  help: 'Current in-flight HTTP requests',
});
const httpDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP request latency in seconds',
  labelNames: ['method', 'path'],
  buckets: [0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5, 5, 10],
});
const dbDuration = new client.Histogram({
  name: 'db_query_duration_seconds',
  help: 'MongoDB query latency in seconds',
  labelNames: ['collection', 'operation'],
  buckets: [0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1],
});
module.exports = { httpRequests, inFlight, httpDuration, dbDuration, register:
client.register };

// src/app.js — middleware
const metrics = require('./metrics');
app.use((req, res, next) => {
  const end = metrics.httpDuration.startTimer({ method: req.method, path: req.route?.path ||
req.path });
  metrics.inFlight.inc();
  res.on('finish', () => {
```

```
    end();   // Histogram-এ সময় রেকর্ড
    metrics.inFlight.dec();
    metrics.httpRequests.inc({ method: req.method, path: req.route?.path || req.path, status:
String(res.statusCode) });
  });
  next();
});
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', metrics.register.contentType);
  res.end(await metrics.register.metrics());
});
```

এখানে `req.route?.path` ব্যবহার করা হয়েছে — যাতে `path` লেবেলে রুটের প্যাটার্ন (`/api/orders/:id`) যায়, আসল ID নয়। `startTimer()` একটি ফাংশন ফেরত দেয়; সেটি কল করলেই সময় রেকর্ড হয় — ভুলে গেলে `Histogram`-এ কিছু জমা হবে না। `dbDuration`-এর বাকেট ছোট — ডাটাবেস কুয়েরি সাধারণত মিলিসেকেন্ডে শেষ হয়, তাই ১ মিলিসেকেন্ড থেকে শুরু করা হয়েছে।

৯.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র MongoDB একবার ধীর হয়ে গেল — `db_query_duration_seconds`-এর p95 ২০ মিলিসেকেন্ড থেকে ৮০০ মিলিসেকেন্ডে উঠল, অথচ অ্যাপের CPU ও মেমরি স্বাভাবিক ছিল। MongoDB Exporter দেখাল সংযোগের সংখ্যা হঠাৎ বেড়ে গেছে — একটি নতুন ডিপ্লয়ে সংযোগ পুলের সীমা না মেনে প্রতিটি অনুরোধে নতুন সংযোগ খুলছিল। সংযোগ পুল ঠিক করতেই লেটেন্সি স্বাভাবিক হলো। এই ঘটনায় তিন স্তরের মেট্রিক একসঙ্গে কাজ করেছে: হোস্ট (Node Exporter), ডাটাবেস (MongoDB Exporter) ও অ্যাপ (prom-client)। একটি স্তর না থাকলে তদন্ত অসম্পূর্ণ থাকত। আরেকটি পরিস্থিতি: Blackbox Exporter এক রাতে জানাল `probe_ssl_earliest_cert_expiry` ৭ দিনের নিচে নেমে এসেছে — TLS সার্টিফিকেটের মেয়াদ শেষ হতে চলেছে। সময়মতো নবায়ন করা হলো; নইলে গ্রাহকরা "নিরাপদ নয়" সতর্কতা দেখতেন।

৯.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Docker Compose-এ Node Exporter চালু করুন; `curl localhost:9100/metrics` দিয়ে `node_cpu_seconds_total` দেখুন। (নমুনা আউটপুট — চিত্রণমূলক)
2. Prometheus-এ `node` job যোগ করুন; `100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))` কুয়েরি করে CPU ব্যবহার দেখুন।
3. Express অ্যাপে উপরের `src/metrics.js` যোগ করুন; `/metrics` খুলে আপনার তিনটি মেট্রিক দেখুন।
4. কৃত্রিম লোড পাঠিয়ে p95 লেটেন্সি ও ত্রুটির হার Grafana-তে নয়, Prometheus UI-তে দেখুন।
5. Blackbox Exporter চালু করুন; `probe_success` দিয়ে `https://technova.example`-এর স্বাস্থ্য পরীক্ষা করুন।
6. একটি ছোট ব্যাচ স্ক্রিপ্ট লিখুন যা Pushgateway-এ মেট্রিক পাঠায়; তারপর ব্যাখ্যা করুন কেন এই স্ক্রিপ্টটি সার্ভিস হিসেবে Pushgateway ব্যবহার করলে ভুল হবে।

৯.১১ সাধারণ ভুল ও ভুল ধারণা

- **Docker-এ /proc মাউন্ট না করা:** Exporter কনটেইনারের মেট্রিক দেখায়, হোস্টের নয়।
- **path লেবেলে আসল ID বসানো:** সিরিজ-বিস্ফোরণ ঘটে; রুটের প্যাটার্ন ব্যবহার করুন।
- **MemFree দিয়ে মেমরি ব্যবহার মাপা:** `MemAvailable_bytes`-ই সঠিক পরিমাপ।
- **Pushgateway দিয়ে সার্ভিস মাপা:** বন্ধ সার্ভিসের পুরোনো মান থেকে যায়; শুধু ব্যাচ জবের জন্য ব্যবহার করুন।

- **Histogram-এর বাকেট না ভাবা:** ডাটাবেস ও HTTP-র বাকেট আলাদা হওয়া উচিত — তাদের লেটেন্সির পরিসর আলাদা।
- **/metrics খোলা রাখা:** প্রোডাকশনে শুধু Prometheus-এর নেটওয়ার্ক থেকে পৌঁছানো যায় এমনভাবে রাখুন।
- **startTimer()-এর ফেরত ফাংশন না ডাকা:** Histogram-এ কিছু জমা হয় না।

৯.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Exporter-গুলো সিস্টেমের অভ্যন্তরীণ তথ্য প্রকাশ করে — চলমান প্রক্রিয়া, সংস্করণ, নেটওয়ার্ক সংযোগ। এগুলো জনসাধারণের জন্য খোলা রাখবেন না। Node Exporter-এর পোর্ট ৯১০০ ফায়ারওয়াল দিয়ে শুধু Prometheus সার্ভারের জন্য খুলুন। `/metrics` এন্ডপয়েন্টে কখনো গোপন তথ্য — টোকেন, পাসওয়ার্ড, ব্যক্তিগত শনাক্তকারী — লেবেল হিসেবে বসাবেন না। MongoDB Exporter-এর জন্য শুধু পড়ার অনুমতিসহ আলাদা ডাটাবেস ব্যবহারকারী তৈরি করুন; অ্যাডমিন ব্যবহারকারী দেবেন না। Blackbox Exporter দিয়ে অভ্যন্তরীণ সেবা পরীক্ষা করলে তার ফলাফলও অভ্যন্তরীণ নেটওয়ার্কে রাখুন।

> **WARNING:** Node Exporter-এর `textfile collector` দিয়ে নিজের মেট্রিক ফাইল লেখা যায় — কিন্তু সেই ফাইলে কখনো গোপন তথ্য লিখবেন না। ফাইলটি যে কেউ পড়তে পারে, আর তার বিষয়বস্তু Prometheus-এ চলে যায়।

৯.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Node Exporter হালকা — সাধারণত কয়েক মেগাবাইট মেমরি ও নগণ্য CPU। কিন্তু collector বেশি চালু করলে খরচ বাড়ে; অপ্রয়োজনীয় collector বন্ধ করুন। অ্যাপের instrumentation-এর খরচও ছোট, তবে `Histogram`-এর প্রতিটি পর্যবেক্ষণে কিছু CPU লাগে — খুব বেশি ট্র্যাফিকে (প্রতি সেকেন্ডে লাখো অনুরোধ) বাকেটের সংখ্যা কম রাখুন। `collectDefaultMetrics()` প্রতি ১০ সেকেন্ডে চলে; তা যথেষ্ট। সবচেয়ে বড় খরচ আবারও `cardinality` — লেবেলের মান সীমিত রাখুন। `scrape_interval` ১৫ সেকেন্ড Node Exporter-এর জন্য যথেষ্ট; ৫ সেকেন্ড করলে ডেটা তিন গুণ হয়, অথচ হোস্টের মেট্রিক ধীরে বদলায়।

৯.১৪ সমস্যা সমাধান (Troubleshooting)

Node Exporter-এর মেট্রিক আসছে না: পোর্ট ৯১০০ খোলা আছে কি না `curl` দিয়ে পরীক্ষা করুন; Docker-এ `/proc` মাউন্ট হয়েছে কি না দেখুন; Prometheus-এর `Targets` পৃষ্ঠায় ত্রুটির বার্তা পড়ুন।

অ্যাপের মেট্রিক ফাঁকা: `/metrics` খুলে দেখুন — মেট্রিক আছে কি না। না থাকলে middleware সব রুটে চলছে কি না, আর `Histogram`-এর `end()` ডাকা হচ্ছে কি না পরীক্ষা করুন।

CPU ব্যবহার ১০০%-এর বেশি দেখাচ্ছে: `node_cpu_seconds_total` একটি Counter — সরাসরি মান নয়, `rate()` দিয়ে ব্যবহার করুন; আর একাধিক কোরের গড় নিতে `avg()` ব্যবহার করুন।

Blackbox probe ব্যর্থ: Blackbox সার্ভার থেকে লক্ষ্য URL-এ `curl` করে দেখুন; TLS সমস্যা হলে `probe_ssl_earliest_cert_expiry` ও সার্টিফিকেট পরীক্ষা করুন।

> **TROUBLESHOOTING:** Exporter-এর নিজের লগ দেখুন — `journalctl -u node_exporter -f`। collector-এর ত্রুটি সেখানে লেখা থাকে; যেমন কোনো `/proc` ফাইল পড়া যাচ্ছে না।

৯.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি হোস্টে Node Exporter চালান; RED পদ্ধতির চারটি মেট্রিক (Counter, Histogram, Gauge, ত্রুটি) প্রতিটি সার্ভিসে রাখুন; `path` লেবেলে রুটের প্যাটার্ন ব্যবহার করুন; ডাটাবেসের জন্য আলাদা Histogram রাখুন; Blackbox দিয়ে বাইরে থেকে গুরুত্বপূর্ণ এন্ডপয়েন্ট পরীক্ষা করুন; Pushgateway শুধু ব্যাচ জবের জন্য ব্যবহার করুন।

অ্যান্টি-প্যাটার্ন: প্রতিটি ফাংশনে মেট্রিক বসানো; লেবেলে `user_id` বসানো; Pushgateway দিয়ে সার্ভিস মাপা; `/metrics` জনসাধারণের জন্য খোলা রাখা; Docker-এ `/proc` মাউন্ট না করে হোস্টের মেট্রিক আশা করা।

৯.১৬ অধ্যায়ের সারাংশ

`Exporter` হলো Prometheus-এর ভাষায় অনুবাদক: Node Exporter হোস্টের মেট্রিক দেয়, Blackbox Exporter বাইরে থেকে সেবা পরীক্ষা করে, MongoDB Exporter ডাটাবেসের পরিসংখ্যান আনে। নিজের অ্যাপের জন্য `prom-client` দিয়ে `Counter`, `Gauge` ও `Histogram` লিখতে হয় — অনুরোধের হার, লেটেন্সির বণ্টন ও চলমান অনুরোধ — আর `/metrics` এন্ডপয়েন্টে সেগুলো প্রকাশ করতে হয়। `Pushgateway` কেবল স্বল্পস্থায়ী ব্যাচ জবের জন্য; দীর্ঘস্থায়ী সার্ভিসের জন্য সরাসরি `scrape`-ই সঠিক পথ। লেবেলে রুটের প্যাটার্ন ব্যবহার করা, `MemAvailable_bytes` দিয়ে মেমরি মাপা ও Exporter-এর পোর্ট সীমিত রাখা — এই তিনটি অভ্যাস অনেক সমস্যা থেকে বাঁচায়।

৯.১৭ মূল প্রযুক্তিগত পরিভাষা

Exporter: কোনো সিস্টেমের মেট্রিক Prometheus-এর exposition format-এ প্রকাশকারী ছোট HTTP সার্ভার।

Node Exporter: Linux হোস্টের CPU, মেমরি, ডিস্ক ও নেটওয়ার্ক মেট্রিক দেওয়া Exporter (পোর্ট ৯১০০)।

Collector: Node Exporter-এর ভেতরে একটি বিষয় মাপার অংশ; যেমন cpu, meminfo, filesystem collector।

Blackbox Exporter: বাইরে থেকে HTTP, DNS বা TCP পরীক্ষা করে সেবার স্বাস্থ্য মাপা Exporter (পোর্ট ৯১১৫)।

Instrumentation: অ্যাপের কোডে মেট্রিক সংগ্রহের ব্যবস্থা যোগ করা।

prom-client: Node.js-এর জন্য Prometheus ক্লায়েন্ট লাইব্রেরি; Counter, Gauge, Histogram তৈরি করে।

Pushgateway: স্বল্পস্থায়ী জবের মেট্রিক গ্রহণ করে Prometheus-এর scrape-এর জন্য রাখা সার্ভার (পোর্ট ৯০৯১)।

RED Method: প্রতিটি সার্ভিসের Rate, Errors ও Duration মাপার পদ্ধতি।

USE Method: প্রতিটি সম্পদের Utilization, Saturation ও Errors মাপার পদ্ধতি।

Default Metrics: prom-client-এর দেওয়া Node.js প্রক্রিয়ার মেট্রিক; যেমন মেমরি ও ইভেন্ট-লুপ বিলম্ব।

৯.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Exporter কী? Prometheus নিজে কেন সব সিস্টেমের মেট্রিক সরাসরি পড়তে পারে না?
2. Node Exporter-এর cpu collector-এর মেট্রিকটি Counter না Gauge? তা দিয়ে CPU ব্যবহার বের করতে কোন ফাংশন লাগে?
3. MemFree-এর বদলে MemAvailable দিয়ে মেমরি ব্যবহার মাপা উচিত কেন?
4. Blackbox Exporter কখন দরকার হয়? probe_success মেট্রিকটি কী বলে?
5. path লেবেলে req.route?.path ব্যবহার করা কেন গুরুত্বপূর্ণ?
6. Pushgateway দিয়ে দীর্ঘস্থায়ী সার্ভিস মাপলে কী ভুল হয়?
7. Docker-এ Node Exporter চালাতে /proc মাউন্ট করা কেন দরকার? না করলে কী দেখা যাবে?
8. ডাটাবেস কুয়েরির Histogram-এর বাকেট HTTP লেটেন্সির বাকেট থেকে আলাদা হওয়া উচিত কেন?
9. collectDefaultMetrics() কী দেয়? ইভেন্ট-লুপ বিলম্ব কেন গুরুত্বপূর্ণ?
10. একটি নতুন সার্ভিসে প্রথমে কোন চারটি মেট্রিক লিখবেন? কেন?

৯.১৯ ব্যবহারিক অনুশীলন

1. Docker Compose-এ Node Exporter চালু করুন ও CPU, মেমরি ও ডিস্কের তিনটি কুয়েরি লিখুন।
2. Express অ্যাপে prom-client দিয়ে Counter, Gauge ও Histogram যোগ করুন; /metrics পড়ে প্রতিটি লাইন ব্যাখ্যা করুন।
3. একটি ধীর এন্ডপয়েন্ট তৈরি করুন; p95 লেটেন্সি কীভাবে বাড়ে তা দেখান।
4. Blackbox Exporter দিয়ে একটি ওয়েবসাইটের probe_success ও probe_duration_seconds পরীক্ষা করুন।
5. একটি ব্যাচ স্ক্রিপ্ট Pushgateway-এ মেট্রিক পাঠান; তারপর ব্যাখ্যা লিখুন কেন সার্ভিসের জন্য এটি ভুল পথ।
6. আপনার অ্যাপের জন্য একটি instrumentation পরিকল্পনা লিখুন: কোন মেট্রিক, কোন ধরন, কোন লেবেল, কোন বাকেট — ও কেন।

অধ্যায় ১০ • PART II — Metrics and Prometheus Engineering

Chapter 10 — Alertmanager, Alert Rules, Routing, and Notifications

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি `alerting rule` লিখতে পারবেন — `for` সময়কাল, লেবেল ও `annotation` সহ — Alertmanager-এর `routing tree`, `grouping`, `inhibition` ও `silence` ব্যাখ্যা করতে পারবেন, ইমেইল, Slack ও webhook `receiver` কনফিগার করতে পারবেন এবং `alert fatigue` এড়িয়ে কার্যকর (actionable) অ্যালার্ট ডিজাইন করতে পারবেন।

- Alerting rule-এর গঠন ও for-এর ভূমিকা বোঝা।
- Routing tree দিয়ে সঠিক দলে অ্যালার্ট পাঠানো।
- Grouping ও inhibition দিয়ে শব্দদূষণ কমানো।
- Silence ও notification template ব্যবহার করা।
- Burn-rate ভিত্তিক SLO অ্যালার্ট লেখা।

১০.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

অ্যালার্ট হলো মনিটরিং-এর উদ্দেশ্য — সংখ্যা সংগ্রহ করা নয়, সঠিক সময়ে সঠিক মানুষকে জাগানো। TechNova-র একবার এমন হয়েছিল: একটি সার্ভিস বন্ধ হয়ে গিয়েছিল, কিন্তু কেউ জানতে পারেনি — কারণ অ্যালার্ট ছিল না। আরেকবার উল্টোটা হয়েছিল: এক রাতে ২০০টি অ্যালার্ট এসেছিল, সবই একই সমস্যার — একটি ডাটাবেস বন্ধ হওয়ায় প্রতিটি সার্ভিস আলাদা অ্যালার্ট পাঠিয়েছিল। অন-কল প্রকৌশলী ফোন নীরব করে ঘুমিয়ে পড়েছিলেন, আর আসল সমস্যাটি সকালে ধরা পড়েছিল। এই দুটি ঘটনাই অ্যালার্ট ডিজাইনের ব্যর্থতা। ভালো অ্যালার্ট মানে: ব্যবহারকারীর ক্ষতি হওয়ার আগেই খবর, প্রতিটি অ্যালার্টে করণীয় লেখা, আর একই ঘটনায় একটি মাত্র বিজ্ঞপ্তি। এই অধ্যায় সেই নকশাই শেখায়।

১০.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬–৮ জানা থাকতে হবে — মেট্রিক ধরন, `PromQL`, `rate()`, `histogram_quantile()` ও `recording rule`। এ ছাড়া প্রয়োজন: YAML-এর ধারণা, ইমেইল ও Slack-এর মতো বিজ্ঞপ্তি মাধ্যমের সাধারণ পরিচিতি, এবং SLO ও `error budget`-এর ধারণা (Part I)। Alertmanager আগে চালানো না থাকলেও চলবে — এই অধ্যায়েই তার কনফিগারেশন আছে।

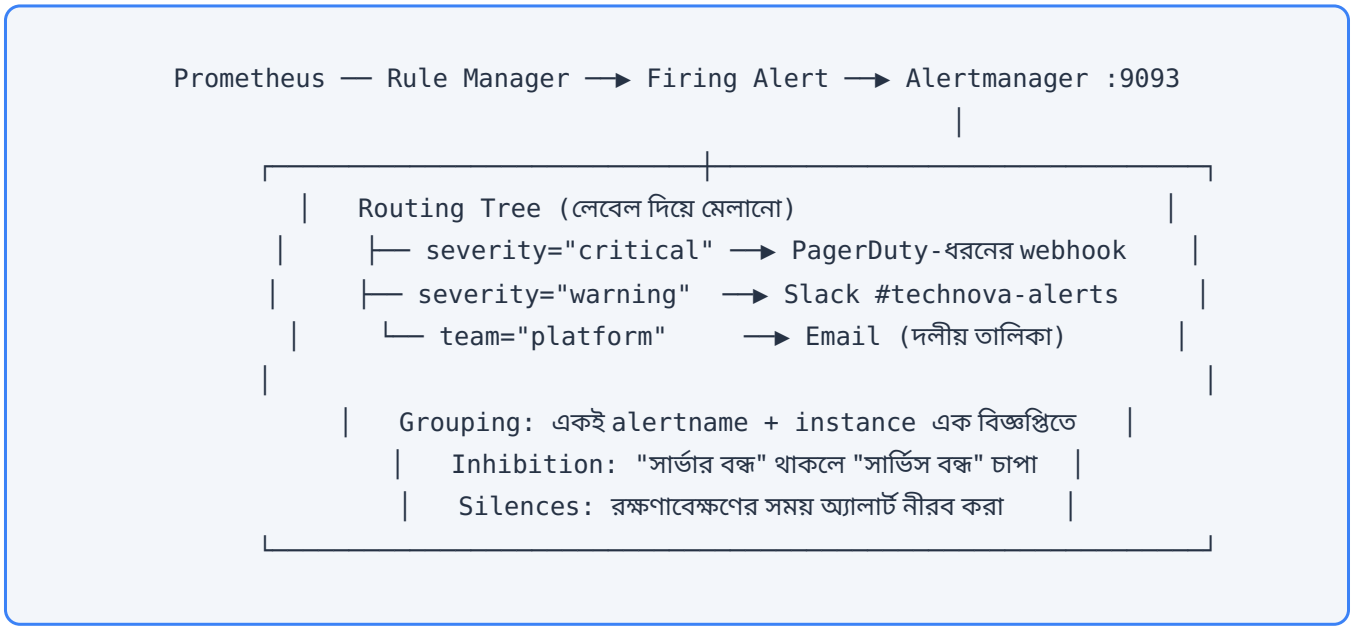
১০.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: অ্যালার্ট কখন দেওয়া উচিত? যখন কোনো মানুষের কিছু করা দরকার, আর তা এখনই করা দরকার। এই দুটি শর্তই মূল কথা। CPU ৯০% হলেই অ্যালার্ট দেওয়া উচিত নয় — যদি ব্যবহারকারীর কোনো ক্ষতি না হয়, আর কেউ এখনই কিছু করতে না পারে, তাহলে সেটি অ্যালার্ট নয়, সেটি একটি টিকিট বা ড্যাশবোর্ডের বিষয়। এই নীতির নাম `actionable alert` — প্রতিটি অ্যালার্টের উত্তর থাকতে হবে: "কী করব?" উত্তর না থাকলে অ্যালার্টটি মুছে ফেলুন বা তার সীমা বদলান। দ্বিতীয় নীতি: অ্যালার্ট উপসর্গে (symptom) দেওয়া উচিত, কারণে নয়। ব্যবহারকারী যা অনুভব করে — ধীরগতি, ত্রুটি, সেবা বন্ধ — সেটাই উপসর্গ। CPU বেশি হওয়া একটি কারণ হতে পারে, কিন্তু ব্যবহারকারী CPU দেখেন না;

তিনি দেখেন ধীরগতি। তাই অ্যালার্ট লিখুন উপসর্গের ওপর — ত্রুটির হার, লেটেন্সি, সেবার উপলভ্যতা — আর কারণ খুঁজুন ড্যাশবোর্ডে।

> **CONCEPT:** Prometheus অ্যালার্ট তৈরি করে, Alertmanager অ্যালার্ট পাঠায় — এই ভাগটি ইচ্ছাকৃত। Prometheus জানে কখন একটি শর্ত সত্য হয়েছে; Alertmanager জানে কাকে, কখন, কীভাবে জানাতে হবে, একই ঘটনার অ্যালার্ট একত্র করতে হবে কি না, আর কোন অ্যালার্ট চাপা দিতে হবে। এই ভাগের কারণে একই Prometheus থেকে অনেক দলে ভিন্ন ভিন্ন নিয়মে বিজ্ঞপ্তি পাঠানো যায়।

১০.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



অ্যালার্টের জীবনচক্রটি লক্ষ্য করুন। `Rule Manager` প্রতি `evaluation_interval`-এ `alerting rule`-এর `expr` মূল্যায়ন করে। শর্ত সত্য হলে অ্যালার্টটি প্রথমে `pending` অবস্থায় যায়; `for` সময়কাল ধরে সত্য থাকলে তবেই `firing` হয়। `Firing` অ্যালার্ট Alertmanager-এ পাঠানো হয়, যেখানে `routing tree` লেবেল দেখে ঠিক করে কোন `receiver`-এ পাঠাবে। `Grouping` একই ধরনের অ্যালার্ট একত্র করে — যেমন দশটি সার্ভারের একই অ্যালার্ট একটি বিজ্ঞপ্তিতে আসে। `Inhibition` একটি অ্যালার্ট দিয়ে আরেকটি অ্যালার্ট চাপা দেয় — যেমন পুরো সার্ভার বন্ধ থাকলে তার ওপরের প্রতিটি সার্ভিসের অ্যালার্ট চাপা পড়ে। `Silence` হলো সাময়িক নীরবতা — রক্ষণাবেক্ষণের সময় অ্যালার্ট আসবে না, কিন্তু Alertmanager-এর UI-তে তা দেখা যাবে।

১০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Alerting Rule — expr, for, labels, annotations

একটি `alerting rule`-এর চারটি অংশ: `expr` (PromQL শর্ত), `for` (কতক্ষণ সত্য থাকলে firing হবে), `labels` (অ্যালার্টের লেবেল — যেমন `severity`), `annotations` (মানুষের পড়ার মতো বার্তা — `summary` ও `description`)। `for` খুব গুরুত্বপূর্ণ: `for: 5m` মানে শর্ত পাঁচ মিনিট ধরে সত্য থাকলে তবেই অ্যালার্ট যাবে — ক্ষণস্থায়ী স্পাইকে ভুল অ্যালার্ট এড়ানো যায়। `annotations`-এ `runbook`-এর লিংক দিন — "এই অ্যালার্ট এলে কী করবেন" — যাতে রাত তিনটায় জেগে ওঠা প্রকৌশলীকে ভাবতে না হয়। `labels`-এ `severity` ছাড়াও `team` রাখুন, যাতে `routing tree` সঠিক দলে পাঠাতে পারে।

Routing Tree

`Routing tree` একটি গাছের মতো: মূল (root) `route` থেকে শাখা বের হয়, প্রতিটি শাখায় `matchers` থাকে। Alertmanager উপর থেকে নিচে যায়; প্রথম যে শাখার `matchers` মেলে, সেই শাখার `receiver`-এ পাঠায়। `continue: true` দিলে মিলে যাওয়া শাখার পরেও পরের শাখাগুলো পরীক্ষা করা হয় — একই অ্যালার্ট একাধিক জায়গায় পাঠাতে চাইলে এটি দরকার। `group_by` নির্ধারণ করে কোন লেবেল দিয়ে অ্যালার্ট একত্র করা হবে; `group_wait` (প্রথম অ্যালার্টের পর কতক্ষণ অপেক্ষা করে একত্র করবে), `group_interval` (পরের বিজ্ঞপ্তিগুলোর বিরতি) ও `repeat_interval` (একই অ্যালার্ট কতক্ষণ পর আবার পাঠাবে) — এই তিনটি সময় শব্দদূষণ নিয়ন্ত্রণ করে। `repeat_interval` খুব ছোট হলে একই অ্যালার্ট বারবার আসবে; খুব বড় হলে চলমান সমস্যা ভুলে যাওয়া যাবে। সাধারণ মান: `critical`-এর জন্য ৪ ঘণ্টা, `warning`-এর জন্য ১২ ঘণ্টা।

Inhibition ও Silences

`Inhibition` নিয়মে দুটি অ্যালার্টের নাম দেওয়া থাকে: `source` (যে অ্যালার্ট চাপা দেয়) ও `target` (যে অ্যালার্ট চাপা পড়ে), আর `equal` লেবেল দিয়ে বলা হয় কোন লেবেল মিললে নিয়মটি প্রযোজ্য। উদাহরণ: `NodeDown` অ্যালার্ট firing থাকলে একই `instance`-এর `ServiceDown` অ্যালার্ট চাপা পড়বে — কারণ সার্ভার বন্ধ থাকলে সার্ভিস তো বন্ধ থাকবেই; দুটি অ্যালার্টের দরকার নেই। `Silence` হলো হাতে তৈরি নীরবতা: Alertmanager-এর UI বা API থেকে একটি সময় ও `matchers` দিয়ে অ্যালার্ট নীরব করা যায়। `Silence`-এর মেয়াদ শেষ হলে অ্যালার্ট আবার আসে — তাই স্থায়ী সমস্যা লুকিয়ে রাখা যায় না। রক্ষণাবেক্ষণের সময় `silence` ব্যবহার করুন, কিন্তু তার মেয়াদ কাজের চেয়ে বেশি রাখবেন না।

Receivers ও Notification Templates

`Receiver` হলো বিজ্ঞপ্তির গন্তব্য: ইমেইল, Slack, PagerDuty-ধরনের webhook, বা অন্য যেকোনো webhook। Slack-এর জন্য `slack_api_url` লাগে — সেটি গোপন তথ্য, তাই কনফিগারেশনে `CHANGE-ME-SLACK-WEBHOOK-URL` আকারে প্লেসহোল্ডার রাখুন, আসল মান গোপন ব্যবস্থায় রাখুন। `Notification template` দিয়ে বার্তার চেহারা বদলানো যায় — Go-র টেমপ্লেট ভাষায় লেখা হয়। ভালো বার্তায় থাকে: অ্যালার্টের নাম, `severity`, কোন সার্ভিস, কখন শুরু হয়েছে, বর্তমান মান, আর `runbook`-এর লিংক। খারাপ বার্তায় থাকে শুধু "অ্যালার্ট!" — তা পড়ে কেউ কিছু করতে পারে না।

Alert Fatigue ও Burn-Rate অ্যালার্ট

`Alert fatigue` হলো সেই অবস্থা, যখন এত অ্যালার্ট আসে যে মানুষ সেগুলো উপেক্ষা করতে শুরু করে — আর তখন আসল বিপদও উপেক্ষিত হয়। এর প্রতিকার: কম অ্যালার্ট লিখুন, প্রতিটি অ্যালার্ট `actionable` করুন, `for` দিয়ে ক্ষণস্থায়ী স্পাইক বাদ দিন, `grouping` দিয়ে একত্র করুন, আর নিয়মিত অ্যালার্টগুলো পর্যালোচনা করুন — যে অ্যালার্টে কেউ কখনো কিছু করেনি, সেটি মুছে ফেলুন বা সীমা বদলান। SLO `burn-rate` অ্যালার্ট এই নীতির ভালো উদাহরণ: `burn rate 14` হলে (বাজেট ১৪ গুণ দ্রুত পুড়ছে) `critical` অ্যালার্ট, `burn rate 6` হলে `warning` — আর দুটি ভিন্ন সময়-উইন্ডো (১ ঘণ্টা ও ৬ ঘণ্টা) একসঙ্গে পরীক্ষা করা হয়, যাতে ক্ষণস্থায়ী স্পাইকে ভুল অ্যালার্ট না যায়।

উপাদান	কাজ	সাধারণ মান
for	কতক্ষণ সত্য থাকলে firing	critical: 5m, warning: 10m
group_wait	প্রথম বিজ্ঞপ্তির আগে অপেক্ষা	30s
group_interval	পরের বিজ্ঞপ্তির বিরতি	5m
repeat_interval	একই অ্যালার্ট আবার পাঠানো	critical: 4h, warning: 12h
severity	অ্যালার্টের গুরুত্ব	critical / warning / info

১০.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# rules/alerts.yml — একটি alerting rule
groups:
  - name: technova-api
    rules:
      - alert: ApiHighErrorRate          # অ্যালার্টের নাম — স্পষ্ট ও বর্ণনামূলক
        expr: |                          # PromQL শর্ত
          100 * sum(rate(http_requests_total{status=~"5.."}[5m]))
            / sum(rate(http_requests_total[5m])) > 5
        for: 5m                          # ৫ মিনিট ধরে সত্য থাকলে firing
        labels:
          severity: critical             # routing tree এই লেবেল দেখে পাঠায়
          team: platform
        annotations:
          summary: "API-তে 5xx ত্রুটির হার ৫%-এর বেশি ({{ $labels.instance }})"
          description: "গত ৫ মিনিটে ত্রুটির হার {{ $value }}% — runbook দেখুন।"
          runbook: "https://technova.example/runbooks/api-high-error-rate"

# alertmanager.yml — routing ও receiver
global:
  resolve_timeout: 5m                    # সমস্যা মিটলে "resolved" জানানোর অপেক্ষা
route:
  receiver: 'team-platform'              # ডিফল্ট receiver
  group_by: ['alertname', 'instance']    # একই নাম ও instance একত্র হবে
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h
  routes:
    - matchers: ['severity="critical"']
      receiver: 'pagerduty-webhook'
      continue: true                     # critical অ্যালার্ট Slack-এও যাবে
    - matchers: ['severity="critical"']
      receiver: 'slack-critical'
receivers:
  - name: 'team-platform'
    email_configs:
      - to: 'platform@technova.example'
        from: 'alerts@technova.example'
        smarthost: 'smtp.technova.example:587'
  - name: 'slack-critical'
    slack_configs:
      - api_url: 'CHANGE-ME-SLACK-WEBHOOK-URL'
        channel: '#technova-alerts'
  - name: 'pagerduty-webhook'
    webhook_configs:
      - url: 'https://hooks.technova.example/pagerduty/CHANGE-ME-KEY'
```

`annotations`-এ `{{ $labels.instance }}` ও `{{ $value }}` হলো টেমপ্লেট চলক — অ্যালার্ট firing হওয়ার সময় সেগুলো আসল মান দিয়ে পূর্ণ হয়। `resolve_timeout` মানে: অ্যালার্টের শর্ত মিথ্যা হওয়ার পাঁচ মিনিট পর Alertmanager "সমস্যা মিটেছে" বিজ্ঞপ্তি পাঠায়। `continue: true` ছাড়া প্রথম মিলে যাওয়া শাখাই শেষ কথা হতো — critical অ্যালার্ট শুধু webhook-এ যেত, Slack-এ যেত না। ইমেইলের `smarthost` ও Slack-এর `api_url` — দুটোই কাল্পনিক ঠিকানা; আসল মান গোপন ব্যবস্থায় রাখুন, কনফিগারেশন ফাইলে কখনো আসল গোপন তথ্য লিখবেন না।

১০.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র সম্পূর্ণ অ্যালার্ট সেট — উপসর্গ-ভিত্তিক, burn-rate সহ:

```
groups:
  - name: technova-slo
    rules:
      - alert: ApiHighErrorRate
        expr: 100 * sum(rate(http_requests_total{status=~"5.."}[5m]))
          / sum(rate(http_requests_total[5m])) > 5
        for: 5m
        labels: { severity: critical, team: platform }
        annotations:
          summary: "API ত্রুটির হার ৫%-এর বেশি"
          description: "ত্রুটির হার {{ $value }}% — runbook দেখুন।"
      - alert: ApiLatencyHigh
        expr: histogram_quantile(0.95,
          sum by (le) (rate(http_request_duration_seconds_bucket[5m]))) > 1
        for: 10m
        labels: { severity: warning, team: platform }
        annotations:
          summary: "API p95 লেটেন্সি ১ সেকেন্ডের বেশি"
          description: "p95 এখন {{ $value }} সেকেন্ড।"
      - alert: ServiceDown
        expr: up{job="technova-api"} == 0
        for: 2m
        labels: { severity: critical, team: platform }
        annotations:
          summary: "TechNova API সাড়া দিচ্ছে না ({{ $labels.instance }})"
      - alert: ErrorBudgetBurnFast
        expr: |
          sum(rate(http_requests_total{status=~"5.."}[1h]))
            / sum(rate(http_requests_total[1h])) > 0.014
          and
          sum(rate(http_requests_total{status=~"5.."}[6h]))
            / sum(rate(http_requests_total[6h])) > 0.014
        for: 5m
        labels: { severity: critical, team: platform }
        annotations:
          summary: "ত্রুটির বাজেট ১৪ গুণ দ্রুত পুড়ছে — SLO ঝুঁকিতে"
  - name: technova-infra
    rules:
      - alert: DiskSpaceLow
        expr: 100 * (1 - node_filesystem_avail_bytes{mountpoint="/"}
          / node_filesystem_size_bytes{mountpoint="/"}) > 85
        for: 10m
        labels: { severity: warning, team: platform }
        annotations:
          summary: "ডিস্ক ৮৫%-এর বেশি পূর্ণ ({{ $labels.instance }})"
```

`ErrorBudgetBurnFast` নিয়মটি লক্ষ্য করুন: ১ ঘণ্টা ও ৬ ঘণ্টা — দুটি উইন্ডোতেই ত্রুটির হার ১.৪%-এর বেশি হলে তবেই অ্যালার্ট যায়। SLO ৯৯.৯% হলে অনুমোদিত ত্রুটি ০.১%; ১.৪% মানে ১৪ গুণ দ্রুত পোড়া। দুটি উইন্ডোর শর্ত `and` দিয়ে জোড়া হয়েছে — একটি উইন্ডোতে স্পাইক হলেই অ্যালার্ট যাবে না। `DiskSpaceLow`-এর `for: 10m` মনে করিয়ে দেয়: ডিস্ক ধীরে ভরে, তাই দশ মিনিট অপেক্ষা করা যায় — তাড়াহুড়ো করে অ্যালার্ট পাঠানোর দরকার নেই।

১০.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র এক রাতে ডাটাবেস সার্ভারটি বন্ধ হয়ে গেল। প্রথমে `ServiceDown` অ্যালার্ট এল — API সাড়া দিচ্ছে না। তারপর `ApiHighErrorRate` এল। তারপর ডাটাবেসের `NodeDown` এল। তিনটি অ্যালার্ট, একটি ঘটনা। `Inhibition` নিয়ম থাকায় `NodeDown` firing হওয়ার পর `ServiceDown` চাপা পড়ল — অন-কল প্রকৌশলী একটি মাত্র বিজ্ঞপ্তি পেলেন: "ডাটাবেস সার্ভার বন্ধ"। তিনি সার্ভারটি পুনরায় চালু করলেন, আর `resolve` বিজ্ঞপ্তি এল। এই ঘটনার আগে, `inhibition` না থাকায় একই ঘটনায় ৪০টি অ্যালার্ট এসেছিল, আর প্রকৌশলী কোনটি আগে দেখবেন বুঝতে পারেননি। আরেকটি পরিস্থিতি: ব্ল্যাক ফ্রাইডে বিক্রির সময় `ApiLatencyHigh` অ্যালার্ট বারবার আসছিল — কিন্তু ট্র্যাফিক পাঁচ গুণ বেড়েছিল, আর p95 ১.২ সেকেন্ডে থাকা স্বাভাবিক ছিল। দলটি সেদিনের জন্য `silence` তৈরি করল — তবে মেয়াদ মাত্র ৬ ঘণ্টা রাখল, যাতে বিক্রি শেষ হলে অ্যালার্ট আবার চালু হয়।

১০.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Docker Compose-এ Alertmanager চালু করুন (পোর্ট ৯০৯৩); Prometheus-এর `alerting` অংশে তার ঠিকানা যোগ করুন।
2. `ApiHighErrorRate` নিয়মটি লিখুন; `promtool check rules` দিয়ে যাচাই করুন; Prometheus পুনরায় লোড করুন।
3. কৃত্রিমভাবে ৫০০ ত্রুটি তৈরি করুন; Prometheus UI-র `Alerts` পৃষ্ঠায় অ্যালার্টটি `pending` থেকে `firing` হতে দেখুন।
4. Alertmanager UI (`http://203.0.113.20:9093`) খুলে অ্যালার্টটি দেখুন; একটি `silence` তৈরি করুন ও তার মেয়াদ শেষ হলে কী হয় লক্ষ্য করুন।
5. একটি Slack webhook (পরীক্ষামূলক) অথবা ইমেইল receiver কনফিগার করুন — আসল গোপন তথ্য নয়, পরীক্ষার মান ব্যবহার করুন — ও বিজ্ঞপ্তি পাওয়ার পরীক্ষা করুন।
6. একটি `inhibition` নিয়ম লিখুন: `NodeDown` থাকলে একই `instance`-এর `ServiceDown` চাপা পড়বে; একটি সার্ভার বন্ধ করে পরীক্ষা করুন।

১০.১১ সাধারণ ভুল ও ভুল ধারণা

- **কারণে অ্যালার্ট লেখা:** CPU ৯০% হলেই অ্যালার্ট নয় — উপসর্গে (ত্রুটির হার, লেটেন্সি) অ্যালার্ট লিখুন।
- **for না দেওয়া:** ক্ষণস্থায়ী স্পাইকে ভুল অ্যালার্ট আসে; `for: 5m` দিয়ে ছাঁকুন।
- **runbook ছাড়া অ্যালার্ট:** রাত তিনটায় কেউ ভেবে কাজ করতে পারে না — প্রতিটি অ্যালার্টে করণীয় লিখুন।
- **severity সবকিছুতে critical:** সবকিছু জরুরি হলে কিছুই জরুরি নয়; সত্যিকারের জরুরি অ্যালার্ট কম রাখুন।
- **inhibition না থাকা:** এক ঘটনায় চল্লিশ অ্যালার্ট আসে; মূল কারণের অ্যালার্ট দিয়ে বাকিগুলো চাপা দিন।
- **repeat_interval খুব ছোট:** একই অ্যালার্ট বারবার এসে fatigue বাড়ায়।
- **অ্যালার্ট পর্যালোচনা না করা:** যে অ্যালার্টে কেউ কখনো কিছু করেনি, সেটি মুছে ফেলুন।

> **WARNING:** Alertmanager-এর কনফিগারেশনে Slack webhook URL বা SMTP পাসওয়ার্ডের মতো গোপন তথ্য থাকে। এই ফাইলটি Git-এ পাঠানোর আগে গোপন মানগুলো `CHANGE-ME-...` প্লেসহোল্ডার দিয়ে বদলে নিন, আর আসল মান গোপন ব্যবস্থায় রাখুন।

১০.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

অ্যালার্ট বার্তায় সংবেদনশীল তথ্য যেতে পারে — যেমন সার্ভারের IP, এন্ডপয়েন্টের নাম, ত্রুটির বার্তায় ব্যবহারকারীর তথ্য। Slack চ্যানেলটি ব্যক্তিগত রাখুন, আর `annotations`-এ কখনো টোকেন বা পাসওয়ার্ড লিখবেন না। Alertmanager-এর UI-তে প্রমাণীকরণ ডিফল্টে থাকে না — প্রোডাকশনে তাকে ব্যক্তিগত নেটওয়ার্কে রাখুন বা বিপরীত প্রক্সির পেছনে প্রমাণীকরণ বসান, যাতে যে কেউ `silence` তৈরি করে অ্যালার্ট বন্ধ করে দিতে না পারে। Webhook receiver-এর URL-টি গোপন রাখুন — যে কেউ সেই URL জানলে ভুয়া অ্যালার্ট পাঠাতে পারে। ইমেইল receiver-এর SMTP পাসওয়ার্ড গোপন ব্যবস্থায় রাখুন।

১০.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

অ্যালার্টের খরচ দুই ধরনের: কম্পিউটিং ও মানুষের মনোযোগ। কম্পিউটিং খরচ ছোট — `Rule Manager` প্রতি ১৫ সেকেন্ডে নিয়মগুলো মূল্যায়ন করে; নিয়মের সংখ্যা শতাধিক হলেও তা সামলানো যায়। আসল খরচ মানুষের মনোযোগ — প্রতিটি ভুল অ্যালার্ট অন-কল প্রকৌশলীর ঘুম ও মনোযোগ কেড়ে নেয়। তাই অ্যালার্টের সংখ্যা কম রাখুন: একটি সার্ভিসের জন্য ৫–১০টি অ্যালার্ট যথেষ্ট। `Recording rule` দিয়ে ভারী কুয়েরি আগে থেকে হিসাব করে রাখলে `alerting rule`-এর মূল্যায়ন দ্রুত হয়। আর `grouping` ও `repeat_interval` ঠিক করে বিজ্ঞপ্তির সংখ্যা নিয়ন্ত্রণ করুন — একই সমস্যায় ঘণ্টায় একবারের বেশি বিজ্ঞপ্তি আসা উচিত নয়।

১০.১৪ সমস্যা সমাধান (Troubleshooting)

অ্যালার্ট firing হচ্ছে না: প্রথমে Prometheus UI-র `Alerts` পৃষ্ঠায় দেখুন — অ্যালার্টটি `pending` অবস্থায় আছে কি না। `pending` থাকলে `for` সময়কাল শেষ হয়নি। না থাকলে `expr`-টি UI-তে চালিয়ে দেখুন — শর্তটি সত্যি কি না।

অ্যালার্ট firing হচ্ছে, কিন্তু বিজ্ঞপ্তি আসছে না: Alertmanager UI-তে অ্যালার্টটি পৌঁছেছে কি না দেখুন। না পৌঁছালে Prometheus-এর `alerting` কনফিগারেশন পরীক্ষা করুন। পৌঁছে থাকলে `route`-এর `matchers` পরীক্ষা করুন — লেবেল না মিললে অ্যালার্টটি ডিফল্ট receiver-এ যায় বা কোথাও যায় না। `amtool` দিয়ে কনফিগারেশন পরীক্ষা করুন: `amtool check-config alertmanager.yml`।

একই অ্যালার্ট বারবার আসছে: `repeat_interval` বাড়ান; অথবা শর্তটি সত্যিই বারবার সত্য-মিথ্যা হচ্ছে — তখন `for` বাড়ান।

Silence কাজ করছে না: `matchers` ঠিক আছে কি না দেখুন — একটি ভুল লেবেল মান পুরো silence-কে অকার্যকর করে।

> **TROUBLESHOOTING:** Alertmanager-এর লগে `notify` ত্রুটি দেখুন — Slack webhook ভুল হলে সেখানে ৪০৪ বা ৪০৩ বার্তা আসে। আর মনে রাখুন: Alertmanager অ্যালার্ট পাঠায়, কিন্তু সে জানে না বিজ্ঞপ্তিটি কেউ পড়েছে কি না — পড়ার নিশ্চয়তা দিতে PagerDuty-ধরনের স্বীকৃতি (acknowledgement) ব্যবস্থা লাগে।

১০.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: উপসর্গে অ্যালার্ট লিখুন; প্রতিটি অ্যালার্টে `severity`, `team`, `summary`, `description` ও `runbook` রাখুন; `for` দিয়ে ক্ষণস্থায়ী স্পাইক ছাঁকুন; `grouping` ও `inhibition` দিয়ে এক ঘটনায় একটি বিজ্ঞপ্তি নিশ্চিত করুন; মাসে একবার অ্যালার্টগুলো পর্যালোচনা করুন — যে অ্যালার্টে কেউ কিছু করেনি, সেটি মুছে ফেলুন।

অ্যান্টি-প্যাটার্ন: প্রতিটি মেট্রিকে অ্যালার্ট লেখা; `severity: critical` সব জায়গায় বসানো; `runbook` ছাড়া অ্যালার্ট; `silence`-কে স্থায়ী সমাধান ভাবা; অ্যালার্টের শর্তে `irate()` ব্যবহার করা।

> **PRO TIP:** নতুন অ্যালার্ট লেখার আগে জিজ্ঞাসা করুন — "এই অ্যালার্ট রাত তিনটায় এলে আমি কী করব?" উত্তর না থাকলে অ্যালার্টটি লিখবেন না। আর প্রতিটি অ্যালার্টের পাশে লিখুন — "এই অ্যালার্ট গত মাসে কতবার এসেছে, আর প্রতিবার কী করা হয়েছে?" — এই পর্যালোচনাই alert fatigue-এর সবচেয়ে ভালো ওষুধ।

১০.১৬ অধ্যায়ের সারাংশ

Prometheus `alerting rule` দিয়ে শর্ত পরীক্ষা করে — `expr` শর্তটি, `for` তার স্থায়িত্ব, `labels` তার পরিচয়, `annotations` তার বার্তা। Alertmanager সেই অ্যালার্ট নিয়ে `routing tree` দিয়ে সঠিক দলে পাঠায়, `grouping` দিয়ে একত্র করে, `inhibition` দিয়ে মূল কারণের অ্যালার্ট ছাড়া বাকিগুলো চাপা দেয়, আর `silence` দিয়ে রক্ষণাবেক্ষণের সময় নীরব করে। ভালো অ্যালার্ট `actionable` ও উপসর্গ-ভিত্তিক হয়; burn-rate অ্যালার্ট SLO-র বাজেট পোড়ার হার ধরে। সবচেয়ে গুরুত্বপূর্ণ শিক্ষা: অ্যালার্ট কম লিখুন, কিন্তু প্রতিটি অ্যালার্ট এমনভাবে লিখুন যেন রাত তিনটায় জেগে ওঠা মানুষটি জানেন কী করতে হবে।

১০.১৭ মূল প্রযুক্তিগত পরিভাষা

Alerting Rule: PromQL শর্ত সত্য হলে অ্যালার্ট তৈরি করার নিয়ম; expr, for, labels ও annotations নিয়ে গঠিত।

Alertmanager: অ্যালার্ট গ্রহণ করে routing, grouping, inhibition ও silence প্রয়োগ করে বিজ্ঞপ্তি পাঠানো সার্ভার।

Routing Tree: লেবেল মিলিয়ে অ্যালার্টকে সঠিক receiver-এ পাঠানোর গাছ-আকৃতির নিয়ম।

Receiver: বিজ্ঞপ্তির গন্তব্য; যেমন ইমেইল, Slack বা webhook।

Grouping: একই ধরনের অ্যালার্ট একত্র করে একটি বিজ্ঞপ্তিতে পাঠানো।

Inhibition: একটি অ্যালার্ট firing থাকলে আরেকটি অ্যালার্ট চাপা দেওয়ার নিয়ম।

Silence: নির্দিষ্ট সময়ের জন্য নির্দিষ্ট অ্যালার্ট নীরব রাখার ব্যবস্থা।

Annotation: অ্যালার্টের মানুষের পড়ার মতো বার্তা; যেমন summary, description ও runbook।

Severity: অ্যালার্টের গুরুত্বের লেবেল; যেমন critical, warning, info।

Alert Fatigue: অতিরিক্ত অ্যালার্টে মানুষের সাড়া দেওয়ার ক্ষমতা কমে যাওয়া।

Actionable Alert: যে অ্যালার্টে স্পষ্ট করণীয় আছে ও এখনই কিছু করা দরকার।

Runbook: একটি অ্যালার্ট এলে ধাপে ধাপে কী করতে হবে তার লিখিত নির্দেশনা।

১০.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Prometheus ও Alertmanager-এর ভূমিকার পার্থক্য কী? অ্যালার্ট কে তৈরি করে, কে পাঠায়?
2. for: 5m-এর কাজ কী? for না দিলে কী সমস্যা হয়?
3. Routing tree-তে continue: true না দিলে কী হয়? কখন এটি দরকার?
4. Grouping, group_wait, group_interval ও repeat_interval — প্রতিটির কাজ কী?
5. Inhibition কীভাবে এক ঘটনায় অনেক অ্যালার্ট আসা ঠেকায়? একটি উদাহরণ দিন।
6. Silence ও inhibition-এর পার্থক্য কী? রক্ষণাবেক্ষণে কোনটি ব্যবহার করবেন?
7. উপসর্গে অ্যালার্ট লেখা কেন ভালো? CPU ৯০% হলে অ্যালার্ট দেওয়া কেন ভুল হতে পারে?
8. Burn-rate অ্যালার্টে দুটি সময়-উইন্ডো (১ ঘণ্টা ও ৬ ঘণ্টা) একসঙ্গে পরীক্ষা করা হয় কেন?
9. অ্যালার্ট বার্তায় কী কী থাকা উচিত? শুধু "অ্যালার্ট!" লিখলে কী সমস্যা?
10. Alert fatigue কী? তা কমাতে তিনটি উপায় লিখুন।

১০.১৯ ব্যবহারিক অনুশীলন

1. Alertmanager চালু করুন; ApiHighErrorRate নিয়মটি লিখে promtool দিয়ে যাচাই করুন।
2. কৃত্রিম ত্রুটি তৈরি করে অ্যালার্টটি pending থেকে firing হতে দেখুন; Alertmanager UI-তে তার অবস্থা লক্ষ্য করুন।
3. একটি silence তৈরি করুন; তার মেয়াদ শেষ হলে অ্যালার্ট আবার আসে কি না দেখুন।
4. একটি inhibition নিয়ম লিখুন ও একটি সার্ভার বন্ধ করে পরীক্ষা করুন।
5. একটি Slack বা ইমেইল receiver কনফিগার করুন (পরীক্ষার মান দিয়ে) ও বিজ্ঞপ্তি পাওয়ার পরীক্ষা করুন।
6. আপনার অ্যালার্টগুলোর পর্যালোচনা লিখুন: প্রতিটি অ্যালার্টে কী করণীয়, আর কোন অ্যালার্টটি মুছে ফেলা যায়।

অধ্যায় ১১ • PART II — Metrics and Prometheus Engineering

Chapter 11 — Kubernetes Monitoring with Prometheus and Service Discovery

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Kubernetes-এ Prometheus দিয়ে মনিটরিং-এর গঠন ব্যাখ্যা করতে পারবেন — `kube-state-metrics`, `cAdvisor` ও Node Exporter-এর ভূমিকা, Kubernetes `service discovery` ও `ServiceMonitor`-ধরনের ধারণা, Pod ও কনটেইনারের মেট্রিক, কন্ট্রোল-প্লেন মনিটরিং এবং সাধারণ Kubernetes অ্যালার্ট লিখতে পারবেন। আপনি `kube-proxy`-র আচরণ নেটওয়ার্কিং বাস্তবায়নভেদে বদলায় — এই সত্যটি ব্যাখ্যা করতে পারবেন।

- Kubernetes-এ মেট্রিকের তিনটি স্তর চেনা: নোড, Pod, কন্ট্রোল-প্লেন।
- kube-state-metrics ও cAdvisor-এর মেট্রিক পড়া।
- Kubernetes service discovery ও relabeling বোঝা।
- ServiceMonitor ধারণা ও Prometheus Operator-এর প্রসঙ্গ ব্যাখ্যা করা।
- সাধারণ Kubernetes অ্যালার্ট লেখা।

১১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Kubernetes-এ সবকিছু গতিশীল — Pod তৈরি হয়, ধ্বংস হয়, IP বদলায়, নোডে ছড়িয়ে যায়। হাতে লেখা `static_configs` এখানে একদিনও টিকবে না। TechNova যখন তাদের MERN অ্যাপ Kubernetes-এ নিল, প্রথম সপ্তাহেই মনিটরিং অন্ধ হয়ে গিয়েছিল — Pod-এর IP বদলে যাওয়ায় Prometheus পুরোনো টার্গেটে `scrape` করতে যাচ্ছিল, আর নতুন Pod-গুলো অদৃশ্য ছিল। সমাধান ছিল Kubernetes `service discovery`: Prometheus Kubernetes API-র কাছে জিজ্ঞাসা করে — "এখন কোন Pod চলছে, তাদের লেবেল কী, তাদের `/metrics` কোথায়?" — আর স্বয়ংক্রিয়ভাবে টার্গেট আপডেট করে। এই অধ্যায় ছাড়া Kubernetes মনিটরিং অসম্ভব; এই অধ্যায়সহ তা স্বয়ংক্রিয়।

১১.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৭–১০ জানা থাকতে হবে — Prometheus আর্কিটেকচার, `service discovery`, `relabeling`, `PromQL` ও অ্যালার্ট। এ ছাড়া প্রয়োজন: Kubernetes-এর মৌলিক ধারণা — Pod, Deployment, Service, Namespace, Node — এবং `kubectl` দিয়ে ক্লাস্টার দেখার অভিজ্ঞতা। kind বা Minikube দিয়ে একটি স্থানীয় ক্লাস্টার থাকলে এই অধ্যায়ের উদাহরণগুলো সরাসরি পরীক্ষা করতে পারবেন। Prometheus Operator আগে না জানলেও চলবে — সেটি এই অধ্যায়েই পরিচিত করা হয়েছে।

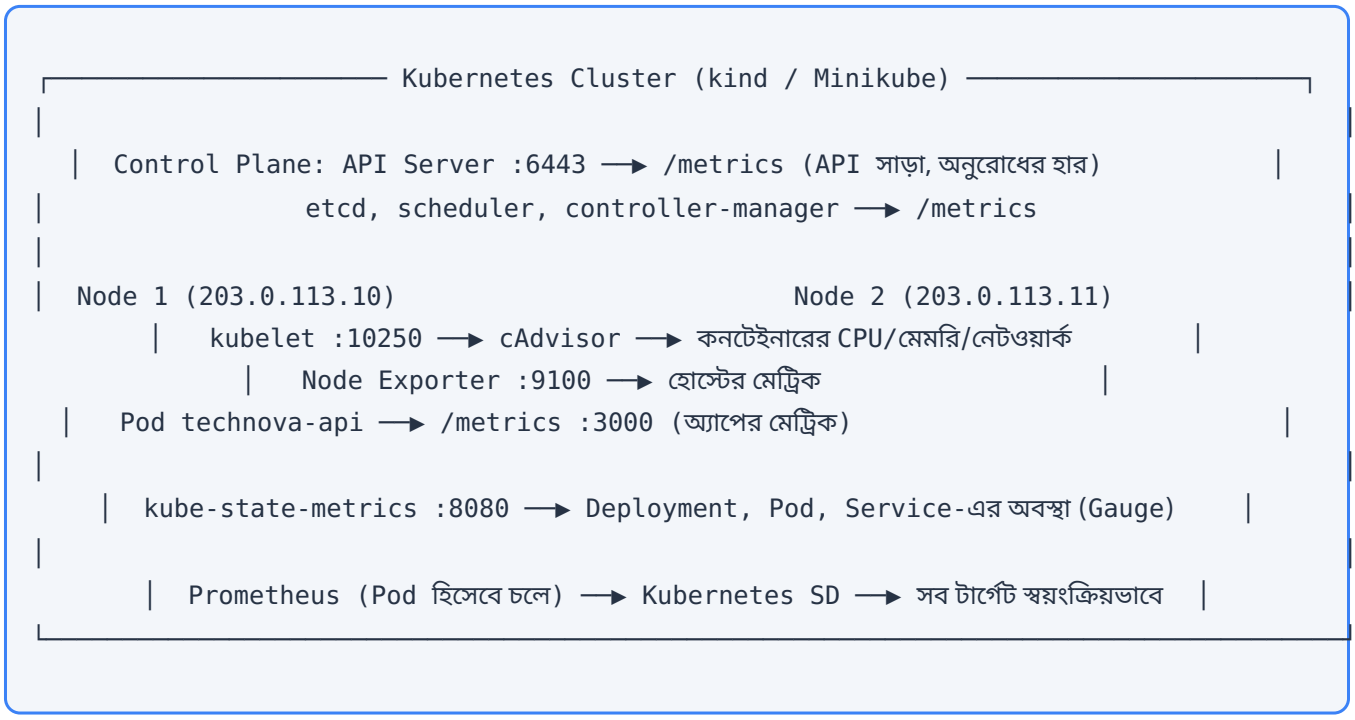
১১.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে ভাবুন: Kubernetes-এ মনিটরিং-এর প্রশ্ন তিনটি স্তরে ভাগ করা যায়। প্রথম স্তর: নোড কি সুস্থ? — CPU, মেমরি, ডিস্ক — এটি Node Exporter-এর কাজ। দ্বিতীয় স্তর: Pod ও কনটেইনার কি সুস্থ? — প্রতিটি কনটেইনার কত CPU ও মেমরি ব্যবহার করছে, কতবার পুনরায় চালু হয়েছে — এটি `cAdvisor` (প্রতিটি নোডের kubelet-এর ভেতরে থাকে) ও `kube-state-metrics`-এর কাজ। তৃতীয় স্তর: Kubernetes নিজে কি সুস্থ? — API সার্ভার সাড়া দিচ্ছে কি না, `scheduler` Pod বসাতে পারছে কি না, `etcd` ধীর কি না — এটি কন্ট্রোল-প্লেন মনিটরিং। এই তিন স্তর একসঙ্গে না দেখলে

ছবি অসম্পূর্ণ থাকে: অ্যাপ ধীর হতে পারে নোডের চাপে, Pod-এর সীমায়, বা কন্ট্রোল-প্লেনের সমস্যায় — কোনটি, তা বোঝা যায় তিন স্তর একসঙ্গে দেখলে।

> **CONCEPT:** Kubernetes-এ Prometheus টার্গেট খোঁজে না, সে Kubernetes API-কে জিজ্ঞাসা করে। `kubernetes_sd_configs`-এ `role: pod` বললে Prometheus সব Pod-এর তালিকা পায়, সঙ্গে প্রতিটি Pod-এর লেবেল ও annotation `__meta_kubernetes_*` লেবেল হিসেবে আসে। তারপর `relabeling` দিয়ে বলা হয় — "শুধু সেই Pod-গুলো scrape করো যাদের annotation-এ `prometheus.io/scrape: "true"` আছে, আর তাদের পোর্টটি annotation থেকে নাও।" এই annotation-ভিত্তিক পদ্ধতিই Kubernetes মনিটরিং-এর সবচেয়ে পুরোনো ও সহজ ধারা।

১১.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



লক্ষ্য করুন, Prometheus নিজেও ক্লাস্টারের ভেতরে একটি Pod হিসেবে চলে — তাতে Kubernetes API-তে পৌঁছানো সহজ হয়। `cAdvisor` আলাদাভাবে ইনস্টল করতে হয় না — প্রতিটি নোডের `kubelet`-এর ভেতরেই থাকে, আর `kubelet`-এর `/metrics/cadvisor` এন্ডপয়েন্টে তার মেট্রিক পাওয়া যায়। `kube-state-metrics` আলাদা একটি Deployment — সে Kubernetes API-কে জিজ্ঞাসা করে প্রতিটি অবজেক্টের অবস্থা (`Deployment`-এ কতটি replica চাওয়া হয়েছে, কতটি চলছে, Pod কি `Pending` অবস্থায় আটকে আছে) `Gauge` মেট্রিক হিসেবে প্রকাশ করে। এই দুটি একসঙ্গে না থাকলে Kubernetes মনিটরিং অসম্পূর্ণ।

১১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

kube-state-metrics — ক্লাস্টারের অবস্থা

`kube-state-metrics` Kubernetes অবজেক্টের অবস্থা মেট্রিক হিসেবে প্রকাশ করে। প্রধান মেট্রিক: `kube_deployment_status_replicas_available` (কতটি replica চলছে), `kube_deployment_spec_replicas` (কতটি চাওয়া হয়েছে), `kube_pod_status_phase{phase="Pending|Running|Failed"}` (Pod কোন অবস্থায়),

`kube_pod_container_status_restarts_total` (কনটেইনার কতবার পুনরায় চালু হয়েছে — একটি Counter), `kube_pod_container_resource_requests{resource="cpu|memory"}` ও `kube_pod_container_resource_limits` (অনুরোধ ও সীমা)। এই মেট্রিকগুলো `Gauge` বা `Counter` — কিন্তু মনে রাখুন, এগুলো Kubernetes-এর ঘোষিত অবস্থা, আসল ব্যবহার নয়। আসল ব্যবহার মাপে `cAdvisor`। দুটি একসঙ্গে দেখলে বোঝা যায় — যেমন, অনুরোধ ২৫৬ মেগাবাইট মেমরি, কিন্তু ব্যবহার ৯০০ মেগাবাইট — মানে সীমা ছাড়িয়ে যাওয়ার ঝুঁকি আছে।

cAdvisor ও kubelet মেট্রিক

`cAdvisor` প্রতিটি কনটেইনারের আসল ব্যবহার মাপে: `container_cpu_usage_seconds_total` (Counter — `rate()` দিয়ে CPU ব্যবহার), `container_memory_working_set_bytes` (Gauge — আসল মেমরি ব্যবহার; `container_memory_usage_bytes`-এর চেয়ে নির্ভরযোগ্য, কারণ এতে ক্যাশ বাদ যায়), `container_network_receive_bytes_total` ও `container_network_transmit_bytes_total`, `container_fs_usage_bytes` (ডিস্ক ব্যবহার)। `kubelet` নিজেও মেট্রিক দেয় — যেমন `kubelet_running_pods`। একটি সাধারণ ভুল: `container_memory_usage_bytes` দিয়ে OOM (Out of Memory) ঝুঁকি মাপা — এতে ক্যাশ অন্তর্ভুক্ত থাকে, তাই `working_set_bytes`-ই সঠিক পরিমাপ।

Service Discovery ও Relabeling

Kubernetes `service discovery`-তে পাঁচটি `role` আছে: `node`, `pod`, `service`, `endpoints` ও `endpointslice`। `role: pod` সব Pod দেয়; `role: endpoints` Service-এর পেছনের Pod-গুলো দেয় — এটি সাধারণত ভালো, কারণ Service-এর লেবেলও সঙ্গে আসে। `Relabeling`-এর সাধারণ ধারা: `__meta_kubernetes_pod_annotation_prometheus_io_scrape` "true" হলে তবেই `scrape` করো (`action: keep`); `__meta_kubernetes_pod_annotation_prometheus_io_path` থেকে `metrics_path` নাও; `__meta_kubernetes_pod_annotation_prometheus_io_port` থেকে পোর্ট নাও; আর `__meta_kubernetes_namespace` থেকে `namespace` লেবেল তৈরি করো। এই নিয়মগুলো একবার লিখলে নতুন Pod এলেই স্বয়ংক্রিয়ভাবে মনিটরিং শুরু হয় — কাউকে Prometheus-এর কনফিগারেশন ছুঁতে হয় না।

ServiceMonitor ও Prometheus Operator

`ServiceMonitor` হলো Prometheus Operator-এর একটি Kubernetes অবজেক্ট (Custom Resource)। এর ধারণা: `scrape` কনফিগারেশন YAML ফাইলে না লিখে Kubernetes অবজেক্ট হিসেবে লেখা — "এই Service-এর পেছনের Pod-গুলোকে এই বিরতিতে scrape করো, এই পথে, এই লেবেলসহ।" Prometheus Operator সেই অবজেক্টগুলো পড়ে Prometheus-এর কনফিগারেশন স্বয়ংক্রিয়ভাবে তৈরি করে। সুবিধা: প্রতিটি দল নিজের `ServiceMonitor` নিজে লিখতে পারে, কেন্দ্রীয় Prometheus কনফিগারেশন ছোঁয়ার দরকার হয় না। অসুবিধা: Operator একটি অতিরিক্ত স্তর — তার নিজেরও রক্ষণাবেক্ষণ লাগে। ছোট ক্লাস্টারে annotation-ভিত্তিক `service discovery` যথেষ্ট; বড় সংস্থায় Operator সাধারণ পছন্দ। এই বইয়ে `ServiceMonitor`-কে ধারণা হিসেবে বুঝুন — তার YAML আপনি পরের অধ্যায়গুলোর ল্যাবে হাতে-কলমে লিখবেন।

কন্ট্রোল-প্লেন মনিটরিং ও kube-proxy

কন্ট্রোল-প্লেনের উপাদানগুলোও `/metrics` প্রকাশ করে: API সার্ভারের `apiserver_request_total` ও `apiserver_request_duration_seconds`, `etcd`-এর `etcd_disk_backend_commit_duration_seconds`, `scheduler`-এর `scheduler_schedule_attempts_total`। API সার্ভার ধীর হলে সবকিছু ধীর হয় — `kubectl` থেকে Pod তৈরি পর্যন্ত। তাই কন্ট্রোল-প্লেনের অ্যালার্ট আলাদা গুরুত্ব পায়। `kube-proxy` সম্পর্কে একটি সতর্কতা: তার আচরণ নেটওয়ার্কিং বাস্তবায়নভেদে বদলায় — কিছু ক্লাস্টারে সে `iptables` নিয়ম লেখে, কিছুতে IPVS ব্যবহার করে, আবার কিছু

নেটওয়ার্ক প্লাগইন (যেমন Cilium) `kube-proxy`-কে পুরোপুরি বদলে দেয়। তাই `kube-proxy`-র মেট্রিক ও আচরণ সম্পর্কে কোনো সার্বজনীন দাবি করা যায় না — আপনার ক্লাস্টারের নেটওয়ার্কিং বাস্তবায়নটি জেনে তার নথি দেখুন।

উৎস	মেট্রিকের উদাহরণ	কী বোঝায়
Node Exporter	node_cpu_seconds_total	নোডের CPU ব্যবহার
cAdvisor	container_memory_working_set_bytes	কনটেইনারের আসল মেমরি
kube-state-metrics	kube_pod_container_status_restarts_total	কনটেইনার পুনরায় চালুর সংখ্যা
API Server	apiserver_request_duration_seconds	API-র লেটেন্সি
অ্যাপ (/metrics)	http_requests_total	অ্যাপের অনুরোধের হার

১১.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# prometheus.yml — Kubernetes service discovery (Pod role)
scrape_configs:
  - job_name: 'kubernetes-pods'
    kubernetes_sd_configs:
      - role: pod                       # সব Pod-এর তালিকা Kubernetes API থেকে
    relabel_configs:
      # শুধু সেই Pod scrape করো যার annotation prometheus.io/scrape: "true"
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
        action: keep
        regex: 'true'
      # annotation থেকে metrics পথ নাও (ডিফল্ট /metrics)
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
        action: replace
        target_label: __metrics_path__
        regex: '(.+)'
      # annotation থেকে পোর্ট নাও
      - source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_port, __address__]
        action: replace
        target_label: __address__
        regex: '(\d+);(.+):(\d+)'
        replacement: '$2:$1'
      # namespace ও pod নাম লেবেল হিসেবে রাখো
      - source_labels: [__meta_kubernetes_namespace]
        target_label: namespace
      - source_labels: [__meta_kubernetes_pod_name]
        target_label: pod

# Pod-এ annotation যোগ করা (Deployment YAML-এর অংশ)
metadata:
  annotations:
    prometheus.io/scrape: "true"
    prometheus.io/port: "3000"
    prometheus.io/path: "/metrics"

# ServiceMonitor ধারণা (Prometheus Operator) — সংক্ষিপ্ত উদাহরণ
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata: { name: technova-api, namespace: technova }
```

```
spec:
  selector: { matchLabels: { app: technova-api } }   # কোন Service-কে scrape করবে
  endpoints:
    - port: web
      path: /metrics
      interval: 15s
```

`relabeling`-এর প্রথম নিয়মটি সবচেয়ে গুরুত্বপূর্ণ: `action: keep` মানে শুধু মিলে যাওয়া Pod-গুলোই রাখা হবে, বাকিগুলো বাদ। `__metrics_path__` ও `__address__` বিশেষ লেবেল — এগুলো `scrape`-এর আচরণ নিয়ন্ত্রণ করে, আর `scrape`-এর পর ফেলে দেওয়া হয়। `ServiceMonitor` উদাহরণে লক্ষ্য করুন — কোনো IP বা পোর্ট নম্বর হাতে লেখা নেই; `selector` লেবেল দিয়ে Service খুঁজে নেওয়া হয়। এটাই Kubernetes-ধরনের ঘোষণা: "কী চাই" বলুন, "কোথায় আছে" সিস্টেম খুঁজে নেবে।

১১.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র Kubernetes মনিটরিং-এর জন্য প্রধান কুয়েরি ও অ্যালার্ট — একটি সংগ্রহ:

```
# Pod-এর CPU ব্যবহার — limit-এর অনুপাতে
sum(rate(container_cpu_usage_seconds_total{namespace="technova"}[5m])) by (pod)
  / sum(kube_pod_container_resource_limits{namespace="technova", resource="cpu"}) by (pod)

# Pod-এর মেমরি ব্যবহার — request-এর অনুপাতে
sum(container_memory_working_set_bytes{namespace="technova"}) by (pod)
  / sum(kube_pod_container_resource_requests{namespace="technova", resource="memory"}) by
(pod)

# কনটেইনার পুনরায় চালুর হার — গত ১ ঘণ্টায়
increase(kube_pod_container_status_restarts_total{namespace="technova"}[1h])

# Deployment-এর উপলভ্য replica কমে গেছে কি না
kube_deployment_status_replicas_available{deployment="technova-api"}
  / kube_deployment_spec_replicas{deployment="technova-api"} < 1

# Pod Pending অবস্থায় আটকে আছে কি না
kube_pod_status_phase{namespace="technova", phase="Pending"} == 1

# rules/alerts-k8s.yml — Kubernetes অ্যালার্ট
groups:
  - name: technova-k8s
    rules:
      - alert: PodCrashLooping
        expr: rate(kube_pod_container_status_restarts_total[15m]) > 0
        for: 5m
        labels: { severity: critical, team: platform }
        annotations:
          summary: "Pod বারবার পুনরায় চালু হচ্ছে ({{ $labels.pod }})"
          description: "গত ১৫ মিনিটে পুনরায় চালু হয়েছে — লগ দেখুন।"
      - alert: PodNotReady
        expr: kube_pod_status_ready{condition="false"} == 1
        for: 5m
        labels: { severity: warning, team: platform }
        annotations:
          summary: "Pod প্রস্তুত নয় ({{ $labels.pod }})"
      - alert: DeploymentReplicasMismatch
        expr: kube_deployment_status_replicas_available
```

```
          / kube_deployment_spec_replicas < 1
        for: 5m
        labels: { severity: warning, team: platform }
        annotations:
          summary: "Deployment-এর replica কম ({{ $labels.deployment }})"
      - alert: NodeMemoryPressure
        expr: 100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) > 90
        for: 5m
        labels: { severity: warning, team: platform }
        annotations:
          summary: "নোডের মেমরি ৯০%-এর বেশি ব্যবহৃত ({{ $labels.instance }})"
```

প্রথম কুয়েরিটি লক্ষ্য করুন: `cAdvisor`-এর আসল ব্যবহারকে `kube-state-metrics`-এর সীমা দিয়ে ভাগ করা হয়েছে — এই অনুপাতই বলে Pod-টি তার সীমার কতটা ব্যবহার করছে। অনুপাত ১-এর কাছাকাছি গেলে CPU `throttling` বা মেমরি OOM-এর ঝুঁকি বাড়ে। `PodCrashLooping` অ্যালার্টে `rate() > 0` ব্যবহার করা হয়েছে — গত ১৫ মিনিটে একবারও পুনরায় চালু হলে অ্যালার্ট যাবে; `for: 5m` ক্ষণস্থায়ী পুনরায় চালু ছেঁকে দেয়।

১১.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র একটি Deployment আপডেটের পর নতুন Pod-গুলো বারবার পুনরায় চালু হচ্ছিল — `CrashLoopBackOff`। `kube_pod_container_status_restarts_total`-এর `rate()` শূন্যের বেশি দেখে `PodCrashLooping` অ্যালার্ট এল। দলটি `kubectl logs` দেখে বুঝল নতুন ইমেজে একটি পরিবেশ চলক (environment variable) অনুপস্থিত — MongoDB-র সংযোগের ঠিকানা দেওয়া হয়নি। Deployment-এ চলকটি যোগ করতেই Pod স্থির হলো। এই ঘটনায় মেট্রিক জানিয়েছে "কী ঘটছে", আর লগ জানিয়েছে "কেন ঘটছে" — দুটি একসঙ্গে কাজ করেছে। আরেকটি পরিস্থিতি: একটি নোডের মেমরি ৯৫% ছুঁয়ে গেল, কারণ একটি Pod-এর মেমরি সীমা নির্ধারণ করা হয়নি — সে যত খুশি মেমরি নিচ্ছিল। `container_memory_working_set_bytes` ও `kube_pod_container_resource_limits` তুলনা করে দলটি দেখল কোন Pod-টি সীমাহীন, আর তার জন্য সীমা বসাল। Kubernetes-এ সীমা না বসানো Pod একটি প্রতিবেশী Pod-কে মেরে ফেলতে পারে — তাই `requests` ও `limits` মনিটরিং-এর অংশ।

১১.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. kind বা Minikube দিয়ে একটি স্থানীয় ক্লাস্টার তৈরি করুন; `kubectl get nodes` দিয়ে নোড দেখুন।
2. `kube-state-metrics` ও Node Exporter (DaemonSet হিসেবে) ইনস্টল করুন — Helm বা YAML যেকোনোভাবে।
3. TechNova API-কে Deployment হিসেবে চালু করুন; Pod-এ `prometheus.io/scrape: "true"` annotation যোগ করুন।
4. Prometheus-এ `kubernetes_sd_configs` (role: pod) যোগ করুন; `Targets` পৃষ্ঠায় Pod-গুলো স্বয়ংক্রিয়ভাবে আসে কি না দেখুন।
5. একটি Pod মুছে ফেলুন (`kubectl delete pod`); নতুন Pod তৈরি হলে Prometheus স্বয়ংক্রিয়ভাবে তাকে `scrape` করে কি না লক্ষ্য করুন — হাতে কনফিগারেশন বদলাতে হয়নি।
6. উপরের কুয়েরিগুলো চালিয়ে Pod-এর CPU ও মেমরি ব্যবহার দেখুন; একটি Pod-এর মেমরি সীমা ইচ্ছা করে কমিয়ে OOM-এর আচরণ পর্যবেক্ষণ করুন (ল্যাবে, সাবধানে)।
7. `PodCrashLooping` অ্যালার্টটি লিখুন; একটি ভুল ইমেজ দিয়ে Pod-কে CrashLoop-এ ফেলে অ্যালার্টটি পরীক্ষা করুন।

১১.১১ সাধারণ ভুল ও ভুল ধারণা

- **static_configs দিয়ে Pod scrape করা:** Pod-এর IP বদলায়; Kubernetes service discovery ব্যবহার করুন।
- **container_memory_usage_bytes দিয়ে OOM ঝুঁকি মাপা:** `working_set_bytes`-ই সঠিক পরিমাপ।
- **requests ও limits না বসানো:** সীমাহীন Pod প্রতিবেশী Pod-কে মেরে ফেলতে পারে; দুটোই বসান ও পর্যবেক্ষণ করুন।
- **kube-state-metrics-কে আসল ব্যবহার ভাবা:** সে ঘোষিত অবস্থা দেয়; আসল ব্যবহার দেয় cAdvisor।
- **সব Pod-কে scrape করা:** `keep` নিয়ম ছাড়া সিস্টেম Pod-গুলোও scrape হবে — অপ্রয়োজনীয় সিরিজ বাড়বে।
- **kube-proxy সম্পর্কে সার্বজনীন দাবি:** তার আচরণ নেটওয়ার্কিং বাস্তবায়নভেদে বদলায়; নিজের ক্লাস্টারের নথি দেখুন।
- **Prometheus-কে ক্লাস্টারের বাইরে রেখে SD আশা করা:** Kubernetes API-তে পৌঁছানোর জন্য RBAC অনুমতি লাগে; Prometheus-এর ServiceAccount-এ সঠিক Role বসান।

> **WARNING:** Prometheus-এর ServiceAccount-কে Kubernetes API-তে অতিরিক্ত অনুমতি দেবেন না। শুধু পড়ার অনুমতি দিন — Pod, Service, Endpoints ও Node পড়ার জন্য। লেখার অনুমতি দিলে Prometheus আপস হলে আক্রমণকারী ক্লাস্টার বদলে দিতে পারবে।

১১.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Kubernetes মনিটরিং-এ তিনটি নিরাপত্তা বিষয় আছে। প্রথমত, RBAC: Prometheus-এর ServiceAccount-কে শুধু পড়ার অনুমতি দিন — `get` ও `list`, কখনো `create` বা `delete` নয়। দ্বিতীয়ত, `/metrics` এন্ডপয়েন্ট: Pod-এর ভেতরে `/metrics` ক্লাস্টারের ভেতর থেকে পৌঁছানো যায়; NetworkPolicy দিয়ে শুধু Prometheus-এর Pod থেকে পৌঁছানো যায় এমনভাবে সীমিত করুন। তৃতীয়ত, লেবেল: Pod-এর লেবেল ও annotation-এ কখনো গোপন তথ্য রাখবেন না — সেগুলো `__meta_kubernetes_*` লেবেল হিসেবে Prometheus-এ আসে ও সংরক্ষিত হয়। `kube-state-metrics`-এর মেট্রিকে Pod-এর নাম ও Namespace থাকে — সেগুলো সংবেদনশীল না হলেও ড্যাশবোর্ড শেয়ার করার আগে পরীক্ষা করুন।

১১.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Kubernetes মনিটরিং-এর খরচের মূল চালক Pod-এর সংখ্যা। প্রতিটি Pod-এর প্রতিটি কনটেইনারের জন্য `cAdvisor` কয়েক ডজন সিরিজ তৈরি করে; ১০০টি Pod মানে হাজারো সিরিজ, ১,০০০টি Pod মানে লাখো সিরিজ। তাই `relabeling` দিয়ে অপ্রয়োজনীয় মেট্রিক ফেলে দিন — যেমন `container_*` মেট্রিকের মধ্যে শুধু প্রয়োজনীয়গুলো রাখুন (`metric_relabel_configs` দিয়ে)। `kube-state-metrics`-এর মেট্রিকও অনেক — শুধু প্রয়োজনীয় অবজেক্টের মেট্রিক রাখুন। `scrape_interval` ১৫ সেকেন্ড Pod-এর জন্য যথেষ্ট; কন্ট্রোল-প্লেনের জন্য ৩০ সেকেন্ডও চলে। আর মনে রাখুন: Prometheus নিজে ক্লাস্টারের ভেতরে চললে তার নিজের CPU ও মেমরি `requests/limits` বসান — নইলে সে নিজেই প্রতিবেশী Pod-কে চাপে ফেলবে।

১১.১৪ সমস্যা সমাধান (Troubleshooting)

Pod টার্গেটে আসছে না: প্রথমে annotation ঠিক আছে কি না দেখুন (`kubectl describe pod`); তারপর Prometheus-এর `Service Discovery` পৃষ্ঠায় Pod-টি `dropped` হিসেবে দেখাচ্ছে কি না দেখুন — সেখানে `relabeling` -এর কোন নিয়মটি তাকে বাদ দিয়েছে তা লেখা থাকে।

RBAC ত্রুটি: Prometheus-এর লগে `forbidden` বার্তা থাকলে ServiceAccount-এর Role পরীক্ষা করুন; `kubectl auth can-i list pods --as=system:serviceaccount:monitoring:prometheus` দিয়ে অনুমতি যাচাই করুন।

Pod বারবার পুনরায় চালু হচ্ছে: `kube_pod_container_status_restarts_total` দেখুন, তারপর `kubectl logs --previous` দিয়ে আগের কনটেইনারের লগ পড়ুন — সেখানেই কারণ লেখা থাকে।

মেমরি ব্যবহার অস্বাভাবিক: `working_set_bytes` দেখুন; `usage_bytes` দেখে ভুল সিদ্ধান্ত নেবেন না — ক্যাশ অন্তর্ভুক্ত থাকে।

API সার্ভার ধীর: `apiserver_request_duration_seconds` -এর p99 দেখুন; `etcd` -এর `commit duration` পরীক্ষা করুন — etcd ধীর হলে API-ও ধীর হয়।

> **TROUBLESHOOTING:** Kubernetes-এ সমস্যার তদন্ত তিন স্তরে করুন — প্রথমে kube-state-metrics (ঘোষিত অবস্থা), তারপর cAdvisor (আসল ব্যবহার), তারপর অ্যাপের মেট্রিক ও লগ (ভেতরের কারণ)। একটি স্তর বাদ দিলে তদন্ত অসম্পূর্ণ থাকে।

১১.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Kubernetes service discovery ব্যবহার করুন; annotation দিয়ে scrape নিয়ন্ত্রণ করুন; প্রতিটি Pod-এ `requests` ও `limits` বসান ও তাদের অনুপাত পর্যবেক্ষণ করুন; `working_set_bytes` দিয়ে মেমরি মাপুন; `ServiceMonitor` বা annotation — একটি পদ্ধতি বেছে নিন, দুটি মিশিয়ে ফেলবেন না; Prometheus-এর RBAC শুধু পড়ার মধ্যে সীমিত রাখুন; কন্ট্রোল-প্লেনের অ্যালার্ট আলাদা গুরুত্ব দিন।

অ্যান্টি-প্যাটার্ন: Pod-এর IP হাতে লেখা; সব Pod-কে scrape করা; `usage_bytes` দিয়ে OOM ঝুঁকি মাপা; সীমা ছাড়া Pod চালানো; Prometheus-কে লেখার অনুমতি দেওয়া; `kube-proxy` -র আচরণ সম্পর্কে না জেনে সার্বজনীন দাবি করা।

> **PRO TIP:** নতুন সার্ভিস Kubernetes-এ নেওয়ার সময় প্রথমে তার `ServiceMonitor` বা annotation লিখুন, তারপর Deployment লিখুন। মনিটরিং পরে যোগ করা যায় — কিন্তু পরে যোগ করা মনিটরিং প্রায়ই ভুলে যাওয়া হয়, আর প্রথম ঘটনাটি ঘটে মনিটরিং ছাড়াই।

১১.১৬ অধ্যায়ের সারাংশ

Kubernetes মনিটরিং তিন স্তরে গঠিত: Node Exporter নোডের স্বাস্থ্য দেয়, `cAdvisor` কনটেইনারের আসল CPU ও মেমরি ব্যবহার দেয়, আর `kube-state-metrics` Deployment, Pod ও Service-এর ঘোষিত অবস্থা দেয়। কন্ট্রোল-প্লেন — API সার্ভার, etcd, scheduler — আলাদাভাবে পর্যবেক্ষণ করা হয়, কারণ তার ধীরগতি পুরো ক্লাস্টারকে ধীর করে। Kubernetes `service discovery` ও `relabeling` Pod-এর গতিশীল জীবনচক্রে টার্গেট ব্যবস্থাপনাকে স্বয়ংক্রিয় করে; `ServiceMonitor` সেই ধারণাকে Kubernetes অবজেক্ট হিসেবে প্রকাশ করে, আর Prometheus Operator তা থেকে কনফিগারেশন তৈরি করে। `PodCrashLooping` , `PodNotReady` ও replica mismatch অ্যালার্ট Kubernetes-এর সাধারণ

সমস্যাগুলো ধরে। মনে রাখবেন: `working_set_bytes`-ই মেমরির সঠিক পরিমাপ, `requests` ও `limits` ছাড়া Pod চালানো ঝুঁকিপূর্ণ, আর `kube-proxy`-র আচরণ নেটওয়ার্কিং বাস্তবায়নভেদে বদলায় — তাই নিজের ক্লাস্টারের নথিই চূড়ান্ত উৎস।

১১.১৭ মূল প্রযুক্তিগত পরিভাষা

kube-state-metrics: Kubernetes অবজেক্টের অবস্থা (replica, Pod phase, restart) মেট্রিক হিসেবে প্রকাশকারী সার্ভিস।

cAdvisor: প্রতিটি kubelet-এর ভেতরে থাকা কনটেইনার সম্পদ ব্যবহার পরিমাপক; CPU, মেমরি, নেটওয়ার্ক মাপে।

kubelet: প্রতিটি নোডে চলা এজেন্ট; Pod চালায় ও cAdvisor মেট্রিক প্রকাশ করে।

ServiceMonitor: Prometheus Operator-এর Custom Resource; কোন Service-কে কীভাবে scrape করতে হবে তা ঘোষণা করে।

Prometheus Operator: Prometheus, Alertmanager ও ServiceMonitor পরিচালনা করা Kubernetes Operator।

Working Set: কনটেইনারের আসল মেমরি ব্যবহার (ক্যাশ বাদে); OOM ঝুঁকি মাপার সঠিক পরিমাপ।

Requests / Limits: Pod-এর জন্য অনুরোধ করা ও সর্বোচ্চ অনুমোদিত CPU/মেমরি।

CrashLoopBackOff: কনটেইনার বারবার চালু হয়ে ব্যর্থ হওয়ার অবস্থা; Kubernetes ক্রমবর্ধমান বিরতিতে পুনরায় চেষ্টা করে।

Endpoints: একটি Service-এর পেছনের Pod-গুলোর ঠিকানার তালিকা; service discovery-র একটি role।

RBAC: Role-Based Access Control — Kubernetes-এ কে কী করতে পারে তার অনুমতি ব্যবস্থা।

Control Plane: Kubernetes-এর নিয়ন্ত্রণ স্তর; API Server, etcd, scheduler ও controller-manager নিয়ে গঠিত।

kube-proxy: Service-এর নেটওয়ার্ক নিয়ম পরিচালনাকারী উপাদান; তার আচরণ নেটওয়ার্কিং বাস্তবায়নভেদে বদলায়।

১১.১৮ জ্ঞান যাচাই — প্রশ্ন

1. kube-state-metrics ও cAdvisor-এর পার্থক্য কী? কোনটি ঘোষিত অবস্থা দেয়, কোনটি আসল ব্যবহার?
2. Pod-এর IP বদলায় বলে static_configs কেন কাজ করে না? Kubernetes service discovery কীভাবে সমস্যাটি সমাধান করে?
3. relabeling-এ action: keep-এর কাজ কী? annotation prometheus.io/scrape: "true" না থাকলে কী হয়?
4. container_memory_usage_bytes-এর বদলে working_set_bytes দিয়ে মেমরি মাপা উচিত কেন?
5. ServiceMonitor কী? Prometheus Operator ছাড়া কি Kubernetes মনিটরিং সম্ভব?
6. requests ও limits না বসালে একটি Pod কীভাবে প্রতিবেশী Pod-কে ক্ষতি করতে পারে?
7. PodCrashLooping অ্যালার্টে rate() > 0 ব্যবহার করা হয়েছে কেন? for: 5m-এর ভূমিকা কী?
8. কন্ট্রোল-প্লেন ধীর হলে কী কী প্রভাব পড়ে? কোন মেট্রিক দিয়ে API সার্ভারের লেটেন্সি দেখবেন?
9. kube-proxy-র আচরণ সম্পর্কে সার্বজনীন দাবি করা যায় না কেন?
10. Prometheus-এর ServiceAccount-কে শুধু পড়ার অনুমতি দেওয়া উচিত কেন?

১১.১৯ ব্যবহারিক অনুশীলন

1. kind বা Minikube ক্লাস্টারে kube-state-metrics ও Node Exporter ইনস্টল করুন; kube_pod_status_phase কুয়েরি করুন।
2. একটি Deployment-এ prometheus.io/scrape annotation যোগ করুন; Prometheus-এর Targets পৃষ্ঠায় Pod-টি আসে কি না দেখুন।
3. একটি Pod মুছে নতুন Pod তৈরি হলে Prometheus স্বয়ংক্রিয়ভাবে তাকে scrape করে কি না পরীক্ষা করুন।
4. container_memory_working_set_bytes ও kube_pod_container_resource_limits তুলনা করে একটি Pod-এর মেমরি অনুপাত বের করুন।
5. PodCrashLooping অ্যালার্ট লিখুন; ভুল ইমেজ দিয়ে Pod-কে CrashLoop-এ ফেলে অ্যালার্ট পরীক্ষা করুন।
6. একটি ServiceMonitor YAML লিখুন (ধারণা হিসেবে) ও ব্যাখ্যা করুন তা annotation পদ্ধতি থেকে কীভাবে আলাদা।

PART III • DEVOPS ENGINEERING MASTERY

Visualization and Logging Engineering

Grafana ড্যাশবোর্ড ও ভিজ্যুয়ালাইজেশন, Variable ও Provisioning, কেন্দ্রীয় লগিং, Loki ও LogQL, ELK ও OpenSearch।

অধ্যায় ১২ • PART III — Visualization and Logging Engineering

Chapter 12 — Grafana Architecture, Dashboards, and Visualization

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Grafana-র আর্কিটেকচার ব্যাখ্যা করতে পারবেন — Grafana নিজে ডেটা সংরক্ষণ করে না, বরং `Data Source` থেকে প্রশ্ন করে এনে তা চিত্রে রূপ দেয়। আপনি `Dashboard`, `Panel`, `Row`, `Visualization` ও `Query`-এর সম্পর্ক বুঝবেন, সঠিক ভিজ্যুয়ালাইজেশন বাছতে পারবেন, এবং TechNova-র MERN অ্যাপ্লিকেশনের জন্য সিদ্ধান্ত নিতে সাহায্য করে এমন ড্যাশবোর্ড বানাতে পারবেন — শুধু সুন্দর গ্রাফ নয়, এমন ড্যাশবোর্ড যা দেখে একজন অন-কল ইঞ্জিনিয়ার বুঝতে পারেন সিস্টেম সুস্থ আছে কি না, কোথায় সমস্যা, এবং পরের পদক্ষেপ কী।

আপনি শিখবেন: Grafana Server, ডেটাবেস (SQLite/PostgreSQL/MySQL), `Plugin`, `Renderer`, `Alerting` ইঞ্জিন কীভাবে কাজ করে; Time Series, Stat, Gauge, Table, Heatmap, Bar Chart, Logs প্যানেল কখন ব্যবহার করতে হয়; ড্যাশবোর্ড ডিজাইনের নীতি — উপরে সারাংশ, নিচে বিস্তারিত; Golden Signals (Latency, Traffic, Errors, Saturation) কীভাবে সাজাতে হয়; এবং Linux সার্ভার, Node.js API, HTTP লেটেন্সি, Kubernetes ক্লাস্টার, ডেটাবেস ও ব্যবসায়িক availability-র জন্য প্রস্তুত ড্যাশবোর্ডের কাঠামো।

- Grafana-র ভূমিকা ব্যাখ্যা করা: visualization layer, storage নয়।
- Data Source, Dashboard, Panel ও Query-এর মধ্যে সম্পর্ক আঁকা।
- প্রতিটি ভিজ্যুয়ালাইজেশন ধরনের উপযুক্ত ব্যবহার চেনা।
- সিদ্ধান্ত-কেন্দ্রিক ড্যাশবোর্ড ডিজাইন করা — RED/USE পদ্ধতিতে।
- TechNova-র জন্য ছয়টি প্রোডাকশন ড্যাশবোর্ডের খসড়া তৈরি করা।

১২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Prometheus মেট্রিক সংগ্রহ করে, কিন্তু মানুষ সংখ্যার সারণি পড়ে সিদ্ধান্ত নিতে পারে না। Grafana সেই সংখ্যাকে চোখে দেখার মতো গল্পে বদলে দেয়। একটি ভালো ড্যাশবোর্ড ইনসিডেন্টের সময় মিনিট বাঁচায়: রাত ৩টায় ফোন বাজলে ইঞ্জিনিয়ার প্রথমে ড্যাশবোর্ড খোলেন — লাল প্যানেলটি কোনটি, কখন থেকে লাল, আর কোন সার্ভিসের সঙ্গে সম্পর্কিত — এই তিনটি উত্তর ৩০ সেকেন্ডে পাওয়া গেলে ভুল সার্ভিস রিস্টার্ট করার মতো ব্যয়বহুল ভুল এড়ানো যায়।

TechNova-র উদাহরণ নিন। তাদের MERN অ্যাপে হঠাৎ চেকআউট API ধীর হয়ে গেল। শুধু Prometheus থাকলে ইঞ্জিনিয়ারকে PromQL লিখে এক এক করে প্রশ্ন করতে হতো। Grafana ড্যাশবোর্ডে একই পাতায় দেখা যায়: HTTP লেটেন্সি বেড়েছে, একই সময়ে MongoDB-র query সময় বেড়েছে, কিন্তু CPU স্বাভাবিক — অর্থাৎ সমস্যা ডেটাবেসে, অ্যাপ সার্ভারে নয়। এই সম্পর্ক-দৃশ্যমানতাই Grafana-র আসল মূল্য।

> **REAL WORLD:** একটি ই-কমার্স কোম্পানিতে Black Friday-র সময় ড্যাশবোর্ডে শুধু গ্রাফ ছিল, কোনো সারাংশ ছিল না। ইঞ্জিনিয়াররা প্রতিটি গ্রাফ খুঁটিয়ে দেখতে ২০ মিনিট নষ্ট করলেন, অথচ উপরে একটি Stat প্যানেলে "p95 latency 2.4s — SLO breach" লেখা থাকলে সিদ্ধান্ত ২ মিনিটে হতো। ড্যাশবোর্ড ডিজাইন একটি ইঞ্জিনিয়ারিং সিদ্ধান্ত, সাজসজ্জা নয়।

আরও একটি কারণ: Grafana একটিমাত্র টুলে বহু উৎস একত্র করে। Prometheus-এর মেট্রিক, Loki-র লগ, Tempo-র ট্রেস, Elasticsearch-এর লগ — সব একই ড্যাশবোর্ডে পাশাপাশি দেখা যায়। এটিই `Single Pane of Glass` — একটি জানালা দিয়ে পুরো সিস্টেম দেখা। ছোট দলে, যেখানে আলাদা আলাদা টুল খুলে দেখার সময় নেই, এই একত্রীকরণ সরাসরি নির্ভরযোগ্যতা বাড়ায়।

১২.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায়ের জন্য আপনার Prometheus ও PromQL-এর মৌলিক ধারণা থাকা দরকার (অধ্যায় ৭–৮): মেট্রিকের ধরন, `rate()`, `histogram_quantile()`, লেবেল। HTTP-র মৌলিক ধারণা — status code, latency, throughput — জানা থাকতে হবে। Linux-এর CPU, মেমোরি, ডিস্ক, নেটওয়ার্কের ধারণা এবং Docker/Kubernetes-এর প্রাথমিক পরিচয় সাহায্য করবে।

Node.js ও Express-এর অভিজ্ঞতা থাকলে API ড্যাশবোর্ডের উদাহরণগুলো সহজ লাগবে, তবে বাধ্যতামূলক নয়। JSON ও YAML পড়তে পারা যথেষ্ট। Grafana আগে কখনো না চালিয়ে থাকলেও চলবে — এই অধ্যায়ে ইনস্টলেশন থেকে ড্যাশবোর্ড বানানো পর্যন্ত ধাপে ধাপে দেখানো হয়েছে।

পূর্বশর্ত	কেন দরকার	কোথায় শিখেছেন
Prometheus ও PromQL	Grafana প্রশ্ন পাঠায় Prometheus-কে; PromQL না জানলে প্যানেল বানানো যাবে না	অধ্যায় ৭–৮
HTTP ও API ধারণা	Latency, error rate, throughput বোঝার জন্য	অধ্যায় ৪–৫
Linux রিসোর্স	CPU, মেমোরি, ডিস্ক প্যানেলের অর্থ বোঝার জন্য	Volume 4
Docker Compose	ল্যাবে Grafana চালানোর জন্য	Volume 6

১২.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি থেকে শুরু করি: কম্পিউটার সিস্টেম সময়ের সঙ্গে বদলানো সংখ্যা তৈরি করে — প্রতি সেকেন্ডে কত অনুরোধ এলো, কত মিলিসেকেন্ড লাগলো, কত শতাংশ CPU ব্যবহৃত হলো। এই সংখ্যাগুলোকে বলে `Time Series` — সময়ের সঙ্গে যুক্ত সংখ্যার ধারা। মানুষ সময়ের ধারা সারণিতে পড়ে বুঝতে পারে না, কিন্তু গ্রাফে দেখলে তৎক্ষণাৎ বোঝে: রেখা উপরে উঠছে মানে খারাপ হচ্ছে, নিচে নামছে মানে ভালো হচ্ছে।

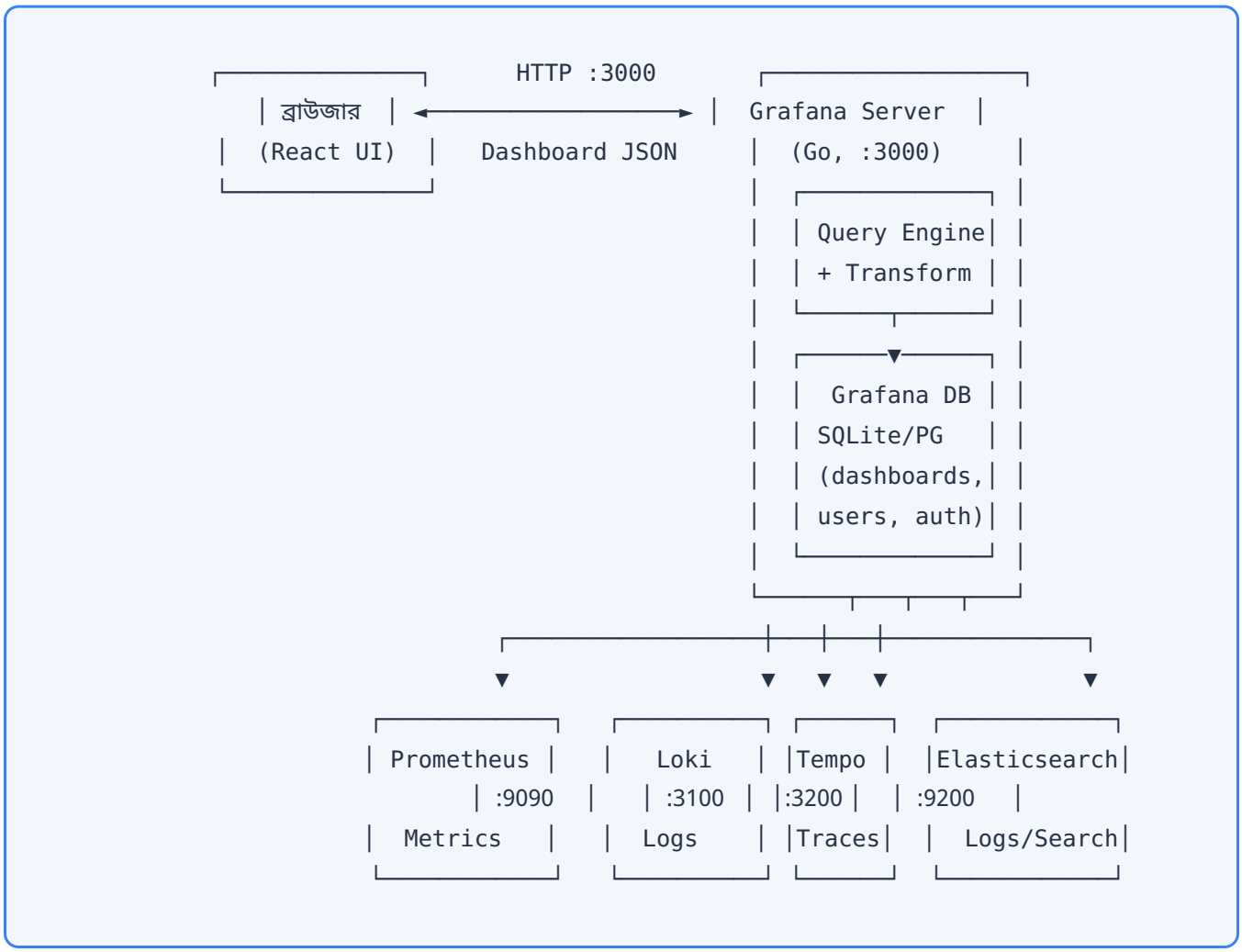
Grafana-র কাজ ঠিক এটাই: (১) কোনো ডেটা উৎসকে প্রশ্ন করা, (২) উত্তর হিসেবে time series পাওয়া, (৩) সেই series-কে মানুষের চোখে বোধগম্য চিত্রে বদলে দেওয়া, এবং (৪) সেই চিত্রগুলোকে এমনভাবে সাজানো যাতে সিদ্ধান্ত নেওয়া যায়। Grafana নিজে ডেটা তৈরি করে না, সংরক্ষণও করে না — সে একজন অনুবাদক ও সংগঠক। এই একটি সত্য বুঝলে Grafana-র সীমা ও শক্তি দুটোই পরিষ্কার হয়: ডেটা না থাকলে Grafana কিছু দেখাতে পারে না, আবার ডেটা থাকলে Grafana তাকে যেকোনো আকারে সাজাতে পারে।

> **CONCEPT:** Visualization মানে সাজানো নয়, মানে চিন্তা করা। একটি ভালো ভিজ্যুয়ালাইজেশন প্রশ্নের উত্তর দেয়: "এখন কি সব ঠিক আছে?" (Stat প্যানেল), "কখন থেকে খারাপ হলো?" (Time Series), "কোন অংশটা সবচেয়ে খারাপ?" (Table সাজানো), "বিতরণ কেমন?" (Heatmap)। ভিজ্যুয়ালাইজেশন বাছার আগে প্রশ্নটি লিখুন, তারপর চিত্রটি বাছুন।

দ্বিতীয় নীতি: ড্যাশবোর্ড একটি নথি নয়, একটি যন্ত্র। নথি পড়া হয় শুরু থেকে শেষ পর্যন্ত; যন্ত্র ব্যবহার করা হয় উদ্দেশ্য নিয়ে। ড্যাশবোর্ড খোলার উদ্দেশ্য সাধারণত তিনটি: (ক) সব ঠিক আছে কি না যাচাই করা, (খ) সমস্যার উৎস খুঁজে বের করা, (গ) পরিবর্তনের প্রভাব দেখা। প্রতিটি ড্যাশবোর্ডের শুরুতেই এই তিন উদ্দেশ্যের উত্তর থাকা উচিত।

১২.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র

Grafana একটি Go ভাষায় লেখা সার্ভার অ্যাপ্লিকেশন। ব্যবহারকারী ব্রাউজার দিয়ে Grafana-র ওয়েব UI খোলেন; ব্রাউজার সরাসরি Prometheus-কে প্রশ্ন করে না — Grafana সার্ভার মধ্যস্থতাকারী হিসেবে Data Source-কে প্রশ্ন করে, ফলাফল প্রক্রিয়া করে ব্রাউজারে পাঠায়। ড্যাশবোর্ডের সংজ্ঞা (কোন প্যানেল, কোন প্রশ্ন, কোন রঙ) Grafana-র নিজস্ব ডেটাবেসে JSON আকারে সংরক্ষিত থাকে।



গুরুত্বপূর্ণ উপাদানগুলো: `HTTP Server` ও `API` — UI ও বাইরের অনুরোধ সামলায়; `Data Source Proxy` — ব্রাউজারকে সরাসরি ডেটাবেসে যেতে না দিয়ে সার্ভার দিয়ে প্রশ্ন পাঠায়, ফলে গোপনীয় ক্রেডেনশিয়াল ব্রাউজারে ফাঁস হয় না; `Dashboard Provisioning` — ফাইল থেকে ড্যাশবোর্ড লোড করে (অধ্যায় ১৩); `Alerting Engine` — নিয়মিত প্রশ্ন চালিয়ে শর্ত মিললে নোটিফিকেশন পাঠায়; `Plugin System` — নতুন Data Source ও Panel ধরন যোগ করে; `Image Renderer` — অ্যালার্টে প্যানেলের ছবি যোগ করে।

Grafana-র নিজস্ব ডেটাবেসে কী থাকে আর কী থাকে না, তা মনে রাখুন: থাকে — ব্যবহারকারী, সংগঠন, ড্যাশবোর্ড JSON, Data Source-এর সংযোগ তথ্য, অ্যালার্ট নিয়ম। থাকে না — মেট্রিক, লগ বা ট্রেস ডেটা। সেগুলো থাকে Prometheus/Loki/

Tempo-তে। তাই Grafana ডাউন হলে ডেটা হারায় না, শুধু দেখার জানালাটি বন্ধ হয়। আবার Prometheus ডাউন হলে Grafana খালি প্যানেল দেখায় — "No data"।

১২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Dashboard, Panel, Row

`Dashboard` হলো প্যানেলের সংগ্রহ, একটি JSON নথি। প্রতিটি `Panel` একটি প্রশ্ন (query) + একটি ভিজ্যুয়ালাইজেশন + কিছু সেটিং (শিরোনাম, থ্রেশহোল্ড, একক)। `Row` প্যানেলগুলোকে দলে সাজায় এবং ভাঁজ করা যায় — যেমন "API Health" রো, "Database" রো। প্যানেলের ভেতরে `Query` চলে Data Source-এর ভাষায়: Prometheus-এর জন্য PromQL, Loki-র জন্য LogQL। একই প্যানেলে একাধিক query থাকতে পারে, এবং `Transform` দিয়ে ফলাফল বদলানো যায় — যেমন দুটি series যোগ করা বা সাজানো।

ভিজ্যুয়ালাইজেশন বাছাই

ভিজ্যুয়ালাইজেশন	যে প্রশ্নের উত্তর দেয়	কখন ব্যবহার করবেন	কখন নয়
Time Series	সময়ের সঙ্গে কীভাবে বদলালো?	Latency, CPU, request rate — বেশিরভাগ মেট্রিক	একটি মাত্র বর্তমান সংখ্যার জন্য
Stat	এখন মান কত?	SLO, বর্তমান error rate, সক্রিয় ব্যবহারকারী	প্রবণতা দেখাতে
Gauge	সীমার তুলনায় কতটা পূর্ণ?	ডিস্ক ব্যবহার, মেমোরি, CPU শতাংশ	সীমা না থাকলে
Table	কোনটি সবচেয়ে খারাপ/বড়?	সার্ভিসভিত্তিক তুলনা, শীর্ষ ধীর endpoint	সময়ের প্রবণতার জন্য
Heatmap	বিতরণ কেমন, কোথায় জমাট?	Latency histogram, অনুরোধের বিতরণ	একটি মাত্র মানের জন্য
Bar Chart	বিভাগভিত্তিক তুলনা	দৈনিক error সংখ্যা, status code	অনেক সময়বিন্দুর জন্য
Logs Panel	কী ঘটেছে, কোন লাইনে?	ড্যাশবোর্ডেই লগ দেখা, মেট্রিকের পাশে	গভীর লগ অনুসন্ধানের জন্য (Explore ব্যবহার করুন)

Golden Signals ও RED/USE

Google-এর SRE ধারণা অনুযায়ী প্রতিটি সার্ভিসের চারটি `Golden Signal` দেখা উচিত: `Latency` (কত সময় লাগে), `Traffic` (কত অনুরোধ), `Errors` (কত ব্যর্থ), `Saturation` (কতটা পূর্ণ)। API-র জন্য RED পদ্ধতি (Rate, Errors, Duration) এবং অবকাঠামোর জন্য USE পদ্ধতি (Utilization, Saturation, Errors) ব্যবহার করুন। TechNova-র API ড্যাশবোর্ডে RED-এর তিনটি প্যানেল উপরে থাকবে, নিচে MongoDB-র USE প্যানেল।

Threshold, Unit ও Legend

`Threshold` রঙের মাধ্যমে অর্থ যোগ করে: সবুজ মানে সুস্থ, হলুদ মানে সতর্ক, লাল মানে বিপদ। কিন্তু থ্রেশহোল্ড SLO থেকে আসা উচিত, আন্দাজ থেকে নয় — যেমন p95 latency 500ms-এর বেশি হলে হলুদ, 1000ms-এর বেশি হলে লাল। `Unit`

সঠিকভাবে দিন: সেকেন্ডকে `s`, বাইটকে `bytes`, শতাংশকে `percent` — ভুল একক ভুল সিদ্ধান্তের জন্ম দেয়। `Legend` সংক্ষিপ্ত রাখুন, লেবেলের অপ্রয়োজনীয় অংশ `Legend` ফরম্যাটে লুকান।

১২.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

Docker Compose দিয়ে Grafana চালানো ল্যাবের সবচেয়ে নির্ভরযোগ্য উপায়। নিচের প্রতিটি লাইনের অর্থ বুঝুন, অন্ধভাবে কপি করবেন না।

```
services:
  grafana:
    image: grafana/grafana:11.4.0
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=CHANGE-ME-GRAFANA-ADMIN
      - GF_USERS_ALLOW_SIGN_UP=false
    volumes:
      - grafana-data:/var/lib/grafana
      - ./provisioning:/etc/grafana/provisioning:ro
    restart: unless-stopped

volumes:
  grafana-data:
```

`image: grafana/grafana:11.4.0` — নির্দিষ্ট সংস্করণ পিন করা হয়েছে; `latest` ব্যবহার করবেন না, কারণ আপগ্রেড হঠাৎ করে আচরণ বদলে দিতে পারে। `ports: "3000:3000"` — হোস্টের 3000 পোর্ট কন্টেইনারের 3000-এ যায়; প্রোডাকশনে এর সামনে reverse proxy (Nginx) বসে। `GF_SECURITY_ADMIN_USER` / `GF_SECURITY_ADMIN_PASSWORD` — প্রথম অ্যাডমিন অ্যাকাউন্ট; পাসওয়ার্ড `CHANGE-ME-...` প্লেসহোল্ডার, আসল মান পরিবেশ ভেরিয়েবল বা secret ম্যানেজার থেকে দিন। `GF_USERS_ALLOW_SIGN_UP=false` — যে কেউ নিজে অ্যাকাউন্ট বানাতে পারবে না; এটি নিরাপত্তার মৌলিক সেটিং। `grafana-data` ভলিউম — ড্যাশবোর্ড ও ব্যবহারকারী কন্টেইনার মুছলেও থেকে যায়। `./provisioning` মাউন্ট — অধ্যায় ১৩-এর provisioning ফাইল এখান থেকে পড়া হয়।

সার্ভিস চালু ও যাচাই:

```
docker compose up -d grafana
docker compose logs -f grafana
# (নমুনা আউটপুট — চিত্রণমূলক)
# logger=server t=2026-11-09T10:00:00+06:00 msg="HTTP Server Listen" addr=0.0.0.0:3000
curl -s http://203.0.113.10:3000/api/health
# {"database":"ok","version":"11.4.0"}
```

`docker compose up -d grafana` কন্টেইনারটি পেছনে চালু করে। `logs -f` দিয়ে দেখুন সার্ভার ঠিকমতো উঠেছে কি না — "HTTP Server Listen" লাইনটি সফলতার চিহ্ন। `/api/health` এন্ডপয়েন্টটি মনিটরিংয়ের জন্য ব্যবহার করা যায়; এটি Grafana-র নিজের স্বাস্থ্য জানায়।

> **WARNING:** Grafana-র ডিফল্ট অ্যাডমিন পাসওয়ার্ড `admin/admin` কখনো প্রোডাকশনে রাখবেন না। প্রথম লগইনের পরপরই বদলে ফেলুন, এবং `GF_USERS_ALLOW_SIGN_UP=false` নিশ্চিত করুন। খোলা Grafana-তে আক্রমণকারী আপনার অবকাঠামোর মানচিত্র পেয়ে যায়।

১২.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র জন্য একটি সম্পূর্ণ Docker Compose — Grafana + Prometheus + Node Exporter একসঙ্গে। এই ফাইলটি ল্যাবে `docker compose up -d` দিয়ে চালানো যায়।

```
services:
  prometheus:
    image: prom/prometheus:v3.0.0
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - prometheus-data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.retention.time=15d"
    restart: unless-stopped

  node-exporter:
    image: prom/node-exporter:v1.8.2
    container_name: node-exporter
    ports:
      - "9100:9100"
    restart: unless-stopped

  grafana:
    image: grafana/grafana:11.4.0
    container_name: grafana
    ports:
      - "3000:3000"
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=CHANGE-ME-GRAFANA-ADMIN
      - GF_USERS_ALLOW_SIGN_UP=false
      - GF_SERVER_ROOT_URL=http://grafana.technova.example/
    volumes:
      - grafana-data:/var/lib/grafana
      - ./provisioning:/etc/grafana/provisioning:ro
    depends_on:
      - prometheus
    restart: unless-stopped

volumes:
  prometheus-data:
  grafana-data:
```

`GF_SERVER_ROOT_URL` অ্যালার্টের লিঙ্ক ও ছবি রেন্ডারিংয়ের জন্য দরকার — এটি না দিলে অ্যালার্টের লিঙ্ক ভুল হয়। `depends_on` শুধু শুরুর ক্রম নিয়ন্ত্রণ করে, Prometheus প্রস্তুত হওয়া পর্যন্ত অপেক্ষা করে না; তাই প্রথমবার Grafana খুলে "No data" দেখলে কয়েক সেকেন্ড অপেক্ষা করুন।

একটি Node.js API ড্যাশবোর্ডের মূল প্যানেলগুলোর PromQL প্রশ্ন (Dashboard JSON-এর ভেতরে এগুলোই থাকে):

```
# অনুরোধের হার (প্রতি সেকেন্ডে)
sum(rate(http_requests_total{job="technova-api"}[5m]))

# Error rate (শতাংশে)
sum(rate(http_requests_total{job="technova-api",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{job="technova-api"}[5m])) * 100

# p95 লেটেন্সি
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket{job="technova-api"}[5m])) by (le))

# CPU ব্যবহার (শতাংশে)
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

# মেমোরি ব্যবহার (শতাংশে)
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
```

১২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — ধীর API। সকাল ১০টায় গ্রাহকরা অভিযোগ করলেন চেকআউট ধীর। API ড্যাশবোর্ডে দেখা গেল p95 latency 180ms থেকে 2.1s-এ উঠেছে, কিন্তু শুধু `/api/checkout` endpoint-এ; অন্য endpoint স্বাভাবিক। Table প্যানেলে endpoint অনুযায়ী সাজানো লেটেন্সি দেখে বোঝা গেল সমস্যা একটি endpoint-এ সীমাবদ্ধ। পাশের MongoDB প্যানেলে দেখা গেল একই সময়ে ধীর query-র সংখ্যা বেড়েছে — মূল কারণ একটি ইনডেক্স ছাড়া query, যা সেদিনের ডিপ্লয়ে যোগ হয়েছিল।

পরিস্থিতি ২ — মেমোরি লিক। Node.js API ড্যাশবোর্ডের মেমোরি Time Series-এ দেখা গেল প্রতিটি ডিপ্লয়ের পর মেমোরি ধীরে ধীরে বাড়ছে, কখনো কমছে না — করাতের দাঁতের মতো (sawtooth) নয়, সিঁড়ির মতো। এটি মেমোরি লিকের চিরায়ত চিহ্ন। ড্যাশবোর্ডে heap ব্যবহারের আলাদা প্যানেল থাকায় দলটি restart-এর আগে লিকটি খুঁজে বের করতে পারল, এবং ডিপ্লয়ের সময়সূচি দেখে কোন রিলিজ থেকে লিক শুরু হয়েছে তা চিহ্নিত করল।

পরিস্থিতি ৩ — ডাউনস্ট্রিম ব্যর্থতা। পেমেন্ট গেটওয়ে ধীর হলে TechNova-র error rate বাড়ল, কিন্তু নিজেদের সার্ভারের CPU ও মেমোরি স্বাভাবিক। ড্যাশবোর্ডে বাইরের সার্ভিসের latency-র আলাদা প্যানেল থাকায় দলটি ৫ মিনিটে বুঝল সমস্যা নিজেদের নয়, গেটওয়ের — এবং গ্রাহকদের জানিয়ে timeout বাড়াল, অকারণে নিজেদের সার্ভার স্কেল করল না।

> **REAL WORLD:** একটি ড্যাশবোর্ডে ৪০টি প্যানেল থাকা মানেই ভালো ড্যাশবোর্ড নয়। ইনসিডেন্টের সময় কেউ ৪০টি গ্রাফ পড়ে না। প্রতিটি ড্যাশবোর্ডে সর্বোচ্চ ১২–১৫টি প্যানেল রাখুন; বিস্তারিত দেখতে আলাদা "drill-down" ড্যাশবোর্ডে লিঙ্ক দিন।

১২.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **ল্যাব চালু করুন:** উপরের Docker Compose ফাইলটি `~/technova-lab/` ডিরেক্টরিতে রেখে `docker compose up -d` চালান। `http://203.0.113.10:3000` খুলে অ্যাডমিন লগইন করুন।
2. **Data Source যোগ করুন:** Connections → Data Sources → Add → Prometheus → URL `http://prometheus:9090` → Save & Test। "Successfully queried" দেখা গেলে সংযোগ ঠিক আছে। একইভাবে Loki (`http://loki:3100`) যোগ করুন।
3. **প্রথম ড্যাশবোর্ড বানান:** Dashboards → New → New Dashboard → Add Visualization → Data Source Prometheus বেছে `sum(rate(http_requests_total[5m]))` লিখুন, Visualization Time Series রাখুন, শিরোনাম দিন "Request Rate"।
4. **Stat প্যানেল যোগ করুন:** আরেকটি প্যানেলে error rate-এর PromQL লিখে Visualization Stat বাছুন, Unit `percent`, Threshold 1 (হলুদ) ও 5 (লাল) দিন।
5. **সারাংশ সারি সাজান:** উপরে ৪টি Stat প্যানেল — Request Rate, Error Rate, p95 Latency, Uptime — তার নিচে Time Series, তার নিচে Table। এই ক্রমটি "সারাংশ → প্রবণতা → বিস্তারিত" নীতি অনুসরণ করে।
6. **সময়সীমা ও রিফ্রেশ ঠিক করুন:** ড্যাশবোর্ডের উপরে Time Range "Last 1 hour", Refresh "30s" দিন। প্রোডাকশনে 5s রিফ্রেশ দেবেন না — তা Prometheus-এর উপর অপ্রয়োজনীয় চাপ ফেলে।
7. **সংরক্ষণ ও ভাগ করা:** Save চাপুন, ফোল্ডার বাছুন, বর্ণনা লিখুন। Share → Link দিয়ে দলের সঙ্গে ভাগ করুন; প্রয়োজনে Snapshot তৈরি করুন, তবে সংবেদনশীল লেবেল থাকলে Snapshot এড়িয়ে চলুন।
8. **যাচাই করুন:** TechNova API-তে কৃত্রিম লোড দিন (`curl` লুপ বা k6), দেখুন গ্রাফ নড়ছে কি না। না নড়লে Data Source বা PromQL যাচাই করুন — ১২.১৪ দেখুন।

১২.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল / ভুল ধারণা	কেন ক্ষতিকর	সঠিক পদ্ধতি
"Grafana ডেটা সংরক্ষণ করে"	Grafana ডাউন হলে ডেটা হারাবে ভেবে ভুল সিদ্ধান্ত	Grafana শুধু দেখায়; ডেটা থাকে Prometheus/Loki-তে
ড্যাশবোর্ডে ৪০+ প্যানেল	ইনসিডেন্টে কিছুই খুঁজে পাওয়া যায় না, লোড ধীর হয়	সর্বোচ্চ ১২–১৫ প্যানেল; drill-down ড্যাশবোর্ড আলাদা
সব প্যানেলে একই Time Range	দীর্ঘ প্রবণতা ও বর্তমান অবস্থা একসঙ্গে দেখা যায় না	প্রয়োজনে প্যানেলে আলাদা Relative Time দিন
থ্রেশহোল্ড আন্দাজে দেওয়া	সবসময় লাল বা কখনো লাল নয় — অ্যালার্ট ক্লান্তি	SLO থেকে থ্রেশহোল্ড নিন
High-cardinality লেবেলে গ্রুপ করা	হাজারো series, ব্রাউজার ও Prometheus দুটোই ধীর	user_id বা request_id দিয়ে গ্রুপ করবেন না
5s রিফ্রেশ সব ড্যাশবোর্ডে	Prometheus-এ অপ্রয়োজনীয় query চাপ	30s–1m রিফ্রেশ; শুধু ইনসিডেন্টে কমান
ড্যাশবোর্ড JSON-এর ব্যাকআপ নেই	Grafana ডেটাবেস নষ্ট হলে সব ড্যাশবোর্ড হারায়	Provisioning বা Git-এ JSON সংরক্ষণ করুন

TROUBLESHOOTING: "No data" দেখলে প্রথমে Explore-এ গিয়ে একই query চালান। Explore-এ ডেটা এলে সমস্যা প্যানেলের সেটিংয়ে (সময়সীমা, transform); Explore-এও না এলে সমস্যা Data Source বা মেট্রিকের নামে।

১২.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Grafana-তে নিরাপত্তা মানে তিনটি স্তর: কে ঢুকতে পারে, কে কী দেখতে পারে, এবং ডেটা কোথায় যায়। প্রথমে `Authentication` — প্রোডাকশনে শুধু ব্যবহারকারী-পাসওয়ার্ড যথেষ্ট নয়; OAuth (Google/GitHub) বা LDAP ব্যবহার করুন, এবং অ্যাডমিন অ্যাকাউন্টের জন্য শক্তিশালী পাসওয়ার্ড ও দ্বি-ধাপ যাচাই চালু করুন। `GF_USERS_ALLOW_SIGN_UP=false` ও `GF_AUTH_DISABLE_LOGIN_FORM` (প্রয়োজনে) সেট করুন।

দ্বিতীয় স্তর `Authorization` — `Organization`, `Folder` ও `Role` (Viewer/Editor/Admin) দিয়ে নিয়ন্ত্রণ করুন। Viewer-রা ড্যাশবোর্ড দেখতে পারে কিন্তু বদলাতে পারে না; Editor-রা বানাতে পারে; Admin-রা Data Source বদলাতে পারে। Data Source-এর ক্রেডেনশিয়াল কখনো Viewer-দের কাছে দৃশ্যমান হওয়া উচিত নয় — Grafana-র Data Source Proxy এই কাজটি করে: ব্রাউজার সরাসরি Prometheus-এ যায় না, সার্ভার হয়ে যায়।

তৃতীয় স্তর গোপনীয়তা: ড্যাশবোর্ডের লেবেলে কখনো পাসওয়ার্ড, টোকেন, ইমেইল বা ব্যক্তিগত তথ্য রাখবেন না। `Snapshot` শেয়ার করার সময় মনে রাখবেন Snapshot-এ ডেটার একটি অনুলিপি থাকে — সংবেদনশীল ড্যাশবোর্ডের Snapshot প্রকাশ্যে শেয়ার করবেন না। Grafana-কে HTTPS-এর পেছনে রাখুন (Nginx reverse proxy), এবং `GF_SERVER_COOKIE_SECURE=true` সেট করুন।

WARNING: Grafana-র `Explore` থেকে কেউ যেকোনো query চালাতে পারে। Viewer-দের জন্য Data Source-এর অনুমতি সীমিত করুন, নইলে একজন Viewer উচ্চ-কার্ডিনালিটি query চালিয়ে Prometheus-কে ধীর করে দিতে পারে।

১২.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Grafana নিজে হালকা, কিন্তু ভুল ড্যাশবোর্ড ভারী হতে পারে। প্রতিটি প্যানেল প্রতিটি রিফ্রেশে একটি query পাঠায়। ২০টি প্যানেল × 5s রিফ্রেশ × ১০ জন দর্শক = প্রতি সেকেন্ডে ৪০টি query — Prometheus-এর উপর বড় চাপ। তাই রিফ্রেশ 30s বা 1m রাখুন, এবং একসঙ্গে অনেক প্যানেল খোলা ড্যাশবোর্ডে `Max data points` সীমিত করুন।

Query-র দিক থেকে: `rate(...[5m])`-এর মতো range vector ব্যবহার করুন, কাঁচা `http_requests_total` কখনো সরাসরি গ্রাফ করবেন না — কাউন্টার ক্রমাগত বাড়ে, গ্রাফটি অর্থহীন হয়। High-cardinality লেবেল (`user_id`, `request_id`) দিয়ে `by` করবেন না; এতে হাজারো series তৈরি হয়, ব্রাউজার ধীর হয় এবং Prometheus-এর মেমোরি বাড়ে। বড় সময়সীমায় (৭ দিন) `Recording Rule` দিয়ে আগে থেকে হিসাব করা মেট্রিক ব্যবহার করুন (অধ্যায় ৮)।

খরচের দিক থেকে Grafana OSS বিনামূল্যে; খরচ আসে অবকাঠামো থেকে — Prometheus-এর ডিস্ক, Loki-র স্টোরেজ। ড্যাশবোর্ডের সংখ্যা বাড়লে Grafana-র ডেটাবেস বড় হয় না বললেই চলে, কিন্তু query চাপ বাড়ে। তাই ড্যাশবোর্ডের সংখ্যা নয়, query-র দক্ষতাই খরচ নির্ধারণ করে।

১২.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	সমাধান

"No data" সব প্যানেলে	Data Source সংযোগ বিচ্ছিন্ন বা Prometheus ডাউন	Data Sources → Test; `curl http://prometheus:9090/-/healthy` চালান
"No data" একটি প্যানেলে	ভুল মেট্রিক নাম, ভুল লেবেল, বা সময়সীমায় ডেটা নেই	Explore-এ query চালিয়ে যাচাই করুন; `up` মেট্রিক দিয়ে scrape হচ্ছে কি না দেখুন
গ্রাফে ফাঁকা জায়গা (gap)	Scrape ব্যর্থ বা Grafana-র সময় Prometheus-এর সময়ের সঙ্গে মিলছে না	Prometheus Targets পাতা দেখুন; সার্ভারের ঘড়ি NTP দিয়ে মেলান
ড্যাশবোর্ড ধীরে লোড হয়	অনেক প্যানেল, ভারী query, বড় সময়সীমা	প্যানেল কমান, Recording Rule ব্যবহার করুন, রিফ্রেশ বাড়ান
ভুল মান দেখাচ্ছে	ভুল Unit বা ভুল aggregation (sum বনাম avg)	Unit যাচাই করুন; Explore-এ কাঁচা মান দেখুন
অ্যালার্টের লিঙ্ক ভুল	`GF_SERVER_ROOT_URL` সেট করা নেই	সঠিক বাইরের URL সেট করুন
লগইন করা যাচ্ছে না	অ্যাডমিন পাসওয়ার্ড ভুলে গেছেন	`grafana-cli admin reset-admin-password CHANGE-ME-NEW-PASSWORD`

একটি কার্যকর তদন্ত ক্রম: (১) Data Source Test → (২) Explore-এ query → (৩) Prometheus-এ সরাসরি query → (৪) Targets/Scrape যাচাই → (৫) Grafana লগ (`docker compose logs grafana`)। এই ক্রম মেনে চললে বেশিরভাগ সমস্যা ১০ মিনিটে চিহ্নিত করা যায়।

১২.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: উপরে সারাংশ, নিচে বিস্তারিত; প্রতিটি ড্যাশবোর্ডে উদ্দেশ্য লেখা Text প্যানেল; SLO-ভিত্তিক থ্রেশহোল্ড; ড্যাশবোর্ড JSON Git-এ সংরক্ষণ; ড্যাশবোর্ডের নামকরণে ধারাবাহিকতা (`[TechNova] API — Overview`); প্রতিটি ড্যাশবোর্ডে মালিকের নাম ও Runbook লিঙ্ক; নিয়মিত পর্যালোচনা — কোন ড্যাশবোর্ড কেউ দেখে না, তা মুছে ফেলা।

অ্যান্টি-প্যাটার্ন: "Wall of graphs" — উদ্দেশ্য ছাড়া গ্রাফের দেওয়াল; কপি-পেস্ট ড্যাশবোর্ড যার নাম ও query মেলে না; থ্রেশহোল্ড ছাড়া লাল-সবুজ রঙের অপব্যবহার; প্রতিটি প্যানেলে আলাদা সময়সীমা; সংবেদনশীল লেবেলের প্যানেল প্রকাশ্যে শেয়ার করা; ড্যাশবোর্ডকে অ্যালার্টের বিকল্প ভাবা — ড্যাশবোর্ড দেখে সমস্যা ধরা যায়, কিন্তু কেউ না দেখলে সমস্যা জানা যায় না; অ্যালার্টই সেই কাজ করে (অধ্যায় ১৩)।

> **CONCEPT:** একটি ড্যাশবোর্ডের সফলতা মাপা হয় "time to understand" দিয়ে — নতুন একজন ইঞ্জিনিয়ার কত সেকেন্ডে বুঝতে পারেন সিস্টেমের অবস্থা। ৩০ সেকেন্ডের বেশি লাগলে ড্যাশবোর্ডটি পুনরায় ডিজাইন করুন।

১২.১৬ অধ্যায়ের সারাংশ

Grafana ডেটা সংরক্ষণ করে না — সে Data Source থেকে প্রশ্ন করে এনে মানুষের বোধগম্য চিত্রে রূপ দেয়। Dashboard হলো প্যানেলের সংগ্রহ, Panel হলো query + visualization, এবং সঠিক ভিজ্যুয়ালাইজেশন বাছাই নির্ভর করে প্রশ্নের উপর: বর্তমান মানের জন্য Stat, প্রবণতার জন্য Time Series, তুলনার জন্য Table, বিতরণের জন্য Heatmap। ভালো ড্যাশবোর্ড উপরে সারাংশ, নিচে বিস্তারিত সাজায়, Golden Signals (Latency, Traffic, Errors, Saturation) দেখায়, SLO-ভিত্তিক থ্রেশহোল্ড ব্যবহার করে, এবং ১২–১৫টি প্যানেলের মধ্যে সীমাবদ্ধ থাকে। TechNova-র জন্য Linux সার্ভার, Node.js API,

HTTP লেটেন্সি, Kubernetes, ডেটাবেস ও ব্যবসায়িক availability ড্যাশবোর্ড একই নীতি অনুসরণ করে। নিরাপত্তা, query দক্ষতা ও ব্যাকআপ — এই তিনটি ছাড়া ড্যাশবোর্ড প্রোডাকশন-প্রস্তুত নয়।

১২.১৭ মূল প্রযুক্তিগত পরিভাষা

Grafana: মেট্রিক, লগ ও ট্রেসকে চিত্রে রূপ দেওয়ার visualization প্ল্যাটফর্ম; নিজে ডেটা সংরক্ষণ করে না।

Data Source: Grafana যেখান থেকে ডেটা প্রশ্ন করে আনে — যেমন Prometheus, Loki, Elasticsearch।

Dashboard: একাধিক প্যানেলের সংগ্রহ, JSON আকারে সংরক্ষিত একটি দৃশ্য।

Panel: ড্যাশবোর্ডের একক একক — একটি query ও একটি ভিজ্যুয়ালাইজেশনের সমন্বয়।

Visualization: ডেটা দেখানোর ধরন — Time Series, Stat, Gauge, Table, Heatmap ইত্যাদি।

Time Series: সময়ের সঙ্গে বদলানো সংখ্যার ধারা; মেট্রিকের মৌলিক রূপ।

Golden Signals: সার্ভিস স্বাস্থ্যের চারটি সংকেত — Latency, Traffic, Errors, Saturation।

Threshold: মানের সীমা, যার ভিত্তিতে প্যানেলের রঙ বদলায় — সবুজ, হলুদ, লাল।

Transform: Query-র ফলাফলকে প্যানেলে দেখানোর আগে বদলানোর ধাপ — যোগ, সাজানো, ফিল্টার।

Single Pane of Glass: একটি জানালা দিয়ে পুরো সিস্টেম দেখা — বহু উৎসের ডেটা এক ড্যাশবোর্ডে।

১২.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Grafana নিজে কি মেট্রিক সংরক্ষণ করে? না করলে ডেটা কোথায় থাকে?
2. Dashboard, Panel ও Query-এর মধ্যে সম্পর্ক ব্যাখ্যা করুন।
3. বর্তমান error rate দেখাতে কোন ভিজ্যুয়ালাইজেশন সবচেয়ে উপযুক্ত, এবং কেন?
4. Latency-র বিতরণ দেখতে Heatmap কেন Time Series-এর চেয়ে ভালো হতে পারে?
5. Golden Signals চারটি কী কী? TechNova-র API-র জন্য প্রতিটির একটি করে উদাহরণ দিন।
6. থ্রেশহোল্ড আন্দাজে দেওয়া কেন বিপজ্জনক?
7. High-cardinality লেবেল দিয়ে গ্রুপ করলে কী সমস্যা হয়?

8. ড্যাশবোর্ডে ৪০টি প্যানেল থাকলে ইনসিডেন্টের সময় কী অসুবিধা হয়?
9. Data Source Proxy কী, এবং এটি নিরাপত্তায় কীভাবে সাহায্য করে?
10. "No data" দেখলে আপনি কোন ক্রমে তদন্ত করবেন?

১২.১৯ ব্যবহারিক অনুশীলন

TechNova ল্যাবে Grafana চালু করে নিচের কাজগুলো করুন:

1. Docker Compose দিয়ে Grafana + Prometheus + Node Exporter চালু করুন এবং `/api/health` যাচাই করুন।
2. Prometheus Data Source যোগ করুন এবং Save & Test সফল করুন।
3. "[TechNova] API — Overview" নামে একটি ড্যাশবোর্ড বানান: উপরে ৪টি Stat প্যানেল (Request Rate, Error Rate, p95 Latency, Uptime), নিচে ২টি Time Series (Latency, CPU/Memory)।
4. একটি Table প্যানেল যোগ করুন যাতে endpoint অনুযায়ী p95 latency সাজানো থাকে।
5. Error Rate প্যানেলে 1% ও 5% থ্রেশহোল্ড দিন এবং রঙ যাচাই করুন।
6. ড্যাশবোর্ডটি JSON আকারে ডাউনলোড করে Git-এ সংরক্ষণ করুন এবং একটি সহকর্মীকে দেখিয়ে ৩০ সেকেন্ডে সিস্টেমের অবস্থা বোঝা যায় কি না যাচাই করুন।
7. ইচ্ছাকৃতভাবে Prometheus থামিয়ে "No data" অবস্থাটি দেখুন, তারপর আবার চালু করে পুনরুদ্ধার যাচাই করুন।

অধ্যায় ১৩ • PART III — Visualization and Logging Engineering

Chapter 13 — Grafana Variables, Alerts, Provisioning, and Data Sources

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি একটি ড্যাশবোর্ডকে শত শত সার্ভিসের জন্য পুনঃব্যবহারযোগ্য করতে পারবেন — `Variable` ও `Templating` দিয়ে। আপনি Grafana Alerting-এর সম্পূর্ণ প্রবাহ বুঝবেন: `Alert Rule`, `Contact Point`, `Notification Policy`, `Silence` ও `Mute Timing`। আপনি `Provisioning` দিয়ে Data Source ও Dashboard-কে কোড হিসেবে সংজ্ঞায়িত করতে পারবেন, `Annotation` দিয়ে ড্যাশবোর্ডে ডিপ্লয় ও ইনসিডেন্ট চিহ্নিত করতে পারবেন, এবং Organization, Folder ও Role দিয়ে অ্যাক্সেস নিয়ন্ত্রণ করতে পারবেন।

- Variable (query, custom, interval, datasource) তৈরি ও ব্যবহার করা।
- Alert rule লেখা, থ্রেশহোল্ড ও evaluation interval ঠিক করা।
- Contact point ও notification policy দিয়ে সঠিক ব্যক্তিকে সঠিক মাধ্যমে জানানো।
- Provisioning YAML দিয়ে reproducible Grafana গড়া।
- Dashboard-as-code ও GitOps প্রবাহ ব্যাখ্যা করা।

১৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Variable ছাড়া প্রতিটি সার্ভিসের জন্য আলাদা ড্যাশবোর্ড বানাতে হয়। TechNova-তে ৫টি সার্ভিস থাকলে ৫টি ড্যাশবোর্ড, ২০টি হলে ২০টি — রক্ষণাবেক্ষণ অসম্ভব হয়ে যায়। Variable দিয়ে একটি ড্যাশবোর্ডেই ড্রপডাউন থেকে সার্ভিস বদলানো যায়। এটি শুধু সুবিধা নয়, এটি নির্ভরযোগ্যতা: একটি ড্যাশবোর্ড ঠিক করলে সব সার্ভিসের দৃশ্য ঠিক হয়।

Alerting আরও গুরুত্বপূর্ণ: ড্যাশবোর্ড কেউ না দেখলে সমস্যা জানা যায় না। Grafana Alerting নিয়মিত query চালায় এবং শর্ত মিললে মানুষকে জানায় — Slack, ইমেইল, PagerDuty বা Webhook-এ। কিন্তু ভুল অ্যালার্ট ভয়ংকর: খুব বেশি অ্যালার্ট এলে মানুষ সব অ্যালার্ট উপেক্ষা করতে শেখে (`Alert Fatigue`), আর খুব কম অ্যালার্ট এলে বিপদ নীরবে বাড়ে। সঠিক অ্যালার্ট ডিজাইন তাই DevOps-এর কেন্দ্রীয় দক্ষতা।

Provisioning গুরুত্বপূর্ণ কারণ হাতে ক্লিক করে বানানো Grafana পুনরায় তৈরি করা যায় না। সার্ভার নষ্ট হলে, নতুন পরিবেশে গেলে, বা দলের নতুন সদস্য এলে — কোড থেকে Grafana গড়ে ওঠা উচিত। এটিই `Infrastructure as Code`-এর ধারাবাহিকতা: ড্যাশবোর্ডও কোড।

> **REAL WORLD:** TechNova-তে একবার হাতে বানানো ৩০টি ড্যাশবোর্ড Grafana ডেটাবেস নষ্ট হয়ে হারিয়ে গিয়েছিল। দলটি দুই সপ্তাহ ধরে আবার বানিয়েছিল। তারপর থেকে সব ড্যাশবোর্ড Git-এ JSON হিসেবে থাকে এবং Provisioning দিয়ে লোড হয় — এখন Grafana মুছে ফেললেও ৫ মিনিটে ফিরে আসে।

১৩.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১২-এর Grafana-র মৌলিক ধারণা, PromQL, এবং YAML-এর প্রাথমিক জ্ঞান দরকার। Alerting বুঝতে অধ্যায় ৪-এর SLI/SLO ও অধ্যায় ১০-এর Alertmanager ধারণা সাহায্য করবে — Grafana Alerting ও Alertmanager আলাদা

হলেও নীতি এক: শর্ত, রাউটিং, নোটিফিকেশন। Git-এর মৌলিক ধারণা (commit, repository) Provisioning অংশের জন্য যথেষ্ট।

পূর্বশর্ত	কেন দরকার
Grafana Dashboard ও Panel	Variable ও Alert প্যানেলের উপরেই বসে
PromQL	Alert rule-এর query লিখতে
SLO ও Error Budget	অ্যালার্টের থ্রেশহোল্ড ঠিক করতে
YAML	Provisioning ফাইল লিখতে

১৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

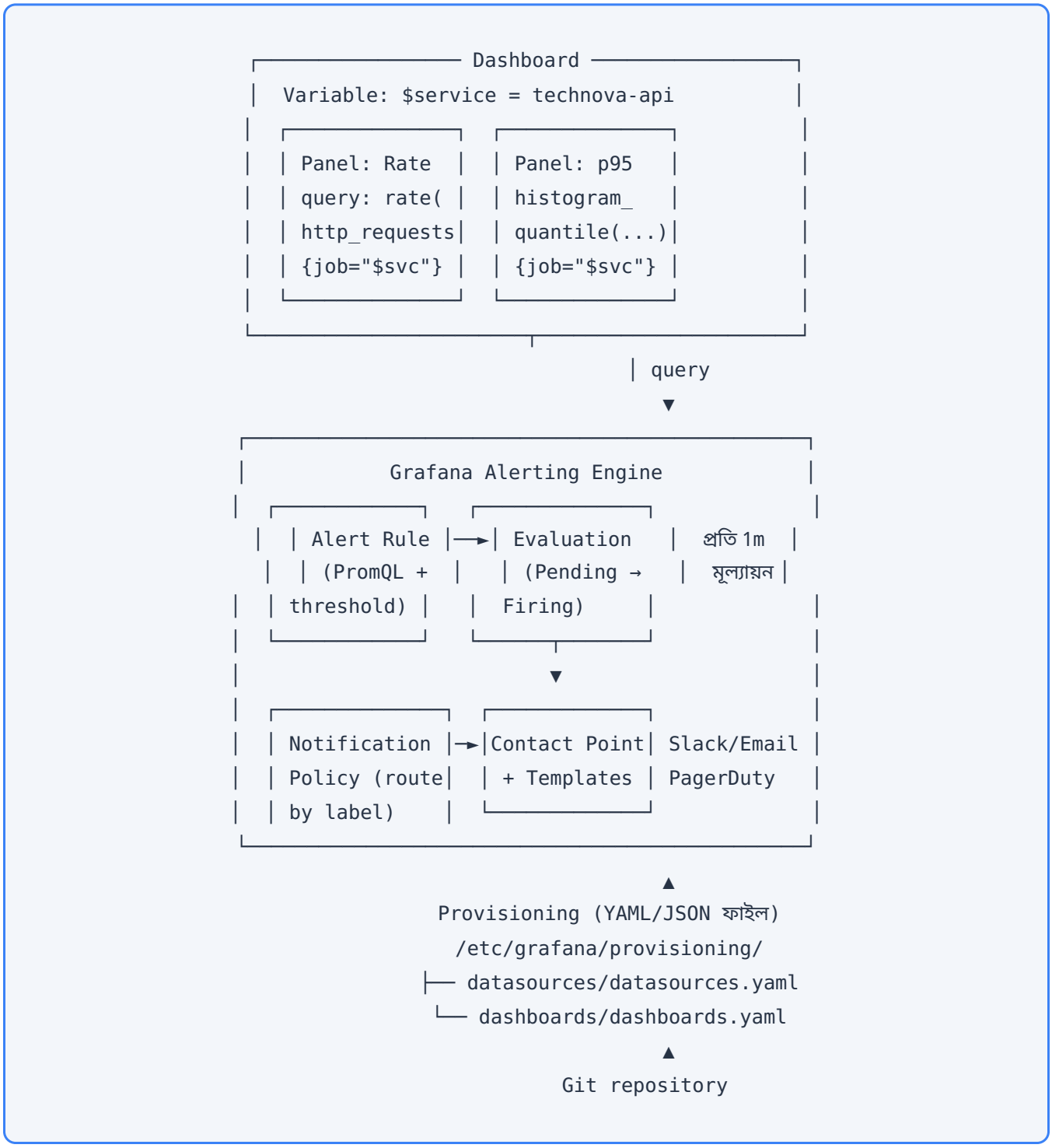
প্রথম নীতি: পুনরাবৃত্তি কমানো। একটি ড্যাশবোর্ডে `job="technova-api"` লেখা থাকলে সেটি শুধু একটি সার্ভিসের জন্য কাজ করে। কিন্তু যদি `job`-এর জায়গায় একটি চলক বসানো যায় — `job="$service"` — তাহলে একই ড্যাশবোর্ড যেকোনো সার্ভিসের জন্য কাজ করে। `Variable` হলো সেই চলক: ড্যাশবোর্ডের উপরে একটি ড্রপডাউন, যার মান query-র ভেতরে বসে যায়। এটি প্রোগ্রামিংয়ের ফাংশন প্যারামিটারের মতো — একই কোড, ভিন্ন ইনপুট।

দ্বিতীয় নীতি: মানুষের নজরদারির বদলে যন্ত্রের নজরদারি। মানুষ ২৪ ঘণ্টা গ্রাফ দেখতে পারে না। তাই নিয়ম লিখে যন্ত্রকে বলা হয়: "প্রতি মিনিটে এই প্রশ্নটি চালাও; উত্তর এই সীমা ছাড়ালে আমাকে জানাও।" এটিই `Alert Rule`। কিন্তু জানানোর কাজটিও নীতিমালা চায়: কাকে জানাবে, কোন মাধ্যমে, কখন চুপ থাকবে — এটিই `Notification Policy`। শর্ত, রাউটিং ও মাধ্যম — এই তিনটি আলাদা করাই Grafana Alerting-এর স্থাপত্যের মূল বুদ্ধি।

তৃতীয় নীতি: যা হাতে করা যায়, তা কোডে লেখা যায়। Grafana-র UI-তে ক্লিক করে যা করা যায়, তার প্রতিটি জিনিসের একটি JSON/YAML প্রতিরূপ আছে। সেই প্রতিরূপ Git-এ রাখলে পরিবর্তন পর্যালোচনা করা যায়, পূর্বাবস্থায় ফেরা যায়, এবং যেকোনো পরিবেশে একই Grafana গড়া যায়। এটিই Provisioning ও Dashboard-as-code-এর প্রথম নীতি।

> **CONCEPT:** Variable, Alert ও Provisioning — তিনটিই একই প্রশ্নের উত্তর: "এই কাজটি কি একবার করে বারবার ব্যবহার করা যায়?" ড্যাশবোর্ডের জন্য Variable, নজরদারির জন্য Alert Rule, আর পুরো Grafana-র জন্য Provisioning।

১৩.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



Alert-এর জীবনচক্র: `Normal` → শর্ত মিললে `Pending` (নির্ধারিত সময় ধরে শর্ত থাকতে হবে, যেমন 5 মিনিট) → তারপর `Firing` → নোটিফিকেশন যায়। `Pending` পর্যায়টি গুরুত্বপূর্ণ: ক্ষণিকের স্পাইকে অ্যালার্ট যায় না। শর্ত শেষ হলে `Resolved` নোটিফিকেশন যায়। `Silence` একটি সময়ের জন্য নোটিফিকেশন থামায়, `Mute Timing` নিয়মিত সময়ে (যেমন রক্ষণাবেক্ষণের জানালা) থামায়।

Provisioning-এর প্রবাহ: Grafana চালু হওয়ার সময় `/etc/grafana/provisioning/` ডিরেক্টরি পড়ে। `datasources.yaml` Data Source তৈরি করে, `dashboards.yaml` বলে দেয় কোন ফোল্ডারের JSON ড্যাশবোর্ড লোড

করতে হবে। ফাইল বদলালে Grafana পুনরায় লোড করে (বা পুনরায় চালু করতে হয়)। Provisioning করা ড্যাশবোর্ড UI থেকে সম্পাদনা করা যায় না — বদলাতে হলে ফাইল বদলাতে হয়, এটিই উদ্দেশ্য।

১৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Variable-এর ধরন

ধরন	অর্থ	উদাহরণ
Query	Data Source থেকে মানের তালিকা আনে	`label_values(http_requests_total, job)`
Custom	হাতে লেখা তালিকা	`prod,staging,dev`
Interval	সময় ব্যবধান বাছাই	`1m,5m,15m,1h` — `rate(...[$interval])`
Datasource	কোন Data Source ব্যবহার হবে তা বাছাই	Prometheus prod বনাম staging
Constant	লুকানো স্থির মান	ড্যাশবোর্ডে পরিবেশের নাম লুকিয়ে রাখা
Textbox	ব্যবহারকারী নিজে মান লেখে	নির্দিষ্ট user_id খোঁজা (সাবধানে)

`Multi-value` চালু করলে একসঙ্গে একাধিক মান বাছা যায়, এবং `Include All option` দিলে সব মান একসঙ্গে দেখা যায়। PromQL-এ Variable ব্যবহারের সময় `=~` (regex match) ব্যবহার করুন: `job=~"$service"` — একক মানেও কাজ করে, বহু মানেও।

Grafana Alerting বনাম Prometheus Alertmanager

দুটোই অ্যালার্ট পাঠায়, কিন্তু জায়গা আলাদা। Prometheus Alertmanager Prometheus-এর নিয়ম থেকে অ্যালার্ট পায়; Grafana Alerting যেকোনো Data Source (Prometheus, Loki, Elasticsearch, এমনকি SQL) থেকে query করে অ্যালার্ট দিতে পারে। TechNova-তে Prometheus-এর নিয়ম Alertmanager-এ যাওয়া ভালো অভ্যাস, আর Loki-র লগ-ভিত্তিক অ্যালার্ট (যেমন "৫ মিনিটে ১০টির বেশি 500 error লগ") Grafana Alerting-এ ভালো। দুটো একসঙ্গে চালানো যায়, তবে একই অ্যালার্ট দুই জায়গা থেকে যাওয়া এড়াতে দায়িত্ব ভাগ করে নিন।

Annotation

`Annotation` হলো ড্যাশবোর্ডের সময়রেখায় চিহ্ন — যেমন ডিপ্লয়ের সময় একটি উল্লম্ব রেখা। ডিপ্লয়ের পর latency বাড়লে রেখাটি দেখে বোঝা যায় ডিপ্লয়ই কারণ। Annotation আসতে পারে Loki-র লগ থেকে ("deploy" শব্দ), Grafana-র নিজস্ব API থেকে, বা ডেটাবেস থেকে। TechNova-তে প্রতিটি ডিপ্লয়ে একটি annotation লেখা হয় — এটি ইনসিডেন্ট তদন্তে সবচেয়ে সস্তা ও কার্যকর টুলগুলোর একটি।

Organization, Folder, Access Control

`Organization` হলো Grafana-র ভেতরে আলাদা জগৎ — প্রতিটি Org-এর নিজস্ব Data Source, Dashboard ও ব্যবহারকারী থাকে। ছোট দলে একটি Org-ই যথেষ্ট। `Folder` ড্যাশবোর্ড সাজায় এবং অনুমতি দেয়: "Payments" ফোল্ডার শুধু পেমেন্ট দল দেখবে। `Role` তিনটি: Viewer (দেখা), Editor (বানানো), Admin (পরিচালনা)। `Data Source Permission` দিয়ে নির্দিষ্ট Data Source শুধু নির্দিষ্ট দলের জন্য সীমিত করা যায় — যেমন প্রোডাকশন Prometheus শুধু অন-কল দল query করতে পারবে।

১৩.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

Data Source Provisioning — `provisioning/datasources/datasources.yaml`:

```
apiVersion: 1
datasources:
  - name: Prometheus-Prod
    type: prometheus
    access: proxy
    url: http://prometheus:9090
    isDefault: true
    editable: false
    jsonData:
      timeInterval: 15s
      httpMethod: POST
  - name: Loki-Prod
    type: loki
    access: proxy
    url: http://loki:3100
    editable: false
```

`apiVersion: 1` — provisioning ফরম্যাটের সংস্করণ। `name` ড্যাশবোর্ডে দেখানো নাম; `type: prometheus` কোন প্লাগইন ব্যবহার হবে তা বলে। `access: proxy` মানে ব্রাউজার সরাসরি যায় না, Grafana সার্ভার হয়ে যায় — ক্রেডেনশিয়াল লুকানো থাকে; `direct` ব্যবহার করবেন না। `isDefault: true` — নতুন প্যানেলে এই Data Source আগে থেকে বাছা থাকে। `editable: false` — UI থেকে কেউ বদলাতে পারবে না; বদলাতে হলে ফাইল বদলাতে হয়। `jsonData.timeInterval` — query-র ন্যূনতম ব্যবধান; `httpMethod: POST` দীর্ঘ query URL-এর সীমা এড়ায়।

Dashboard Provisioning — `provisioning/dashboards/dashboards.yaml`:

```
apiVersion: 1
providers:
  - name: technova-dashboards
    orgId: 1
    folder: TechNova
    type: file
    disableDeletion: false
    updateIntervalSeconds: 30
    allowUiUpdates: false
    options:
      path: /etc/grafana/provisioning/dashboards/json
      foldersFromFilesStructure: true
```

`folder: TechNova` — ড্যাশবোর্ডগুলো এই ফোল্ডারে যাবে। `updateIntervalSeconds: 30` — প্রতি ৩০ সেকেন্ডে ফাইল বদলেছে কি না দেখে। `allowUiUpdates: false` — UI-তে বদলালে তা সংরক্ষিত হবে না; এতে Git-ই সত্যের উৎস থাকে। `foldersFromFilesStructure: true` — JSON ফাইলের ডিরেক্টরি কাঠামো ফোল্ডারে রূপ নেয়।

একটি Alert Rule-এর query (UI-তে বা API-তে):

```
# ৫ মিনিটে error rate ৫%-এর বেশি থাকলে Firing
sum(rate(http_requests_total{job="technova-api",status=~"5.."}[5m]))
  / sum(rate(http_requests_total{job="technova-api"}[5m])) * 100
# শর্ত: IS ABOVE 5
```

```
# Evaluate every: 1m, For: 5m
# Labels: severity=critical, service=technova-api, team=backend
```

`Evaluate every: 1m` মানে প্রতি মিনিটে query চলে; `For: 5m` মানে ৫ মিনিট ধরে শর্ত থাকলে তবেই Firing হয় — ক্ষণিকের স্পাইক বাদ যায়। `Labels` নোটিফিকেশন রাউটিংয়ে ব্যবহৃত হয়: `severity=critical` PagerDuty-তে যাবে, `severity=warning` Slack-এ।

১৩.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র সম্পূর্ণ provisioning ডিরেক্টরি কাঠামো ও একটি Variable-সহ ড্যাশবোর্ডের মূল অংশ:

```
provisioning/
├── datasources/
│   └── datasources.yaml
├── dashboards/
│   ├── dashboards.yaml
│   └── json/
│       ├── api-overview.json
│       └── server-health.json
└── alerting/
    ├── alert-rules.yaml
    └── contact-points.yaml

# api-overview.json — Variable সংজ্ঞা (সংক্ষিপ্ত)
{
  "title": "[TechNova] API — Overview",
  "templating": {
    "list": [
      {
        "name": "service",
        "type": "query",
        "datasource": "Prometheus-Prod",
        "query": "label_values(http_requests_total, job)",
        "multi": true,
        "includeAll": true
      },
      {
        "name": "interval",
        "type": "interval",
        "query": "1m,5m,15m,1h",
        "current": { "text": "5m", "value": "5m" }
      }
    ]
  }
}
```

একটি প্যানেলের query Variable ব্যবহার করে:

```
sum(rate(http_requests_total{job=~"$service"}[$interval]))
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket{job=~"$service"}[$interval])) by (le))
```

Notification Policy-র ধারণা (UI-তে সাজানো হয়): `severity=critical` → PagerDuty contact point; `severity=warning` → Slack #technova-alerts; `team=backend` → Slack #backend। `Group by: service,`

`alertname` দিলে একই সার্ভিসের অনেক অ্যালার্ট একটি নোটিফিকেশনে আসে — অ্যালার্ট ঝড় কমে। `Group wait: 30s`, `Group interval: 5m`, `Repeat interval: 4h` — বারবার একই অ্যালার্ট না পাঠানোর জন্য এই তিনটি মান গুরুত্বপূর্ণ।

> **PRO TIP:** অ্যালার্টের বার্তায় Runbook লিঙ্ক ও ড্যাশবোর্ড লিঙ্ক যোগ করুন। রাত ৩টায় ঘুম থেকে উঠে কেউ Runbook খুঁজবে না — লিঙ্কটি বার্তার ভেতরেই থাকা চাই। Annotation হিসেবে `summary` ও `description` লিখুন, যেমন: "API error rate 5%-এর বেশি ৫ মিনিট ধরে — ড্যাশবোর্ড দেখুন, MongoDB প্যানেল যাচাই করুন।"

১৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — এক ড্যাশবোর্ড, বিশ সার্ভিস। TechNova মাইক্রোসার্ভিসে যাওয়ার সময় প্রতিটি সার্ভিসের জন্য ড্যাশবোর্ড বানাতে গিয়ে দলটি হিমশিম খাচ্ছিল। Variable যোগ করার পর একটি ড্যাশবোর্ডেই ড্রপডাউন থেকে সার্ভিস বদলানো যায় — নতুন সার্ভিস এলে ড্যাশবোর্ড বানাতে হয় না, শুধু মেট্রিক পাঠালেই হয়।

পরিস্থিতি ২ — অ্যালার্ট ঝড়। এক রাতে ডেটাবেস ধীর হলে ৪০টি অ্যালার্ট একসঙ্গে এলো — API, ওয়েব, মোবাইল, প্রতিটি endpoint থেকে। দলটি সব অ্যালার্ট mute করে দিল, এবং আসল সমস্যা (ডেটাবেস) চাপা পড়ে গেল। সমাধান: `Group by: service` ও সঠিক `For` সময়, এবং শুধু উপসর্গ নয়, মূল কারণের উপর অ্যালার্ট — ডেটাবেস latency-র উপর একটি অ্যালার্ট, প্রতিটি endpoint-এর উপর নয়।

পরিস্থিতি ৩ — ভুল ব্যক্তিকে অ্যালার্ট। ফ্রন্টএন্ড দলের Slack-এ ডেটাবেস অ্যালার্ট যাচ্ছিল, কারণ লেবেল ভুল ছিল। Notification Policy-তে `team` লেবেল দিয়ে রাউটিং ঠিক করার পর প্রতিটি অ্যালার্ট সঠিক দলে যায়। অ্যালার্টের লেবেল তাই শুধু তথ্য নয় — এটি রাউটিংয়ের ঠিকানা।

১৩.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **Variable যোগ করুন:** ড্যাশবোর্ড Settings → Variables → Add → Query ধরন, Data Source Prometheus, Query `label_values(http_requests_total, job)` → নাম `service`। প্যানেলের query-তে `job="$service"` লিখুন (একক) বা `job=~"$service"` (বহু)।
2. **Interval Variable যোগ করুন:** নাম `interval`, মান `1m,5m,15m,1h`। `rate(...[$interval])` লিখে যাচাই করুন।
3. **Alert Rule বানান:** Alerting → Alert rules → New → query লিখুন, শর্ত IS ABOVE 5, Evaluate every 1m, For 5m → Labels `severity=critical` → Save। Preview দিয়ে দেখুন এখন Firing হবে কি না।
4. **Contact Point যোগ করুন:** Alerting → Contact points → Slack Webhook URL দিন (প্লেসহোল্ডার: `https://hooks.slack.com/services/CHANGE-ME-WEBHOOK`) → Test চাপুন। আসল Webhook URL কখনো Git-এ লিখবেন না।
5. **Notification Policy সাজান:** `severity=critical` → PagerDuty, `severity=warning` → Slack। Group by `alertname, service` দিন।
6. **Provisioning করুন:** উপরের YAML ফাইল দুটি লিখে `docker compose restart grafana` করুন। Data Source ও Dashboard নিজে নিজে আসছে কি না দেখুন।
7. **Dashboard JSON Git-এ রাখুন:** ড্যাশবোর্ড Save → JSON Model কপি করে `provisioning/dashboards/json/`-এ রাখুন, Git-এ commit করুন।
8. **Annotation যোগ করুন:** Dashboard Settings → Annotations → Loki থেকে `{job="deploy"} |= "deploy"` যোগ করুন; একটি ডিপ্লয় করে রেখাটি দেখুন।

১৩.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	কেন ক্ষতিকর	সঠিক পদ্ধতি
Variable-এ `=` দিয়ে বহু মান	বহু মান বাছলে query ভেঙে যায়	বহু মানের জন্য `=~` ব্যবহার করুন
For: 0m অ্যালার্ট	ক্ষণিকের স্পাইকেই অ্যালার্ট যায়	For: 5m (critical) বা 10m (warning)
সব অ্যালার্ট critical	সবকিছু জরুরি মানে কিছুই জরুরি নয়	ব্যবহারকারী-প্রভাব অনুযায়ী severity ভাগ করুন
Contact Point-এ আসল secret Git-এ	Webhook URL ফাঁস হলে যে কেউ অ্যালার্ট পাঠাতে পারে	Secret পরিবেশ ভেরিয়েবল বা secret ম্যানেজারে রাখুন
Provisioning ছাড়া হাতে ড্যাশবোর্ড	সার্ভার হারালে সব হারায়	Dashboard-as-code, Git-ই সত্যের উৎস
অ্যালার্টে Runbook লিঙ্ক নেই	রাতে কী করতে হবে কেউ জানে না	প্রতিটি অ্যালার্টে summary, description, runbook
Silence স্থায়ীভাবে রেখে দেওয়া	অ্যালার্ট চিরতরে বন্ধ, বিপদ অজানা থাকে	Silence-এর মেয়াদ দিন, মেয়াদ শেষে পর্যালোচনা করুন

১৩.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Data Source-এর ক্রেডেনশিয়াল সবচেয়ে সংবেদনশীল অংশ। `access: proxy` ব্যবহার করুন, `basicAuth` পাসওয়ার্ড বা টোকেন কখনো Git-এ লিখবেন না — পরিবেশ ভেরিয়েবল (`$__env{...}` বা Grafana-র secure JSON data) ব্যবহার করুন। Viewer-দের Data Source-এর "Explore" অনুমতি সীমিত করুন, নইলে তারা যেকোনো query চালাতে পারে।

Contact Point-এর Webhook URL একটি গোপন তথ্য — এটি জানলে বাইরের কেউ আপনার Slack-এ বার্তা পাঠাতে পারে। URL ঘোরানোর (rotate) প্রক্রিয়া রাখুন। Alert Rule-এর লেবেলে ব্যবহারকারীর ব্যক্তিগত তথ্য (ইমেইল, ফোন) রাখবেন না। Grafana-র API Key বা Service Account Token-এর মেয়াদ ও অনুমতি সীমিত রাখুন — শুধু যা দরকার, ততটুকু।

> **WARNING:** Provisioning YAML-এ পাসওয়ার্ড লিখে Git-এ push করবেন না। একবার Git ইতিহাসে গেলে পাসওয়ার্ড মুছে ফেলা কঠিন। Secret-এর জন্য Grafana-র পরিবেশ ভেরিয়েবল বা বাইরের secret ম্যানেজার ব্যবহার করুন।

১৩.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Alert Rule প্রতিটি evaluation-এ একটি query চালায়। ১০০টি নিয়ম × প্রতি মিনিটে = প্রতি মিনিটে ১০০টি query — Prometheus-এর উপর স্থায়ী চাপ। তাই নিয়মের সংখ্যা নিয়ন্ত্রণে রাখুন, `Evaluate every` 1m-এর কম করবেন না, এবং ভারী query-র জন্য Recording Rule ব্যবহার করুন। Variable-র query-ও প্রতিবার ড্যাশবোর্ড খুললে চলে — `label_values` হালকা, কিন্তু জটিল query Variable-এ দেবেন না।

Notification-এর খরচ: PagerDuty বা SMS প্রতি অ্যালার্টে খরচ হতে পারে। `Repeat interval` 4h রাখলে একই সমস্যায় বারবার পেজ যাবে না। Grouping দিয়ে অনেক অ্যালার্ট একত্র করলে নোটিফিকেশনের সংখ্যা ও খরচ দুটোই কমে।

১৩.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	কারণ	সমাধান
Variable ড্রপডাউন খালি	Query-তে ভুল মেট্রিক বা Data Source ভুল	Explore-এ `label_values(...)` চালিয়ে যাচাই করুন
বহু মান বাছলে "No data"	`=` ব্যবহার করা হয়েছে	`=~` ব্যবহার করুন
অ্যালার্ট Firing হচ্ছে না	For সময় শেষ হয়নি বা query-তে ডেটা নেই	Preview দিয়ে বর্তমান মান দেখুন; For কমিয়ে পরীক্ষা করুন
অ্যালার্ট যাচ্ছে না	Contact Point ভুল বা Policy রাউটিং মিলছে না	Contact Point Test করুন; Policy-র matcher লেবেল মিলিয়ে দেখুন
Provisioning ড্যাশবোর্ড আসছে না	YAML-এ ভুল পথ বা JSON-এ ভুল	`docker compose logs grafana`-এ provisioning error দেখুন
UI-তে বদল সংরক্ষিত হচ্ছে না	`allowUiUpdates: false`	এটিই উদ্দেশ্য — ফাইল বদলে Git-এ commit করুন
অনেক নোটিফিকেশন একসঙ্গে	Grouping নেই	Group by alertname, service দিন; Group wait বাড়ান

১৩.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: একটি ড্যাশবোর্ড, অনেক সার্ভিস (Variable); অ্যালার্ট শুধু ব্যবহারকারী-প্রভাবিত সমস্যায়; প্রতিটি অ্যালার্টে severity, Runbook, ড্যাশবোর্ড লিঙ্ক; Notification Policy-তে grouping; Provisioning + Git; Contact Point-এর secret পরিবেশে; নিয়মিত অ্যালার্ট পর্যালোচনা — কোন অ্যালার্ট কখনো Firing হয়নি, তা মুছে ফেলা।

অ্যান্টি-প্যাটার্ন: প্রতিটি সার্ভিসের জন্য আলাদা ড্যাশবোর্ড; For: 0m অ্যালার্ট; সব অ্যালার্ট critical; অ্যালার্টে শুধু "something wrong" লেখা; Silence স্থায়ী করে রাখা; Provisioning YAML-এ secret লেখা; একই অ্যালার্ট Grafana ও Alertmanager দুই জায়গা থেকে পাঠানো।

১৩.১৬ অধ্যায়ের সারাংশ

Variable একটি ড্যাশবোর্ডকে পুনঃব্যবহারযোগ্য করে — একই ড্যাশবোর্ড, ভিন্ন সার্ভিস। Grafana Alerting-এর তিনটি স্তম্ভ: Alert Rule (শর্ত), Notification Policy (রাউটিং), Contact Point (মাধ্যম)। Pending পর্যায় ক্ষণিকের স্পাইক থেকে রক্ষা করে, Grouping অ্যালার্ট ঝড় কমায়। Provisioning Data Source ও Dashboard-কে কোডে রূপ দেয় — Git-ই সত্যের উৎস, Grafana শুধু তা পড়ে। Annotation ডিপ্লয় ও ইনসিডেন্টকে সময়রেখায় চিহ্নিত করে। Organization, Folder ও Role দিয়ে অ্যাক্সেস নিয়ন্ত্রণ করুন, এবং secret কখনো Git-এ রাখবেন না।

১৩.১৭ মূল প্রযুক্তিগত পরিভাষা

Variable (Templating): ড্যাশবোর্ডের ড্রপডাউন মান, যা query-র ভেতরে বসে একই ড্যাশবোর্ডকে বহু সার্ভিসের জন্য ব্যবহারযোগ্য করে।

Alert Rule: নিয়মিত মূল্যায়ন করা শর্ত — query-র মান সীমা ছাড়ালে অ্যালার্ট তৈরি হয়।

Contact Point: নোটিফিকেশন পাঠানোর মাধ্যম — Slack, ইমেইল, PagerDuty, Webhook।

Notification Policy: কোন অ্যালার্ট কোন Contact Point-এ যাবে, তা লেবেল দিয়ে নির্ধারণের নিয়ম।

Provisioning: YAML/JSON ফাইল থেকে Data Source ও Dashboard স্বয়ংক্রিয়ভাবে লোড করা।

Dashboard-as-Code: ড্যাশবোর্ডকে কোড হিসেবে Git-এ রাখার অভ্যাস — পর্যালোচনা ও পুনরুদ্ধার সহজ হয়।

Annotation: ড্যাশবোর্ডের সময়রেখায় চিহ্ন — যেমন ডিপ্লয় বা ইনসিডেন্টের মুহূর্ত।

Alert Fatigue: অতিরিক্ত অ্যালার্টের কারণে মানুষ সব অ্যালার্ট উপেক্ষা করতে শুরু করা।

Silence / Mute Timing: নির্দিষ্ট সময়ের জন্য নোটিফিকেশন থামানোর ব্যবস্থা।

Grouping: একই ধরনের অনেক অ্যালার্টকে একটি নোটিফিকেশনে একত্র করা।

১৩.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Variable ছাড়া ২০টি সার্ভিসের জন্য ড্যাশবোর্ড বানাতে কী সমস্যা হয়?
2. Query Variable ও Custom Variable-এর পার্থক্য কী?
3. বহু মানের Variable-এর জন্য PromQL-এ `=`-এর বদলে `=~` কেন দরকার?
4. Alert-এর Pending পর্যায়ের উদ্দেশ্য কী?
5. Contact Point, Notification Policy ও Alert Rule-এর ভূমিকা আলাদা করে লিখুন।
6. Grafana Alerting ও Prometheus Alertmanager কখন কোনটি ব্যবহার করবেন?
7. Provisioning কেন Dashboard-as-code-এর জন্য গুরুত্বপূর্ণ?
8. Annotation কীভাবে ইনসিডেন্ট তদন্তে সাহায্য করে?
9. Grouping না থাকলে অ্যালার্ট ঝড় কীভাবে তৈরি হয়?

10. Provisioning YAML-এ secret লেখা কেন বিপজ্জনক?

১৩.১৯ ব্যবহারিক অনুশীলন

1. TechNova ড্যাশবোর্ডে `service` ও `interval` Variable যোগ করুন এবং দুটি ভিন্ন সার্ভিসে বদলে দেখুন।
2. Error rate ৫%-এর বেশি ৫ মিনিট থাকলে Firing হয় — এমন একটি Alert Rule বানান ও Preview যাচাই করুন।
3. একটি Slack Contact Point (পরীক্ষামূলক Webhook) যোগ করে Test নোটিফিকেশন পাঠান।
4. Notification Policy-তে `severity=warning` Slack-এ ও `severity=critical` আলাদা মাধ্যমে যাওয়ার নিয়ম লিখুন।
5. datasources.yaml ও dashboards.yaml লিখে Grafana পুনরায় চালু করে Provisioning যাচাই করুন।
6. ড্যাশবোর্ড JSON Git-এ commit করুন এবং একটি সহকর্মীকে দিয়ে শুধু Git থেকে Grafana পুনর্গঠন করান।
7. একটি ডিপ্লয় Annotation যোগ করে ডিপ্লয়ের পর latency-র পরিবর্তন দেখুন।

অধ্যায় ১৪ • PART III — Visualization and Logging Engineering

Chapter 14 — Centralized Logging Architecture and Structured Logging

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি ব্যাখ্যা করতে পারবেন কেন লগ প্রতিটি সার্ভারে ছড়িয়ে রাখা যায় না, কেন্দ্রীয় লগিং কীভাবে কাজ করে, এবং `Structured Logging` কেন আধুনিক প্রোডাকশনের মান। আপনি JSON লগ লিখতে পারবেন, সঠিক `Log Level` বাছতে পারবেন, `Correlation ID` ও `Request ID` দিয়ে একটি অনুরোধকে সব সার্ভিসে অনুসরণ করতে পারবেন, এবং লগ ingestion, parsing, storage, retention, indexing ও search-এর সম্পূর্ণ প্রবাহ ডিজাইন করতে পারবেন।

- কেন্দ্রীয় লগিং আর্কিটেকচারের উপাদানগুলো আঁকা ও ব্যাখ্যা করা।
- অসংগঠিত লগের বদলে structured JSON লগ লেখা।
- Log level (DEBUG/INFO/WARN/ERROR/FATAL) সঠিকভাবে ব্যবহার করা।
- Correlation ID দিয়ে মাইক্রোসার্ভিসে অনুরোধ অনুসরণ করা।
- সংবেদনশীল তথ্য লগ থেকে বাদ দেওয়া ও redaction করা।
- উচ্চ-ভলিউম লগের খরচ ও retention পরিকল্পনা করা।

১৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

মেট্রিক বলে "কী" ঘটেছে — error rate বেড়েছে। লগ বলে "কেন" ঘটেছে — কোন অনুরোধ, কোন ব্যবহারকারী, কোন ভুল বার্তা, কোন স্ট্যাক ট্রেস। মেট্রিক ছাড়া লগ অন্ধ, লগ ছাড়া মেট্রিক বোবা। প্রোডাকশন ইনসিডেন্টে শেষ পর্যন্ত একজন ইঞ্জিনিয়ার লগের লাইন পড়েই মূল কারণ খুঁজে পান।

কিন্তু লগ যদি প্রতিটি সার্ভারে আলাদা ফাইলে থাকে, তাহলে ইনসিডেন্টের সময় ইঞ্জিনিয়ারকে পাঁচটি সার্ভারে SSH করে পাঁচটি ফাইল grep করতে হয় — ততক্ষণে গ্রাহক চলে যায়। কেন্দ্রীয় লগিং সব লগ এক জায়গায় এনে অনুসন্ধানযোগ্য করে। আর লগ যদি অসংগঠিত হয় — "User 123 failed" বনাম "Failed for user 123" — তাহলে অনুসন্ধান করা যায় না। Structured logging প্রতিটি লগকে একই আকার দেয়, ফলে মেশিন তা পড়তে, ফিল্টার করতে ও গণনা করতে পারে।

> **REAL WORLD:** TechNova-তে একবার পেমেন্ট ব্যর্থ হচ্ছিল, কিন্তু কোনো সার্ভারে স্পষ্ট ভুল ছিল না। কেন্দ্রীয় লগে Correlation ID দিয়ে খুঁজে দেখা গেল একই অনুরোধ API-তে সফল, কিন্তু পেমেন্ট সার্ভিসে timeout — সমস্যাটি নেটওয়ার্কে, কোডে নয়। ছড়ানো লগে এই অনুসন্ধান ঘণ্টা লাগত; কেন্দ্রীয় লগে লেগেছিল ৪ মিনিট।

১৪.৩ পূর্বশর্ত জ্ঞান

Node.js ও Express-এর মৌলিক ধারণা, JSON, এবং Linux-এ ফাইল ও `stdout` / `stderr`-এর ধারণা দরকার। HTTP request/response চক্র জানা থাকলে Correlation ID বোঝা সহজ হবে। Docker-এ লগ কীভাবে বের হয় (`docker logs`) জানা থাকলে ingestion অংশ সহজ লাগবে।

পূর্বশর্ত	কেন দরকার

Node.js / Express	লগিং কোডের উদাহরণ বুঝতে
JSON	Structured log-এর ফরম্যাট
Linux stdout/stderr	Twelve-Factor লগিং নীতি বুঝতে
HTTP headers	Correlation ID হেডারে পাঠানো বুঝতে

১৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

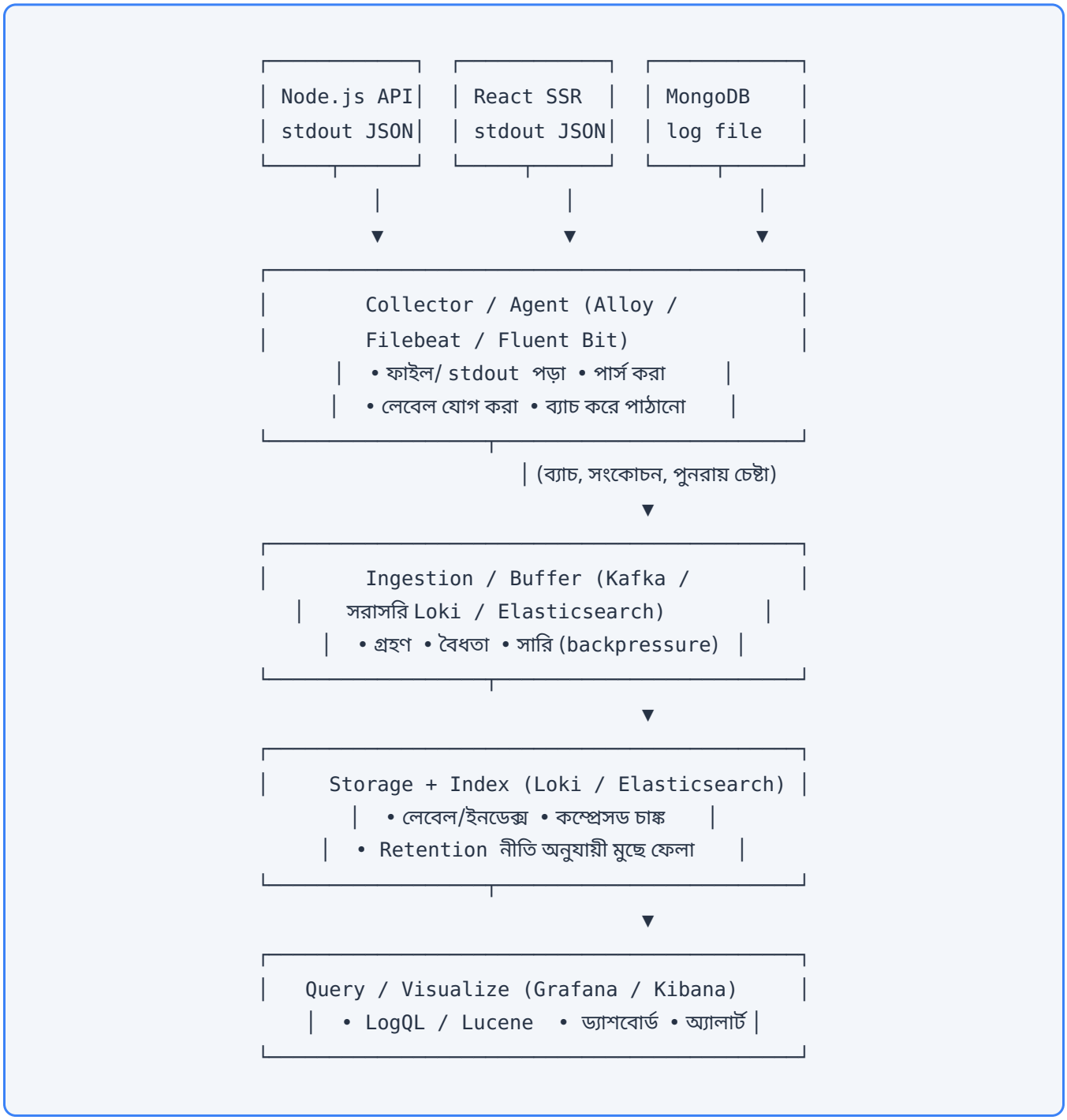
প্রথম নীতি: কম্পিউটার যা করে, তার চিহ্ন রেখে যায়। সেই চিহ্নই লগ — সময়ের সঙ্গে যুক্ত একটি ঘটনার বিবরণ। লগের তিনটি কাজ: (১) কী ঘটেছে তা জানা, (২) কখন ঘটেছে তা জানা, (৩) কেন ঘটেছে তা বোঝা। এই তিনটি কাজের জন্য প্রতিটি লগে তিনটি জিনিস থাকা চাই: সময় (`timestamp`), ঘটনা (`message`), ও প্রসঙ্গ (`context` — কে, কোথায়, কোন অনুরোধ)। প্রসঙ্গ ছাড়া লগ শুধু শব্দ; সময় ছাড়া লগ শুধু গল্প।

দ্বিতীয় নীতি: মানুষের জন্য লেখা লগ মেশিন পড়তে পারে না, মেশিনের জন্য লেখা লগ মানুষ পড়তে পারে না — structured logging দুটোর মধ্যে সেতু। JSON-এ লেখা লগ মানুষ পড়তে পারে (কীগুলো বোধগম্য), মেশিনও পড়তে পারে (কী দিয়ে ফিল্টার করা যায়)। তাই প্রতিটি লগ একটি JSON অবজেক্ট: `{"level":"info","msg":"order created","orderId":"...","userId":"..."}`।

তৃতীয় নীতি: লগ একটি প্রবাহ (stream), ফাইল নয়। অ্যাপ্লিকেশন লগ ফাইলে লিখে নিজে ঘোরাবে (rotate) — এই ধারণা পুরোনো। আধুনিক নীতি (`Twelve-Factor App`): অ্যাপ শুধু `stdout`-এ লগ লেখে, সংগ্রহ, ঘোরানো ও সংরক্ষণের দায়িত্ব প্ল্যাটফর্মের। এতে অ্যাপ সহজ থাকে, আর লগ যেকোনো কেন্দ্রীয় সিস্টেমে পাঠানো যায়।

> **CONCEPT:** লগ, মেট্রিক ও ট্রেস একই ঘটনার তিনটি দৃষ্টিভঙ্গি। মেট্রিক বলে "কত", লগ বলে "কী", ট্রেস বলে "কোথা দিয়ে গেল"। তিনটি একসঙ্গে থাকলে তবেই সম্পূর্ণ ছবি পাওয়া যায় — এজন্য Correlation ID ও Trace ID একই অনুরোধে ভাগ করা হয়।

১৪.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



প্রতিটি স্তরের কাজ আলাদা: `Collector` লগ সংগ্রহ করে ও সমৃদ্ধ করে (হোস্ট, সার্ভিস, পরিবেশ লেবেল যোগ করে); `Buffer` কেন্দ্রীয় সিস্টেম ডাউন থাকলে লগ হারাতে দেয় না — ডিস্কে সারি করে রাখে; `Storage` লগ সংরক্ষণ করে ও অনুসন্ধানের জন্য ইনডেক্স বানায়; `Query` স্তর মানুষকে অনুসন্ধান করতে দেয়। কোনো স্তর বাদ গেলে সমস্যা হয়: Buffer না থাকলে কেন্দ্রীয় সিস্টেম ডাউন থাকলে লগ হারায়; Index না থাকলে অনুসন্ধান ধীর হয়।

`Parsing` মানে অসংগঠিত লেখাকে কাঠামোতে বদলানো — যেমন Nginx access log থেকে IP, status, latency আলাদা করা। Structured JSON লগে parsing-এর দরকার কম, কারণ লগটি আগে থেকেই কাঠামোবদ্ধ। এজন্যই structured logging এত শক্তিশালী: parsing-এর খরচ ও ভুল দুটোই কমে।

১৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Log Level

Level	অর্থ	উদাহরণ	প্রোডাকশনে
DEBUG	বিস্তারিত, বিকাশের সময়	ফাংশনে ঢোকা/বেরোনো, ভেরিয়েবলের মান	সাধারণত বন্ধ
INFO	স্বাভাবিক গুরুত্বপূর্ণ ঘটনা	সার্ভার চালু, অর্ডার তৈরি, ডিপ্লয়	চালু
WARN	অস্বাভাবিক কিন্তু সামলানো গেছে	পুনরায় চেষ্টা, ধীর query, অবচয়িত API	চালু
ERROR	অনুরোধ ব্যর্থ হয়েছে	ডেটাবেস সংযোগ ব্যর্থ, 500 error	চালু + অ্যালার্ট
FATAL	প্রক্রিয়া আর চলতে পারছে না	পোর্ট দখল, কনফিগ ভুল	চালু + জরুরি অ্যালার্ট

Level-এর নিয়ম: প্রোডাকশনে DEBUG বন্ধ রাখুন — এতে ভলিউম ও খরচ দুটোই বাড়ে, আর সংবেদনশীল তথ্য ফাঁসের ঝুঁকি বাড়ে। ERROR মানে "কেউ কিছু করতে পারে" — প্রতিটি ERROR-এর জন্য হয় সমাধান, নয়তো অ্যালার্ট থাকা উচিত। যে ERROR কেউ দেখে না, সেটি লগ নয়, শব্দদূষণ।

Correlation ID ও Request ID

`Request ID` একটি অনুরোধের অনন্য পরিচয় — API গেটওয়ে বা প্রথম সার্ভিস এটি তৈরি করে (`X-Request-Id` হেডার)। `Correlation ID` একই ধারণা, তবে একাধিক সার্ভিস জুড়ে একই মান বয়ে যায়: API অনুরোধটি পেলে ডেটাবেস কল, পেমেন্ট কল, নোটিফিকেশন কলে একই ID পাঠায়। ফলে কেন্দ্রীয় লগে একটি ID খুঁজলেই পুরো যাত্রা দেখা যায়। ID তৈরি করুন `crypto.randomUUID()` দিয়ে; ক্লায়েন্ট পাঠালে সেটি গ্রহণ করুন, না পাঠালে নিজে তৈরি করুন।

Ingestion, Indexing, Retention

`Ingestion` — লগ গ্রহণ ও বৈধতা। `Indexing` — অনুসন্ধান দ্রুত করতে সূচি বানানো। Elasticsearch পুরো লেখা ইনডেক্স করে (full-text), Loki শুধু লেবেল ইনডেক্স করে (অধ্যায় ১৫)। `Retention` — কতদিন লগ রাখা হবে: সাধারণত ৭–৩০ দিন গরম স্টোরেজে, তারপর ঠান্ডা স্টোরেজ বা মুছে ফেলা। Retention ঠিক করুন আইনি প্রয়োজন, খরচ ও তদন্তের প্রয়োজনের ভারসাম্যে। `Log Rotation` — ফাইলে লেখা লগের ক্ষেত্রে পুরোনো ফাইল সংকুচিত ও মুছে ফেলার নীতি; stdout-ভিত্তিক লগিংয়ে এটি প্ল্যাটফর্মের দায়িত্ব।

সংবেদনশীল তথ্য ও Redaction

লগে কখনো পাসওয়ার্ড, টোকেন, API key, ক্রেডিট কার্ড নম্বর, OTP লিখবেন না। ভুল করে লেখা হলে তা লগ সিস্টেমে ছড়িয়ে পড়ে — ব্যাকআপেও থেকে যায়। `Redaction` মানে সংবেদনশীল ক্ষেত্র মুছে বা মাস্ক করে দেওয়া: `password: "****"`, কার্ড নম্বরের শেষ ৪ সংখ্যা ছাড়া বাকি মাস্ক করা। Redaction করুন লগ লেখার সময় (অ্যাপে), শুধু সংরক্ষণের সময় নয় — কারণ লগ পাঠানোর পথেও ফাঁস হতে পারে।

১৪.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

Node.js-এ `pino` দিয়ে Structured Logging — TechNova API-র উদাহরণ:

```
const pino = require('pino');
const crypto = require('crypto');
```

```
// লগার তৈরি: JSON আকারে, ISO সময়, সার্ভিস নামসহ
const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  base: { service: 'technova-api', env: process.env.NODE_ENV || 'dev' },
  redact: ['req.headers.authorization', 'req.body.password', 'req.body.cardNumber'],
  timestamp: pino.stdTimeFunctions.isoTime,
});

// প্রতিটি অনুরোধে Request ID যোগ করা (Express middleware)
function requestId(req, res, next) {
  req.id = req.headers['x-request-id'] || crypto.randomUUID();
  res.setHeader('x-request-id', req.id);
  req.log = logger.child({ requestId: req.id, correlationId: req.id });
  next();
}

// ব্যবহার: প্রতিটি লগে স্বয়ংক্রিয়ভাবে requestId যুক্ত হয়
// req.log.info({ userId: 'u-123', orderId: 'o-456' }, 'order created');
// req.log.error({ err: err }, 'payment failed');
```

`level: process.env.LOG_LEVEL || 'info'` — পরিবেশ ভেরিয়েবল দিয়ে level বদলানো যায়, কোড বদলাতে হয় না। `base` — প্রতিটি লগে সার্ভিস ও পরিবেশের নাম যুক্ত হয়, ফলে কেন্দ্রীয় লগে ফিল্টার করা যায়। `redact` — সংবেদনশীল ক্ষেত্র স্বয়ংক্রিয়ভাবে মাস্ক হয়; এটি নিরাপত্তার সবচেয়ে গুরুত্বপূর্ণ লাইন। `timestamp: isoTime` — সময় ISO 8601 আকারে, যাতে সব সিস্টেমে একইভাবে বোঝা যায়। `logger.child({requestId})` — প্রতিটি অনুরোধের নিজস্ব লগার, যাতে সব লগে একই ID থাকে।

আউটপুটের উদাহরণ (একটি লাইন = একটি JSON অবজেক্ট):

```
{"level":30,"time":"2026-11-09T10:15:00.123+06:00","service":"technova-
api","env":"prod","requestId":"9f2c...","userId":"u-123","orderId":"o-456","msg":"order
created"}
{"level":50,"time":"2026-11-09T10:16:02.400+06:00","service":"technova-
api","env":"prod","requestId":"9f2c...","msg":"payment failed","err":
{"type":"TimeoutError","message":"gateway timeout after 5000ms"}}
```

`level: 30` মানে INFO, `50` মানে ERROR (pino-র সংখ্যাভিত্তিক level)। দুটি লগে একই `requestId` থাকায় কেন্দ্রীয় লগে এই ID খুঁজলেই পুরো অনুরোধের গল্প পাওয়া যায়।

Docker-এ লগ দেখা ও সংগ্রহ যাচাই:

```
docker logs --tail 20 technova-api
docker logs -f technova-api --since 10m
# JSON লগ যাচাই: প্রতিটি লাইন বৈধ JSON কি না
docker logs technova-api 2>&1 | head -5 | python3 -m json.tool
# (নমুনা আউটপুট — চিত্রণমূলক)
```

১৪.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র জন্য সম্পূর্ণ উদাহরণ: Winston দিয়ে structured logging, Express middleware, ও Docker Compose-এ Alloy দিয়ে Loki-তে পাঠানো (বিস্তারিত Alloy কনফিগ অধ্যায় ১৫-এ)।

```
// logger.js — winston দিয়ে
const winston = require('winston');
const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  defaultMeta: { service: 'technova-api' },
  transports: [new winston.transports.Console()],
});
module.exports = logger;

// app.js — middleware
const express = require('express');
const crypto = require('crypto');
const app = express();
app.use(express.json());
app.use((req, res, next) => {
  req.id = req.headers['x-request-id'] || crypto.randomUUID();
  res.setHeader('x-request-id', req.id);
  const start = Date.now();
  res.on('finish', () => {
    logger.info({
      requestId: req.id,
      method: req.method,
      path: req.path,
      status: res.statusCode,
      durationMs: Date.now() - start,
      userAgent: req.headers['user-agent'],
    }, 'http request');
  });
  next();
});
```

এই middleware প্রতিটি HTTP অনুরোধের একটি লগ লেখে — method, path, status, সময়। এটিই access log-এর structured রূপ। `durationMs` দিয়ে ধীর অনুরোধ খুঁজে বের করা যায়; `status` দিয়ে error rate গোনা যায় (অধ্যায় ১৫-এর LogQL metric query দেখুন)। `userAgent` লগ করা ঠিক আছে, কিন্তু IP ঠিকানা লগ করার আগে গোপনীয়তা নীতি যাচাই করুন।

Log rotation — ফাইলে লেখা লগের জন্য `logrotate` উদাহরণ (stdout ব্যবহার করলে দরকার নেই):

```
/var/log/technova/*.log {
  daily
  rotate 14
  compress
  delaycompress
  missingok
  notifempty
  create 0640 technova technova
}
```

`daily` প্রতিদিন নতুন ফাইল; `rotate 14` ১৪ দিনের ফাইল রাখে; `compress` পুরোনো ফাইল সংকুচিত করে। প্রোডাকশনে কেন্দ্রীয় লগিং থাকলে ফাইলের retention কম রাখুন — ফাইল শুধু ব্যাকআপ, মূল ভরসা কেন্দ্রীয় সিস্টেম।

১৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — ধীর API, লগে প্রমাণ। p95 latency বাড়ল। মেট্রিক বলল "ধীর", লগ বলল "কেন": `durationMs > 2000` ফিল্টার করে দেখা গেল সব ধীর অনুরোধে একই MongoDB query — একটি `find()` যাতে ইনডেক্স নেই। লগে query-র নাম থাকায় দলটি ১০ মিনিটে ইনডেক্স যোগ করল।

পরিস্থিতি ২ — ব্যর্থ পেমেন্ট, Correlation ID-র জয়। গ্রাহক অভিযোগ করলেন পেমেন্ট কেটেছে কিন্তু অর্ডার হয়নি। Correlation ID দিয়ে খুঁজে দেখা গেল: API সফল, পেমেন্ট সফল, কিন্তু অর্ডার সার্ভিসে বার্তা পৌঁছায়নি — মেসেজ কিউতে আটকে ছিল। ID না থাকলে তিনটি সার্ভিসের লগ আলাদা আলাদা খুঁজতে হতো।

পরিস্থিতি ৩ — লগে পাসওয়ার্ড ফাঁস। একবার একটি DEBUG লগে পুরো request body ছাপা হচ্ছিল — পাসওয়ার্ডসহ। নিরাপত্তা পর্যালোচনায় ধরা পড়ল। সমাধান: `redact` যোগ করা, DEBUG প্রোডাকশনে বন্ধ করা, এবং পুরোনো লগ মুছে ফেলা। এই ঘটনার পর দলটি লগ পর্যালোচনাকে কোড রিভিউয়ের অংশ করল — প্রতিটি নতুন লগ লাইন রিভিউয়ে যাচাই করা হয়।

> **WARNING:** লগে পাসওয়ার্ড, টোকেন বা API key লিখলে তা শুধু ফাঁস নয় — তা ব্যাকআপ, রেপ্লিকা ও লগ অ্যাগ্রিগেটরে ছড়িয়ে পড়ে। মুছে ফেলা প্রায় অসম্ভব। তাই লগ লেখার আগেই redact করুন, এবং নিয়মিত লগে সংবেদনশীল তথ্য খুঁজে দেখুন (যেমন `password`, `token` শব্দের অনুসন্ধান)।

১৪.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **pino ইনস্টল করুন:** `npm install pino` — TechNova API-তে `logger.js` তৈরি করুন (১৪.৭-এর কোড)।
2. **Request ID middleware যোগ করুন:** প্রতিটি অনুরোধে ID তৈরি ও response হেডারে ফেরত দিন। `curl -i` দিয়ে হেডারটি দেখুন।
3. **সব `console.log` বদলান:** `console.log("User created")`-এর বদলে `logger.info({userId}, "user created")` লিখুন — প্রসঙ্গসহ।
4. **Redaction যাচাই করুন:** পাসওয়ার্ডসহ একটি অনুরোধ পাঠিয়ে লগে `****` দেখা যাচ্ছে কি না যাচাই করুন।
5. **Level পরীক্ষা করুন:** `LOG_LEVEL=debug npm start` দিয়ে DEBUG লগ দেখুন, তারপর `LOG_LEVEL=info` দিয়ে দেখুন DEBUG গায়েব হয়েছে।
6. **Docker লগ দেখুন:** `docker logs technova-api | python3 -m json.tool` দিয়ে JSON বৈধতা যাচাই করুন।
7. **Correlation পরীক্ষা করুন:** দুটি সার্ভিসে একই Request ID পাঠিয়ে দুই জায়গার লগে একই ID খুঁজুন।
8. **লগ পর্যালোচনা চালু করুন:** দলের কোড রিভিউ চেকলিস্টে যোগ করুন — "নতুন লগ লাইনে সংবেদনশীল তথ্য আছে কি?"

১৪.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	কেন ক্ষতিকর	সঠিক পদ্ধতি
`console.log` দিয়ে অসংগঠিত লগ	অনুসন্ধান ও ফিল্টার করা যায় না	Structured JSON logger (pino/winston)

প্রসঙ্গ ছাড়া লগ: "failed"	কী ব্যর্থ হলো, কার জন্য — জানা যায় না	প্রতিটি লগে ID, ব্যবহারকারী, প্রসঙ্গ
সবকিছু ERROR level-এ	আসল ERROR চাপা পড়ে যায়	Level-এর সংজ্ঞা মেনে চলুন
লগে পাসওয়ার্ড/টোকেন	গোপন তথ্য ফাঁস, মুছে ফেলা কঠিন	Redact করুন, লগ রিভিউ করুন
প্রতিটি সার্ভারে আলাদা লগ	ইনসিডেন্টে খুঁজে পাওয়া যায় না	কেন্দ্রীয় লগিং
DEBUG প্রোডাকশনে চালু	ভলিউম, খরচ ও ফাঁসের ঝুঁকি বাড়ে	প্রোডাকশনে INFO থেকে শুরু
সময় timezone ছাড়া	সার্ভিসগুলোর সময় মেলে না, ক্রম বোঝা যায় না	ISO 8601, UTC বা স্পষ্ট offset
লগে অতিরিক্ত তথ্য (পুরো body)	স্টোরেজ খরচ বাড়ে, গোপনীয়তা ঝুঁকি	শুধু যা দরকার, তাই লগ করুন

১৪.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

লগ নিরাপত্তার সবচেয়ে অবহেলিত অংশ। প্রথম নিয়ম: লগে কখনো secret লিখবেন না — পাসওয়ার্ড, টোকেন, API key, session cookie, OTP, ক্রেডিট কার্ড নম্বর। দ্বিতীয় নিয়ম: ব্যক্তিগত তথ্য (PII) — নাম, ইমেইল, ফোন, ঠিকানা — শুধু প্রয়োজনে ও ন্যূনতম আকারে লগ করুন, এবং গোপনীয়তা নীতি মেনে retention ঠিক করুন। তৃতীয় নিয়ম: লগে অ্যাক্সেস নিয়ন্ত্রণ করুন — সবাই সব লগ দেখতে পারবে না; প্রোডাকশন লগে শুধু অনুমোদিত দলের অ্যাক্সেস।

লগ পাঠানোর পথও সুরক্ষিত করুন: Collector থেকে কেন্দ্রীয় সিস্টেমে TLS ব্যবহার করুন। লগ স্টোরেজে অ্যাক্সেস লগ রাখুন — কে কখন কোন লগ দেখল, তা জানা উচিত। পুরোনো লগ মুছে ফেলার সময় নিশ্চিত করুন ব্যাকআপ থেকেও মুছেছে কি না — সংবেদনশীল তথ্য ফাঁসের ঘটনায় এটিই সবচেয়ে কঠিন অংশ।

১৪.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

লগের খরচ ভলিউমের সঙ্গে সরাসরি বাড়ে। প্রতিটি অনুরোধে ৫টি লগ লাইন × ১০,০০০ অনুরোধ/দিন = ৫০,০০০ লাইন/দিন। প্রতিটি লাইন ৫০০ বাইট হলে দিনে ২৫ MB, মাসে ৭৫০ MB — শুধু একটি সার্ভিসের। DEBUG চালু করলে এটি ১০ গুণ হতে পারে। তাই: (১) প্রোডাকশনে DEBUG বন্ধ, (২) প্রতিটি অনুরোধে একটি access log, ভেতরের প্রতিটি ধাপে নয়, (৩) বড় অবজেক্ট লগ করবেন না, (৪) `Sampling` বিবেচনা করুন — সফল অনুরোধের ১% লগ করা, ব্যর্থ অনুরোধের ১০০%।

কর্মক্ষমতার দিক থেকে: লগ লেখা যেন অনুরোধকে ধীর না করে। pino-র মতো asynchronous logger ব্যবহার করুন; synchronous ফাইল লেখা এড়িয়ে চলুন। লগের ব্যাচিং ও সংকোচন নেটওয়ার্ক খরচ কমায়। Retention পরিকল্পনা করুন: ৭ দিন গরম, ৩০ দিন ঠান্ডা, তারপর মুছে ফেলা — খরচ ও তদন্তের প্রয়োজনের ভারসাম্য।

১৪.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	কারণ	সমাধান
কেন্দ্রীয় লগে কিছু লগ আসছে না	Collector ডাউন বা ফিল্টারে বাদ পড়ছে	Collector লগ দেখুন; সার্ভারে `docker logs` দিয়ে যাচাই করুন
লগ আসছে কিন্তু খুঁজে পাওয়া যাচ্ছে না	সময় ভুল বা লেবেল ভুল	সময়সীমা বাড়ান; লেবেল (service, env) যাচাই করুন
লগের সময় মিলছে না	সার্ভারের ঘড়ি বা timezone ভিন্ন	NTP চালু করুন; ISO 8601 UTC ব্যবহার করুন

JSON লগ ভাঙা	বহু লাইনের stack trace এক লাইনে নয়	Stack trace-কে একটি ক্ষেত্রে রাখুন; multiline পার্সিং সেট করুন
লগ ভলিউম হঠাৎ বেড়েছে	DEBUG চালু বা লুপে লগ	Level যাচাই করুন; কোন logger সবচেয়ে বেশি লিখছে তা গুনুন
সংবেদনশীল তথ্য লগে পাওয়া গেল	Redaction নেই	Redact যোগ করুন; পুরোনো লগ মুছুন; ঘটনা নথিভুক্ত করুন

১৪.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: সব লগ JSON-এ; প্রতিটি লগে timestamp, level, service, requestId; Level-এর সংজ্ঞা মেনে চলা; Redaction ডিফল্ট; Correlation ID সব সার্ভিসে; কেন্দ্রীয় লগিং; Retention নীতি লিখিত; লগ রিভিউ কোড রিভিউয়ের অংশ; ERROR লগের জন্য অ্যালার্ট।

অ্যান্টি-প্যাটার্ন: `console.log`-এ অসংগঠিত লগ; "failed" বলে প্রসঙ্গ ছাড়া লগ; লগে পুরো request body; DEBUG প্রোডাকশনে; লগ ফাইলে নিজে rotation লেখা; লগে secret; প্রতিটি সার্ভিসে আলাদা লগ ফরম্যাট; যে ERROR কেউ দেখে না, তা লেখা।

১৪.১৬ অধ্যায়ের সারাংশ

লগ বলে "কেন" ঘটেছে — মেট্রিকের পরিপূরক। কেন্দ্রীয় লগিং সব লগ এক জায়গায় এনে অনুসন্ধানযোগ্য করে; ছড়ানো লগ ইনসিডেন্টে অকেজো। Structured JSON logging লগকে মেশিন-পাঠযোগ্য করে — ফিল্টার, গণনা ও অ্যালার্ট সম্ভব হয়। Log Level সঠিকভাবে ব্যবহার করুন, প্রতিটি অনুরোধে Request/Correlation ID দিন, সংবেদনশীল তথ্য redact করুন, এবং retention ও খরচের পরিকল্পনা করুন। অ্যাপ শুধু stdout-এ লগ লেখে; সংগ্রহ ও সংরক্ষণের দায়িত্ব প্ল্যাটফর্মের।

১৪.১৭ মূল প্রযুক্তিগত পরিভাষা

Structured Logging: লগকে নির্দিষ্ট কাঠামোতে (সাধারণত JSON) লেখা, যাতে মেশিন তা পড়তে ও ফিল্টার করতে পারে।

Centralized Logging: সব সার্ভিসের লগ এক জায়গায় সংগ্রহ করে অনুসন্ধানযোগ্য করা।

Log Level: লগের গুরুত্বের স্তর — DEBUG, INFO, WARN, ERROR, FATAL।

Correlation ID: একাধিক সার্ভিস জুড়ে একই অনুরোধকে চিহ্নিত করার অনন্য ID।

Request ID: একটি HTTP অনুরোধের অনন্য পরিচয়, সাধারণত X-Request-Id হেডারে।

Ingestion: লগ গ্রহণ, বৈধতা ও সারিতে রাখার প্রক্রিয়া।

Retention: লগ কতদিন সংরক্ষণ করা হবে, তার নীতি।

Redaction: লগ থেকে সংবেদনশীল তথ্য মুছে বা মাস্ক করে দেওয়া।

Log Rotation: পুরোনো লগ ফাইল সংকুচিত ও মুছে ফেলে ডিস্ক পূর্ণ হওয়া ঠেকানো।

Parsing: অসংগঠিত লেখাকে কাঠামোবদ্ধ ক্ষেত্রে বদলানো।

১৪.১৮ জ্ঞান যাচাই — প্রশ্ন

1. মেট্রিক "কী" বলে, লগ "কেন" বলে — এই কথার অর্থ ব্যাখ্যা করুন।
2. ছড়ানো লগের বদলে কেন্দ্রীয় লগিং কেন দরকার?
3. Structured logging অসংগঠিত লগের চেয়ে কেন ভালো?
4. DEBUG, INFO, WARN, ERROR — প্রতিটির একটি করে উদাহরণ দিন।
5. Correlation ID কীভাবে মাইক্রোসার্ভিসে অনুরোধ অনুসরণে সাহায্য করে?
6. লগে পাসওয়ার্ড লেখা কেন বিপজ্জনক, এমনকি লগ সুরক্ষিত থাকলেও?
7. Redaction লগ লেখার সময় করা উচিত কেন, সংরক্ষণের সময় নয়?
8. Twelve-Factor নীতি অনুযায়ী অ্যাপ কোথায় লগ লিখবে, এবং কেন?
9. লগ ভলিউম কমাতে তিনটি উপায় লিখুন।
10. Retention নীতি ঠিক করার সময় কোন তিনটি বিষয় বিবেচনা করবেন?

১৪.১৯ ব্যবহারিক অনুশীলন

1. TechNova API-তে pino দিয়ে structured logger যোগ করুন ও একটি অনুরোধের JSON লগ দেখুন।
2. Request ID middleware লিখুন; `curl -i` দিয়ে হেডার ও লগে একই ID যাচাই করুন।
3. একটি ERROR লগে stack trace-সহ লগ লিখুন এবং JSON বৈধতা যাচাই করুন।
4. Redaction পরীক্ষা করুন: পাসওয়ার্ডসহ অনুরোধ পাঠিয়ে লগে মাস্ক দেখুন।
5. `LOG_LEVEL` বদলে DEBUG চালু/বন্ধ করে ভলিউমের পার্থক্য মাপুন।
6. একটি ভুল লগ লাইন লিখুন ("failed" — প্রসঙ্গ ছাড়া), তারপর তা সঠিক আকারে আবার লিখুন; পার্থক্য লিখে রাখুন।
7. দলের জন্য এক পাতার লগিং নীতি লিখুন: level, ক্ষেত্র, redaction, retention।

অধ্যায় ১৫ • PART III — Visualization and Logging Engineering

Chapter 15 — Loki, Grafana Alloy, and LogQL

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Loki-র আর্কিটেকচার ব্যাখ্যা করতে পারবেন — কেন Loki পুরো লেখা ইনডেক্স করে না, শুধু লেবেল ইনডেক্স করে, এবং এতে কী সুবিধা ও সীমা তৈরি হয়। আপনি `Grafana Alloy` দিয়ে লগ সংগ্রহ করতে পারবেন, `LogQL`-এ লগ খুঁজতে ও মেট্রিকে রূপ দিতে পারবেন, retention ঠিক করতে পারবেন, এবং Grafana-তে মেট্রিক ও লগ পাশাপাশি দেখে তদন্ত করতে পারবেন।

- Loki-র label-ভিত্তিক ইনডেক্সিং ব্যাখ্যা করা ও Elasticsearch-এর সঙ্গে তুলনা করা।
- Alloy কনফিগ (River) লিখে Docker ও ফাইল থেকে লগ সংগ্রহ করা।
- LogQL-এ label filter, line filter, parser ও metric query লেখা।
- `rate()` ও `count_over_time()` দিয়ে লগ থেকে অ্যালার্টের মেট্রিক বানানো।
- Retention, cardinality ও খরচের ভারসাম্য বোঝা।

১৫.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Elasticsearch শক্তিশালী, কিন্তু ব্যয়বহুল ও ভারী — পুরো লেখা ইনডেক্স করতে প্রচুর ডিস্ক, মেমোরি ও পরিচালনা লাগে। ছোট ও মাঝারি দলের জন্য এই খরচ প্রায়ই লগিংয়ের সুবিধাকে ছাড়িয়ে যায়। Loki এই সমস্যার উত্তর: শুধু লেবেল ইনডেক্স করে, লেখাটি সংকুচিত চাঙ্কে রাখে। ফলে Loki হালকা, সস্তা, এবং Prometheus-এর মতোই পরিচালনা করা সহজ — একই লেবেল মডেল, একই Grafana।

TechNova-র মতো ছোট দলের জন্য এটি নির্ণায়ক। তাদের লগ ভলিউম মাঝারি, তদন্তের ধরন সাধারণত "এই সার্ভিসের, এই সময়ের, এই error-এর লগ দেখাও" — এই প্রশ্নের উত্তর লেবেল দিয়েই পাওয়া যায়। পুরো লেখায় অনুসন্ধান (full-text search) কদাচিৎ দরকার হয়। Loki সেই সাধারণ প্রশ্নের উত্তর সস্তায় দেয়।

> **REAL WORLD:** একটি স্টার্টআপ Elasticsearch চালাতে মাসে তিনটি বড় সার্ভার ব্যবহার করছিল, শুধু লগ রাখতে। Loki-তে যাওয়ার পর একটি ছোট সার্ভারেই চলল, আর Grafana-তেই মেট্রিক ও লগ পাশাপাশি দেখা গেল। সঞ্চয় শুধু টাকা নয় — পরিচালনার সময়ও কমল।

Grafana Alloy-ও একই গল্পের অংশ: আগে Promtail, Fluent Bit, Logstash — প্রতিটি কাজের জন্য আলাদা এজেন্ট। Alloy একটি এজেন্টে মেট্রিক, লগ ও ট্রেস সংগ্রহ করে — একটি কনফিগ, একটি প্রক্রিয়া। এতে শেখার খরচ কমে, ভুল কমে।

১৫.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৪-এর structured logging ও কেন্দ্রীয় লগিং, অধ্যায় ৭–৮-এর Prometheus লেবেল ও PromQL, এবং অধ্যায় ১২-এর Grafana Data Source ধারণা দরকার। LogQL PromQL-এর মতোই — `{label="value"}` নির্বাচক ও `rate()` ফাংশন চেনা থাকলে LogQL সহজ লাগবে।

পূর্বশর্ত	কেন দরকার

Structured JSON logging	Loki-তে JSON parser ব্যবহার করতে
Prometheus লেবেল মডেল	Loki-র stream ও লেবেল একই ধারণা
PromQL	LogQL-এর metric query PromQL-এর মতো
Docker Compose	Loki ও Alloy ল্যাবে চালাতে

১৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

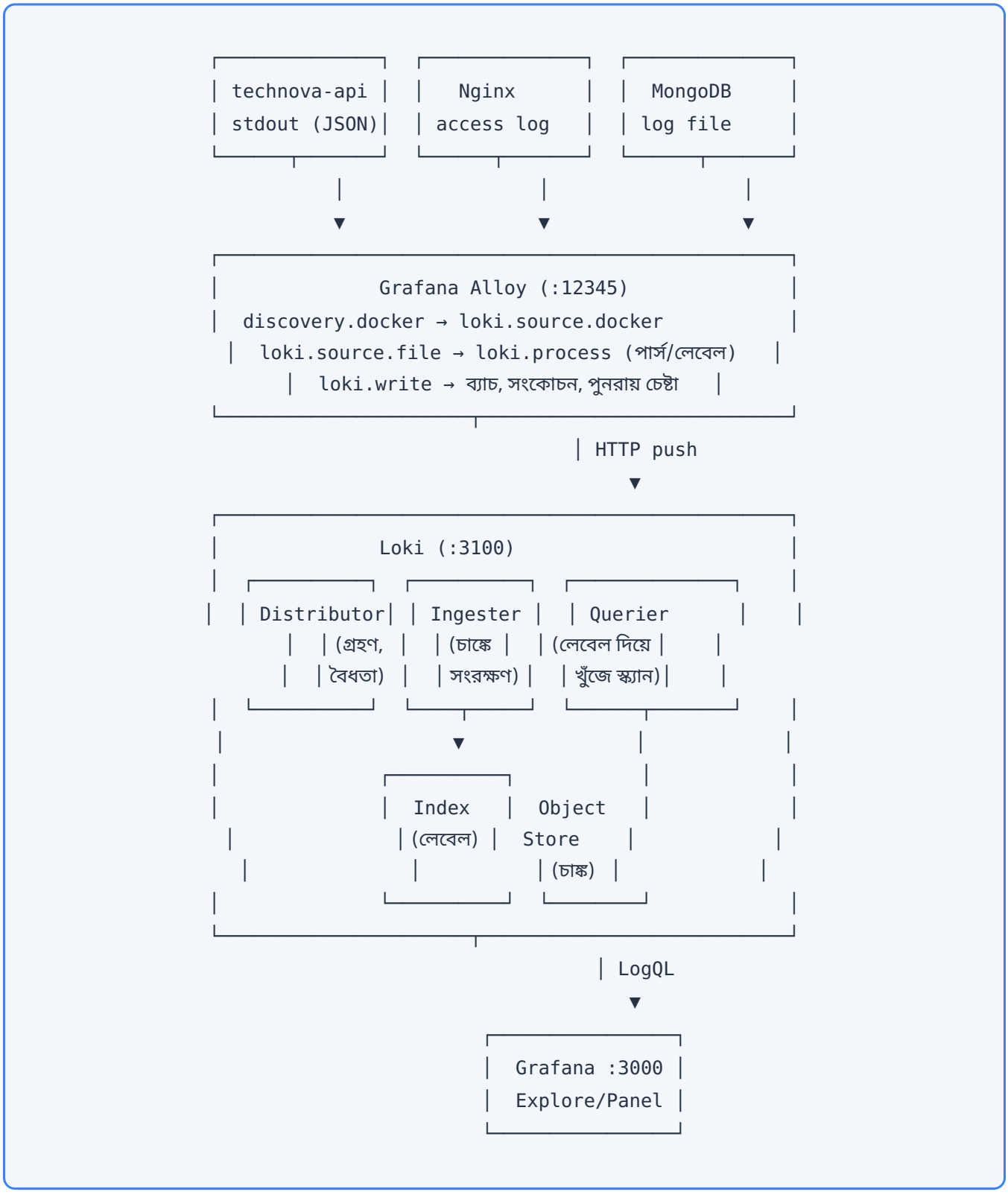
প্রথম নীতি: অনুসন্ধান দ্রুত করতে সূচি (index) লাগে, কিন্তু সূচি নিজেও জায়গা নেয়। বইয়ের শেষে সূচিপত্র থাকলে শব্দ খুঁজে পাওয়া যায়, কিন্তু সূচিপত্র বইয়ের মতো মোটা হলে লাভ নেই। Elasticsearch প্রতিটি শব্দের সূচি বানায় — সূচিটি ডেটার চেয়ে বড় হতে পারে। Loki ভিন্ন সিদ্ধান্ত নেয়: শুধু কয়েকটি লেবেলের সূচি বানায় (কোন সার্ভিস, কোন পরিবেশ, কোন level), আর লেখাটি যেমন আছে তেমন সংকুচিত করে রাখে। অনুসন্ধানের সময় প্রথমে লেবেল দিয়ে সঠিক stream খুঁজে নেয়, তারপর সেই stream-এর ভেতরে লেখা স্ক্যান করে।

দ্বিতীয় নীতি: লগের মৌলিক একক `Stream` — একই লেবেলের সব লগ লাইন একটি stream। যেমন `{service="technova-api", level="error"}` একটি stream। Stream-এর লাইনগুলো সময়ের ক্রমে সাজানো চাঙ্কে সংরক্ষিত হয়। এই মডেল Prometheus-এর series-এর মতোই — যারা Prometheus বোঝেন, তারা Loki বোঝেন।

তৃতীয় নীতি: সংগ্রহ ও সংরক্ষণ আলাদা। Alloy লগ সংগ্রহ করে, সমৃদ্ধ করে, পাঠায়; Loki সংরক্ষণ করে ও উত্তর দেয়। এই বিভাজনের ফলে Alloy বদলানো যায় Loki না বদলে, আর Loki বদলানো যায় অ্যাপ না বদলে।

> **CONCEPT:** Loki বনাম Elasticsearch-এর পার্থক্য একটি বাণিজ্য (trade-off): Loki সস্তা ও হালকা, কিন্তু পুরো লেখায় অনুসন্ধান ধীর; Elasticsearch শক্তিশালী অনুসন্ধান দেয়, কিন্তু ব্যয়বহুল। কোনটি ভালো, তা নির্ভর করে আপনার প্রশ্নের ধরনের উপর — লেবেল দিয়ে খুঁজলে Loki, যেকোনো শব্দ দিয়ে খুঁজলে Elasticsearch।

১৫.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



ছোট স্থাপনায় (single binary / monolithic mode) Distributor, Ingester ও Querier একই প্রক্রিয়ায় চলে — ল্যাব ও ছোট প্রোডাকশনের জন্য এটিই যথেষ্ট। বড় স্থাপনায় এগুলো আলাদা সার্ভিসে ভাগ হয় (microservices mode)। `Ingester` লগ চাঙ্কে জমায় ও সংকুচিত করে Object Store-এ (ফাইল সিস্টেম বা S3-সদৃশ) পাঠায়; `Index`-এ শুধু লেবেল থাকে। `Querier` লেবেল দিয়ে stream খুঁজে, চাঙ্ক নামিয়ে, লেখা স্ক্যান করে উত্তর দেয়।

Promtail প্রসঙ্গে স্পষ্ট কথা: Promtail আগে Loki-র ডিফল্ট সংগ্রাহক ছিল, কিন্তু এখন এটি maintenance mode-এ আছে — নতুন বৈশিষ্ট্য আসছে না, এবং Grafana-র সুপারিশকৃত সংগ্রাহক এখন `Grafana Alloy`। নতুন স্থাপনায় Promtail দিয়ে শুরু করবেন না; Alloy ব্যবহার করুন। পুরোনো Promtail কনফিগ থাকলে Alloy-তে রূপান্তরের সরঞ্জাম (converter) আছে। এই বইয়ে Alloy-ই ডিফল্ট।

১৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Label ও Stream — Cardinality সতর্কতা

Loki-তে লেবেল কম রাখুন: `service`, `env`, `level`, `host` — এই কয়টি যথেষ্ট। `user_id`, `request_id`, `ip` কখনো লেবেল করবেন না — প্রতিটি মান নতুন stream তৈরি করে, stream-এর সংখ্যা বিস্ফোরিত হয়, ইনডেক্স বড় হয়, Loki ধীর হয়। এটিই `High Cardinality` সমস্যা। বিস্তারিত তথ্য লগের লেখার ভেতরে রাখুন (JSON ক্ষেত্র), লেবেলে নয় — লেখা স্ক্যান করে খোঁজা যায়, কিন্তু ইনডেক্স ভারী হয় না।

LogQL — তিন ধাপ

LogQL query-র তিনটি ধাপ: (১) `Stream selector` — `{service="technova-api"}` দিয়ে stream বাছা; (২) `Line filter` — `|= "error"` (আছে), `!= "health"` (নেই), `|~ "timeout|failed"` (regex); (৩) `Parser` — `| json` বা `| logfmt` দিয়ে লেখা থেকে ক্ষেত্র বের করা, তারপর `| level="error"` দিয়ে ক্ষেত্রে ফিল্টার করা। Metric query-তে চতুর্থ ধাপ: `rate()`, `count_over_time()`, `sum by()` দিয়ে সংখ্যায় রূপ দেওয়া।

```
# সব error লগ
{service="technova-api"} |= "error"

# JSON লগ থেকে ক্ষেত্র বের করে ফিল্টার
{service="technova-api"} | json | level="error" | status >= 500

# ৫ মিনিটে error-র হার (প্রতি সেকেন্ডে)
sum(rate({service="technova-api"} | json | level="error" [5m]))

# ১ ঘণ্টায় 500 status-এর সংখ্যা
sum(count_over_time({service="technova-api"} | json | status="500" [1h]))

# ধীর অনুরোধ: duration 2 সেকেন্ডের বেশি
{service="technova-api"} | json | durationMs > 2000
```

Retention ও Compaction

Loki-তে retention নির্ধারণ করা হয় `compactor`-এর মাধ্যমে — পুরোনো চাঙ্ক মুছে ফেলা হয়। সাধারণত ৩০ দিন রাখা হয়; আইনি প্রয়োজনে বেশি। Retention কমালে খরচ কমে, কিন্তু পুরোনো ঘটনার তদন্ত অসম্ভব হয়। Stream-ভিত্তিক retention দিয়ে গুরুত্বপূর্ণ সার্ভিসের লগ বেশি দিন রাখা যায়।

১৫.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

Grafana Alloy কনফিগ — `config.river` (River ভাষায় লেখা, Alloy-র নিজস্ব কনফিগ ভাষা):

```
// Docker কন্টেইনার আবিষ্কার: কোন কন্টেইনার চলছে তা জানা
discovery.docker "technova" {
  host = "unix:///var/run/docker.sock"
}
```

```
// Docker লগ পড়া: প্রতিটি কন্টেইনারের stdout/stderr
loki.source.docker "technova" {
  host       = "unix:///var/run/docker.sock"
  targets    = discovery.docker.technova.targets
  forward_to = [loki.process.technova.receiver]
}

// প্রক্রিয়াকরণ: JSON পার্স, লেবেল যোগ, অপ্রয়োজনীয় লাইন বাদ
loki.process "technova" {
  forward_to = [loki.write.loki.receiver]

  stage.json {
    expressions = {
      level   = "level",
      service = "service",
      msg     = "msg",
    }
  }
  stage.labels {
    values = {
      level   = "",
      service = "",
    }
  }
  stage.drop {
    source = "msg"
    value  = "health check ok"
  }
}

// Loki-তে পাঠানো
loki.write "loki" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
```

`discovery.docker` — Docker-কে জিজ্ঞেস করে কোন কন্টেইনার চলছে; নতুন কন্টেইনার এলে স্বয়ংক্রিয়ভাবে লগ সংগ্রহ শুরু হয়। `loki.source.docker` — কন্টেইনারের লগ পড়ে। `stage.json` — JSON লগ থেকে ক্ষেত্র বের করে; `stage.labels` — সেই ক্ষেত্র থেকে Loki লেবেল বানায় (শুধু কম-কার্ডিনালিটি ক্ষেত্র!)। `stage.drop` — স্বাস্থ্য পরীক্ষার মতো শব্দদূষণ লগ বাদ দেয় — ভলিউম ও খরচ কমে। `loki.write` — ব্যাচ করে Loki-তে পাঠায়; Loki ডাউন থাকলে পুনরায় চেষ্টা করে।

Loki কনফিগ — `loki-config.yaml` (সংক্ষিপ্ত):

```
auth_enabled: false
server:
  http_listen_port: 3100
common:
  path_prefix: /loki
  storage:
    filesystem:
      chunks_directory: /loki/chunks
      rules_directory: /loki/rules
  ring:
```

```
    kvstore:
      store: inmemory
schema_config:
  configs:
    - from: 2024-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h
limits_config:
  retention_period: 720h   # ৩০ দিন
  max_streams_per_user: 10000
  ingestion_rate_mb: 10
  ingestion_burst_size_mb: 20
```

`retention_period: 720h` — ৩০ দিনের পুরোনো লগ মুছে যায়। `max_streams_per_user` — stream বিস্ফোরণ ঠেকায়। `ingestion_rate_mb` — লগের বন্যা ঠেকায়; হঠাৎ ভলিউম বাড়লে Loki গ্রহণ সীমিত করে।

Docker Compose-এ Loki + Alloy যোগ করা:

```
  loki:
    image: grafana/loki:3.2.0
    container_name: loki
    ports:
      - "3100:3100"
    volumes:
      - ./loki-config.yaml:/etc/loki/local-config.yaml:ro
      - loki-data:/loki
    command: -config.file=/etc/loki/local-config.yaml
    restart: unless-stopped

  alloy:
    image: grafana/alloy:v1.4.0
    container_name: alloy
    volumes:
      - ./config.river:/etc/alloy/config.river:ro
      - /var/run/docker.sock:/var/run/docker.sock:ro
      - /var/log:/var/log:ro
    command: run --server.http.listen-addr=0.0.0.0:12345 /etc/alloy/config.river
    restart: unless-stopped
```

Alloy-কে `docker.sock` দেওয়া হয় কন্টেইনার আবিষ্কারের জন্য — এটি শক্তিশালী অনুমতি, তাই শুধু বিশ্বস্ত পরিবেশে দিন। `/var/log:ro` হোস্টের ফাইল লগ পড়তে দেয়।

১৫.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র সম্পূর্ণ স্ট্যাক: Prometheus + Loki + Alloy + Grafana। Loki-তে Grafana Data Source যোগ করা (provisioning-এ আগেই দেখানো হয়েছে — `url: http://loki:3100`)। Grafana Explore-এ LogQL যাচাই:

```
# একটি Correlation ID-র পুরো যাত্রা
{service="technova-api"} | json | requestId="9f2c-..."

# গত ১ ঘণ্টায় সব সার্ভিসের ERROR
```

```
{env="prod"} | json | level="error"

# MongoDB-র ধীর query লগ
{service="mongodb"} |~ "slow query|ms$"

# লগ থেকে মেট্রিক: প্রতি ৫ মিনিটে error সংখ্যা (Grafana প্যানেলে)
sum by (service) (
  count_over_time({env="prod"} | json | level="error" [5m])
)
```

Grafana ড্যাশবোর্ডে মেট্রিক ও লগ একসঙ্গে: উপরে Prometheus-এর error rate Time Series, নিচে Loki-র Logs প্যানেল — একই সময়সীমা, একই Variable (`$service`)। Time Series-এ স্পাইক দেখলে নিচের লগে ক্লিক করে সেই সময়ের লাইন দেখা যায়। এটিই মেট্রিক-লগ সংযোগ (correlation) — আলাদা টুলে গেলে এই সংযোগ হারিয়ে যায়।

Loki-ভিত্তিক অ্যালার্ট (Grafana Alerting-এ):

```
# ৫ মিনিটে ১০টির বেশি ERROR লগ হলে অ্যালার্ট
sum(count_over_time({service="technova-api"} | json | level="error" [5m])) > 10
# শর্ত: IS ABOVE 10, Evaluate every 1m, For 5m
# Labels: severity=warning, service=technova-api
```

> **PRO TIP:** লগ-ভিত্তিক অ্যালার্ট মেট্রিক অ্যালার্টের পরিপূরক, বিকল্প নয়। মেট্রিক অ্যালার্ট দ্রুত ও সস্তা; লগ অ্যালার্ট তখনই ব্যবহার করুন যখন মেট্রিকে সেই তথ্য নেই — যেমন নির্দিষ্ট ভুল বার্তার সংখ্যা।

১৫.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — মেট্রিক থেকে লগে। Error rate বেড়েছে (Grafana Stat লাল)। একই ড্যাশবোর্ডের Logs প্যানেলে দেখা গেল সব error-এ একই বার্তা: `gateway timeout after 5000ms` — পেমেন্ট গেটওয়ে ধীর। মেট্রিক বলল "কত", লগ বলল "কেন" — একই পাতায়।

পরিস্থিতি ২ — মেমোরি লিকের লগ প্রমাণ। মেমোরি বাড়ছে। লগে `durationMs > 2000` ফিল্টার করে দেখা গেল ধীর অনুরোধগুলো সব একটি নতুন endpoint-এ — সেই endpoint-এ একটি query প্রতিবার পুরো সংগ্রহ (collection) স্ক্যান করছে। লগে query-র নাম থাকায় মূল কারণ ১৫ মিনিটে পাওয়া গেল।

পরিস্থিতি ৩ — Alloy-র drop stage-এ সঞ্চয়। TechNova-র স্বাস্থ্য পরীক্ষা প্রতি ১০ সেকেন্ডে লগ লিখছিল — দিনে ৮,৬৪০ লাইন, কোনো মূল্য ছাড়া। `stage.drop` দিয়ে সেগুলো বাদ দেওয়ার পর লগ ভলিউম ৩০% কমল, খরচও কমল।

১৫.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **Loki চালু করুন:** উপরের Docker Compose অংশ যোগ করে `docker compose up -d loki` করুন; `curl http://203.0.113.10:3100/ready` দিয়ে প্রস্তুতি যাচাই করুন।
2. **Alloy চালু করুন:** `config.river` লিখে `docker compose up -d alloy` করুন; Alloy UI (`:12345`) খুলে component-গুলো সবুজ কি না দেখুন।
3. **Grafana-তে Loki Data Source যোগ করুন:** URL `http://loki:3100` → Save & Test।
4. **Explore-এ প্রথম query:** `{service="technova-api"}` লিখে লগ আসছে কি না দেখুন। না এলে Alloy লগ দেখুন।

5. **Line filter শিখুন:** `|= "error"`, `!= "health"`, `|~ "timeout|failed"` — তিনটি চেষ্টা করুন।
6. **JSON parser ব্যবহার করুন:** `| json | level="error"` লিখে ক্ষেত্রে ফিল্টার করুন।
7. **Metric query বানান:** `sum(rate({service="technova-api"} | json | level="error" [5m]))` দিয়ে Time Series প্যানেল বানান।
8. **ড্যাশবোর্ডে সংযোগ করুন:** API ড্যাশবোর্ডে Logs প্যানেল যোগ করুন; একই Variable ও সময়সীমা ব্যবহার করুন।

১৫.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	কেন ক্ষতিকর	সঠিক পদ্ধতি
request_id লেবেল করা	Stream বিস্ফোরণ, Loki ধীর	ID লেখার ভেতরে রাখুন, লেবেলে নয়
Promtail দিয়ে নতুন স্থাপনা	Maintenance mode, নতুন বৈশিষ্ট্য নেই	Alloy ব্যবহার করুন
Loki-কে Elasticsearch ভাবা	Full-text অনুসন্ধান ধীর হবে ভেবে হতাশা	Loki লেবেল-ভিত্তিক; শব্দ খুঁজতে লেবেল দিয়ে stream ছোট করুন
সব লগে উচ্চ cardinality লেবেল	ইনডেক্স বড়, query ধীর	লেবেল সর্বোচ্চ ৫–৭টি, কম-কার্ডিনালিটি
Line filter ছাড়া বড় stream স্ক্যান	ধীর query, বেশি খরচ	প্রথমে লেবেল, তারপর line filter, তারপর parser
Retention না ঠিক করা	ডিস্ক ভরে যায়	retention_period স্পষ্টভাবে সেট করুন
স্বাস্থ্য পরীক্ষার লগ সংগ্রহ করা	ভলিউম ও খরচ বাড়ে, মূল্য নেই	stage.drop দিয়ে বাদ দিন

১৫.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Loki-তে লগ যাওয়ার আগেই redact করুন — Alloy-র `stage.replace` দিয়ে সংবেদনশীল প্যাটার্ন মাস্ক করা যায়, তবে সবচেয়ে ভালো অ্যাপেই redact করা (অধ্যায় ১৪)। Loki-র `auth_enabled` ছোট স্থাপনায় বন্ধ থাকে, কিন্তু প্রোডাকশনে multi-tenant হলে চালু করুন — নইলে এক দল আরেক দলের লগ দেখতে পারে। Grafana-তে Loki Data Source-এর অনুমতি সীমিত করুন। `docker.sock` Alloy-কে দেওয়া শক্তিশালী অনুমতি — শুধু বিশ্বস্ত হোস্টে দিন, এবং Alloy-র নিজের লগে গোপন তথ্য যাচ্ছে কি না দেখুন।

১৫.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Loki-র খরচ তিনটি জায়গায়: ইনডেক্স (লেবেলের সংখ্যা), চাঙ্ক স্টোরেজ (ভলিউম × retention), ও query (স্ক্যানের পরিমাণ)। লেবেল কম রাখলে ইনডেক্স ছোট থাকে; অপ্রয়োজনীয় লগ বাদ দিলে স্টোরেজ কমে; লেবেল দিয়ে stream ছোট করে query করলে স্ক্যান কমে। বড় সময়সীমায় (৭ দিন) query করলে Loki অনেক চাঙ্ক স্ক্যান করে — তখন metric query-র জন্য Recording Rule-এর মতো আগে থেকে হিসাব করা মেট্রিক ব্যবহার করুন, বা সময়সীমা ছোট করুন। `ingestion_rate_mb` সীমা দিয়ে লগের বন্যা ঠেকান।

১৫.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	কারণ	সমাধান
Explore-এ "No data"	Alloy পাঠাচ্ছে না বা লেবেল ভুল	Alloy UI-তে component স্বাস্থ্য দেখুন; `{service=~".+"}` দিয়ে সব stream দেখুন
লগ আসছে কিন্তু দেরিতে	ব্যাচিং বা Loki চাপে আছে	Alloy-র batch সেটিং দেখুন; Loki-র ingestion rate যাচাই করুন
"too many streams"	High cardinality লেবেল	লেবেল কমান; `max_streams_per_user` যাচাই করুন
Query ধীর	বড় stream, বড় সময়সীমা, parser আগে	লেবেল দিয়ে ছোট করুন; line filter parser-এর আগে দিন
Alloy component লাল	কনফিগ ভুল বা Loki পৌঁছাচ্ছে না	`docker compose logs alloy` দেখুন; River সিনট্যাক্স যাচাই করুন
পুরোনো লগ পাওয়া যাচ্ছে না	Retention শেষ	retention_period যাচাই করুন; প্রয়োজনে বাড়ান

১৫.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Alloy ডিফল্ট সংগ্রাহক; লেবেল কম ও কম-কার্ডিনালিটি; প্রথমে লেবেল, তারপর line filter, তারপর parser; JSON লগ; অপ্রয়োজনীয় লগ drop; retention স্পষ্ট; মেট্রিক ও লগ একই ড্যাশবোর্ডে; লগ-ভিত্তিক অ্যালার্ট শুধু প্রয়োজনে।

অ্যান্টি-প্যাটার্ন: Promtail দিয়ে নতুন স্থাপনা; request_id লেবেল; সবকিছু লেবেল করা; parser আগে, filter পরে; Loki-কে full-text search ইঞ্জিন ভাবা; retention না ঠিক করা; স্বাস্থ্য পরীক্ষার লগ জমা করা।

১৫.১৬ অধ্যায়ের সারাংশ

Loki শুধু লেবেল ইনডেক্স করে, লেখা সংকুচিত চাঙ্কে রাখে — তাই হালকা ও সস্তা, Prometheus-এর মতোই পরিচালনা সহজ। Stream হলো একই লেবেলের লগের ধারা; লেবেল কম ও কম-কার্ডিনালিটি রাখা অপরিহার্য। Grafana Alloy বর্তমান সুপারিশকৃত সংগ্রাহক — Promtail maintenance mode-এ আছে, নতুন স্থাপনায় Alloy ব্যবহার করুন। LogQL-এর তিন ধাপ: stream selector, line filter, parser; metric query-তে rate ও count_over_time। Grafana-তে মেট্রিক ও লগ পাশাপাশি দেখে তদন্ত করুন, এবং retention ও খরচের পরিকল্পনা করুন।

১৫.১৭ মূল প্রযুক্তিগত পরিভাষা

Loki: Grafana-র লগ সংগ্রহ ও অনুসন্ধান সিস্টেম — শুধু লেবেল ইনডেক্স করে, লেখা সংকুচিত রাখে।

Grafana Alloy: Loki-র বর্তমান সুপারিশকৃত সংগ্রাহক — মেট্রিক, লগ ও ট্রেস এক এজেন্টে সংগ্রহ করে।

Promtail: Loki-র পুরোনো সংগ্রাহক, এখন maintenance mode-এ — নতুন স্থাপনায় ব্যবহার করবেন না।

LogQL: Loki-র query ভাষা — লগ খোঁজা ও লগ থেকে মেট্রিক বানানোর ভাষা।

Stream: একই লেবেলের সব লগ লাইনের ধারা — Loki-র মৌলিক একক।

Label: লগ stream-কে চিহ্নিত করার কী-মান জোড়া — যেমন service, level, env।

Line Filter: লগের লেখায় শব্দ খোঁজা — `|=`, `!=`, `|~`।

Parser: লগের লেখা থেকে ক্ষেত্র বের করা — `| json`, `| logfmt`।

Cardinality: একটি লেবেলের ভিন্ন মানের সংখ্যা — বেশি হলে stream বিস্ফোরণ হয়।

River: Alloy-র কনফিগ ভাষা — component ও তাদের সংযোগ লেখার ভাষা।

১৫.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Loki কেন পুরো লেখা ইনডেক্স করে না? এতে কী সুবিধা ও সীমা তৈরি হয়?
2. Stream কী? `request_id` লেবেল করলে কী সমস্যা হয়?
3. Promtail-এর বর্তমান অবস্থা কী? নতুন স্থাপনায় কী ব্যবহার করবেন?
4. LogQL-এর তিন ধাপ লিখুন ও প্রতিটির উদাহরণ দিন।
5. `|= "error"` ও `| json | level="error"`-এর পার্থক্য কী?
6. `count_over_time()` দিয়ে কীভাবে লগ থেকে অ্যালার্টের মেট্রিক বানাবেন?
7. লেবেল দিয়ে stream ছোট না করে বড় সময়সীমায় query করলে কী হয়?
8. Alloy-র `stage.drop` কেন খরচ কমায়?
9. Loki বনাম Elasticsearch — কখন কোনটি বাছবেন?
10. Grafana-তে মেট্রিক ও লগ পাশাপাশি দেখার সুবিধা কী?

১৫.১৯ ব্যবহারিক অনুশীলন

1. Loki + Alloy Docker Compose দিয়ে চালু করুন ও `/ready` যাচাই করুন।
2. Grafana-তে Loki Data Source যোগ করে `{service="technova-api"}` query চালান।
3. Line filter ও JSON parser দিয়ে শুধু ERROR লগ খুঁজে বের করুন।

4. `rate()` দিয়ে error rate-এর Time Series প্যানেল বানান।
5. একটি Correlation ID দিয়ে পুরো অনুরোধের যাত্রা খুঁজুন।
6. `stage.drop` দিয়ে স্বাস্থ্য পরীক্ষার লগ বাদ দিয়ে ভলিউমের পার্থক্য মাপুন।
7. একটি লগ-ভিত্তিক Alert Rule বানান (৫ মিনিটে ১০টির বেশি ERROR) ও পরীক্ষা করুন।

অধ্যায় ১৬ • PART III — Visualization and Logging Engineering

Chapter 16 — ELK Stack, Elastic Stack, and OpenSearch Logging

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Elastic Stack (Elasticsearch, Logstash, Kibana, Beats/Elastic Agent) ও OpenSearch-এর আর্কিটেকচার ব্যাখ্যা করতে পারবেন, ইনডেক্সিং ও অনুসন্ধান কীভাবে কাজ করে তা বুঝবেন, ingestion pipeline লিখতে পারবেন, এবং Loki-র সঙ্গে সৎ তুলনা করে কোন পরিস্থিতিতে কোনটি বাছবেন তা সিদ্ধান্ত নিতে পারবেন।

- Elasticsearch-এর index, shard, document ও mapping ব্যাখ্যা করা।
- Logstash pipeline (input → filter → output) ও Filebeat/Elastic Agent দিয়ে লগ পাঠানো।
- Kibana/OpenSearch Dashboards-এ অনুসন্ধান ও ভিজ্যুয়ালাইজেশন করা।
- OpenSearch-এর আর্কিটেকচার ও Elastic Stack-এর সঙ্গে সম্পর্ক বোঝা।
- Loki বনাম Elastic/OpenSearch — খরচ, query শক্তি ও পরিচালনার ভার তুলনা করা।

১৬.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Loki ছোট ও মাঝারি দলের জন্য যথেষ্ট, কিন্তু কিছু পরিস্থিতিতে পূর্ণ-লেখা অনুসন্ধান (full-text search) অপরিহার্য: নিরাপত্তা তদন্তে যেকোনো শব্দ খোঁজা, আইনি অনুসন্ধানে সব ক্ষেত্রে খোঁজা, বড় ভলিউমে জটিল aggregation। Elasticsearch ও OpenSearch এই কাজের জন্য তৈরি — তারা প্রতিটি শব্দের সূচি বানায়, ফলে যেকোনো শব্দে মুহূর্তে খোঁজা যায়। বড় প্রতিষ্ঠান, নিরাপত্তা দল (SIEM) ও compliance-প্রয়োজনীয় পরিবেশে এটিই মান।

আরও একটি কারণ: বাজারে অনেক চাকরি ও বিদ্যমান সিস্টেম এখনো ELK-ভিত্তিক। একজন DevOps ইঞ্জিনিয়ার হিসেবে আপনি Loki জানলেই চলবে না — Elasticsearch-এর ধারণা, Logstash pipeline, Kibana-র অনুসন্ধানও জানতে হবে, নইলে পুরোনো সিস্টেম রক্ষণাবেক্ষণ বা স্থানান্তরের কাজে পিছিয়ে পড়বেন।

> **REAL WORLD:** TechNova-র একটি ক্লায়েন্ট ব্যাংকিং খাতে কাজ করে — নিয়ন্ত্রক সংস্থা চায় যেকোনো লেনদেনের লগ যেকোনো ক্ষেত্র দিয়ে খুঁজে বের করা যাক, ৯০ দিন ধরে। Loki-র লেবেল-ভিত্তিক মডেলে এই প্রয়োজন মেটানো কঠিন; OpenSearch-এর full-text ইনডেক্স সেখানে সঠিক পছন্দ। প্রযুক্তি বাছাই ব্যবসায়িক প্রয়োজন থেকে আসে, পছন্দ থেকে নয়।

১৬.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৪-এর লগিং ধারণা, অধ্যায় ১৫-এর Loki তুলনা, JSON, YAML, এবং REST API-র মৌলিক ধারণা দরকার। Elasticsearch-এর query ভাষা (Query DSL) JSON-ভিত্তিক — JSON পড়তে পারলেই শুরু করা যায়। Docker Compose দিয়ে তিনটি সার্ভিস চালানোর অভিজ্ঞতা সাহায্য করবে।

পূর্বশর্ত	কেন দরকার
কেন্দ্রীয় লগিং	Ingestion → Storage → Search প্রবাহ বুঝতে

Loki আর্কিটেকচার	তুলনা করে পার্থক্য বুঝতে
JSON ও YAML	Mapping, pipeline ও Query DSL লিখতে
REST API	Elasticsearch API-তে query পাঠাতে

১৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

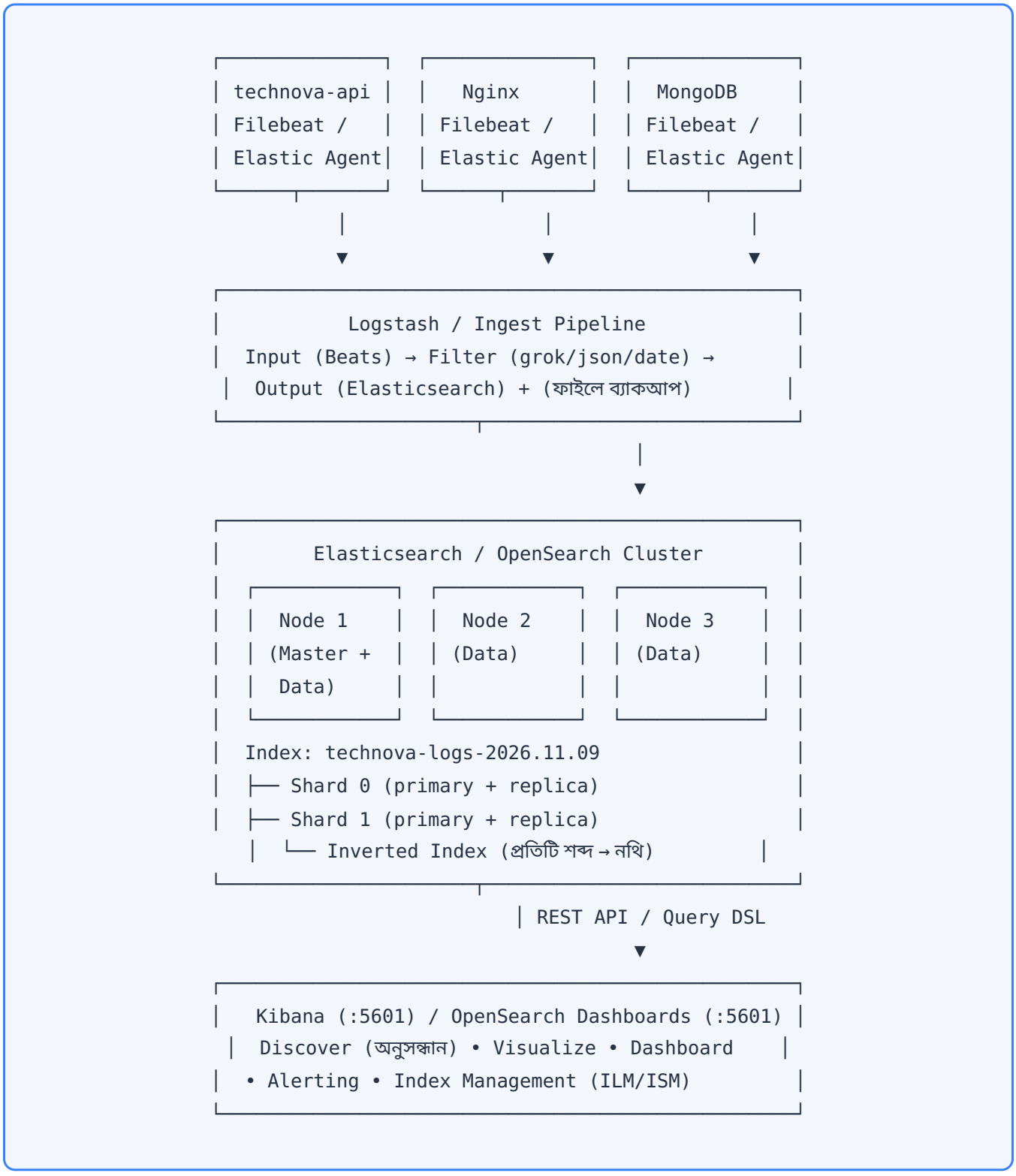
প্রথম নীতি: অনুসন্ধান মানে আগে থেকে সাজিয়ে রাখা। বইয়ের প্রতিটি শব্দের পাতা নম্বর লিখে রাখলে যেকোনো শব্দ খুঁজে পাওয়া যায় — এটিই `Inverted Index`। Elasticsearch প্রতিটি লগের প্রতিটি শব্দের জন্য লেখে: "এই শব্দটি এই নথিতে আছে।" ফলে "timeout" লিখে খুঁজলে মুহূর্তে সব নথি পাওয়া যায়। এই সুবিধার মূল্য: ইনডেক্সটি ডেটার মতোই বড়, কখনো বড়তর — ডিস্ক, মেমোরি ও CPU-র খরচ।

দ্বিতীয় নীতি: লগ আসার পথে বদলানো যায়। Logstash-এর `Pipeline` তিন ধাপ: `Input` (কোথা থেকে আসবে), `Filter` (কীভাবে বদলাবে — পার্স, সমৃদ্ধ, বাদ), `Output` (কোথায় যাবে)। এই পাইপলাইন মডেল শক্তিশালী: একই লগ Elasticsearch-এ যাবে, আবার ফাইলে ব্যাকআপও হবে। কিন্তু পাইপলাইন নিজেই একটি সার্ভিস — পরিচালনা, মনিটরিং ও স্কেলিং লাগে।

তৃতীয় নীতি: উন্মুক্ত উৎসের ইতিহাস বোঝা জরুরি। Elasticsearch একসময় সম্পূর্ণ উন্মুক্ত ছিল; লাইসেন্স পরিবর্তনের পর সম্প্রদায় OpenSearch তৈরি করে — একই ধারণা, আলাদা প্রকল্প। দুটির API ও ধারণা কাছাকাছি, কিন্তু তারা আলাদা পথে এগোচ্ছে। নতুন স্থাপনায় লাইসেন্স ও পরিচালনার ভার যাচাই করে সিদ্ধান্ত নিন — শুধু নাম দেখে নয়।

> **CONCEPT:** Elasticsearch/OpenSearch বনাম Loki — একটি বাণিজ্য: পুরো লেখা ইনডেক্স করা মানে যেকোনো প্রশ্নের উত্তর, কিন্তু বেশি খরচ; শুধু লেবেল ইনডেক্স করা মানে কম খরচ, কিন্তু সীমিত প্রশ্ন। আপনার প্রশ্নের ধরনই বলে দেবে কোনটি সঠিক।

১৬.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



মূল ধারণাগুলো: `Document` — একটি JSON লগ, ইনডেক্সের একক। `Index` — একই ধরনের নথির সংগ্রহ, সাধারণত দিনভিত্তিক (`technova-logs-2026.11.09`)। `Shard` — ইনডেক্সের ভাগ; প্রতিটি shard একটি Lucene ইনডেক্স। `Replica` — shard-এর অনুলিপি, ব্যর্থতা থেকে রক্ষা করে। `Mapping` — প্রতিটি ক্ষেত্রের ধরন (text, keyword, date, long) — ভুল mapping-এ অনুসন্ধান ভুল হয়। `Node` — ক্লাস্টারের একক সার্ভার; `Cluster` — একসঙ্গে কাজ করা নোডের দল।

OpenSearch-এর আর্কিটেকচার একই ধারণা অনুসরণ করে — Document, Index, Shard, Node, Cluster — তবে নিজস্ব Dashboards, Ingestion (Data Prepper) ও Index State Management (ISM) আছে। Elasticsearch-এর Index Lifecycle Management (ILM)-এর অনুরূপ ISM দিয়ে পুরোনো ইনডেক্স স্বয়ংক্রিয়ভাবে মুছে বা ঠান্ডা স্টোরেজে পাঠানো যায়।

সংগ্রাহকের বর্তমান অবস্থা: Beats (Filebeat) এখনো ব্যবহৃত হয়, কিন্তু Elastic-এর বর্তমান সুপারিশ `Elastic Agent` — একটি এজেন্টে বহু integration, Fleet দিয়ে কেন্দ্রীয় পরিচালনা। Logstash এখনো শক্তিশালী pipeline-এর জন্য ব্যবহৃত হয়, তবে হালকা কাজে Filebeat/Elastic Agent সরাসরি Elasticsearch-এ পাঠাতে পারে। নতুন স্থাপনায় Elastic Agent দিয়ে শুরু করুন; জটিল রূপান্তর দরকার হলে Logstash যোগ করুন।

১৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Indexing ও Searching — কীভাবে কাজ করে

একটি লগ Elasticsearch-এ এলে: (১) `Ingest Pipeline` ক্ষেত্র বের করে ও সমৃদ্ধ করে; (২) `Mapping` অনুযায়ী প্রতিটি ক্ষেত্র বিশ্লেষিত (analyzed) হয় — `text` ক্ষেত্র শব্দে ভাঙা হয় ও inverted index-এ যায়, `keyword` ক্ষেত্র হুবহু থাকে (ফিল্টার ও aggregation-এর জন্য); (৩) নথিটি shard-এ লেখা হয় ও replica-তে অনুলিপি হয়। অনুসন্ধানে Query DSL চলে: `match` (শব্দ খোঁজা), `term` (হুবহু মান), `range` (সময়/সংখ্যা), `bool` (একাধিক শর্ত), `aggs` (গণনা — যেমন status অনুযায়ী সংখ্যা)।

Mapping — সবচেয়ে সাধারণ ভুলের উৎস

Elasticsearch প্রথম নথি দেখে mapping অনুমান করে (`Dynamic Mapping`) — কিন্তু অনুমান ভুল হতে পারে: `"500"` সংখ্যা হিসেবে নিলে পরে `"500 Internal"` লেখা ভেঙে যাবে। তাই গুরুত্বপূর্ণ ক্ষেত্রে `Explicit Mapping` লিখুন: `status` → `keyword`, `durationMs` → `long`, `@timestamp` → `date`, `message` → `text`। Mapping ইনডেক্স তৈরির পর বদলানো কঠিন — ভুল হলে নতুন ইনডেক্সে পুনরায় ইনডেক্স (reindex) করতে হয়।

Logstash Pipeline

Logstash-এর শক্তি `Filter` প্লাগইনে: `grok` (অসংগঠিত লেখা থেকে ক্ষেত্র বের করা), `json` (JSON পার্স করা), `date` (সময় পার্স করা), `mutate` (ক্ষেত্র যোগ/বাদ/বদলানো), `geoip` (IP থেকে দেশ/শহর)। তবে grok জটিল ও ধীর — structured JSON লগ থাকলে grok-এর দরকার নেই, `json` filter-ই যথেষ্ট। এজন্যই structured logging Elasticsearch-এও লাভজনক।

Loki বনাম Elastic/OpenSearch — সৎ তুলনা

দিক	Loki	Elasticsearch / OpenSearch
ইনডেক্স	শুধু লেবেল	পুরো লেখা (inverted index)
অনুসন্ধান শক্তি	লেবেল + লেখা স্ক্যান	যেকোনো শব্দ, জটিল aggregation
খরচ (ডিস্ক/মেমোরি)	কম	বেশি — ইনডেক্স ডেটার মতো বড়
পরিচালনার ভার	হালকা, Prometheus-এর মতো	ভারী — ক্লাস্টার, shard, heap পরিচালনা
Grafana সংযোগ	নিজস্ব, গভীর	Grafana Data Source আছে, তবে Kibana/Dashboards মূল

সেরা ব্যবহার	অ্যাপ লগ, মেট্রিকের পাশে তদন্ত	নিরাপত্তা, compliance, বড় ভলিউম, জটিল অনুসন্ধান

১৬.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

Elasticsearch mapping — `technova-mapping.json`:

```
{
  "mappings": {
    "properties": {
      "@timestamp": { "type": "date" },
      "service":    { "type": "keyword" },
      "env":        { "type": "keyword" },
      "level":      { "type": "keyword" },
      "requestId":  { "type": "keyword" },
      "status":     { "type": "integer" },
      "durationMs": { "type": "long" },
      "message":    { "type": "text" },
      "userId":     { "type": "keyword" }
    }
  }
}
```

`keyword` মানে হুবহু মান — ফিল্টার ও গণনার জন্য; `text` মানে বিশ্লেষিত লেখা — শব্দে খোঁজার জন্য। `status`-কে `integer` করলে `status >= 500` অনুসন্ধান করা যায়; `keyword` করলে যেত না। `@timestamp` `date` ধরনের হলে সময়সীমায় অনুসন্ধান দ্রুত হয়।

Logstash pipeline — `logstash.conf`:

```
input {
  beats {
    port => 5044
  }
}
filter {
  json {
    source => "message"
  }
  date {
    match => ["time", "ISO8601"]
    target => "@timestamp"
  }
  mutate {
    remove_field => ["host", "agent"]
  }
}
output {
  elasticsearch {
    hosts => ["http://elasticsearch:9200"]
    index => "technova-logs-%{+YYYY.MM.dd}"
    user => "elastic"
    password => "CHANGE-ME-ELASTIC-PASSWORD"
  }
}
```

`input beats port 5044` — Filebeat/Elastic Agent এখানে পাঠায়। `filter json` — structured লগ পার্স করে। `filter date` — লগের সময়কে `@timestamp`-এ বসায়; নইলে Logstash-এর গ্রহণের সময় ব্যবহৃত হয়, যা ভুল হতে পারে। `mutate remove_field` — অপ্রয়োজনীয় ক্ষেত্র বাদ দিয়ে ইনডেক্স ছোট রাখে। `output index => "technova-logs-%{+YYYY.MM.dd}"` — দিনভিত্তিক ইনডেক্স, ফলে পুরোনো দিন মুছে ফেলা সহজ। পাসওয়ার্ড `CHANGE-ME-...` প্লেসহোল্ডার — আসল মান secret ম্যানেজার থেকে দিন।

Filebeat কনফিগ — `filebeat.yml` (সংক্ষিপ্ত):

```
filebeat.inputs:
  - type: container
    paths:
      - /var/lib/docker/containers/*/*.log
    processors:
      - add_docker_metadata: ~
output.logstash:
  hosts: ["logstash:5044"]
```

OpenSearch-এ অনুসন্ধান (Query DSL):

```
GET /technova-logs-*/_search
{
  "query": {
    "bool": {
      "must": [
        { "term": { "level": "error" } },
        { "range": { "@timestamp": { "gte": "now-1h" } } }
      ]
    }
  },
  "aggs": {
    "by_service": { "terms": { "field": "service" } }
  }
}
```

`term` হুবহু মিল খোঁজে, `range` সময়সীমা দেয়, `aggs` সার্ভিস অনুযায়ী গণনা করে — কোন সার্ভিসে সবচেয়ে বেশি error, তা এক query-তে পাওয়া যায়।

১৬.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র জন্য Docker Compose — Elasticsearch (single-node, ল্যাবের জন্য) + Kibana + Logstash:

```
services:
  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.15.0
    container_name: elasticsearch
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=true
      - ELASTIC_PASSWORD=CHANGE-ME-ELASTIC-PASSWORD
      - ES_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"
    volumes:
```

```
      - elasticsearch-data:/usr/share/elasticsearch/data
    restart: unless-stopped

  kibana:
    image: docker.elastic.co/kibana/kibana:8.15.0
    container_name: kibana
    environment:
      - ELASTICSEARCH_HOSTS=http://elasticsearch:9200
      - ELASTICSEARCH_USERNAME=kibana_system
      - ELASTICSEARCH_PASSWORD=CHANGE-ME-KIBANA-PASSWORD
    ports:
      - "5601:5601"
    depends_on:
      - elasticsearch
    restart: unless-stopped

  logstash:
    image: docker.elastic.co/logstash/logstash:8.15.0
    container_name: logstash
    ports:
      - "5044:5044"
    volumes:
      - ./logstash.conf:/usr/share/logstash/pipeline/logstash.conf:ro
    depends_on:
      - elasticsearch
    restart: unless-stopped

volumes:
  elasticsearch-data:
```

`discovery.type=single-node` — ল্যাবের জন্য এক নোড; প্রোডাকশনে কমপক্ষে ৩ নোডের ক্লাস্টার দরকার। `ES_JAVA_OPTS=-Xms512m -Xmx512m` — ল্যাবের জন্য ছোট heap; প্রোডাকশনে heap মোট মেমোরির অর্ধেকের বেশি দেবেন না, এবং ৩২ GB-এর বেশি নয়। `xpack.security.enabled=true` — নিরাপত্তা চালু; ডিফল্ট পাসওয়ার্ড কখনো রাখবেন না।

OpenSearch-এর সমতুল্য (সংক্ষিপ্ত):

```
services:
  opensearch:
    image: opensearchproject/opensearch:2.17.0
    container_name: opensearch
    environment:
      - discovery.type=single-node
      - OPENSEARCH_INITIAL_ADMIN_PASSWORD=CHANGE-ME-OPENSEARCH-ADMIN
      - OPENSEARCH_JAVA_OPTS=-Xms512m -Xmx512m
    ports:
      - "9200:9200"
    volumes:
      - opensearch-data:/usr/share/opensearch/data
    restart: unless-stopped

  opensearch-dashboards:
    image: opensearchproject/opensearch-dashboards:2.17.0
    container_name: opensearch-dashboards
    environment:
      - OPENSEARCH_HOSTS=http://opensearch:9200
    ports:
```

```
      - "5601:5601"
    depends_on:
      - opensearch
    restart: unless-stopped

volumes:
  opensearch-data:
```

স্বাস্থ্য যাচাই:

```
curl -s -u elastic:CHANGE-ME-ELASTIC-PASSWORD http://203.0.113.10:9200/_cluster/health
# (নমুনা আউটপুট — চিত্রণমূলক)
# {"cluster_name":"technova","status":"green","number_of_nodes":1,"active_shards":5}
curl -s http://203.0.113.10:9200/_cat/indices?v
# (নমুনা আউটপুট — চিত্রণমূলক)
# health status index                 docs.count store.size
# green  open   technova-logs-2026.11.09  15230      12mb
```

`status: green` মানে সব shard সুস্থ; `yellow` মানে replica অনুপস্থিত (single-node-এ স্বাভাবিক); `red` মানে primary shard অনুপস্থিত — জরুরি অবস্থা।

১৬.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি ১ — নিরাপত্তা তদন্ত। সন্দেহজনক লগইন চেষ্টা ধরা পড়ল। OpenSearch-এ IP ঠিকানা দিয়ে খুঁজে দেখা গেল একই IP থেকে ২০০টি ব্যর্থ লগইন, ভিন্ন ভিন্ন ব্যবহারকারীর নামে — credential stuffing আক্রমণ। Full-text অনুসন্ধান ছাড়া এই প্যাটার্ন ধরা যেত না। IP-টি ব্লক করা হলো ও অ্যালার্ট নিয়ম যোগ করা হলো।

পরিস্থিতি ২ — ভুল mapping-এর বিপদ। একটি নতুন সার্ভিস `status`-কে লেখা হিসেবে পাঠাল ("500"), আরেকটি সংখ্যা হিসেবে (500)। Dynamic mapping প্রথমটিকে keyword ধরল, দ্বিতীয়টির লগ প্রত্যাখ্যাত হলো। সমাধান: explicit mapping ও index template — সব ইনডেক্সে একই mapping।

পরিস্থিতি ৩ — ক্লাস্টার হলুদ থেকে লাল। ডিস্ক ৮৫% পূর্ণ হলে Elasticsearch নতুন shard বরাদ্দ বন্ধ করে (watermark)। দলটি ILM দিয়ে পুরোনো ইনডেক্স মুছে ডিস্ক খালি করল। শিক্ষা: ডিস্ক মনিটরিং ও ILM নীতি শুরুতেই ঠিক করুন, ডিস্ক ভরার পরে নয়।

> **WARNING:** Elasticsearch-কে ইন্টারনেটে খোলা রাখবেন না। অতীতে খোলা Elasticsearch ক্লাস্টার থেকে বিপুল তথ্য ফাঁস হয়েছে। সবসময় authentication চালু করুন, firewall দিয়ে শুধু বিশ্বস্ত নেটওয়ার্ক থেকে অ্যাক্সেস দিন, এবং নিয়মিত নিরাপত্তা আপডেট করুন।

১৬.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. **ল্যাব চালু করুন:** উপরের Docker Compose দিয়ে Elasticsearch + Kibana চালু করুন; `/_cluster/health` দিয়ে `green` / `yellow` যাচাই করুন।
2. **Mapping তৈরি করুন:** Index template-এ উপরের mapping লিখে `technova-logs-*` প্যাটার্নে প্রয়োগ করুন।
3. **Logstash চালু করুন:** `logstash.conf` দিয়ে Filebeat থেকে লগ পাঠান; Kibana Discover-এ লগ আসছে কি না দেখুন।

4. **Kibana-তে অনুসন্ধান করুন:** `level: error` লিখে ফিল্টার করুন; সময়সীমা বদলে দেখুন; `service` দিয়ে aggregation বানান।
5. **OpenSearch চেষ্টা করুন:** একই লগ OpenSearch-এ পাঠিয়ে Dashboards-এ একই অনুসন্ধান করুন; পার্থক্য লিখে রাখুন।
6. **ILM/ISM নীতি লিখুন:** ৩০ দিন পর ইনডেক্স মুছে ফেলার নীতি সেট করুন ও যাচাই করুন।
7. **Grafana-তে Elasticsearch Data Source যোগ করুন:** একই লগ Grafana-তেও দেখুন; Loki-র সঙ্গে অভিজ্ঞতার তুলনা করুন।
8. **তুলনা নথিভুক্ত করুন:** আপনার দলের জন্য এক পাতার সিদ্ধান্ত নথি লিখুন — কখন Loki, কখন OpenSearch।

১৬.১১ সাধারণ ভুল ও ভুল ধারণা

ভুল	কেন ক্ষতিকর	সঠিক পদ্ধতি
Elasticsearch ইন্টারনেটে খোলা	তথ্য ফাঁস, ransomware	Authentication + firewall, শুধু ব্যক্তিগত নেটওয়ার্ক
Dynamic mapping-এর উপর ভরসা	ভুল ধরন, লগ প্রত্যাখ্যাত	Explicit mapping + index template
একটি বিশাল ইনডেক্স	Shard বড়, অনুসন্ধান ধীর, মুছে ফেলা কঠিন	দিনভিত্তিক ইনডেক্স, ILM/ISM
Heap খুব বড় (৩২ GB-এর বেশি)	JVM compressed pointer হারায়, কর্মক্ষমতা কমে	Heap মোট মেমোরির ৫০%, সর্বোচ্চ ৩২ GB
সব ক্ষেত্র `text` হিসেবে	Aggregation করা যায় না, ইনডেক্স বড়	ফিল্টারের ক্ষেত্র `keyword`, খোঁজার ক্ষেত্র `text`
Logstash ছাড়া জটিল grok	ধীর, ভুল-প্রবণ	Structured JSON লগ, grok শুধু পুরোনো লগে
ILM/ISM ছাড়া retention	ডিস্ক ভরে ক্লাস্টার লাল হয়	শুরুতেই lifecycle নীতি
সব লগ Elasticsearch-এ, Loki-ও আছে	দ্বিগুণ খরচ, দ্বিগুণ পরিচালনা	একটি মূল সিস্টেম বাছুন, অন্যটি শুধু প্রয়োজনে

১৬.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Elasticsearch/OpenSearch-এর নিরাপত্তা তিন স্তরে: `Authentication` (ব্যবহারকারী ও ভূমিকা — শুধু যা দরকার, ততটুকু অনুমতি), `Transport security` (নোডের মধ্যে TLS), ও `Network` (firewall, শুধু বিশ্বস্ত নেটওয়ার্ক)। Kibana/Dashboards-এর অ্যাডমিন পাসওয়ার্ড শক্তিশালী করুন ও নিয়মিত ঘোরান। লগে সংবেদনশীল তথ্য থাকলে Index-ভিত্তিক অ্যাক্সেস নিয়ন্ত্রণ করুন — যেমন পেমেন্ট লগ শুধু পেমেন্ট দল দেখবে। PII-র retention আইন মেনে ঠিক করুন, এবং মুছে ফেলার সময় replica ও snapshot থেকেও মুছেছে কি না যাচাই করুন।

১৬.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Elasticsearch-এর খরচের মূল কারণ ইনডেক্সের আকার: প্রতিটি ক্ষেত্র ইনডেক্স করা মানে বেশি ডিস্ক। অপ্রয়োজনীয় ক্ষেত্র `index: false` করুন বা Logstash-এ বাদ দিন। Shard-এর সংখ্যা নিয়ন্ত্রণ করুন — অনেক ছোট shard ক্লাস্টারকে ধীর করে; প্রতিটি shard ১০–৫০ GB লক্ষ্য রাখুন। Heap সঠিকভাবে দিন (৫০%, সর্বোচ্চ ৩২ GB)। ILM/ISM দিয়ে পুরোনো ইনডেক্স ঠান্ডা স্টোরেজে পাঠান বা মুছে ফেলুন। Query-তে সময়সীমা সবসময় দিন — সময়সীমা ছাড়া query পুরো ক্লাস্টার স্ক্যান করে।

খরচের তুলনা: একই ভলিউমে Elasticsearch-এর ডিস্ক Loki-র ৩–৫ গুণ হতে পারে। তাই সিদ্ধান্ত নিন প্রয়োজনে: full-text অনুসন্ধান সত্যিই দরকার কি না। দরকার হলে খরচটি যুক্তিসঙ্গত; না হলে Loki-ই যথেষ্ট।

১৬.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	কারণ	সমাধান
Cluster status red	Primary shard অনুপস্থিত	`/_cat/shards?v` দেখুন; নোড ও ডিস্ক যাচাই করুন
Cluster status yellow	Replica বরাদ্দ হয়নি	Single-node-এ স্বাভাবিক; প্রোডাকশনে নোড যোগ করুন
লগ আসছে না	Logstash বা Filebeat থেমে আছে	Logstash লগ দেখুন; Filebeat-এর registry যাচাই করুন
লগ প্রত্যাখ্যাত (rejected)	Mapping দ্বন্দ্ব	Elasticsearch লগে mapping error দেখুন; template ঠিক করুন
ডিস্ক পূর্ণ, লেখা বন্ধ	Watermark ছাড়িয়েছে	পুরোনো ইনডেক্স মুছুন; ILM নীতি ঠিক করুন
Query ধীর	সময়সীমা নেই বা অনেক shard	সময়সীমা দিন; shard সংখ্যা কমান
Kibana-তে ইনডেক্স দেখা যাচ্ছে না	Index pattern তৈরি হয়নি	Stack Management → Index Patterns-এ `technova-logs-*` যোগ করুন

১৬.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: Explicit mapping + index template; দিনভিত্তিক ইনডেক্স + ILM/ISM; Structured JSON লগ; Elastic Agent দিয়ে শুরু, প্রয়োজনে Logstash; Authentication + TLS + firewall; Heap ৫০%/৩২ GB সীমা; ডিস্ক মনিটরিং; একটি মূল লগ সিস্টেম বাছা।

অ্যান্টি-প্যাটার্ন: খোলা ক্লাস্টার; Dynamic mapping-এর উপর ভরসা; একটি বিশাল ইনডেক্স; অতিরিক্ত shard; সব ক্ষেত্র text; ILM ছাড়া retention; grok-এর উপর অতিরিক্ত নির্ভরতা; Loki ও Elasticsearch দুটোতেই সব লগ পাঠানো।

১৬.১৬ অধ্যায়ের সারাংশ

Elastic Stack — Elasticsearch (সংরক্ষণ ও অনুসন্ধান), Logstash (pipeline), Kibana (ভিজ্যুয়ালাইজেশন), Elastic Agent/Filebeat (সংগ্রহ) — পুরো লেখা ইনডেক্স করে যেকোনো শব্দে অনুসন্ধান দেয়, কিন্তু খরচ ও পরিচালনার ভার বেশি। OpenSearch একই ধারণার উন্মুক্ত বিকল্প, নিজস্ব Dashboards ও ISM নিয়ে। সিদ্ধান্ত নিন প্রয়োজনে: লেবেল-ভিত্তিক তদন্তে Loki, যেকোনো শব্দে অনুসন্ধান, নিরাপত্তা ও compliance-এ Elastic/OpenSearch। Mapping, দিনভিত্তিক ইনডেক্স, ILM/ISM, নিরাপত্তা ও heap — এই পাঁচটি ঠিক না করলে কোনো ক্লাস্টারই প্রোডাকশন-প্রস্তুত নয়।

১৬.১৭ মূল প্রযুক্তিগত পরিভাষা

Elasticsearch: বিতরণকৃত অনুসন্ধান ও বিশ্লেষণ ইঞ্জিন — লগের পুরো লেখা ইনডেক্স করে দ্রুত অনুসন্ধান দেয়।

Logstash: লগ প্রক্রিয়াকরণের পাইপলাইন — input → filter → output ধাপে লগ রূপান্তর করে।

Kibana: Elasticsearch-এর ভিজ্যুয়ালাইজেশন ও অনুসন্ধান ইন্টারফেস।

Beats / Filebeat: হালকা লগ সংগ্রাহক; Filebeat ফাইল থেকে লগ পড়ে পাঠায়।

Elastic Agent: Elastic-এর বর্তমান সুপারিশকৃত সংগ্রাহক — বহু integration এক এজেন্টে, Fleet দিয়ে পরিচালিত।

OpenSearch: Elasticsearch-ধারণার উন্মুক্ত উৎসের বিকল্প — নিজস্ব Dashboards ও ISM নিয়ে।

Inverted Index: প্রতিটি শব্দ কোন নথিতে আছে, তার সূচি — দ্রুত full-text অনুসন্ধানের ভিত্তি।

Shard: ইনডেক্সের ভাগ — প্রতিটি shard একটি স্বাধীন Lucene ইনডেক্স।

Mapping: প্রতিটি ক্ষেত্রের ধরন (text, keyword, date) নির্ধারণ — অনুসন্ধানের আচরণ নিয়ন্ত্রণ করে।

ILM / ISM: Index Lifecycle/State Management — পুরোনো ইনডেক্স স্বয়ংক্রিয়ভাবে সরানো বা মোছার নীতি।

Query DSL: Elasticsearch/OpenSearch-এর JSON-ভিত্তিক query ভাষা।

Ingest Pipeline: লগ Elasticsearch-এ ঢোকার সময় ক্ষেত্র বের করা ও সমৃদ্ধ করার ধাপ।

১৬.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Inverted Index কী, এবং এটি কীভাবে দ্রুত অনুসন্ধান সম্ভব করে?
2. Document, Index, Shard ও Replica-র সম্পর্ক ব্যাখ্যা করুন।
3. `text` ও `keyword` ক্ষেত্রের পার্থক্য কী? `status`-কে কোনটি করা উচিত?
4. Logstash pipeline-এর তিন ধাপ কী কী? প্রতিটির একটি উদাহরণ দিন।
5. Dynamic mapping কেন বিপজ্জনক? Index template কীভাবে সাহায্য করে?

6. Elastic Agent ও Filebeat-এর মধ্যে বর্তমান সুপারিশ কোনটি, এবং কেন?
7. OpenSearch কীভাবে Elasticsearch-এর সঙ্গে সম্পর্কিত?
8. Cluster status green, yellow ও red-এর অর্থ কী?
9. Loki বনাম Elasticsearch — তিনটি দিক থেকে তুলনা করুন।
10. Heap ৩২ GB-এর বেশি দেওয়া কেন ক্ষতিকর?

১৬.১৯ ব্যবহারিক অনুশীলন

1. Docker Compose দিয়ে Elasticsearch + Kibana চালু করুন ও cluster health যাচাই করুন।
2. Explicit mapping-সহ একটি index template তৈরি করুন।
3. Logstash pipeline দিয়ে Filebeat থেকে লগ পাঠিয়ে Kibana Discover-এ দেখুন।
4. Query DSL দিয়ে গত ১ ঘণ্টার ERROR লগ খুঁজুন ও সার্ভিস অনুযায়ী aggregation করুন।
5. OpenSearch + Dashboards চালু করে একই অনুসন্ধান করুন; পার্থক্য লিখুন।
6. ILM/ISM নীতি লিখুন: ৩০ দিন পর ইনডেক্স মুছে ফেলা।
7. এক পাতার সিদ্ধান্ত নথি লিখুন: TechNova-র জন্য কখন Loki, কখন OpenSearch — খরচ, query শক্তি ও পরিচালনার ভারসহ।

PART IV • DEVOPS ENGINEERING MASTERY

Distributed Tracing and Advanced Observability

Distributed Tracing, OpenTelemetry, Jaeger ও Tempo, মাইক্রোসার্ভিস Observability, APM ও Profiling।

অধ্যায় ১৭ • PART IV — Distributed Tracing and Advanced Observability

Chapter 17 — Distributed Tracing Fundamentals and Trace Context

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: **Trace** (একটি অনুরোধের সম্পূর্ণ যাত্রার রেকর্ড) ও **Span** (সেই যাত্রার একটি ধাপ) ব্যাখ্যা করতে; **Trace Context** কীভাবে এক সার্ভিস থেকে আরেক সার্ভিসে যায় তা বর্ণনা করতে; **W3C Trace Context** হেডার `traceparent` ও `tracestate` পড়তে ও লিখতে; **Sampling** (কোন Trace সংরক্ষণ করবেন) নীতি বেছে নিতে; এবং TechNova-র MERN অ্যাপে একটি ধীর API অনুরোধের দেরি কোন সার্ভিসে হচ্ছে তা Trace দিয়ে খুঁজে বের করতে।

- Trace ID, Span ID, Parent Span ID-র সম্পর্ক আঁকতে পারবেন।
- Head Sampling ও Tail Sampling-এর পার্থক্য ও ট্রেড-অফ বলতে পারবেন।
- Span Attribute, Event ও Baggage-র সঠিক ব্যবহার জানবেন।
- Service Map পড়ে নির্ভরতা (dependency) বিশ্লেষণ করতে পারবেন।

১৭.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একটি Monolith-এ একটি অনুরোধ একটিই প্রসেসে চলে; Log পড়লেই বোঝা যায় কোথায় সময় গেল। Microservices-এ TechNova-র একটি অনুরোধ যায় React Frontend → API Gateway → Auth Service → Order Service → MongoDB → Payment Service — ছয়টি জায়গায় ছড়িয়ে থাকা Log একত্র না করলে আপনি জানতেই পারবেন না ২.৪ সেকেন্ড দেরির ২.১ সেকেন্ড কোথায় গেল। **Distributed Tracing** সেই ছড়িয়ে থাকা ধাপগুলোকে একটি Trace ID দিয়ে জোড়া দেয়। Metrics বলে কী খারাপ হয়েছে (p95 latency বেড়েছে), Log বলে কী ঘটেছে (error লেখা আছে), Trace বলে কোথায় ও কেন — কোন Span-এ সময় আটকে আছে। এ ছাড়া প্রোডাকশনে ধীর অনুরোধের তদন্ত অন্ধকারে হাতড়ানোর মতো হয়।

> **REAL WORLD:** TechNova-র /api/orders অনুরোধ ৮০০ মিলিসেকেন্ড থেকে ২.৪ সেকেন্ডে উঠল। Metrics শুধু জানাল latency বেড়েছে। Trace খুলে দেখা গেল Payment Service-এর Span ১.৯ সেকেন্ড — তার ভেতরে ডেটাবেস Span মাত্র ৪০ মিলিসেকেন্ড, বাকিটা বাইরের Payment Gateway-র অপেক্ষা। Log দিয়ে এই বিভাজন ঘণ্টায় মিলত না, Trace-এ মিনিটে মিলল।

১৭.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায়ের জন্য আপনার জানা থাকা উচিত: HTTP অনুরোধ-উত্তর ও হেডার কী; Microservices আর্কিটেকচারের ধারণা; Log ও Metrics-এর মৌলিক ধারণা (Volume 10-এর আগের অধ্যায়গুলো); Node.js/Express-এ একটি API কীভাবে কাজ করে; এবং JSON পড়তে পারা। W3C, Hexadecimal বা Sampling-এর গাণিতিক জ্ঞান লাগবে না — এখানেই শেখানো হবে।

১৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

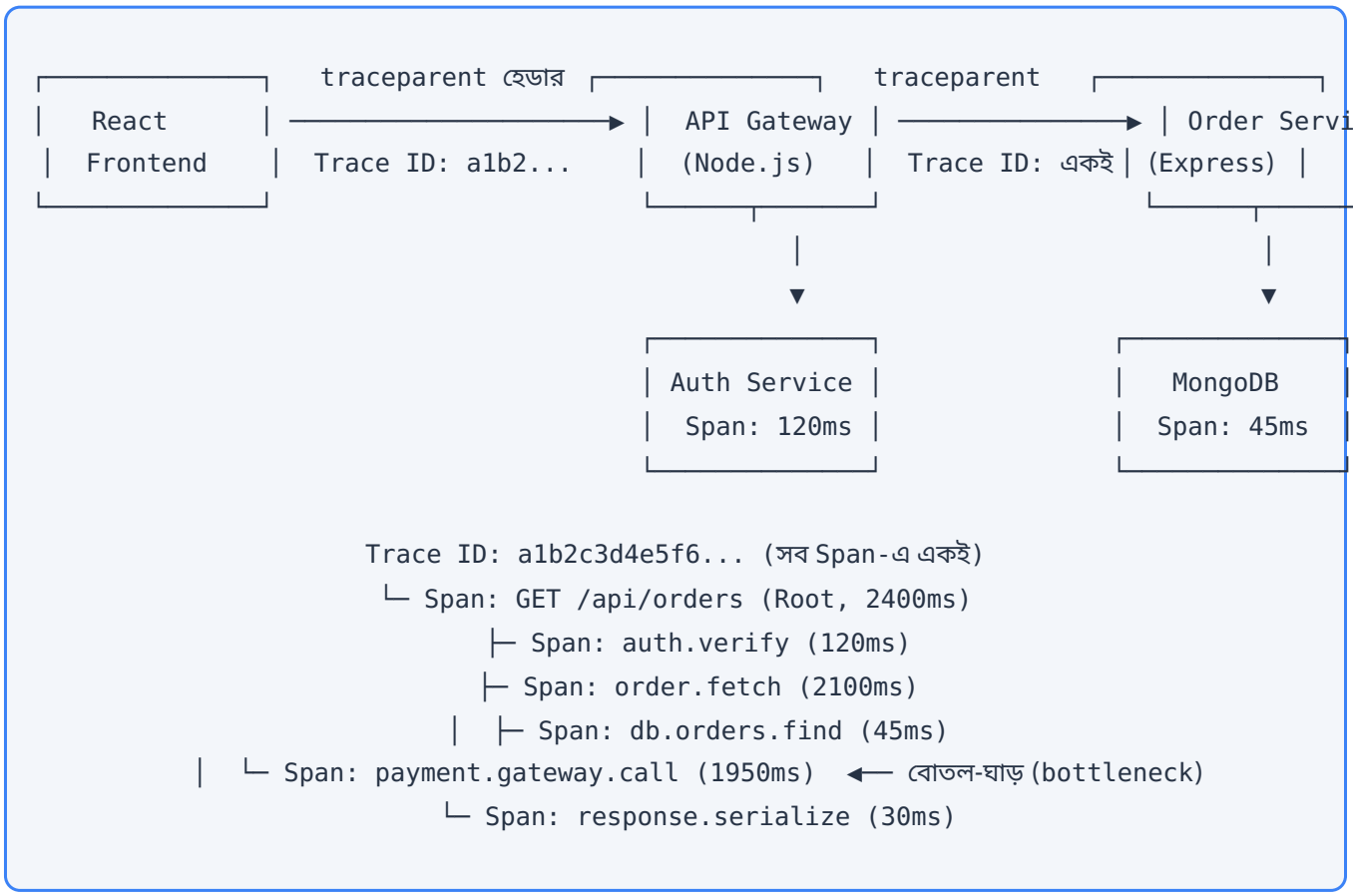
প্রথম নীতি থেকে ভাবুন: একটি অনুরোধ আসলে কী? এটি একটি কাজের শৃঙ্খল — প্রতিটি কাজের শুরু ও শেষ সময় আছে। যদি প্রতিটি কাজকে একটি নাম, শুরু-সময়, শেষ-সময় ও একটি অনন্য পরিচয় (ID) দেন, এবং প্রতিটি কাজ জানে তার আগের কাজটি কে, তবে পুরো শৃঙ্খলটি পুনর্গঠন করা যায়। এটাই Tracing-এর মূল ধারণা।

Span হলো সেই একটি কাজের রেকর্ড: যেমন "MongoDB-তে order খোঁজা" — এর নাম, শুরু ও শেষ, সময়কাল (duration), কোন সার্ভিসে চলেছে, এবং কী কী বৈশিষ্ট্য (attribute) ছিল, যেমন `db.collection = "orders"`। **Trace** হলো একটি অনুরোধের সব Span-এর গাছ (tree): Root Span (প্রথম Span, যেমন HTTP অনুরোধ গ্রহণ) থেকে শাখা-প্রশাখা। **Trace ID** হলো পুরো গাছটির একটিই পরিচয় — সব Span একই Trace ID বহন করে। **Span ID** প্রতিটি Span-এর নিজস্ব পরিচয়, আর **Parent Span ID** বলে দেয় এই Span-টি কার সন্তান। এই তিনটি ID দিয়েই একটি ছড়িয়ে থাকা অনুরোধকে একত্র করা যায়।

কিন্তু এক সার্ভিস কীভাবে জানে আগের সার্ভিসের Trace ID কী? উত্তর: **Context Propagation** — প্রসঙ্গ (context) পাশ করা। HTTP অনুরোধের হেডারে Trace ID লিখে পাঠানো হয়, পরের সার্ভিস সেটি পড়ে নিজের Span-এ একই Trace ID ব্যবহার করে। এটাই Trace Context। কোনো জাদু নেই — এটি হেডারে ID বহন করার একটি চুক্তি (standard)।

> **CONCEPT:** Trace হলো অনুরোধের গল্প, Span হলো গল্পের একেকটি বাক্য, আর Trace Context হলো সেই গল্পের শিরোনাম যা প্রতিটি সার্ভিস পরের সার্ভিসের হাতে তুলে দেয়। শিরোনাম হারিয়ে গেলে গল্পটি ছিন্নভিন্ন Log-এর স্তূপ হয়ে যায়।

১৭.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



উপরের চিত্রে Root Span ২৪০০ মিলিসেকেন্ড, কিন্তু তার সন্তানদের যোগফল দেখলেই বোঝা যায় সময় কোথায় গেল। Jaeger বা Tempo-তে এই গাছটি Waterfall আকারে দেখা যায় — প্রতিটি Span একটি বার, যার দৈর্ঘ্য তার সময়কাল।

১৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Trace ID, Span ID ও Parent Span ID

Trace ID সাধারণত ১২৮-বিট (৩২টি hexadecimal অক্ষর), Span ID ৬৪-বিট (১৬টি hexadecimal অক্ষর)। এগুলো এলোমেলো (random) তৈরি হয় — দুটি অনুরোধের একই ID হওয়ার সম্ভাবনা কার্যত শূন্য। Root Span-এর Parent Span ID থাকে না; বাকি সব Span-এর Parent থাকে।

W3C Trace Context

W3C Trace Context একটি আন্তর্জাতিক মান (standard)। `traceparent` হেডারের গঠন: `version-trace-id-parent-id-trace-flags`, যেমন `00-a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6-1234567890abcdef-01`। এখানে `00` সংস্করণ, মাঝের ৩২ অক্ষর Trace ID, পরের ১৬ অক্ষর Parent Span ID, শেষে `01` মানে Sampled (এই Trace সংরক্ষণ করো), `00` মানে Sampled নয়। `tracestate` হেডারে ভেন্ডর-নির্দিষ্ট তথ্য থাকে। এই মান থাকার ফলে Jaeger, Tempo, Datadog — যেকোনো টুল একে অপরের Trace বুঝতে পারে।

Span-এর ভেতরে কী থাকে

একটি Span-এ থাকে: নাম (Operation Name), শুরু ও শেষ Timestamp, Duration, Trace ID, Span ID, Parent Span ID, **Attributes** (key-value তথ্য, যেমন `http.method=GET`, `http.status_code=200`), **Events** (সময়-চিহ্নিত ঘটনা, যেমন "cache miss ঘটল ১২:০৩:০১-এ"), **Status** (OK / Error ও বার্তা), এবং **Links** (অন্য Trace-এর সঙ্গে সম্পর্ক)। Attributes-এ কখনো Password, Token বা ব্যক্তিগত তথ্য লিখবেন না — Trace সংরক্ষিত হয়, Log-এর মতোই সংবেদনশীল।

Sampling — সব Trace রাখা যায় না

TechNova প্রতিদিন ১ কোটি অনুরোধ পেলে সব Trace সংরক্ষণ করা অসম্ভব — খরচ ও স্টোরেজ আকাশ ছোঁবে। **Sampling** মানে বেছে বেছে Trace রাখা। **Head Sampling**: অনুরোধ শুরুতেই সিদ্ধান্ত — যেমন ১০%-এ এলোমেলোভাবে রাখা হবে; সিদ্ধান্তটি `traceparent`-এর flag-এ লেখা থাকে, তাই সব সার্ভিস একই সিদ্ধান্ত মানে। সুবিধা: সহজ ও সস্তা; অসুবিধা: বিরল Error Trace হারিয়ে যেতে পারে (১০%-এ না পড়লে)। **Tail Sampling**: অনুরোধ শেষ হওয়ার পর সিদ্ধান্ত — Error হয়েছে, বা ধীর হয়েছে, এমন Trace রাখা হয়। সুবিধা: গুরুত্বপূর্ণ Trace ধরা পড়ে; অসুবিধা: সব Span সাময়িকভাবে ধরে রাখতে হয়, Collector-এ বেশি মেমরি ও জটিলতা লাগে। প্রোডাকশনে সাধারণত দুটির মিশ্রণ: Head-এ ১০% সাধারণ, Tail-এ সব Error ও ধীর Trace।

Baggage

Baggage হলো Trace Context-এর সঙ্গে বহন করা অতিরিক্ত key-value তথ্য, যেমন `user-tier=premium` বা `tenant-id=technova-acme` — যা সব সার্ভিস দেখতে পায়। এটি ব্যবসায়িক প্রসঙ্গ বহনের জন্য দরকারি, কিন্তু এতে সংবেদনশীল তথ্য রাখবেন না, কারণ এটি হেডারে ঘোরে এবং Log-এ উঠতে পারে।

Service Map

সব Trace একত্র করলে স্বয়ংক্রিয়ভাবে একটি **Service Map** তৈরি হয় — কোন সার্ভিস কাকে ডাকে, কতবার, কত দেরিতে। এটি আর্কিটেকচার ডকুমেন্টেশনের চেয়ে সত্য — কারণ এটি আসল ট্রাফিক থেকে তৈরি। নতুন ইঞ্জিনিয়ার প্রথমে Service Map দেখলেই বুঝে যান সিস্টেমের আকৃতি।

ধারণা	কী করে	মনে রাখার কথা
Trace	এক অনুরোধের সব Span-এর গাছ	একটিই Trace ID
Span	একটি কাজের সময়-রেকর্ড	নাম, শুরু, শেষ, Attribute
Context Propagation	Trace ID পরের সার্ভিসে পাঠানো	traceparent হেডার
Head Sampling	শুরুতেই রাখা/না-রাখার সিদ্ধান্ত	সহজ, Error হারাতে পারে
Tail Sampling	শেষে, ফল দেখে সিদ্ধান্ত	Error ধরে, ব্যয়বহুল
Baggage	প্রসঙ্গ তথ্য বহন	সংবেদনশীল তথ্য নয়

১৭.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

Node.js-এ হাতে Span তৈরি করার ন্যূনতম উদাহরণ — প্রতিটি লাইনের অর্থ বুঝুন:

```
// (নমুনা কোড — চিত্রণমূলক)
const { trace } = require('@opentelemetry/api');          // 1. OTel API থেকে tracer নেওয়ার হুক
const tracer = trace.getTracer('technova-order-service'); // 2. এই সার্ভিসের নামে tracer
const span = tracer.startSpan('order.fetch');             // 3. নতুন Span শুরু; সময় এখন থেকে গোনা
হয়
span.setAttribute('order.id', 'ORD-1042');                // 4. Attribute যোগ — খোঁজার জন্য দরকারি
span.addEvent('db.query.started');                        // 5. Event — একটি মুহূর্ত চিহ্নিত করা
try { /* ... ডেটাবেস কল ... */ span.setStatus({ code: 1 }); } // 6. সফল হলে Status OK (code 1)
catch (err) { span.recordException(err); span.setStatus({ code: 2, message:
err.message }); } // 7. ব্যর্থতা Span-এ লেখা
finally { span.end(); }                                   // 8. Span শেষ — ভুলে গেলে Span ঝুলে থাকে
```

লাইন ৮ সবচেয়ে গুরুত্বপূর্ণ: `span.end()` না ডাকলে Span কখনো Export হয় না, Trace অসম্পূর্ণ দেখায়। `try/finally` ব্যবহার করুন যাতে Error হলেও Span শেষ হয়।

```
# traceparent হেডার নিজে পরীক্ষা করা (নমুনা আউটপুট — চিত্রণমূলক)
curl -s -D - https://api.technova.example/orders -H "traceparent: 00-
a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6-1234567890abcdef-01" -o /dev/null
# -D - : হেডার দেখাও; -H : traceparent হেডার পাঠাও; 01 মানে এই Trace সংরক্ষণ করো
```

১৭.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র Order Service-এ Express মিডলওয়্যার দিয়ে প্রতিটি অনুরোধে স্বয়ংক্রিয় Span (পূর্ণাঙ্গ নয়, মূল কাঠামো):

```
// tracing.js — TechNova Order Service (Node.js 20)
const { NodeSDK } = require('@opentelemetry/sdk-node');
const { getNodeAutoInstrumentations } = require('@opentelemetry/auto-instrumentations-node');
```

```
const { OTLPMetricExporter } = require('@opentelemetry/exporter-metrics-otlp-http');
const sdk = new NodeSDK({
  serviceName: 'technova-order-service',   // Service Map-এ এই নামে দেখাবে
  traceExporter: new (require('@opentelemetry/exporter-trace-otlp-http').OTLPTraceExporter)({
    url: 'http://203.0.113.10:4318/v1/traces', // Collector-এর OTLP HTTP ঠিকানা
  }),
  instrumentations: [getNodeAutoInstrumentations()], // Express, HTTP, MongoDB স্বয়ংক্রিয়
});
sdk.start();
```

১৭.৯ বাস্তব প্রোডাকশন পরিস্থিতি

পরিস্থিতি: শুক্রবার সন্ধ্যায় TechNova-র চেকআউট ধীর। Support টিম জানায় কিছু গ্রাহকের Payment আটকে যাচ্ছে। Trace-এ দেখা গেল: `payment.gateway.call` Span-এ ৯৫% সময়; কিন্তু শুধু `tenant-id` Baggage-যুক্ত Trace-গুলোতেই দেরি — একটি নির্দিষ্ট Tenant-এর Gateway অ্যাকাউন্টে সমস্যা। Metrics এই বিভাজন দিতে পারত না। সিদ্ধান্ত: ওই Tenant-এর ট্রাফিক বিকল্প Gateway-তে সরানো হলো, বাকিদের কোনো ব্যাঘাত ঘটল না। এই ধরনের সিদ্ধান্তই Tracing-এর আসল মূল্য — শুধু "ধীর" জানা নয়, কার জন্য ধীর জানা।

> **WARNING:** Trace-এ কখনো Authorization হেডার, Password বা Token Attribute হিসেবে লিখবেন না। Trace ডেটা অনেকেই দেখেন — Developer, SRE, Vendor — এবং এটি দীর্ঘদিন সংরক্ষিত থাকে। একবার লেখা Secret ঘুরিয়ে (rotate) ফেলতে হয়।

১৭.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. একটি সার্ভিস বেছে নিন — TechNova-র Order Service দিয়ে শুরু করুন, সব সার্ভিস একসঙ্গে নয়।
2. OpenTelemetry SDK বসান ও Collector-এর দিকে OTLP Export চালু করুন (অধ্যায় ১৮ দেখুন)।
3. Jaeger বা Tempo-তে একটি Trace খুলে Root Span ও সন্তানদের গাছ মিলিয়ে দেখুন।
4. একটি ধীর অনুরোধ খুঁজে বের করুন ও সবচেয়ে দীর্ঘ Span চিহ্নিত করুন।
5. সেই Span-এ `db.statement` বা `http.url` Attribute আছে কি না দেখুন; না থাকলে যোগ করুন।
6. Head Sampling ১০% ও Tail Sampling-এ Error Trace রাখার নীতি চালু করুন।
7. দ্বিতীয় সার্ভিস (Payment) যোগ করুন ও দেখুন Trace ID দুই সার্ভিস জুড়ে একই আছে কি না — না থাকলে Context Propagation ভাঙা।

১৭.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "Tracing বসালেই Root Cause আপনাআপনি বেরিয়ে আসবে।" — না। Tracing দেখায় কোথায় সময় গেল; কেন গেল তা বোঝার জন্য Attribute, Log ও ডোমেইন জ্ঞান লাগে।
- **ভুল:** প্রতিটি ফাংশনে Span তৈরি করা — Trace এত বড় হয় যে পড়া যায় না; Span তৈরি করুন সার্ভিস সীমানা, ডেটাবেস কল ও বাইরের API কলে।
- **ভুল:** Span-এ সংবেদনশীল তথ্য লেখা — উপরে সতর্কতা দেখুন।
- **ভুল:** Sampling ১০০% রেখে দেওয়া — প্রোডাকশনে খরচ ও Collector চাপে সিস্টেম ধীর হয়।

- **ভুল:** Async কাজে Context হারানো — Node.js-এ `async/await` ছাড়া Callback-এ Context হারিয়ে Trace ভেঙে যায়; OTel-এর Context Manager এটি সামলায়, কিন্তু নিজে হাতে Context না বুঝে Thread/Callback পাল্টাবেন না।

১৭.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Trace ডেটায় URL, Query Parameter, Header ও Attribute থাকতে পারে — এর ভেতরে ব্যক্তিগত তথ্য (ইমেইল, ফোন, ঠিকানা) বা Secret ঢুকে যেতে পারে। Collector-এ Processor দিয়ে সংবেদনশীল Attribute মুছে ফেলুন বা লাল (redact) করুন; Trace সংরক্ষণে Retention সীমিত রাখুন (যেমন ৭–১৪ দিন); Jaeger/Tempo-তে প্রবেশাধিকার (access control) দিন — সব Developer-কে সব Tenant-এর Trace দেখতে দেবেন না। GDPR-ধরনের নিয়মে Trace-ও ব্যক্তিগত ডেটা হতে পারে, তাই নীতিমালা লিখে রাখুন।

১৭.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Instrumentation-এর খরচ সাধারণত ১–৫% CPU ও সামান্য মেমরি — কিন্তু খারাপ Instrumentation (প্রতিটি লুপে Span) ২০% পর্যন্ত যেতে পারে। Span সংখ্যা সীমিত রাখুন: প্রতি অনুরোধে ১০–৫০টি Span স্বাভাবিক, ১০০০টি নয়। Sampling দিয়ে Export-এর পরিমাণ নিয়ন্ত্রণ করুন; Collector-এ Batch Processor দিয়ে ছোট ছোট Export একত্র করুন। Storage খরচ Trace সংখ্যা ও Retention-এর গুণফল — Retention বাড়ানোর আগে খরচ হিসাব করুন।

১৭.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Trace-এ একটি সার্ভিসের Span নেই	সেই সার্ভিসে SDK বসানো হয়নি, বা Context Propagation ভাঙা	traceparent হেডার পাঠানো হচ্ছে কি না curl দিয়ে পরীক্ষা করুন
Span আছে কিন্তু Parent ভুল	Async Context হারিয়েছে	OTel Context Manager ও auto-instrumentation সংস্করণ পরীক্ষা করুন
Trace অসম্পূর্ণ (কিছু Span অনুপস্থিত)	Sampling-এ বাদ পড়েছে, বা Export ব্যর্থ	Collector Log-এ Export Error দেখুন; Sampled flag পরীক্ষা করুন
Jaeger-এ Trace খুঁজে পাওয়া যায় না	Collector থেকে Backend-এ Export হয়নি	Collector ও Jaeger-এর সংযোগ ও Storage পরীক্ষা করুন
সব Trace-ই ধীর দেখায়	Clock Skew — সার্ভিসগুলোর ঘড়ি মেলেনি	সব হোস্টে NTP চালু করুন

TROUBLESHOOTING: Trace ভাঙা মনে হলে প্রথমে Trace ID দিয়ে খুঁজুন — একই Trace ID-র Span সব সার্ভিসে আছে কি না। যে সার্ভিসে নেই, সেখানেই Propagation ভেঙেছে; সেখান থেকে তদন্ত শুরু করুন, Jaeger-এর দোষ দিয়ে নয়।

১৭.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: W3C Trace Context মানুন; সার্ভিস সীমানায় Span তৈরি করুন; অর্থপূর্ণ Attribute যোগ করুন (`http.route`, `db.system`); Error Trace Tail Sampling-এ রাখুন; Service Map নিয়মিত দেখুন। **অ্যান্টি-প্যাটার্ন:**

সবকিছুতে Span; Secret Attribute-এ লেখা; ১০০% Sampling প্রোডাকশনে; Trace ছাড়া শুধু Log দিয়ে Microservices তদন্ত করার চেষ্টা; Trace ডেটা কাউকে না দেখিয়ে শুধু নিজে দেখা — Trace টিমের সম্পদ।

১৭.১৬ অধ্যায়ের সারাংশ

Distributed Tracing একটি অনুরোধের যাত্রাকে Span-এর গাছ হিসেবে ধরে, Trace ID দিয়ে সব সার্ভিস জোড়া দেয়, W3C Trace Context হেডারে সেই ID বহন করে, Sampling দিয়ে খরচ নিয়ন্ত্রণ করে, আর Service Map দিয়ে সিস্টেমের আসল চিত্র দেখায়। এটি Root Cause-এর জাদুকাঠি নয় — এটি একটি মানচিত্র, যা দিয়ে আপনি দ্রুত সঠিক জায়গায় পৌঁছান।

১৭.১৭ মূল প্রযুক্তিগত পরিভাষা

Trace: একটি অনুরোধের সব Span-এর সমষ্টি; পুরো যাত্রার রেকর্ড।

Span: একটি কাজের সময়-রেকর্ড — নাম, শুরু, শেষ, Attribute সহ।

Trace ID: পুরো Trace-এর অনন্য পরিচয়, সব Span-এ একই থাকে।

Span ID: প্রতিটি Span-এর নিজস্ব অনন্য পরিচয়।

Trace Context: Trace ID ও Span ID-র মতো প্রসঙ্গ যা সার্ভিসগুলোর মধ্যে পাঠানো হয়।

W3C Trace Context: Trace Context বহনের আন্তর্জাতিক মান; traceparent ও tracestate হেডার।

Sampling: কোন Trace সংরক্ষণ করা হবে তা বেছে নেওয়ার নীতি।

Head Sampling: অনুরোধের শুরুতে নেওয়া Sampling সিদ্ধান্ত।

Tail Sampling: অনুরোধ শেষে ফল দেখে নেওয়া Sampling সিদ্ধান্ত।

Baggage: Trace Context-এর সঙ্গে বহন করা অতিরিক্ত key-value তথ্য।

Service Map: Trace থেকে তৈরি সার্ভিসগুলোর নির্ভরতার চিত্র।

Span Attribute: Span-এর সঙ্গে যুক্ত key-value বৈশিষ্ট্য, যেমন http.status_code।

১৭.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Trace ও Span-এর পার্থক্য কী? একটি অনুরোধের উদাহরণ দিয়ে বোঝান।
2. দুটি সার্ভিসের Span একই Trace-এর অংশ — তা কীভাবে বোঝা যায়?
3. traceparent হেডারের চারটি অংশ কী কী? `01` flag-এর অর্থ কী?
4. Head Sampling-এ একটি Error Trace হারিয়ে যেতে পারে কেন? Tail Sampling কীভাবে তা সামলায়?
5. Root Span-এর Parent Span ID থাকে না কেন?
6. Span Attribute-এ Password লেখা কেন বিপজ্জনক?
7. Service Map কীভাবে তৈরি হয়? এটি আর্কিটেকচার ডকুমেন্ট থেকে কেন বেশি নির্ভরযোগ্য?
8. একটি Trace-এ Order Service-এর Span নেই — প্রথমে কী পরীক্ষা করবেন?

১৭.১৯ ব্যবহারিক অনুশীলন

1. TechNova-র একটি Express API-তে OpenTelemetry SDK বসান (অধ্যায় ১৮-এর কোড ব্যবহার করুন)।
2. Jaeger বা Tempo-তে একটি Trace খুলে Root Span ও অন্তত ৩টি Child Span চিহ্নিত করুন।
3. curl দিয়ে traceparent হেডার পাঠিয়ে দেখুন Trace ID দুই সার্ভিসে একই থাকে কি না।
4. একটি ইচ্ছাকৃত ধীর ডেটাবেস কল (যেমন ২ সেকেন্ড sleep) যোগ করে Trace-এ সবচেয়ে দীর্ঘ Span খুঁজুন।
5. Head Sampling ১০% চালু করে ২০টি অনুরোধ পাঠান; কয়টি Trace সংরক্ষিত হলো তা লিখে রাখুন।
6. একটি Span-এ ভুল করে সংবেদনশীল Attribute যোগ করলে কী ঝুঁকি হয় — এক প্যারায় লিখুন।

অধ্যায় ১৮ • PART IV — Distributed Tracing and Advanced Observability

Chapter 18 — OpenTelemetry Architecture, SDKs, and Collector

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: **OpenTelemetry** (সংক্ষেপে OTel) কী ও কেন এটি শিল্প-মান (industry standard) তা ব্যাখ্যা করতে; API, SDK ও Collector-এর ভূমিকা আলাদা করতে; Node.js-এ **Manual** ও **Auto-Instrumentation** ব্যবহার করতে; Collector-এর **Receiver, Processor, Exporter ও Pipeline** দিয়ে একটি সম্পূর্ণ Telemetry প্রবাহ বানাতে; এবং **OTLP** প্রোটোকল দিয়ে Trace, Metric ও Log পাঠাতে।

১৮.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

আগে প্রতিটি পর্যবেক্ষণ টুলের নিজস্ব Agent ও লাইব্রেরি ছিল — Jaeger-এর জন্য এক লাইব্রেরি, Prometheus-এর জন্য আরেকটি, Vendor বদলালে কোড বদলাতে হতো। OpenTelemetry এই Vendor Lock-in ভাঙে: একবার Instrument করলে Telemetry যেকোনো Backend-এ (Jaeger, Tempo, Prometheus, Vendor) পাঠানো যায়, শুধু Collector কনফিগারেশন বদলে। TechNova-র জন্য এর অর্থ — আজ Tempo ব্যবহার করলেও কাল অন্য Backend-এ যাওয়া যায় কোড না বদলে। এটি DevOps-এর সবচেয়ে গুরুত্বপূর্ণ নিরপেক্ষ স্তর (vendor-neutral layer)।

১৮.৩ পূর্বশর্ত জ্ঞান

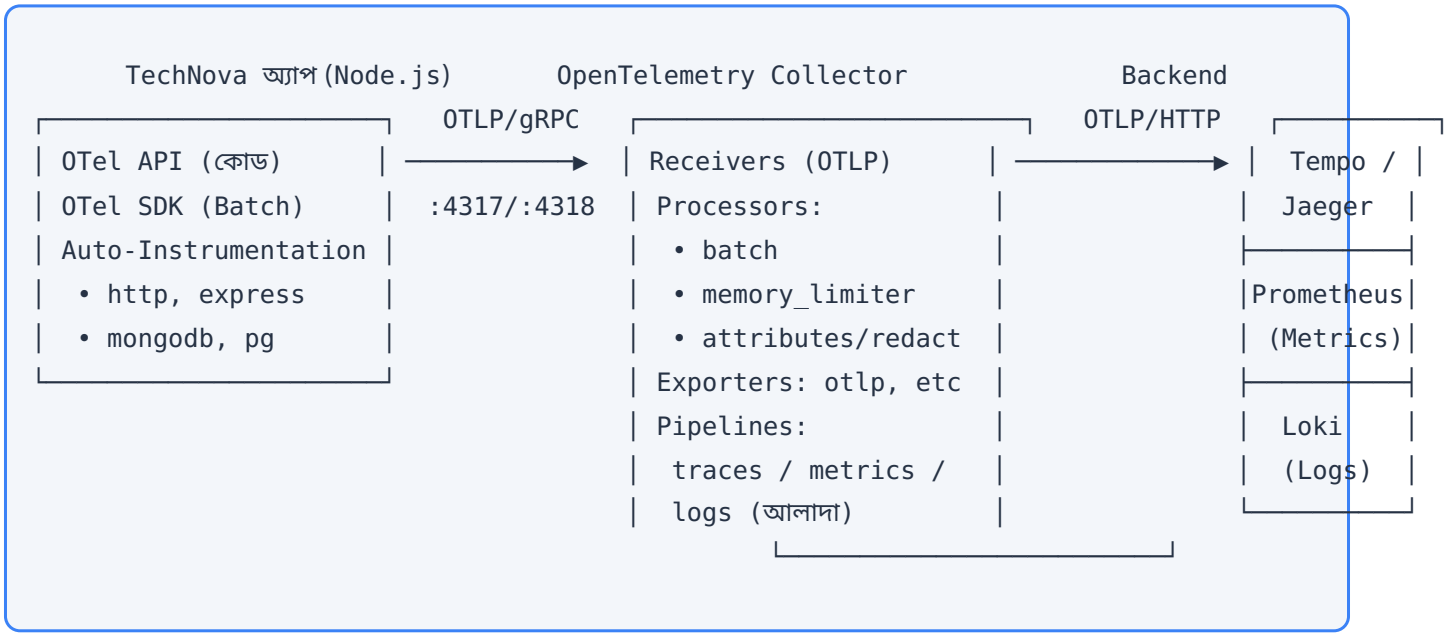
অধ্যায় ১৭-এর Trace/Span ধারণা, Node.js ও npm, YAML ফাইলের গঠন, HTTP/gRPC-এর মৌলিক ধারণা, এবং Docker-এ একটি সার্ভিস চালানোর অভিজ্ঞতা থাকলে ভালো হয়। gRPC না জানলেও চলবে — এখানে প্রয়োজনমতো ব্যাখ্যা দেওয়া হয়েছে।

১৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Telemetry তৈরি করা (আপনার কোডে কী ঘটছে তা রেকর্ড করা) আর Telemetry পাঠানো/সংরক্ষণ করা — দুটি আলাদা কাজ। প্রথমটি আপনার অ্যাপের ভেতরে হয়, দ্বিতীয়টি বাইরে। OTel এই দুটি কাজকে আলাদা করে: **API** হলো কোড লেখার চুক্তি (কীভাবে Span তৈরি করবেন), **SDK** হলো সেই চুক্তির বাস্তবায়ন (Span কীভাবে তৈরি, Batch ও Export কীভাবে হবে), আর **Collector** হলো অ্যাপের বাইরে বসা একটি মধ্যবর্তী সার্ভিস — যা Telemetry গ্রহণ করে, প্রক্রিয়া করে, ও Backend-এ পাঠায়। অ্যাপ সরাসরি Backend-এর সঙ্গে কথা বলে না; Collector মাঝখানে থাকায় Backend বদলানো, Sampling করা বা Secret মুছে ফেলা — সবই অ্যাপ না ছুঁয়ে করা যায়।

> **CONCEPT:** API হলো ভাষা, SDK হলো সেই ভাষায় কথা বলার যন্ত্র, Collector হলো ডাকঘর — চিঠি (Telemetry) সংগ্রহ করে, সাজিয়ে, সঠিক ঠিকানায় (Backend) পৌঁছে দেয়। চিঠি লেখককে ডাকঘরের ঠিকানা বদল নিয়ে ভাবতে হয় না।

১৮.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



একটি Pipeline হলো Receiver → Processor → Exporter-এর শৃঙ্খল। Traces, Metrics ও Logs-এর জন্য আলাদা Pipeline থাকে, কিন্তু একই Collector-এ চলে।

১৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

API বনাম SDK

API (`@opentelemetry/api`) হালকা — এতে শুধু Interface: `trace.getTracer()`, `span.setAttribute()`। আপনার ব্যবসায়িক কোড শুধু API ব্যবহার করে, তাই SDK না থাকলেও কোড চলে (No-op — কিছুই রেকর্ড হয় না)। **SDK** (`@opentelemetry/sdk-node`) ভারী — এতে Span তৈরি, Sampling, Batching, Export-এর বাস্তবায়ন। এই বিভাজনের ফলে লাইব্রেরি লেখকরা API দিয়ে Instrument করতে পারেন, আর অ্যাপ মালিক SDK দিয়ে নিয়ন্ত্রণ করেন।

Auto-Instrumentation বনাম Manual

Auto-Instrumentation আপনার কোড না বদলে জনপ্রিয় লাইব্রেরি (Express, HTTP, MongoDB ড্রাইভার, pg) থেকে স্বয়ংক্রিয়ভাবে Span তৈরি করে। সুবিধা: দ্রুত শুরু, কম কোড। সীমা: এটি শুধু লাইব্রেরি সীমানা দেখে — আপনার ব্যবসায়িক ধাপ (যেমন "discount গণনা") দেখে না। **Manual Instrumentation** নিজে Span তৈরি করা — ব্যবসায়িক ধাপ ও গুরুত্বপূর্ণ সিদ্ধান্ত চিহ্নিত করতে। ভালো প্রোডাকশন সেটআপে দুটিই থাকে: Auto দিয়ে ভিত্তি, Manual দিয়ে অর্থ।

Collector-এর উপাদান

Receiver: Telemetry গ্রহণ করে — OTLP Receiver (gRPC :4317, HTTP :4318) সবচেয়ে সাধারণ; এছাড়া Prometheus Receiver, Jaeger Receiver, Host Metrics Receiver আছে। **Processor:** মাঝপথে প্রক্রিয়া — `batch` (ছোট Export একত্র করা), `memory_limiter` (মেমরি বেশি হলে থামানো), `attributes` (Attribute যোগ/মোছা), `tail_sampling` (শেষে Sampling)। **Exporter:** Backend-এ পাঠায় — OTLP Exporter, Prometheus Exporter, Logging Exporter (ডিবাগের জন্য)। **Pipeline:** এই তিনটির শৃঙ্খল; `traces`, `metrics`, `logs` Pipeline আলাদা।

OTLP

OTLP (OpenTelemetry Protocol) হলো Telemetry পাঠানোর মান — gRPC বা HTTP-তে, Protobuf বা JSON-এ। একই প্রোটোকলে Trace, Metric ও Log — তিনটি Signal-ই যায়। পোর্ট মনে রাখুন: gRPC 4317, HTTP 4318।

Context Propagation ও Baggage

OTel-এ Context Propagation হয় **Propagator** দিয়ে — W3C TraceContext Propagator ডিফল্ট। HTTP Client লাইব্রেরি (auto-instrumented) অনুরোধ পাঠানোর সময় স্বয়ংক্রিয়ভাবে `traceparent` হেডার যোগ করে; সার্ভার তা পড়ে। Baggage Propagator `baggage` হেডার বহন করে।

উপাদান	কাজ	উদাহরণ
API	কোডে Span তৈরির চুক্তি	@opentelemetry/api
SDK	Span তৈরি, Batch, Export	@opentelemetry/sdk-node
Receiver	Telemetry গ্রহণ	otlp (4317/4318)
Processor	মাঝপথে প্রক্রিয়া	batch, tail_sampling
Exporter	Backend-এ পাঠানো	otlp, prometheus
Pipeline	Receiver → Processor → Exporter	traces/metrics/logs

১৮.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# OpenTelemetry Collector চালানো (Docker, নমুনা আউটপুট — চিত্রণমূলক)
docker run -d --name otel-collector \
  -p 4317:4317 -p 4318:4318 \   # 1. OTLP gRPC ও HTTP পোর্ট প্রকাশ করা
  -v $(pwd)/otel-collector.yaml:/etc/otelcol.yaml \  # 2. কনফিগারেশন ফাইল মাউন্ট করা
  otel/opentelemetry-collector-contrib:0.96.0 \       # 3. Contrib সংস্করণ — বেশি Receiver/
Processor
  --config=/etc/otelcol.yaml                          # 4. কোন কনফিগারেশন ব্যবহার করবে
docker logs otel-collector  # Collector চালু হয়েছে কি না দেখুন
# (নমুনা আউটপুট — চিত্রণমূলক)
# Everything is ready. Begin running and processing data.
```

```
# Node.js SDK ইনস্টল করা
npm install @opentelemetry/sdk-node @opentelemetry/api \
  @opentelemetry/auto-instrumentations-node \
  @opentelemetry/exporter-trace-otlp-http
# api: কোডের চুক্তি; sdk-node: বাস্তবায়ন; auto-instrumentations: Express/MongoDB স্বয়ংক্রিয়;
# exporter-trace-otlp-http: Collector-এ HTTP-তে Trace পাঠানো
```

১৮.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র সম্পূর্ণ Collector কনফিগারেশন — Traces, Metrics, Logs তিনটি Pipeline:

```
# otel-collector.yaml — TechNova (Ubuntu 24.04, Docker)
receivers:
```

```
  otlp:
    protocols:
      grpc: { endpoint: "0.0.0.0:4317" }
      http: { endpoint: "0.0.0.0:4318" }
  hostmetrics:  # হোস্টের CPU/মেমরি/ডিস্ক মেট্রিক
    collection_interval: 30s
    scrapers: { cpu: {}, memory: {}, disk: {}, filesystem: {} }
processors:
  memory_limiter: { check_interval: 1s, limit_mib: 512 }  # মেমরি সুরক্ষা
  batch: { timeout: 5s, send_batch_size: 512 }             # Export একত্র করা
  attributes/redact:  # সংবেদনশীল Attribute মোছা
    actions: [{ key: "http.request.header.authorization", action: delete }]
exporters:
  otlp/tempo: { endpoint: "tempo.technova.example:4317", tls: { insecure: true } }
  prometheus: { endpoint: "0.0.0.0:8889" }
  logging: { loglevel: warn }  # শুধু সতর্কতা দেখা — ডিবাগে debug করুন
service:
  pipelines:
    traces: { receivers: [otlp], processors: [memory_limiter, attributes/redact, batch],
exporters: [otlp/tempo] }
    metrics: { receivers: [otlp, hostmetrics], processors: [memory_limiter, batch], exporters:
[prometheus] }
    logs: { receivers: [otlp], processors: [memory_limiter, batch], exporters: [logging] }
```

> **WARNING:** উপরের উদাহরণে `tls: { insecure: true }` শুধু অভ্যন্তরীণ ল্যাবের জন্য। প্রোডাকশনে OTLP-তে TLS চালু করুন, নইলে Telemetry (যাতে URL ও Attribute থাকে) নেটওয়ার্কে খোলা ঘুরবে। Endpoint-এ কখনো আসল Secret বা Token লিখবেন না — প্লেসহোল্ডার ব্যবহার করুন, যেমন `CHANGE-ME-TEMPO-ENDPOINT`।

১৮.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova Collector ছাড়া সরাসরি Tempo-তে Trace পাঠাত। Tempo রক্ষণাবেক্ষণের জন্য এক ঘণ্টা বন্ধ হলো — সব অ্যাপের Export ব্যর্থ হতে লাগল, কিছু অ্যাপে Export Retry-তে মেমরি বাড়তে লাগল। Collector বসানোর পর চিত্র বদলে গেল: Collector সাময়িকভাবে Telemetry ধরে রাখল (buffer), Tempo ফিরলে পাঠিয়ে দিল; অ্যাপ কিছুই টের পেল না। পরে Backend Tempo থেকে অন্য সিস্টেমে বদলানো হলো — অ্যাপের কোডে এক লাইনও বদলাতে হয়নি, শুধু Collector-এর Exporter বদলানো হলো। এটাই Collector-এর আসল মূল্য: অ্যাপ ও Backend-এর মাঝে একটি স্থিতিস্থাপক (resilient) স্তর।

১৮.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Docker-এ Collector চালু করুন ও `docker logs`-এ "Everything is ready" দেখুন।
2. Node.js অ্যাপে SDK বসান, `OTEL_EXPORTER_OTLP_ENDPOINT=http://203.0.113.10:4318` পরিবেশ চলক (environment variable) দিন।
3. একটি অনুরোধ পাঠান ও Collector Log-এ (debug স্তরে) Trace গ্রহণ দেখুন।
4. Tempo/Jaeger-এ Trace দেখা গেলে Manual Span যোগ করুন — যেমন "discount.calculate"।
5. `attributes/redact` Processor দিয়ে একটি পরীক্ষামূলক সংবেদনশীল Attribute মুছে দেখুন।
6. Collector বন্ধ করে দেখুন অ্যাপ থামে কি না — থামা উচিত নয়; Export ব্যর্থ হলেও অ্যাপ চলবে।
7. Tail Sampling Processor যোগ করে শুধু Error Trace রাখার পরীক্ষা করুন।

১৮.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** SDK ইনস্টল করলেই Auto-Instrumentation চালু — না, SDK `start()` করতে হয়, এবং `require`-এর আগে Tracing ফাইল লোড করতে হয় (নইলে লাইব্রেরি ইতিমধ্যে লোড হয়ে যায়)।
- **ভুল ধারণা:** "Collector বসালেই সব সমস্যা মিটে যাবে।" — Collector নিজেও একটি সার্ভিস; তার মেমরি, CPU ও ব্যর্থতা পর্যবেক্ষণ করতে হয়।
- **ভুল:** সব Signal এক Pipeline-এ ঢোকানো — Traces, Metrics, Logs আলাদা Pipeline রাখুন।
- **ভুল:** `memory_limiter` ছাড়া Collector চালানো — ট্রাফিক বাড়লে Collector মেমরি খেয়ে মারা যায়।
- **ভুল:** প্রোডাকশনে Logging Exporter `debug` স্তরে রাখা — Log-এর বন্যা বয়ে যায়।

১৮.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

OTLP Endpoint-এ প্রমাণীকরণ (authentication) যোগ করুন — খোলা Collector-এ যে কেউ ভুয়া Telemetry পাঠাতে পারে। TLS দিয়ে পরিবহন সুরক্ষিত করুন। Processor দিয়ে Secret, Token, Password, ব্যক্তিগত তথ্য মুছুন — Exporter-এর আগেই। Collector-এর কনফিগারেশন ফাইলে কখনো আসল Credential লিখবেন না; পরিবেশ চলক বা Secret Manager ব্যবহার করুন।

১৮.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Collector-এর খরচ নির্ভর করে Throughput-এর ওপর। Batch Processor Export-এর সংখ্যা কমিয়ে নেটওয়ার্ক ও Backend চাপ কমায় — `send_batch_size` ও `timeout` ঠিক করুন। এক Collector-এ সব ট্রাফিক না ঢেলে প্রতি হোস্টে একটি Agent Collector (DaemonSet-ধরনের) ও কেন্দ্রে একটি Gateway Collector — এই দুই স্তরের নকশা বড় সিস্টেমে ভালো। Collector-এর নিজের Metrics (Prometheus Endpoint :8888) পর্যবেক্ষণ করুন — Dropped Span সংখ্যা বাড়লে Sampling বা স্কেলিং দরকার।

১৮.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Collector-এ কোনো Telemetry আসে না	অ্যাপ ভুল Endpoint/পোর্টে পাঠাচ্ছে	OTEL_EXPORTER_OTLP_ENDPOINT ও ফায়ারওয়াল পরীক্ষা করুন
Trace আসে, Metric আসে না	Metrics Pipeline-এ Exporter নেই	service.pipelines.metrics যাচাই করুন
Collector বারবার Restart হয়	মেমরি সীমা ছাড়িয়েছে	memory_limiter ও Batch আকার দেখুন; মেমরি বাড়ান
Span-এ Service নাম ভুল	serviceName দেওয়া হয়নি	SDK-তে serviceName বা OTEL_SERVICE_NAME দিন
Auto-Instrumentation কাজ করে না	SDK অ্যাপের পরে লোড হয়েছে	Tracing ফাইল সবার আগে require করুন

১৮.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ব্যবসায়িক কোডে শুধু API ব্যবহার করুন; কনফিগারেশন পরিবেশ চলকে রাখুন; Collector-এ Batch ও Memory Limiter সবসময় দিন; প্রতি Signal-এ আলাদা Pipeline; Collector-এর নিজের স্বাস্থ্য পর্যবেক্ষণ করুন। **অ্যান্টি-প্যাটার্ন:** অ্যাপ থেকে সরাসরি Backend-এ Export; Secret কনফিগারেশনে লেখা; একটিমাত্র Collector-এ সব নির্ভরতা; Sampling ছাড়া ১০০% Export।

১৮.১৬ অধ্যায়ের সারাংশ

OpenTelemetry Telemetry তৈরির নিরপেক্ষ মান: API দিয়ে কোডে Span, SDK দিয়ে Batch ও Export, Collector দিয়ে গ্রহণ-প্রক্রিয়া-প্রেরণ। Receiver, Processor, Exporter ও Pipeline — এই চারটি উপাদান দিয়েই যেকোনো Telemetry প্রবাহ বানানো যায়, আর OTLP দিয়ে তিনটি Signal একই পথে চলে। একবার Instrument করলে Backend বদলানো যায় কোড না ছুঁয়ে।

১৮.১৭ মূল প্রযুক্তিগত পরিভাষা

OpenTelemetry (OTel): Trace, Metric ও Log সংগ্রহের নিরপেক্ষ (vendor-neutral) মান ও টুলসেট।

OTLP: OpenTelemetry Protocol — Telemetry পাঠানোর মান, gRPC (4317) বা HTTP (4318)।

API: কোডে Telemetry লেখার চুক্তি (Interface); হালকা, No-op হতে পারে।

SDK: API-র বাস্তবায়ন — Span তৈরি, Sampling, Batching, Export।

Collector: অ্যাপের বাইরে Telemetry গ্রহণ, প্রক্রিয়া ও প্রেরণকারী সার্ভিস।

Receiver: Collector-এর গ্রহণকারী উপাদান, যেমন OTLP Receiver।

Processor: Collector-এ মাঝপথে Telemetry প্রক্রিয়াকারী, যেমন Batch।

Exporter: Collector থেকে Backend-এ Telemetry পাঠানোকারী উপাদান।

Pipeline: Receiver → Processor → Exporter শৃঙ্খল; প্রতি Signal-এ আলাদা।

Auto-Instrumentation: কোড না বদলে লাইব্রেরি থেকে স্বয়ংক্রিয় Span তৈরি।

Manual Instrumentation: নিজে কোড লিখে Span তৈরি করা।

Propagator: Context (Trace ID) সার্ভিসগুলোর মধ্যে বহনকারী উপাদান।

১৮.১৮ জ্ঞান যাচাই — প্রশ্ন

1. API ও SDK-এর পার্থক্য কী? ব্যবসায়িক কোডে শুধু API ব্যবহার করা কেন ভালো?
2. Collector ছাড়া সরাসরি Backend-এ Export করলে কী কী সমস্যা হয়?
3. Receiver, Processor, Exporter ও Pipeline — প্রতিটির একটি করে উদাহরণ দিন।
4. OTLP-র gRPC ও HTTP পোর্ট কোনটি?
5. Auto-Instrumentation কী দেখে, আর কী দেখে না? Manual কখন দরকার?
6. memory_limiter Processor না থাকলে কী হতে পারে?
7. একটি Trace আসে কিন্তু Metric আসে না — কোথায় তাকাবেন?
8. Backend বদলালে অ্যাপের কোড বদলাতে হয় না কেন?

১৮.১৯ ব্যবহারিক অনুশীলন

1. Docker-এ OpenTelemetry Collector চালু করুন ও otel-collector.yaml লিখুন (Traces Pipeline সহ)।
2. Node.js অ্যাপে Auto-Instrumentation বসিয়ে Tempo/Jaeger-এ Trace দেখুন।
3. একটি Manual Span ("discount.calculate") যোগ করুন ও Trace-এ খুঁজুন।
4. attributes Processor দিয়ে `http.request.header.authorization` মুছে ফেলুন।
5. Collector বন্ধ করে অ্যাপ চালু রাখুন — অ্যাপ থামে কি না লিখুন।
6. Batch Processor-এর timeout বদলে Export-এর আচরণ পর্যবেক্ষণ করুন।

অধ্যায় ১৯ • PART IV — Distributed Tracing and Advanced Observability

Chapter 19 — Jaeger, Grafana Tempo, and Trace Analysis

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: **Jaeger** ও **Grafana Tempo**-র আর্কিটেকচার ব্যাখ্যা করতে; Trace খুঁজতে, Waterfall পড়তে ও ধীর Span চিহ্নিত করতে; দুটি Trace তুলনা করে Regression খুঁজতে; **TraceQL**-এর মৌলিক Query লিখতে; এবং Grafana-তে Trace, Metric ও Log একত্রে দেখতে।

১৯.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Trace সংগ্রহ করা অর্ধেক কাজ; বাকি অর্ধেক হলো তা থেকে উত্তর বের করা। Jaeger ও Tempo দুটি জনপ্রিয় Trace Backend — Jaeger পুরোনো ও স্বয়ংসম্পূর্ণ, Tempo Grafana ইকোসিস্টেমের সঙ্গে গভীরভাবে জোড়া ও Object Storage-ভিত্তিক। TechNova-র SRE-কে রাত ২টায় একটি ধীর অনুরোধের উত্তর মিনিটে বের করতে হয় — সেই দক্ষতাই এই অধ্যায়: কোথায় খুঁজবেন, কীভাবে পড়বেন, কীভাবে তুলনা করবেন।

১৯.৩ পূর্বশর্ত জ্ঞান

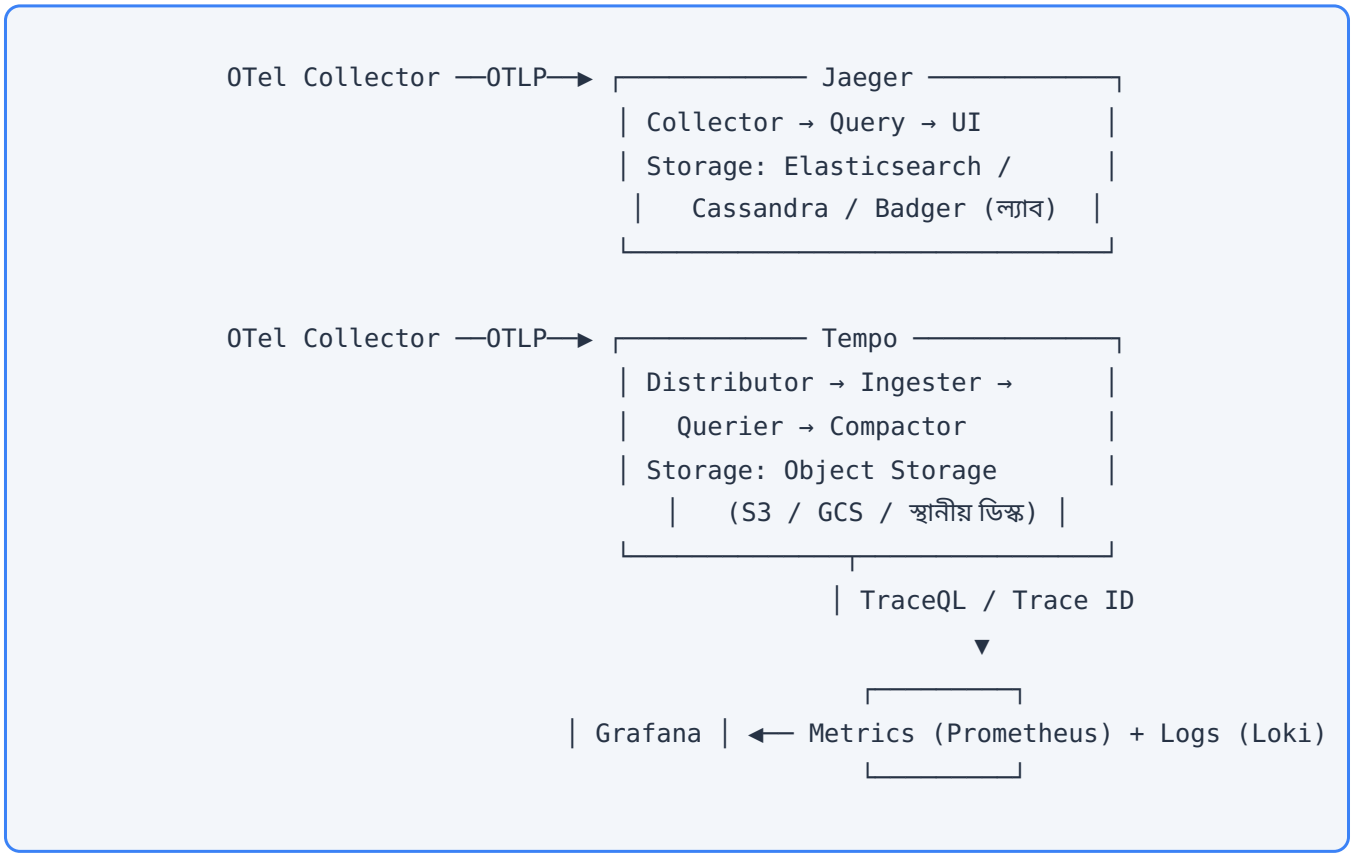
অধ্যায় ১৭–১৮-এর Trace, Span, OTel Collector ও OTLP ধারণা; Grafana-তে Dashboard দেখার অভিজ্ঞতা; Docker Compose-এর মৌলিক জ্ঞান। TraceQL আগে না জানলেও চলবে।

১৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Trace Backend-এর কাজ তিনটি — (১) Span গ্রহণ করা, (২) Trace ID দিয়ে সেগুলো একত্র করে সংরক্ষণ করা, (৩) প্রশ্নের উত্তরে দ্রুত ফিরিয়ে দেওয়া। Jaeger ও Tempo এই তিনটি কাজ ভিন্নভাবে করে। Jaeger Span-গুলোকে Index করে রাখে — যেমন Service নাম, Operation নাম, Tag দিয়ে খোঁজা যায়; এতে খোঁজা দ্রুত, কিন্তু Index-এর খরচ বেশি। Tempo উল্টো পথে যায়: এটি শুধু Trace ID দিয়ে খোঁজে, বড় Index রাখে না, Trace-গুলো Object Storage-এ (যেমন S3-ধরনের) সস্তায় রাখে — তাই খরচ কম, কিন্তু Tag দিয়ে খোঁজা সীমিত। Tempo-র TraceQL সেই সীমা কাটাতে Span-এর ভেতরে Query করার ভাষা দেয়। কোনটি ভালো — তা নির্ভর করে আপনার খোঁজার ধরনের ওপর, "সেরা" বলে কিছু নেই।

> **CONCEPT:** Jaeger হলো সাজানো লাইব্রেরি — বইয়ের তালিকা (Index) আছে, তাই বিষয় দিয়ে খোঁজা যায়। Tempo হলো গুদাম — বাক্সে Trace ID লেখা, ID জানলে সস্তায় পাওয়া যায়। TraceQL হলো গুদামে ঢুকে বাক্স খুলে দেখার উপায়।

১৯.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



১৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Jaeger আর্কিটেকচার

Jaeger-এর উপাদান: **Collector** (Span গ্রহণ), **Query Service** (খোঁজা), **UI** (Waterfall দেখা), **Storage** (Elasticsearch/Cassandra)। ছোট সেটআপে `all-in-one` ইমেজে সব একসঙ্গে চলে — ল্যাবের জন্য আদর্শ, প্রোডাকশনে আলাদা উপাদান। Jaeger-এ Service, Operation, Tag, সময়সীমা ও Duration দিয়ে খোঁজা যায় — যেমন "গত ১ ঘণ্টায় Order Service-এ ১ সেকেন্ডের বেশি সময় নেওয়া Trace"।

Tempo আর্কিটেকচার

Tempo-র উপাদান: **Distributor** (Span গ্রহণ ও বণ্টন), **Ingester** (সাময়িক সংরক্ষণ), **Querier** (খোঁজা), **Compactor** (পুরোনো Trace গুছিয়ে Object Storage-এ রাখা)। Tempo নিজে কোনো UI দেয় না — Grafana-ই তার UI। Tempo-র শক্তি: Object Storage-এ সস্তা দীর্ঘ Retention, আর Grafana-তে Metric/Log-এর সঙ্গে একই Trace ID-তে লাফ দেওয়া (correlation)।

Waterfall পড়া

Waterfall-এ প্রতিটি Span একটি অনুভূমিক বার: বামের অবস্থান শুরু সময়, দৈর্ঘ্য Duration। Root Span সবচেয়ে উপরে। ধীর Span খুঁজতে: সবচেয়ে দীর্ঘ বারটি দেখুন, তারপর তার সন্তানদের দেখুন — সময় কি সন্তানে, নাকি নিজের কাজে (Self Time)? একটি Span দীর্ঘ কিন্তু সন্তান ছোট মানে সেই সার্ভিসের নিজের কাজ ধীর; সন্তান দীর্ঘ মানে দেরি নিচে। **Critical Path** হলো Root থেকে শেষ পর্যন্ত দীর্ঘতম শৃঙ্খল — সেটি ছোট করলেই মোট latency কমে।

Trace তুলনা করা

একটি দ্রুত Trace ও একটি ধীর Trace পাশাপাশি রাখুন — কোন Span-এ পার্থক্য? একই Operation-এর Duration দুটিতে কত? এই তুলনাই Regression ধরার সবচেয়ে সরাসরি উপায় — "গতকাল এই Span ৪০ মিলিসেকেন্ড ছিল, আজ ৯০০"। Jaeger-এ দুটি Trace ID খুলে হাতে তুলনা করুন; Grafana-তেও একই কাজ করা যায়।

TraceQL basics

TraceQL হলো Tempo-তে Trace খোঁজার Query ভাষা। উদাহরণ: `{ span.http.status_code = 500 }` — Error Span-যুক্ত Trace খুঁজুন; `{ resource.service.name = "technova-order-service" && duration > 1s }` — Order Service-এ ১ সেকেন্ডের বেশি Trace; `{ span.db.system = "mongodb" }` — MongoDB Span-যুক্ত Trace। TraceQL Span-এর Attribute, Duration ও Status দিয়ে ফিল্টার করে — Index ছাড়াই, Trace-এর ভেতরে খুঁজে।

বৈশিষ্ট্য	Jaeger	Grafana Tempo
UI	নিজস্ব UI আছে	Grafana-ই UI
Storage	Elasticsearch/Cassandra (Index-ভিত্তিক)	Object Storage (সস্তা, দীর্ঘ Retention)
খোঁজা	Service/Tag/Index দিয়ে দ্রুত	Trace ID + TraceQL
Grafana সংযোগ	ডেটা সোর্স হিসেবে যোগ করা যায়	গভীর সংযোগ — Metric/Log-এ লাফ
ল্যাব শুরু	all-in-one ইমেজে সহজ	Docker Compose-এ সহজ

১৯.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# Jaeger all-in-one — ল্যাবের জন্য (নমুনা আউটপুট — চিত্রণমূলক)
docker run -d --name jaeger \
  -p 16686:16686 \  # 1. Jaeger UI পোর্ট
  -p 4317:4317 -p 4318:4318 \  # 2. OTLP গ্রহণ পোর্ট
  jaegertracing/all-in-one:1.55
# UI: http://203.0.113.10:16686 — Service বেছে Trace খুঁজুন
# প্রোডাকশনে all-in-one নয় — আলাদা Storage (Elasticsearch) ব্যবহার করুন
```

```
# Tempo + Grafana — Docker Compose (মূল অংশ)
# tempo: image: grafana/tempo:latest, ports: 4317, 3200 (query)
# grafana: image: grafana/grafana:latest, ports: 3000
# Grafana-তে Data Source যোগ করুন: Tempo, URL http://tempo:3200
# তারপর Explore → Tempo → Trace ID দিয়ে খুঁজুন, বা TraceQL লিখুন
```

TraceQL উদাহরণ — লাইন ধরে ধরে:

```
{ resource.service.name = "technova-order-service" && duration > 1s }
# resource.service.name : কোন সার্ভিসের Trace চাই
# duration > 1s : ১ সেকেন্ডের বেশি সময় নেওয়া Trace
{ span.http.status_code = 500 }   # Error Trace
{ span.db.system = "mongodb" && span.db.operation = "find" }  # MongoDB find Span
```

১৯.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

```
# docker-compose.yml — TechNova ল্যাব: Tempo + Grafana + Prometheus
services:
  tempo:
    image: grafana/tempo:2.5.0
    command: ["-config.file=/etc/tempo.yaml"]
    ports: ["4317:4317", "4318:4318", "3200:3200"]
    volumes: ["./tempo.yaml:/etc/tempo.yaml", "tempo-data:/tmp/tempo"]
  grafana:
    image: grafana/grafana:11.0.0
    ports: ["3000:3000"]
    environment: ["GF_AUTH_ANONYMOUS_ENABLED=false"]
    volumes: ["grafana-data:/var/lib/grafana"]
volumes: { tempo-data: {}, grafana-data: {} }
# tempo.yaml-এ storage: trace: backend: local, path: /tmp/tempo/traces
# Grafana-তে Tempo Data Source: http://tempo:3200; Prometheus: http://prometheus:9090
```

১৯.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র /api/checkout হঠাৎ ধীর — p95 ৩ সেকেন্ড। Grafana Dashboard-এ Metric দেখে সময়সীমা চিহ্নিত করা হলো, তারপর সেই সময়ের একটি ধীর Trace Tempo-তে TraceQL দিয়ে খোঁজা হলো: `{ resource.service.name="technova-order-service" && duration > 2s }`। Waterfall-এ দেখা গেল `inventory.reserve` Span ২.২ সেকেন্ড — তার ভেতরে MongoDB Span মাত্র ৩০ মিলিসেকেন্ড, বাকিটা Inventory Service-এর উত্তরের অপেক্ষা। Inventory Service-এর নিজের Trace খুলে দেখা গেল তার ডেটাবেস সংযোগ Pool শেষ হয়ে গেছে (connection pool saturation)। Metric বলেছিল "ধীর", Trace বলল "কোথায়", Inventory-র Trace বলল "কেন"।

> **REAL WORLD:** ধীর Trace খুঁজতে সবসময় সবচেয়ে দীর্ঘ Trace নয় — প্রতিনিধিত্বমূলক Trace নিন। একটি অস্বাভাবিক Trace (যেমন একজন ব্যবহারকারীর বিশাল Cart) ভুল সিদ্ধান্তে নিয়ে যেতে পারে। একই সময়ের ৩–৫টি ধীর Trace-এ একই Span ধীর কি না দেখুন।

১৯.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Docker Compose-এ Tempo + Grafana চালু করুন; Collector-এর Exporter Tempo-র দিকে নির্দেশ করুন।
2. অ্যাপে কয়েকটি অনুরোধ পাঠান; Grafana Explore-এ Trace ID দিয়ে একটি Trace খুলুন।
3. Waterfall-এ সবচেয়ে দীর্ঘ Span ও Critical Path চিহ্নিত করুন।
4. TraceQL-এ `{ span.http.status_code = 500 }` লিখে Error Trace খুঁজুন।
5. একটি ধীর ও একটি দ্রুত Trace তুলনা করে পার্থক্যের Span লিখুন।
6. Grafana-তে একটি Log লাইন থেকে Trace ID-তে লাফ দিন (Loki + Tempo সংযোগ)।
7. Jaeger all-in-one-ও চালিয়ে একই Trace দুই UI-তে দেখে পার্থক্য লিখুন।

১৯.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** শুধু সবচেয়ে দীর্ঘ Trace দেখে সিদ্ধান্ত নেওয়া — অস্বাভাবিক Trace ভুল পথে নেয়।

- **ভুল ধারণা:** "Tempo-তে Tag দিয়ে Jaeger-এর মতো খোঁজা যায়।" — Tempo-র খোঁজা Trace ID ও TraceQL-ভিত্তিক; Index-ভিত্তিক খোঁজা Jaeger-এর শক্তি।
- **ভুল:** Trace না পেয়ে Backend-এর দোষ দেওয়া — আগে Collector-এ Export হচ্ছে কি না দেখুন।
- **ভুল:** Self Time না দেখে শুধু মোট Duration দেখা — দেরি সন্তানে না নিজের কাজে, তা আলাদা করুন।

১৯.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Jaeger UI (16686) ও Tempo Query পোর্ট ইন্টারনেটে খোলা রাখবেন না — অভ্যন্তরীণ নেটওয়ার্ক বা VPN-এর পেছনে রাখুন। Grafana-তে প্রমাণীকরণ ও ভূমিকা (role) দিয়ে কে Trace দেখতে পারে তা নিয়ন্ত্রণ করুন। Trace-এ সংবেদনশীল Attribute থাকলে Collector-এ Redact করুন (অধ্যায় ১৮)। Retention নীতি লিখে রাখুন — Trace-ও ব্যক্তিগত ডেটা হতে পারে।

১৯.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Jaeger-এর Index খরচ বেশি — High Cardinality Tag (যেমন প্রতিটি User ID) Index করলে Storage ও খোঁজা দুটোই ধীর হয়। Tempo Object Storage-এ সস্তা, তাই দীর্ঘ Retention (৩০ দিন) সম্ভব — কিন্তু পুরোনো Trace খুঁজতে সময় বেশি লাগতে পারে। Sampling ও Retention একসঙ্গে ঠিক করুন: বেশি Sampling + দীর্ঘ Retention = বেশি খরচ। Compactor নিয়মিত চালান, নইলে Storage ছড়িয়ে যায়।

১৯.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Jaeger-এ Trace নেই	Collector Exporter ভুল ঠিকানা	Collector Log ও Jaeger Log দেখুন
Tempo-তে Trace ID দিয়ে পাওয়া যায় না	Trace Ingest হয়নি, বা Retention পার হয়েছে	সময়সীমা ও Ingester Log পরীক্ষা করুন
TraceQL কোনো ফল দেয় না	Attribute নাম ভুল, বা Sampling-এ বাদ	একটি পরিচিত Trace-এর Attribute নাম মিলিয়ে দেখুন
Waterfall-এ Span-এর সময় মেলে না	Clock Skew	সব হোস্টে NTP যাচাই করুন
Grafana থেকে Tempo সংযোগ ব্যর্থ	Data Source URL ভুল	http://tempo:3200 ঠিকানা ও নেটওয়ার্ক পরীক্ষা করুন

১৯.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ধীর Trace-এর সঙ্গে দ্রুত Trace তুলনা করুন; Critical Path দেখুন; TraceQL সংরক্ষণ করে রাখুন (পুনরাবৃত্ত তদন্তের জন্য); Grafana-তে Trace-Metric-Log সংযোগ চালু করুন। **অ্যান্টি-প্যাটার্ন:** একটি Trace দেখে সিদ্ধান্ত; High Cardinality Tag Index করা; প্রোডাকশনে all-in-one Jaeger; UI ইন্টারনেটে খোলা রাখা।

১৯.১৬ অধ্যায়ের সারাংশ

Jaeger Index-ভিত্তিক, নিজস্ব UI-সহ Trace Backend; Tempo Object Storage-ভিত্তিক, Grafana-কেন্দ্রিক, TraceQL দিয়ে খোঁজা যায়। দুটিতেই Waterfall পড়া, Critical Path খোঁজা ও ধীর-দ্রুত Trace তুলনা করা — তদন্তের মূল দক্ষতা। Trace, Metric ও Log একত্রে দেখতে পারলেই তদন্ত সম্পূর্ণ হয়।

১৯.১৭ মূল প্রযুক্তিগত পরিভাষা

Jaeger: Index-ভিত্তিক Distributed Tracing Backend; নিজস্ব UI আছে।

Grafana Tempo: Object Storage-ভিত্তিক Tracing Backend; Grafana-ই তার UI।

Waterfall: Span-গুলোকে সময়-অনুযায়ী বার আকারে দেখানো চিত্র।

Critical Path: Root থেকে শেষ পর্যন্ত দীর্ঘতম Span শৃঙ্খল; মোট latency নির্ধারণ করে।

Self Time: একটি Span-এর নিজের কাজে ব্যয়িত সময়, সন্তানদের সময় বাদে।

TraceQL: Tempo-তে Trace খোঁজার Query ভাষা।

Ingester: Tempo-তে Span গ্রহণ ও সাময়িক সংরক্ষণকারী উপাদান।

Compactor: পুরোনো Trace গুছিয়ে Object Storage-এ রাখার উপাদান।

Distributor: Tempo-তে Span গ্রহণ ও Ingester-এ বণ্টনকারী উপাদান।

Querier: Tempo-তে Trace খোঁজার অনুরোধ সামলানোকারী উপাদান।

Object Storage: S3-ধরনের সস্তা, স্কেলযোগ্য সংরক্ষণ (যেমন Tempo-র Backend)।

Regression: আগে দ্রুত ছিল এমন কিছু ধীর বা খারাপ হয়ে যাওয়া।

১৯.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Jaeger ও Tempo-র Storage পদ্ধতির পার্থক্য কী? কোনটি দীর্ঘ Retention-এ সস্তা?
2. Waterfall-এ একটি Span দীর্ঘ কিন্তু তার সন্তান ছোট — এর অর্থ কী?
3. Critical Path কী? এটি ছোট করলে কী হয়?
4. TraceQL-এ Error Trace খুঁজতে কী লিখবেন?
5. একটি ধীর Trace দেখে সিদ্ধান্ত নেওয়া কেন ঝুঁকিপূর্ণ?
6. Tempo-র নিজস্ব UI নেই কেন? এর UI কী?
7. Clock Skew Waterfall-এ কী সমস্যা তৈরি করে?
8. Jaeger all-in-one কখন ব্যবহার করবেন, কখন নয়?

১৯.১৯ ব্যবহারিক অনুশীলন

1. Tempo + Grafana চালু করে একটি Trace খুলে Critical Path লিখুন।
2. TraceQL-এ তিনটি Query লিখুন: Error Trace, ১ সেকেন্ডের বেশি Trace, MongoDB Span-যুক্ত Trace।
3. ইচ্ছাকৃতভাবে একটি API ধীর করে (sleep যোগ করে) ধীর ও দ্রুত Trace তুলনা করুন।
4. Grafana-তে Log থেকে Trace ID-তে লাফ দিয়ে দেখান।
5. Jaeger all-in-one-এ একই Trace খুঁজে দুই UI-র পার্থক্য লিখুন।

অধ্যায় ২০ • PART IV — Distributed Tracing and Advanced Observability

Chapter 20 — Microservices Observability and Telemetry Correlation

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Metrics, Log ও Trace-কে একই Trace ID দিয়ে জোড়া দিতে (**Correlation**); Log-এ Trace ID ও **Exemplar** দিয়ে Metric থেকে Trace-এ লাফ দিতে; Service Map ও Dependency বিশ্লেষণ করতে; এবং একটি API অনুরোধ Frontend → API → Database পর্যন্ত Trace করে তার যাত্রা ব্যাখ্যা করতে — সেই সঙ্গে Tracing-এর সীমাবদ্ধতা স্বীকার করতে।

২০.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

তিনটি Signal আলাদা থাকলে তদন্ত তিনবার করতে হয়: Metric-এ সময় দেখুন, Log-এ Error খুঁজুন, Trace-এ Span দেখুন — প্রতিবার আলাদা খোঁজা। Correlation এই তিনটিকে এক সুতোয় গাঁথে: একটি Trace ID দিয়ে Log, Metric ও Trace — সব এক জায়গায়। TechNova-র SRE রাত ২টায় একটি Alert পেয়ে Metric থেকে সরাসরি সেই অনুরোধের Trace-এ, Trace থেকে সেই অনুরোধের Log-এ যেতে পারেন — মিনিটে, ঘণ্টায় নয়। এটাই Microservices Observability-র আসল শক্তি।

২০.৩ পূর্বশর্ত জ্ঞান

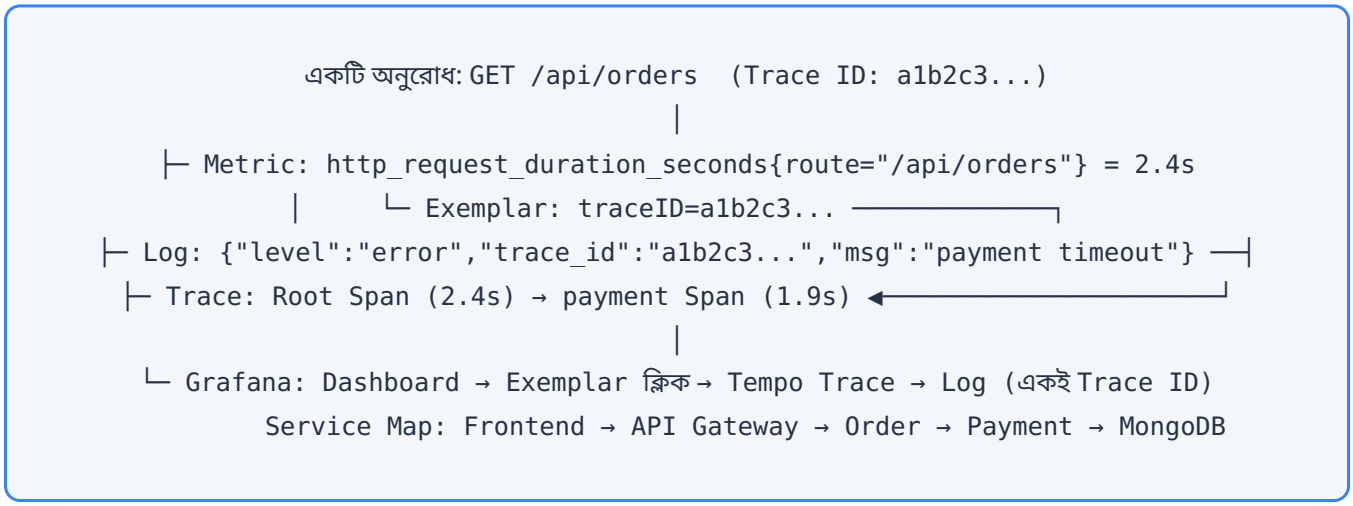
অধ্যায় ১৭–১৯-এর Trace, OTel, Jaeger/Tempo; Prometheus Metric ও Grafana Dashboard-এর ধারণা; Structured Log (JSON Log) কী তা জানা। Exemplar আগে না জানলেও চলবে।

২০.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Correlation মানে একই ঘটনার বিভিন্ন রেকর্ডে একই পরিচয় লেখা। একটি অনুরোধের Metric (কত সময় লেগেছে), Log (কী ঘটেছে) ও Trace (কোথায় সময় গেল) — তিনটি আলাদা রেকর্ড, কিন্তু যদি তিনটিতেই একই Trace ID লেখা থাকে, তবে তারা একই ঘটনার তিনটি দৃষ্টিভঙ্গি। Log-এ Trace ID লেখা মানে Log লাইন থেকে Trace খুঁজে পাওয়া; Metric-এ **Exemplar** (একটি নমুনা Trace ID যা Metric-এর সঙ্গে সংরক্ষিত থাকে) মানে Histogram-এর একটি Bucket থেকে সরাসরি একটি আসল Trace-এ লাফ দেওয়া। কোনো জাদু নেই — শুধু ভাগ করা পরিচয় (shared identifier)।

> **CONCEPT:** Metric হলো শহরের মানচিত্র (কোন রাস্তায় যানজট), Log হলো ঘটনার বিবরণ (কোথায় দুর্ঘটনা), Trace হলো একটি গাড়ির যাত্রাপথ। Trace ID হলো গাড়ির নম্বর প্লেট — তিনটি নথিতেই একই নম্বর থাকলে আপনি গাড়িটিকে সব জায়গায় অনুসরণ করতে পারেন।

২০.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



২০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Log-এ Trace ID

Structured Log-এ `trace_id` ও `span_id` ফিল্ড যোগ করুন। OTel SDK Context থেকে বর্তমান Trace ID নিয়ে Logger-এ দেওয়া যায় (যেমন Pino/Winston-এ)। তখন Loki-তে `{ trace_id="a1b2c3..." }` লিখে সেই অনুরোধের সব Log পাওয়া যায়। Log-এ Trace ID না থাকলে Log ও Trace আলাদা জগৎ হয়ে থাকে।

Exemplar

Exemplar হলো Metric-এর সঙ্গে সংরক্ষিত একটি নমুনা — যেমন Histogram-এর একটি Observation-এর সঙ্গে তার Trace ID। Prometheus-এ Exemplar চালু থাকলে Grafana Dashboard-এর Graph-এ একটি বিন্দুতে ক্লিক করে সরাসরি সেই Trace-এ যাওয়া যায়। এটি Metric ("p95 বেড়েছে") থেকে Trace ("কোন অনুরোধটি ধীর ছিল")-এর সেতু।

Service Map ও Dependency বিশ্লেষণ

Trace থেকে তৈরি Service Map বলে: কোন সার্ভিস কাকে কতবার ডাকে, গড় latency কত, Error হার কত। Dependency বিশ্লেষণে প্রশ্ন করুন: কোন সার্ভিস সবচেয়ে বেশি ডাকা হয় (Single Point of Failure)? কোন নির্ভরতা সবচেয়ে ধীর? কোন সার্ভিসের Error অন্য সার্ভিসে ছড়ায় (Error Propagation)? একটি নতুন সার্ভিস যোগ করার আগে Service Map দেখুন — অজানা নির্ভরতা প্রায়ই এখানে ধরা পড়ে।

কাজ করা উদাহরণ: একটি অনুরোধের যাত্রা

গ্রাহক React Frontend-এ "Place Order" চাপলেন। (১) Frontend `traceparent` হেডারসহ POST /api/orders পাঠাল। (২) API Gateway Trace গ্রহণ করে Auth Service-কে ডাকল — Span ১২০ মিলিসেকেন্ড। (৩) Order Service MongoDB-তে Order লিখল — Span ৪৫ মিলিসেকেন্ড। (৪) Payment Service বাইরের Gateway-কে ডাকল — Span ১৯৫০ মিলিসেকেন্ড, Timeout-এর কাছাকাছি। (৫) Order Service উত্তর সাজিয়ে ফেরত পাঠাল। মোট ২.৪ সেকেন্ডের মধ্যে ১.৯৫ সেকেন্ড Payment Gateway-তে — সমস্যা ডেটাবেসে নয়, বাইরের নির্ভরতায়। Trace, Log (Payment Timeout বার্তা) ও Metric (p95 বৃদ্ধি) — তিনটি একই Trace ID-তে মিলে গেল।

Tracing-এর সীমাবদ্ধতা — সৎভাবে

Tracing সব Root Cause ধরিয়ে দেয় না। **সীমাবদ্ধতা:** (১) Instrument করা হয়নি এমন জায়গা Trace-এ দেখা যায় না — যেমন একটি লাইব্রেরির ভেতরের কাজ, বা ডেটাবেসের ভেতরের Lock। (২) Sampling-এ বাদ পড়া Trace হারিয়ে যায় — বিরল Error ধরা নাও পড়তে পারে। (৩) Trace দেখায় কোথায় সময় গেল, কেন তা নয় — Payment Gateway ধীর কেন (তাদের সমস্যা, নেটওয়ার্ক, নাকি আমাদের Payload) তা Trace বলে না। (৪) Trace-এ ভুল Attribute থাকলে ভুল সিদ্ধান্ত হয়। তাই Tracing-কে Metric, Log, Profiling ও ডোমেইন জ্ঞানের সঙ্গে ব্যবহার করুন — একা নয়।

সংযোগ	কীভাবে	কী পাওয়া যায়
Log → Trace	Log-এ trace_id ফিল্ড	এক অনুরোধের সব Log ও Trace
Metric → Trace	Exemplar (Trace ID Metric-এ)	ধীর অনুরোধের আসল Trace
Trace → Log	Grafana-তে Trace ID দিয়ে Loki খোঁজা	Span-এর সময়ের Log
Trace → Service Map	সব Trace একত্র করা	নির্ভরতার চিত্র

২০.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
// Node.js: Pino Logger-এ Trace ID যোগ করা (নমুনা কোড — চিত্রণমূলক)
const { trace } = require('@opentelemetry/api');
const pino = require('pino');
const logger = pino({ formatters: { log(obj) {
  const span = trace.getActiveSpan();  // 1. বর্তমান Span নাও
  if (span) {  // 2. Span থাকলে তার ID Log-এ যোগ করো
    const ctx = span.spanContext();
    obj.trace_id = ctx.traceId; obj.span_id = ctx.spanId;
  }
  return obj;
}}});
logger.info('order created'); // Log লাইন: {"level":30,"trace_id":"a1b2...","msg":"order
created"}
```

```
# Loki-তে Trace ID দিয়ে Log খোঁজা (LogQL)
{app="technova-order-service"} | json | trace_id="a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6"
# {app=...} : কোন সার্ভিসের Log; | json : JSON পার্স করা; trace_id=... : সেই অনুরোধের Log
```

২০.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

Grafana-তে Tempo Data Source-এ Trace থেকে Log-এ লাফ দেওয়ার সংযোগ (ধারণাগত কনফিগারেশন): Tempo Data Source-এর "Trace to logs" বিভাগে Loki Data Source বেছে নিন, Query লিখুন `{app="technova-order-service"} | json | trace_id="${__trace.traceId}"` — তখন Tempo-তে একটি Trace খুলে "Logs for this span" ক্লিক করলেই সেই Trace ID-র Log দেখা যায়। Prometheus Data Source-এ Exemplar চালু থাকলে Histogram Graph-এ বিন্দুতে ক্লিক করে Trace-এ যাওয়া যায়। এই সংযোগগুলো একবার বসালে তদন্তের পথটি এক ক্লিকের হয়।

২০.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র Support টিম জানাল একজন গ্রাহকের Order দুবার চার্জ হয়েছে। Order ID দিয়ে Log খুঁজে Trace ID পাওয়া গেল; Trace খুলে দেখা গেল Payment Span দুবার চলেছে — Frontend Retry-এর কারণে একই অনুরোধ দুবার গেছে, কিন্তু Idempotency Key ছিল না। Metric-এ শুধু Payment অনুরোধের সংখ্যা বেড়েছে দেখা গিয়েছিল; Log-Trace Correlation ছাড়া "দুবার চার্জ" ও "Retry" যোগ করা যেত না। সমাধান: Payment API-তে Idempotency Key বাধ্যতামূলক করা হলো।

> **WARNING:** Correlation-এর জন্য Trace ID Log-এ লেখা দরকার — কিন্তু Log-এ Trace ID-র সঙ্গে User-এর ব্যক্তিগত তথ্য (ইমেইল, ঠিকানা) একসঙ্গে লিখবেন না। Trace ID নিজে সংবেদনশীল নয়, কিন্তু তা দিয়ে ব্যক্তিগত Log খুঁজে পাওয়া যায় — তাই Log-এর প্রবেশাধিকার নিয়ন্ত্রণ করুন।

২০.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. একটি সার্ভিসের Logger-এ Trace ID যোগ করুন ও একটি অনুরোধের Log-এ তা দেখুন।
2. Loki-তে সেই Trace ID দিয়ে খুঁজে সব Log একত্র করুন।
3. Grafana-তে Tempo ও Loki সংযোগ করে Trace থেকে Log-এ লাফ দিন।
4. Prometheus-এ Exemplar চালু করে Histogram থেকে Trace-এ যান।
5. Service Map খুলে সবচেয়ে বেশি ডাকা সার্ভিস ও সবচেয়ে ধীর নির্ভরতা চিহ্নিত করুন।
6. উপরের "একটি অনুরোধের যাত্রা" উদাহরণটি নিজের সিস্টেমে পুনরায় তৈরি করে লিখুন।

২০.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "Tracing বসালেই সব Root Cause বেরিয়ে আসবে।" — না; Instrument করা হয়নি এমন জায়গা, Sampling-এ হারানো Trace, ও বাইরের সার্ভিসের ভেতরের কারণ Trace দেখায় না।
- **ভুল:** Log-এ Trace ID না লেখা — তাহলে Log ও Trace আলাদা থাকে।
- **ভুল:** Exemplar ছাড়া Metric থেকে Trace খোঁজার চেষ্টা — সময় মিলিয়ে অনুমান করতে হয়, নির্ভরযোগ্য নয়।
- **ভুল:** Service Map-কে স্থির ডকুমেন্ট মনে করা — এটি আসল ট্রাফিকের চিত্র, নিয়মিত বদলায়।

২০.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Trace ID দিয়ে Log খুঁজে পাওয়া যায় — তাই Log ও Trace-এর প্রবেশাধিকার একসঙ্গে নিয়ন্ত্রণ করুন। Tenant-ভিত্তিক বিচ্ছিন্নতা (isolation) থাকলে এক Tenant-এর Trace ID দিয়ে অন্য Tenant-এর Log দেখা যাবে না — তা পরীক্ষা করুন। Baggage-এ Tenant ID থাকলে তা Log-এও উঠতে পারে — সংবেদনশীল Baggage এড়িয়ে চলুন।

২০.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

প্রতিটি Log লাইনে Trace ID যোগ করার খরচ নগণ্য। Exemplar-এর খরচ Prometheus-এ সামান্য বেশি Storage — কিন্তু তদন্তের সময় বাঁচায়। Service Map তৈরির খরচ Backend-এর — Tempo/Jaeger তা Trace থেকে গণনা করে। Correlation-এর আসল খরচ হলো তিনটি সিস্টেম (Metric, Log, Trace) চালানো — ছোট টিমে Grafana স্ট্যাক (Prometheus + Loki + Tempo) একসঙ্গে চালানো সাশ্রয়ী।

২০.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Log-এ Trace ID নেই	Logger-এ Formatter যোগ হয়নি	OTel Context থেকে Trace ID নেওয়ার কোড যাচাই করুন
Trace ID আছে কিন্তু Trace পাওয়া যায় না	Sampling-এ বাদ, বা Backend-এ Export হয়নি	Collector Log ও Sampling নীতি দেখুন
Exemplar ক্লিক কাজ করে না	Prometheus-এ Exemplar বন্ধ, বা Grafana সংযোগ নেই	Prometheus-এ Exemplar চালু ও Tempo Data Source যাচাই করুন
Service Map-এ সার্ভিস নেই	সেই সার্ভিস Instrument করা হয়নি	SDK ও Propagation যাচাই করুন

TROUBLESHOOTING: Correlation ভাঙা মনে হলে একটি অনুরোধের Trace ID নিন ও তিন জায়গায় খুঁজুন — Loki-তে Log, Tempo-তে Trace, Prometheus-এ Exemplar। যেখানে পাওয়া যায় না, সেখানেই সংযোগ ভাঙা।

২০.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: সব Log-এ Trace ID; Metric-এ Exemplar; Grafana-তে তিন Signal-এর সংযোগ; Service Map নিয়মিত পর্যালোচনা; Tracing-এর সীমা দলকে জানানো। **অ্যান্টি-প্যাটার্ন:** Trace ছাড়া Log দিয়ে Microservices তদন্ত; "Tracing সব বলে দেবে" মনে করা; Correlation না বসিয়ে তিনটি টুল আলাদা রাখা।

২০.১৬ অধ্যায়ের সারাংশ

Microservices Observability-র কেন্দ্রে আছে Correlation: Trace ID Log-এ, Exemplar Metric-এ, Trace Backend-এ — তিনটি এক সুতোয়। Service Map নির্ভরতা দেখায়, একটি অনুরোধের যাত্রা তদন্তের পথ দেখায়। কিন্তু Tracing-এর সীমা মনে রাখুন: যা Instrument করা হয়নি, যা Sample করা হয়নি, ও বাইরের সার্ভিসের ভেতরের কারণ — Trace তা দেখায় না।

২০.১৭ মূল প্রযুক্তিগত পরিভাষা

Correlation: ভিন্ন Signal-এর রেকর্ডকে ভাগ করা পরিচয় (যেমন Trace ID) দিয়ে জোড়া দেওয়া।

Exemplar: Metric-এর সঙ্গে সংরক্ষিত নমুনা Trace ID; Metric থেকে Trace-এ লাফ দেওয়ার সেতু।

Structured Log: JSON-এর মতো কাঠামোবদ্ধ Log, যাতে ফিল্ড দিয়ে খোঁজা যায়।

Service Map: Trace থেকে তৈরি সার্ভিস নির্ভরতার চিত্র।

Dependency Analysis: কোন সার্ভিস কাকে ডাকে ও তার প্রভাব বিশ্লেষণ করা।

Error Propagation: একটি সার্ভিসের Error অন্য সার্ভিসে ছড়িয়ে পড়া।

Idempotency Key: একই অনুরোধ দুবার এলে দ্বিতীয়বার কাজ না করার জন্য অনন্য চাবি।

Single Point of Failure: যে উপাদান নষ্ট হলে পুরো সিস্টেম থামে।

Trace to Logs: Trace থেকে সেই Trace ID-র Log-এ যাওয়ার Grafana সংযোগ।

Histogram: মানের বণ্টন দেখানো Metric ধরন; Exemplar এখানে যুক্ত হয়।

২০.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Correlation কী? Trace ID ছাড়া Log ও Trace জোড়া দেওয়া যায় কি?
2. Exemplar কী? Metric থেকে Trace-এ যেতে এটি কীভাবে সাহায্য করে?
3. Service Map থেকে Single Point of Failure কীভাবে চিনবেন?
4. একটি অনুরোধ Frontend → API → Database গেল — প্রতিটি ধাপে Trace ID কী থাকে?
5. Tracing-এর তিনটি সীমাবদ্ধতা লিখুন।
6. Log-এ Trace ID থাকা সত্ত্বেও Trace পাওয়া যাচ্ছে না — সম্ভাব্য দুটি কারণ কী?
7. Idempotency Key না থাকলে Retry কী সমস্যা তৈরি করে?
8. Error Propagation-এর একটি উদাহরণ দিন।

২০.১৯ ব্যবহারিক অনুশীলন

1. Logger-এ Trace ID যোগ করে একটি অনুরোধের সব Log এক Trace ID-তে খুঁজুন।
2. Grafana-তে Trace থেকে Log-এ লাফ দেওয়ার সংযোগ বসান।
3. একটি API অনুরোধের যাত্রা (Frontend → API → Database) লিখে প্রতিটি Span-এর সময় নোট করুন।
4. Service Map থেকে সবচেয়ে ধীর নির্ভরতা চিহ্নিত করুন।
5. একটি Retry-জনিত দুবার চার্জের পরিস্থিতি বর্ণনা করে Idempotency Key-র সমাধান লিখুন।
6. Tracing কী দেখায় না — তিনটি উদাহরণ লিখুন।

অধ্যায় ২১ • PART IV — Distributed Tracing and Advanced Observability

Chapter 21 — Application Performance Monitoring and Profiling

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: **APM** (Application Performance Monitoring) কী ও তা Trace থেকে কীভাবে আলাদা তা ব্যাখ্যা করতে; API Latency, Throughput ও Error Rate বিশ্লেষণ করতে; Database Query Latency, Memory, CPU Profiling ও **Event Loop** বিলম্ব ব্যাখ্যা করতে; Garbage Collection, Connection Pool Saturation ও ধীর বাইরের নির্ভরতা চিনতে; এবং Node.js-এ Continuous Profiling-এর ধারণা দিয়ে Bottleneck খুঁজতে।

২১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Trace বলে একটি অনুরোধ কোথায় সময় নিল; APM বলে পুরো অ্যাপটি কেমন চলছে — সব অনুরোধ মিলিয়ে। একটি API ধীর হতে পারে ডেটাবেসের জন্য, মেমরি লিকের জন্য, Event Loop আটকে থাকার জন্য, বা Connection Pool শেষ হওয়ার জন্য — কারণ ভিন্ন, সমাধান ভিন্ন। APM ছাড়া আপনি শুধু "সার্ভার ধীর" জানেন, কেন ধীর তা জানেন না। TechNova-র প্রোডাকশনে মেমরি লিক ধরা পড়েছিল APM-এর Memory Graph-এ — Trace-এ তা দেখা যেত না, কারণ প্রতিটি অনুরোধ আলাদাভাবে ঠিক ছিল, কিন্তু সময়ের সঙ্গে মেমরি বাড়ছিল।

২১.৩ পূর্বশর্ত জ্ঞান

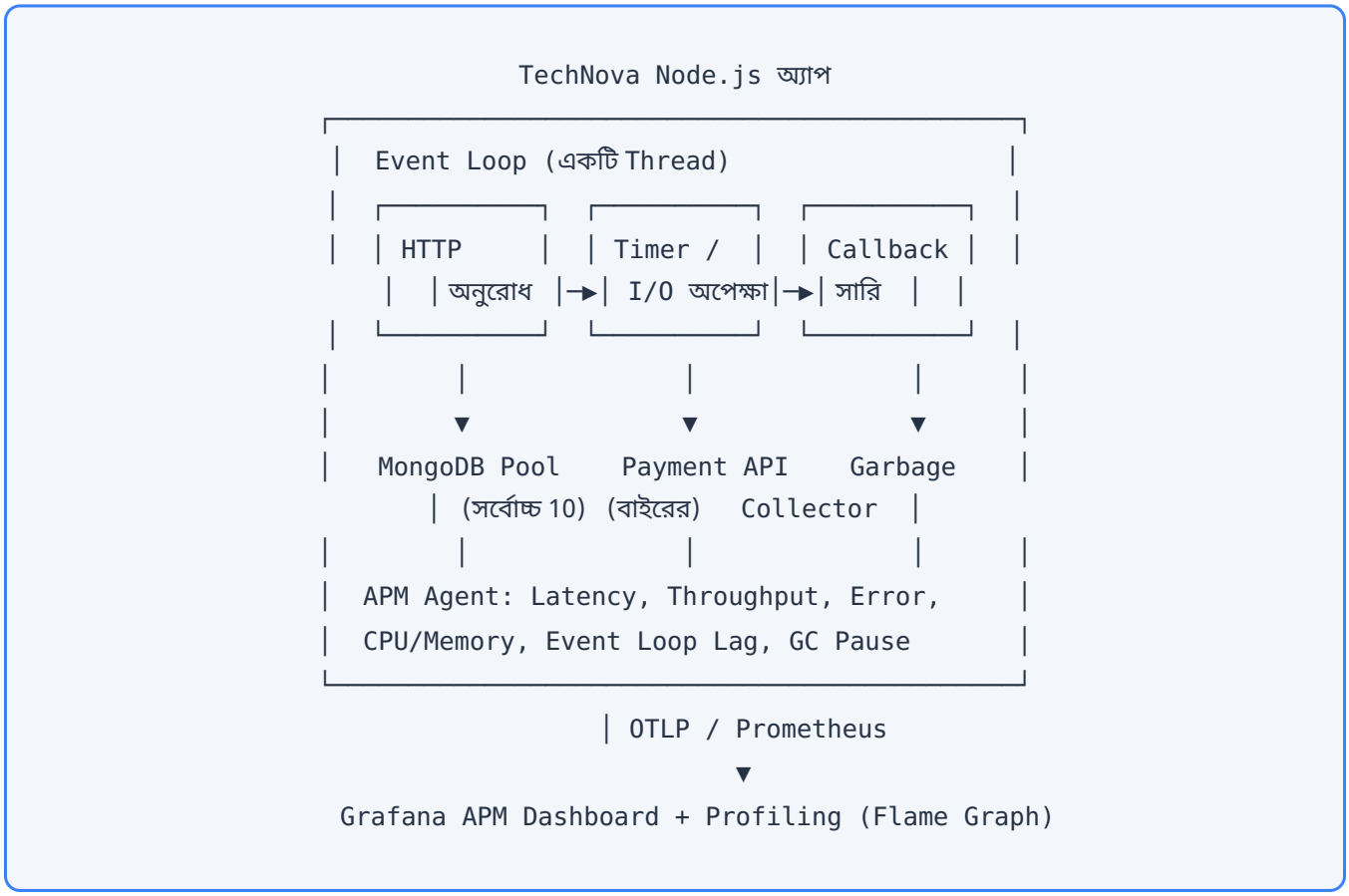
অধ্যায় ১৭–২০-এর Trace ও Correlation; Node.js-এর মৌলিক ধারণা (Single Thread, Event Loop-এর নাম শুনে থাকা); MongoDB Query-এর ধারণা; Prometheus Metric-এর ধারণা। Profiling আগে না জানলেও চলবে।

২১.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি অ্যাপের কর্মক্ষমতা নির্ভর করে চারটি সম্পদের ওপর — CPU (গণনা), Memory (সংরক্ষণ), I/O (ডেটাবেস/নেটওয়ার্ক অপেক্ষা), ও Event Loop (Node.js-এ কাজের সারি)। যেকোনো ধীরতা এই চারটির একটিতে আটকে থাকে। **APM** এই চারটি সম্পদ ও ব্যবসায়িক মেট্রিক (Latency, Throughput, Error) একসঙ্গে মাপে। **Profiling** আরও গভীরে যায়: CPU Profiling বলে কোন ফাংশনে CPU সময় গেল; Memory Profiling (Heap Snapshot) বলে কোন Object মেমরি ধরে রেখেছে। Trace অনুরোধের গল্প বলে, Profiling কোডের ভেতরের গল্প বলে — দুটি একসঙ্গে ব্যবহার করতে হয়।

> **CONCEPT:** Trace হলো রোগীর দিনলিপি (কখন কী হলো), APM হলো নিয়মিত স্বাস্থ্য পরীক্ষা (রক্তচাপ, ওজন — সময়ের সঙ্গে প্রবণতা), Profiling হলো স্ক্যান (শরীরের ভেতরে কোথায় সমস্যা)। তিনটি ছাড়া রোগ নির্ণয় অসম্পূর্ণ।

২১.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



Node.js Single Thread-এ চলে: Event Loop-এ একটি ভারী গণনা (যেমন বড় JSON পার্স) আটকে গেলে সব অনুরোধ অপেক্ষা করে — একে **Event Loop Lag** বলে।

২১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

APM-এর মূল মেট্রিক: Latency, Throughput, Error Rate

Latency: একটি অনুরোধে কত সময় লাগে — গড় (average) নয়, p50/p95/p99 দেখুন; গড় অস্বাভাবিক মান লুকিয়ে ফেলে। **Throughput:** প্রতি সেকেন্ডে কত অনুরোধ (RPS) — হঠাৎ কমে গেলে সার্ভিস থেমে যাচ্ছে বুঝুন। **Error Rate:** ৫xx বা ব্যর্থ অনুরোধের হার — ১% থেকে ৫% ওঠা মানে কিছু ভাঙছে। এই তিনটি একসঙ্গে দেখুন: Latency বাড়ল কিন্তু Throughput একই মানে ধীরতা; Throughput কমল মানে সার্ভিস সাড়া দিচ্ছে না।

Database Query Latency

ধীর Query-র লক্ষণ: একটি Span-এ `db.statement` দেখে Query চিনুন, তার Duration দেখুন। MongoDB-তে Index না থাকলে Collection Scan হয় — হাজার হাজার ডকুমেন্ট পড়ে একটি খুঁজতে হয়। `explain()` দিয়ে Query Plan দেখুন। N+1 সমস্যা: একটি তালিকার প্রতিটি আইটেমের জন্য আলাদা Query — Trace-এ একই Span বারবার দেখা যায়; সমাধান: এক Query-তে সব আনা (batching)।

Memory, Garbage Collection ও Memory Leak

Node.js-এ অব্যবহৃত Object **Garbage Collection** (GC) মুছে ফেলে। Memory Graph সময়ের সঙ্গে ধীরে ধীরে বাড়তে থাকলে ও GC-র পরও না কমলে — **Memory Leak** সন্দেহ করুন। সাধারণ কারণ: Global Cache-এ সীমাহীন জমা,

Event Listener না সরানো, বন্ধ না করা Timer/Connection। Heap Snapshot নিয়ে কোন Object বাড়ছে তা দেখুন। GC Pause বেশি হলে Event Loop থেমে যায় — Latency বাড়ে।

CPU Profiling ও Event Loop Delay

CPU Profiling বলে কোন ফাংশনে CPU সময় গেল — **Flame Graph**-এ চওড়া বার মানে বেশি সময়। Node.js-এ ভারী Synchronous কাজ (বড় লুপ, Crypto, JSON পার্স) Event Loop আটকে দেয় — **Event Loop Lag** মেট্রিক তা মাপে (যেমন ১০০ মিলিসেকেন্ডের বেশি Lag মানে সমস্যা)। সমাধান: ভারী কাজ Worker Thread-এ বা আলাদা সার্ভিসে সরানো।

Connection Pool Saturation

ডেটাবেস সংযোগ সীমিত — যেমন Pool-এ ১০টি সংযোগ। ১০টি একসঙ্গে ব্যবহৃত হলে ১১তম অনুরোধ অপেক্ষা করে — Trace-এ ডেটাবেস Span-এর আগে অপেক্ষার সময় দেখা যায়। লক্ষণ: ডেটাবেস দ্রুত, কিন্তু অনুরোধ ধীর; Pool Wait Time বাড়ে। সমাধান: Pool আকার ঠিক করা, Query দ্রুত করা, বা সংযোগ ফেরত দেওয়া (leak) ঠিক করা।

ধীর বাইরের নির্ভরতা ও Continuous Profiling

Payment Gateway বা Email Service ধীর হলে আপনার অ্যাপও ধীর — Trace-এ বাইরের Span দীর্ঘ দেখায়। Timeout ও Circuit Breaker (বারবার ব্যর্থ হলে সাময়িকভাবে ডাকা বন্ধ) ব্যবহার করুন। **Continuous Profiling** মানে প্রোডাকশনে সবসময় হালকা Profiling চালু রাখা — হঠাৎ Regression ধরা পড়ে, শুধু সমস্যার সময় Profiling চালু করার অপেক্ষা করতে হয় না।

সমস্যা	লক্ষণ	যাচাই
ধীর Query	DB Span দীর্ঘ	explain(), Index যাচাই
Memory Leak	Memory সময়ের সঙ্গে বাড়ে	Heap Snapshot
Event Loop আটকে	Event Loop Lag বেশি	CPU Profile / Flame Graph
Pool Saturation	DB দ্রুত, অনুরোধ ধীর	Pool Wait Time
ধীর বাইরের API	বাইরের Span দীর্ঘ	Timeout, Circuit Breaker
N+1 Query	একই Span বারবার	Trace-এ Span গণনা

২১.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# Node.js Event Loop Lag ও Memory মাপা (নমুনা কোড — চিত্রণমূলক)
const { monitorEventLoopDelay } = require('perf_hooks'); // 1. Node.js-এর নিজস্ব মাপক
const h = monitorEventLoopDelay({ resolution: 20 });      // 2. 20ms রেজোলিউশনে Lag মাপা
h.enable();                                                // 3. মাপা শুরু
setInterval(() => {
  console.log('eventLoopLag p99 (ms):', h.percentile(99) / 1e6); // 4. p99 Lag দেখা
  console.log('heapUsed (MB):', process.memoryUsage().heapUsed / 1024 / 1024); // 5. Heap
ব্যবহার
}, 30000);
# Event Loop Lag p99 > 100ms হলে তদন্ত করুন; Heap ক্রমাগত বাড়লে Leak সন্দেহ করুন
```

```
# CPU Profiling — Node.js Inspector দিয়ে (ল্যাব)
node --inspect --cpu-prof --cpu-prof-dir=./profiles app.js
# --cpu-prof : CPU Profile তৈরি করা; প্রোফাইল ফাইল .cpuprofile — Chrome DevTools-এ খুলে Flame Graph দেখুন
# প্রোডাকশনে ভারী Profiling চালাবেন না — Continuous Profiling টুলের হালকা Agent ব্যবহার করুন
```

২১.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র APM Dashboard-এর জন্য Prometheus Metric-এর উদাহরণ — Express অ্যাপে `prom-client` দিয়ে HTTP Histogram ও Event Loop Lag:

```
// apm-metrics.js — TechNova Order Service
const client = require('prom-client');
const httpDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'HTTP অনুরোধের সময়কাল',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.05, 0.1, 0.3, 0.5, 1, 2, 5], // Latency Bucket — p95/p99 গণনার ভিত্তি
});
const poolWait = new client.Gauge({
  name: 'db_pool_wait_seconds', help: 'ডেটাবেস Pool-এ অপেক্ষার সময়',
});
module.exports = { httpDuration, poolWait };
// Grafana-তে: histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
// দিয়ে p95 Latency দেখুন; Route ও Status Code দিয়ে ভাগ করুন
```

২১.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র সার্ভার প্রতি ৩ দিনে Restart হতো — Memory শেষ হয়ে যেত। প্রতিটি অনুরোধের Trace স্বাভাবিক ছিল। APM-এর Memory Graph-এ দেখা গেল Heap প্রতিদিন ২০০ MB করে বাড়ছে, GC-র পরও কমছে না। Heap Snapshot-এ দেখা গেল একটি Cache Object — Order-এর হিসাব Cache-এ জমা হচ্ছে, কিন্তু কখনো মোছা হচ্ছে না, সীমাও নেই। সমাধান: Cache-এ TTL (Time To Live) ও সর্বোচ্চ আকার (max size) দেওয়া হলো — Restart বন্ধ হলো। এই সমস্যা Trace দিয়ে ধরা যেত না — এটি সময়ের প্রবণতা (trend), একটি অনুরোধের গল্প নয়।

> **REAL WORLD:** "সার্ভার ধীর" শুনে প্রথমে জিজ্ঞেস করুন: CPU বেশি, নাকি Memory বাড়ছে, নাকি Event Loop Lag বেশি, নাকি ডেটাবেস ধীর? চারটির উত্তর চারটি আলাদা সমাধান — অনুমানে Restart দিলে সমস্যা ফিরে আসে।

২১.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Express অ্যাপে HTTP Duration Histogram বসান ও Grafana-তে p50/p95/p99 দেখুন।
2. Event Loop Lag মাপার কোড যোগ করে ৩০ সেকেন্ড পরপর Log করুন।
3. একটি ভারী Synchronous লুপ (ইচ্ছাকৃত) যোগ করে Event Loop Lag বাড়তে দেখুন, তারপর সরান।
4. MongoDB-তে একটি ধীর Query তৈরি করে Trace-এ তার Span ও explain() দেখুন।
5. একটি সীমাহীন Cache যোগ করে Memory বাড়তে দেখুন, তারপর TTL দিয়ে ঠিক করুন।
6. Flame Graph-এ (CPU Profile) সবচেয়ে চওড়া বারটি চিহ্নিত করুন।

২১.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "গড় Latency ঠিক আছে মানে সব ঠিক।" — গড় p99 লুকায়; ১% ব্যবহারকারী ৫ সেকেন্ড অপেক্ষা করলেও গড় ভালো দেখাতে পারে।
- **ভুল:** Memory বাড়লেই Restart দেওয়া — Leak-এর কারণ না খুঁজে Restart করলে সমস্যা ফিরে আসে।
- **ভুল:** প্রোডাকশনে ভারী Profiling চালু রাখা — Profiling-এর নিজের খরচে অ্যাপ ধীর হয়।
- **ভুল:** N+1 Query-কে "ডেটাবেস ধীর" বলা — সমস্যা Query-র সংখ্যায়, ডেটাবেসে নয়।
- **ভুল ধারণা:** "APM মানেই Trace।" — APM বৃহত্তর: Metric, Profiling, Error Tracking — Trace তার একটি অংশ।

২১.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Profiling ডেটায় ফাংশন নাম, Query ও কখনো কখনো ডেটা থাকতে পারে — Heap Snapshot-এ User-এর তথ্য থাকতে পারে। Snapshot শুধু সুরক্ষিত জায়গায় সংরক্ষণ করুন, প্রবেশাধিকার সীমিত রাখুন। APM Dashboard-এ ব্যক্তিগত তথ্য (ইমেইল, ঠিকানা) Label হিসেবে ব্যবহার করবেন না — High Cardinality-ও হয়, গোপনীয়তাও ভাঙে।

২১.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

APM Agent-এর খরচ সাধারণত ১–৫% CPU; ভারী Profiling ১০% ছাড়াতে পারে — তাই Continuous Profiling হালকা Sampling-এ চালান। Histogram Bucket-এর সংখ্যা বেশি হলে Prometheus Storage বাড়ে — Bucket সীমিত রাখুন। Profiling ডেটা Retention কম রাখুন (যেমন ৭ দিন), কারণ তা বড় হয়।

২১.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
p99 হঠাৎ বেড়েছে, p50 ঠিক	কিছু অনুরোধে ধীর নির্ভরতা	ধীর Trace-এর Exemplar দেখুন
Memory ক্রমাগত বাড়ছে	Memory Leak	Heap Snapshot নিয়ে বড় Object খুঁজুন
CPU ১০০%, Throughput কম	ভারী Synchronous কাজ	Flame Graph-এ চওড়া বার দেখুন
DB দ্রুত, অনুরোধ ধীর	Pool Saturation	Pool Wait Time ও Pool আকার দেখুন
হঠাৎ Error Rate বেড়েছে	ডিপ্লয় বা নির্ভরতা ভাঙা	ডিপ্লয় সময়ের সঙ্গে মিলিয়ে দেখুন

TROUBLESHOOTING: Latency বাড়লে প্রথমে ভাগ করুন: ডেটাবেস Span, বাইরের Span, নিজের কাজ (Self Time) — কোনটি বেড়েছে? তারপর সেই অংশের গভীরে যান। সব একসঙ্গে তদন্ত করবেন না।

PRO TIP: প্রতিটি ডিপ্লয়ের পর p95 ও Error Rate ১৫ মিনিট দেখুন। Regression-এর সবচেয়ে ভালো ধরা পড়ে ডিপ্লয়ের ঠিক পরে — তখন Profiling চালু রাখুন।

২১.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: p50/p95/p99 দেখুন, গড় নয়; Event Loop Lag ও Pool Wait Time Dashboard-এ রাখুন; Cache-এ TTL ও সীমা দিন; বাইরের API-তে Timeout ও Circuit Breaker দিন; ডিপ্লয়ের পর APM দেখুন। **অ্যান্টি-প্যাটার্ন:** গড় Latency-তে ভরসা; Leak না খুঁজে Restart; ভারী কাজ Event Loop-এ রাখা; N+1 Query; Profiling ছাড়া "অনুমানে" অপ্টিমাইজ করা।

২১.১৬ অধ্যায়ের সারাংশ

APM অ্যাপের সামগ্রিক স্বাস্থ্য মাপে — Latency (p95/p99), Throughput, Error Rate — আর Profiling কোডের ভেতরে কোথায় সময় ও মেমরি যাচ্ছে তা দেখায়। Event Loop Lag, GC, Pool Saturation ও ধীর বাইরের নির্ভরতা — প্রতিটির লক্ষণ ও সমাধান আলাদা। Trace অনুরোধের গল্প বলে, APM ও Profiling অ্যাপের গল্প বলে — দুটি একসঙ্গে ব্যবহার করুন।

২১.১৭ মূল প্রযুক্তিগত পরিভাষা

APM: Application Performance Monitoring — অ্যাপের কর্মক্ষমতা পর্যবেক্ষণ।

Latency: একটি অনুরোধ সম্পূর্ণ হতে কত সময় লাগে।

Throughput: প্রতি সেকেন্ডে কত অনুরোধ সামলানো হয়।

Error Rate: ব্যর্থ অনুরোধের হার।

Profiling: কোডের কোন অংশে CPU/Memory ব্যয় হচ্ছে তা মাপা।

Flame Graph: CPU সময়ের চিত্র — চওড়া বার মানে বেশি সময়।

Event Loop Lag: Node.js Event Loop-এ কাজ শুরু হতে বিলম্ব।

Garbage Collection (GC): অব্যবহৃত Object মুছে মেমরি ফেরত দেওয়া।

Memory Leak: অব্যবহৃত মেমরি ফেরত না পাওয়া — সময়ের সঙ্গে মেমরি বাড়ে।

Connection Pool Saturation: ডেটাবেস সংযোগ Pool শেষ হয়ে অনুরোধ অপেক্ষা করা।

Continuous Profiling: প্রোডাকশনে সবসময় হালকা Profiling চালু রাখা।

Circuit Breaker: বারবার ব্যর্থ নির্ভরতাকে সাময়িকভাবে ডাকা বন্ধ করার প্যাটার্ন।

২১.১৮ জ্ঞান যাচাই — প্রশ্ন

1. APM ও Distributed Tracing-এর পার্থক্য কী?
2. গড় Latency ভালো দেখালেও p99 খারাপ হতে পারে কেন?
3. Event Loop Lag কী? এটি বাড়লে কী হয়?
4. Memory Leak-এর লক্ষণ কী? Heap Snapshot কীভাবে সাহায্য করে?
5. N+1 Query সমস্যা Trace-এ কীভাবে চেনা যায়?
6. Connection Pool Saturation-এর লক্ষণ কী? ডেটাবেস দ্রুত হলেও অনুরোধ ধীর হতে পারে কেন?
7. Flame Graph-এ চওড়া বারের অর্থ কী?
8. Continuous Profiling-এর সুবিধা কী?

২১.১৯ ব্যবহারিক অনুশীলন

1. Express অ্যাপে HTTP Histogram বসিয়ে p95 বের করুন।
2. Event Loop Lag মাপুন ও একটি ভারী Synchronous কাজের প্রভাব দেখুন।
3. একটি ধীর MongoDB Query-র explain() দেখে Index-এর প্রভাব লিখুন।
4. সীমাহীন Cache-এ Memory Leak তৈরি করে Heap ব্যবহারের Graph আঁকুন, তারপর TTL দিয়ে ঠিক করুন।
5. CPU Profile নিয়ে Flame Graph-এ সবচেয়ে ব্যয়বহুল ফাংশন চিহ্নিত করুন।
6. একটি বাইরের API-তে Timeout ও Circuit Breaker-এর নকশা লিখুন।

PART V • DEVOPS ENGINEERING MASTERY

Production Observability Engineering

Incident Detection, Alert Fatigue, On-call, Docker/Kubernetes/Cloud Observability, এন্টারপ্রাইজ আর্কিটেকচার ও নির্ভরযোগ্যতা।

অধ্যায় ২২ • PART V — Production Observability Engineering

Chapter 22 — Incident Detection, Alert Fatigue, and On-call Engineering

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: **Incident** (সেবা ব্যাহত হওয়ার ঘটনা) শনাক্ত করার নীতি ব্যাখ্যা করতে; **Alert Fatigue** (অতিরিক্ত Alert-এ ক্লান্তি) কী ও কেন বিপজ্জনক তা বলতে; কার্যকর (actionable) Alert লিখতে — যে Alert একটি করণীয় অবস্থা বোঝায়, শুধু Threshold পার হওয়া নয়; Severity ঠিক করতে; On-call দায়িত্ব, Incident Lifecycle, Escalation ও **Postmortem** (ব্লেমলেস পর্যালোচনা) ব্যাখ্যা করতে।

২২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

পর্যবেক্ষণ ব্যবস্থার উদ্দেশ্য Dashboard সাজানো নয় — সমস্যা ধরা ও মানুষকে সময়মতো জানানো। কিন্তু Alert বেশি হলে উল্টো হয়: On-call ইঞ্জিনিয়ার রাতে ৩০টি Alert পেয়ে সব উপেক্ষা করতে শেখেন, তারপর আসল Incident-এর Alert-ও উপেক্ষা করেন — একে Alert Fatigue বলে, এটি প্রোডাকশন দুর্ঘটনার অন্যতম প্রধান কারণ। TechNova-তে একবার CPU ৮০% ছাড়ালেই Alert আসত — দিনে ২০ বার; কেউ আর দেখত না। তারপর একদিন সত্যিকারের ডিস্ক পূর্ণ হওয়ার Alert সেই ভিড়ে হারিয়ে গেল। ভালো Alerting কম Alert, কিন্তু সঠিক Alert — এই অধ্যায় সেই শিল্প শেখায়।

২২.৩ পূর্বশর্ত জ্ঞান

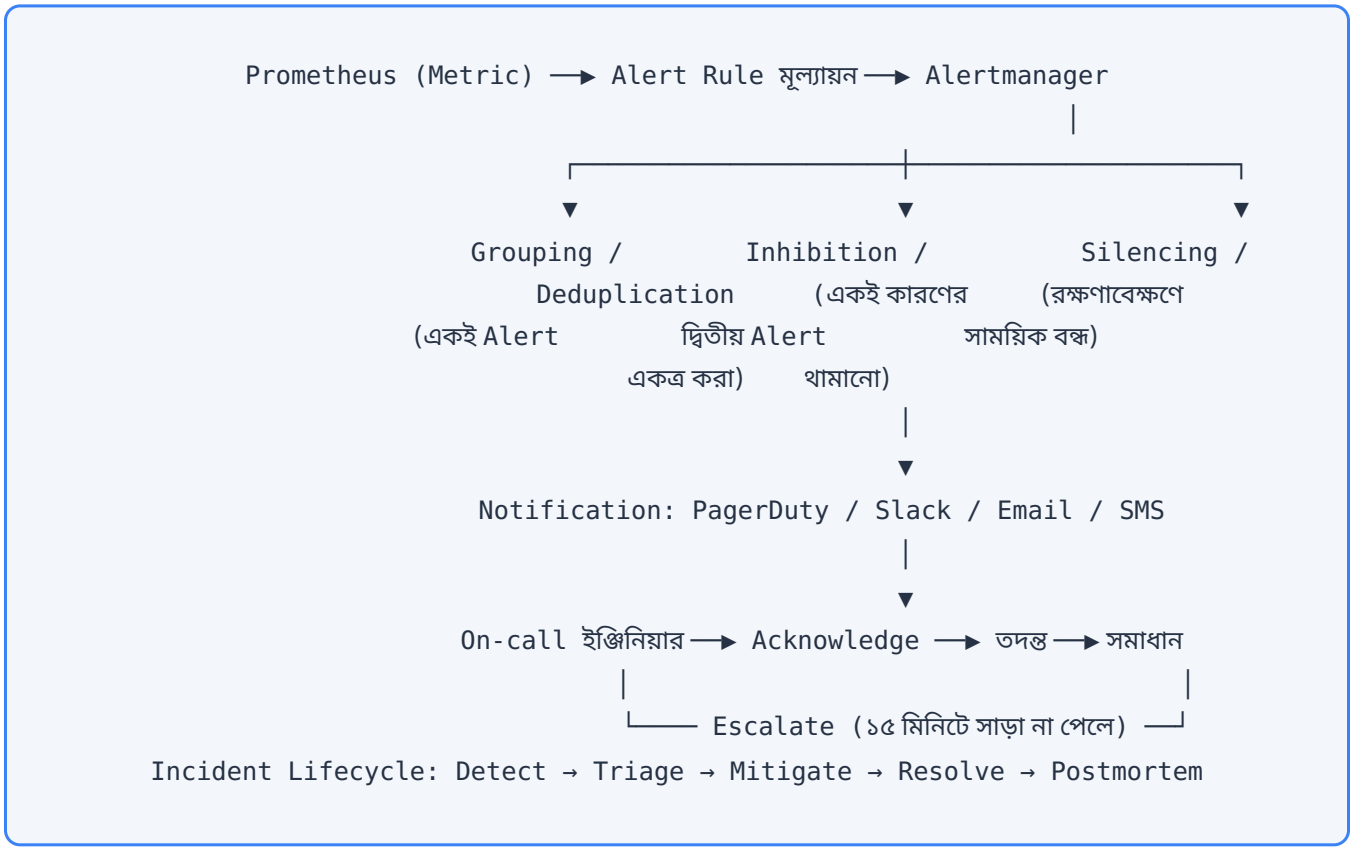
Prometheus, Alertmanager ও Grafana-র মৌলিক ধারণা; SLI/SLO-র নাম শুনে থাকা (বিস্তারিত অধ্যায় ২৪-এ); TechNova-র সেবা কীভাবে চলে তার ধারণা। On-call অভিজ্ঞতা লাগবে না।

২২.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: একটি Alert হলো মানুষের মনোযোগ চাওয়ার অনুরোধ — মনোযোগ সীমিত সম্পদ। প্রতিটি Alert-এর খরচ আছে: ঘুম ভাঙা, কাজ থামানো, মানসিক চাপ। তাই Alert পাঠানো উচিত শুধু তখনই, যখন (১) ব্যবহারকারী ক্ষতিগ্রস্ত হচ্ছেন বা হতে যাচ্ছেন, (২) এখনই কিছু করার আছে, (৩) তা না করলে ক্ষতি বাড়বে। CPU ৮০% — এতে ব্যবহারকারী ক্ষতিগ্রস্ত হচ্ছেন না, এখনই কিছু করার নেই — এটি Alert নয়, এটি Dashboard-এ দেখার Metric। "Checkout API-র Error Rate ৫ মিনিটে ৫% ছাড়িয়েছে" — ব্যবহারকারী ক্ষতিগ্রস্ত হচ্ছেন, এখনই তদন্ত দরকার — এটি Alert। এই পার্থক্যই Actionable Alert-এর মূলনীতি: একটি *Alert* একটি করণীয় অবস্থা বোঝাবে, শুধু *Threshold* পার হওয়া নয়।

> **CONCEPT:** Alert হলো ফায়ার অ্যালার্ম — আগুন লাগলে বাজবে, রান্নাঘরে ধোঁয়া উঠলেই নয়। প্রতিদিন ভুল অ্যালার্ম বাজলে মানুষ অ্যালার্ম শুনে আর বেরোয় না — তখন আসল আগুনে সব পুড়ে যায়।

২২.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



২২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Incident Detection — কীভাবে ধরা পড়ে

Incident ধরা পড়ে তিন পথে: (১) **Alert** (স্বয়ংক্রিয় — Metric/Log/Trace থেকে), (২) **ব্যবহারকারীর অভিযোগ** (Support টিকিট, সোশ্যাল মিডিয়া), (৩) **Synthetic Monitoring** (বাইরে থেকে নিয়মিত পরীক্ষামূলক অনুরোধ পাঠিয়ে দেখা সেবা চলছে কি না)। ভালো ব্যবস্থায় Alert-ই প্রথমে ধরে — ব্যবহারকারী জানার আগেই। Detection-এর গতি (Time to Detect) কমানোই Alerting-এর লক্ষ্য।

Alert Fatigue ও তার প্রতিকার

Alert Fatigue হলো অতিরিক্ত, অপ্রয়োজনীয় বা ভুল Alert-এ মানুষের সংবেদনশীলতা হারানো। লক্ষণ: Alert দেখে আর তড়িঘড়ি করা হয় না, Alert Mute করে রাখা হয়, On-call-এ কেউ থাকতে চায় না। প্রতিকার: (১) শুধু Actionable Alert রাখুন — বাকিগুলো Dashboard বা Ticket-এ নামান; (২) Threshold-এ `for: 5m` দিন — ক্ষণিকের স্পাইকে Alert নয়; (৩) Grouping করুন — একই কারণের ৫০টি Alert একটিতে; (৪) নিয়মিত Alert পর্যালোচনা (Alert Review) — কোন Alert-এ কেউ সাড়া দেয়নি, তা মুছে ফেলুন।

Alert Severity

Severity মানে কত জরুরি: **SEV-1 (Critical):** সেবা বন্ধ বা তথ্য হারানোর ঝুঁকি — এখনই সব কাজ ফেলে সাড়া দিন, পেজার বাজবে। **SEV-2 (High):** সেবা আংশিক ব্যাহত — কর্মঘণ্টায় দ্রুত, রাতে পরদিন সকালে নয়, নির্দিষ্ট সময়ে সাড়া। **SEV-3 (Medium):** অবনতি হচ্ছে, কিন্তু সেবা চলছে — Ticket-এ, কর্মঘণ্টায়। **SEV-4 (Low):** তথ্যমূলক — Dashboard-এ দেখুন। Severity ঠিক না করলে সব Alert-ই Critical মনে হয়, আর কোনোটিই Critical থাকে না।

On-call দায়িত্ব

On-call ইঞ্জিনিয়ার নির্দিষ্ট সময়ে (যেমন এক সপ্তাহ) সেবার প্রথম সাড়াদানকারী। দায়িত্ব: Alert-এ নির্দিষ্ট সময়ে সাড়া দেওয়া (যেমন SEV-1-এ ৫ মিনিট), Triage করা (কত গুরুতর, কাকে ডাকতে হবে), Mitigate করা (সমস্যা থামানো — পুরো সমাধান নয়), যোগাযোগ রাখা (Status আপডেট), ও হস্তান্তর (Handover) লিখে যাওয়া। On-call টেকসই হতে হয়: সপ্তাহে একজনের বেশি নয়, ঘুমের সময় সুরক্ষা, On-call-এর পর বিশ্রাম, ও ক্ষতিপূরণ/স্বীকৃতি — নইলে ভালো ইঞ্জিনিয়ার চলে যান।

Incident Lifecycle ও Escalation

Lifecycle: **Detect** (ধরা) → **Triage** (গুরুত্ব নির্ধারণ) → **Mitigate** (ক্ষতি থামানো — যেমন ট্রাফিক সরানো, Rollback) → **Resolve** (মূল সমস্যা ঠিক করা) → **Postmortem** (শেখা)। **Escalation:** নির্দিষ্ট সময়ে সাড়া না পেলে পরের ব্যক্তিকে ডাকা; SEV-1-এ ১৫ মিনিটে সমাধান না হলে ব্যবস্থাপক ও বিশেষজ্ঞকে ডাকা। Escalation লজ্জার নয় — এটি নিরাপত্তা জাল।

Root Cause Analysis ও Blameless Postmortem

Root Cause Analysis মানে "কেন" পাঁচবার জিজ্ঞেস করা (5 Whys): সেবা থেমেছে কেন? ডিস্ক পূর্ণ কেন? Log Rotation বন্ধ কেন? কনফিগারেশন বদল কেন? পর্যালোচনা হয়নি কেন? **Blameless Postmortem** মানে দোষারোপ ছাড়া পর্যালোচনা — "কে ভুল করেছে" নয়, "কোন ব্যবস্থা ভুল করতে দিল" — এই প্রশ্ন। মানুষ ভুল করবেই; ব্যবস্থা এমন হওয়া উচিত যাতে একটি ভুলে সেবা না থামে। Postmortem-এ লেখা হয়: কী ঘটেছে (Timeline), প্রভাব, Root Cause, ও Action Item (যার মালিক ও সময়সীমা আছে)। Action Item ছাড়া Postmortem শুধু গল্প।

Severity	অর্থ	সাড়ার সময়
SEV-1 Critical	সেবা বন্ধ / তথ্য ঝুঁকি	৫ মিনিট, পেজার
SEV-2 High	আংশিক ব্যাহত	৩০ মিনিট
SEV-3 Medium	অবনতি, সেবা চলছে	কর্মঘণ্টায় Ticket
SEV-4 Low	তথ্যমূলক	Dashboard

২২.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# Prometheus Alert Rule — Actionable Alert-এর উদাহরণ (YAML)
groups:
  - name: technova-checkout
    rules:
      - alert: CheckoutErrorRateHigh   # 1. Alert-এর নাম — কী সমস্যা, স্পষ্ট
        expr: |                        # 2. PromQL — Error হার
          sum(rate(http_requests_total{route="/api/checkout",status=~"5.."}[5m]))
          / sum(rate(http_requests_total{route="/api/checkout"}[5m])) > 0.05
        for: 5m                        # 3. ৫ মিনিট ধরে থাকলে তবেই Alert — ক্ষণিকের স্পাইক নয়
        labels: { severity: "sev-1", team: "platform" }  # 4. Severity ও দল — রাউটিং-এর জন্য
        annotations:                   # 5. মানুষের জন্য নির্দেশনা
          summary: "Checkout Error Rate 5% ছাড়িয়েছে"
          runbook_url: "https://wiki.technova.example/runbooks/checkout-errors"
          # Runbook লিংক থাকলে রাত ২টায় নতুন ইঞ্জিনিয়ারও শুরু করতে পারেন
```

```
# Alertmanager — Grouping ও Silencing (মূল অংশ)
route: { group_by: ["alertname", "team"], group_wait: 30s, group_interval: 5m,
repeat_interval: 4h }
# group_by: একই Alert একত্র করা; group_wait: প্রথম Alert-এর জন্য ৩০ সেকেন্ড অপেক্ষা;
# repeat_interval: ৪ ঘণ্টা পর আবার জানানো — প্রতি ৫ মিনিটে নয়
# Silencing: রক্ষণাবেক্ষণের সময় amtool silence add alertname="..." --comment="deploy"
```

২২.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র On-call Rotation ও Escalation-এর ধারণাগত নকশা: Primary On-call (১ সপ্তাহ, একজন), Secondary (Primary সাড়া না দিলে ১৫ মিনিট পর), Manager Escalation (SEV-1-এ ৩০ মিনিট পর)। Rotation-এ সপ্তাহান্তে দুজন নয় — একজন Primary, একজন Backup; ছুটির দিনে আগেই বদলি ঠিক করা হয়। Alertmanager-এ Receiver: SEV-1 → PagerDuty + Slack #incidents; SEV-2 → Slack; SEV-3 → Email/Ticket; SEV-4 → Dashboard মাত্র। এই নকশা লিখিত Runbook-এ রাখুন — মুখের কথায় নয়।

২২.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র ডিস্ক ৯০% পূর্ণ হওয়ার Alert দিনে তিনবার আসত — কেউ দেখত না, কারণ ডিস্ক পরিষ্কার করে আবার ৮৫% হতো। একদিন রাতে ডিস্ক ১০০% হলো — MongoDB লিখতে পারল না, Checkout বন্ধ। Postmortem-এ দেখা গেল: Alert-টি Actionable ছিল না — এতে "কী করবেন" লেখা ছিল না, Threshold-ও ভুল ছিল (৯০% নয়, ৭ দিনের পূর্বাভাস দরকার)। সমাধান: Alert বদলে "৭ দিনের মধ্যে ডিস্ক শেষ হবে" (পূর্বাভাস-ভিত্তিক) করা হলো, Runbook-এ পরিষ্কারের ধাপ লেখা হলো, আর Log Rotation স্বয়ংক্রিয় করা হলো — যাতে Alert-ই না আসে। ভালো Incident ব্যবস্থা Alert কমায়, বাড়ায় না।

> **WARNING:** On-call-এ ঘুমন্ত মানুষকে ডাকার আগে জিজ্ঞেস করুন: ব্যবহারকারী কি এখন ক্ষতিগ্রস্ত হচ্ছেন? এখনই কিছু করার আছে কি? দুটির উত্তর "না" হলে সকালে Ticket করুন — রাতে ডাকবেন না। ঘুমন্ত ইঞ্জিনিয়ার ভুল সিদ্ধান্ত নেন, আর ক্লান্ত দল ভালো সেবা দিতে পারে না।

২২.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. বর্তমান সব Alert-এর তালিকা করুন — কোনটিতে গত মাসে কেউ সাড়া দেয়নি, তা চিহ্নিত করুন।
2. প্রতিটি Alert-কে প্রশ্ন করুন: ব্যবহারকারী ক্ষতিগ্রস্ত হচ্ছেন? এখনই করণীয় আছে? না হলে Dashboard/Ticket-এ নামান।
3. বাকি Alert-গুলোতে `for: 5m`, Severity Label ও Runbook URL যোগ করুন।
4. Alertmanager-এ Grouping চালু করুন — একই কারণের Alert একত্র করুন।
5. একটি On-call Rotation লিখুন: Primary, Secondary, Escalation সময়।
6. একটি কাল্পনিক Incident-এর মহড়া (Game Day) করুন ও Postmortem লিখুন — Blameless ভাষায়।

২২.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "বেশি Alert মানেই ভালো পর্যবেক্ষণ।" — উল্টো: বেশি Alert মানে Fatigue, আর আসল Alert হারিয়ে যায়।

- **ভুল:** Threshold-এ Alert, `for` ছাড়া — ক্ষণিকের স্পাইকে রাতে ডাকা হয়।
- **ভুল:** Alert-এ Runbook না থাকা — রাত ২টায় নতুন ইঞ্জিনিয়ার জানেন না কী করবেন।
- **ভুল:** Postmortem-এ দোষারোপ — তাতে মানুষ ভুল লুকোতে শেখে, শেখে না।
- **ভুল:** Mitigate না করে Root Cause খোঁজা — প্রথমে ক্ষতি থামান (Rollback/ট্রাফিক সরানো), তারপর কারণ খুঁজুন।

২২.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Alert-এর বার্তায় (Slack/Email) সংবেদনশীল তথ্য (User-এর ইমেইল, Order-এর বিবরণ, Token) লিখবেন না — Alert অনেক চ্যানেলে যায়। PagerDuty-ধরনের টুলে প্রবেশাধিকার নিয়ন্ত্রণ করুন — কে On-call, কে Incident দেখতে পারে। Incident-এর Timeline-এ ব্যক্তিগত তথ্য থাকলে Postmortem শেয়ার করার আগে মুছে ফেলুন।

২২.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Alerting-এর আর্থিক খরচ কম, মানবিক খরচ বেশি: On-call ক্লান্তি, চাকরি ছাড়া, ভুল সিদ্ধান্ত। Alert কমানোই সবচেয়ে সস্তা উন্নতি। PagerDuty-ধরনের সেবার খরচ ব্যবহারকারী সংখ্যা অনুযায়ী — ছোট দলে Slack + Alertmanager যথেষ্ট হতে পারে। Incident-এর খরচ মাপুন: Downtime × ব্যবহারকারী — এই সংখ্যাই Alerting-এ বিনিয়োগের যুক্তি।

২২.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Alert আসে না, সেবা বন্ধ	Rule-এর Expr ভুল, বা Threshold বেশি	Prometheus-এ Expr হাতে চালিয়ে দেখুন
Alert আসে কিন্তু কেউ পায় না	Receiver/রাউটিং ভুল	Alertmanager UI-তে Route পরীক্ষা করুন
একই Alert বারবার আসে	repeat_interval কম, বা সমস্যা ঠিক হয়নি	Grouping ও Repeat Interval ঠিক করুন
Alert Fatigue — সবাই উপেক্ষা করে	অপ্রয়োজনীয় Alert বেশি	Alert Review করে Actionable ছাড়া বাকি মুছুন
Escalation হয় না	Secondary ঠিক করা হয়নি	Rotation ও Escalation নীতি যাচাই করুন

TROUBLESHOOTING: Alert আসছে না মনে হলে Prometheus UI-তে Alerts পাতায় Rule-এর অবস্থা দেখুন: Inactive (শর্ত পূরণ হয়নি), Pending (for সময় চলছে), Firing (Alertmanager-এ গেছে)। কোন ধাপে আটকে আছে, সেখানেই সমস্যা।

২২.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: শুধু Actionable Alert; প্রতিটি Alert-এ Runbook; Severity ঠিক করা; নিয়মিত Alert Review; Blameless Postmortem ও Action Item; Mitigate আগে, Root Cause পরে; On-call টেকসই রাখা। **অ্যান্টি-প্যাটার্ন:** সব Metric-এ Alert; `for` ছাড়া Alert; দোষারোপের Postmortem; একই ব্যক্তিকে মাসের পর মাস On-call; Alert-এ সংবেদনশীল তথ্য।

২২.১৬ অধ্যায়ের সারাংশ

ভালো Incident ব্যবস্থা মানে কম, কিন্তু সঠিক Alert: প্রতিটি Alert একটি করণীয় অবস্থা, Severity-সহ, Runbook-সহ। Alert Fatigue এড়াতে Grouping, `for` সময় ও নিয়মিত পর্যালোচনা দরকার। On-call টেকসই হতে হয়, Escalation নিরাপত্তা জাল, আর Blameless Postmortem-ই আসল শেখার জায়গা — Action Item ছাড়া তা অসম্পূর্ণ।

২২.১৭ মূল প্রযুক্তিগত পরিভাষা

Incident: সেবা ব্যাহত হওয়ার ঘটনা — ব্যবহারকারী ক্ষতিগ্রস্ত হন।

Alert Fatigue: অতিরিক্ত Alert-এ মানুষের সাড়া দেওয়ার ক্ষমতা হারানো।

Actionable Alert: যে Alert একটি করণীয় অবস্থা বোঝায়, শুধু Threshold পার হওয়া নয়।

Severity (SEV): Incident-এর গুরুত্বের স্তর (SEV-1 থেকে SEV-4)।

On-call: নির্দিষ্ট সময়ে Incident-এর প্রথম সাড়াদানকারী ইঞ্জিনিয়ার।

Escalation: সাড়া না পেলে বা সমাধান না হলে পরের স্তরে ডাকা।

Runbook: একটি সমস্যার ধাপে ধাপে সমাধানের লিখিত নির্দেশনা।

Postmortem: Incident-এর পর Blameless পর্যালোচনা ও শেখা।

Root Cause Analysis: সমস্যার মূল কারণ খুঁজে বের করা (যেমন 5 Whys)।

Mitigation: ক্ষতি থামানো — পুরো সমাধান নয়, প্রথম পদক্ষেপ।

Triage: Incident-এর গুরুত্ব ও প্রভাব নির্ধারণ করা।

Grouping: একই কারণের Alert একত্র করা (Alertmanager)।

২২.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Actionable Alert কী? CPU ৮০% Alert কেন সাধারণত Actionable নয়?
2. Alert Fatigue কী? এটি কেন বিপজ্জনক?
3. SEV-1 ও SEV-3-এর পার্থক্য কী? উদাহরণ দিন।
4. `for: 5m` Alert Rule-এ কী করে?
5. Mitigate ও Resolve-এর পার্থক্য কী? কোনটি আগে?
6. Blameless Postmortem কেন দরকার? দোষারোপ করলে কী হয়?
7. Escalation কখন হয়? এটি লজ্জার কেন নয়?
8. একটি Alert-এ Runbook URL থাকা কেন গুরুত্বপূর্ণ?

২২.১৯ ব্যবহারিক অনুশীলন

1. Checkout Error Rate-এর জন্য একটি Actionable Alert Rule লিখুন (for, severity, runbook সহ)।
2. নিজের সিস্টেমের (বা কাল্পনিক) ৫টি Alert-কে Actionable / Dashboard / Ticket-এ ভাগ করুন।
3. একটি On-call Rotation ও Escalation নীতি লিখুন।
4. একটি কাল্পনিক Incident-এর (ডিস্ক পূর্ণ) Timeline ও Blameless Postmortem লিখুন।
5. 5 Whys দিয়ে একটি Incident-এর Root Cause বের করুন।
6. Alertmanager-এ Grouping কনফিগার করে একই Alert-এর বন্যা কমানোর পরিকল্পনা লিখুন।

অধ্যায় ২৩ • PART V — Production Observability Engineering

Chapter 23 — Observability for Docker, Kubernetes, and Cloud Platforms

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: Docker Container-এর Metric ও Log সংগ্রহ করতে; Kubernetes-এ Control Plane, Node ও Pod/Container Metric ব্যাখ্যা করতে; **kube-state-metrics** ও **Prometheus Operator**-এর ধারণা বলতে; AWS/Azure/GCP/IBM Cloud-এর Managed Monitoring, Cloud Log ও Tracing-এর ধারণা উচ্চ-স্তরে ব্যাখ্যা করতে — কোনো মূল্য বা সুবিধার বিষয়ে অনুমান না করে।

২৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Container-এ অ্যাপ চললে হোস্টের Metric আর যথেষ্ট নয় — Container-এর CPU/Memory সীমা (limit), Restart, ও Pod-এর অবস্থা দেখতে হয়। Kubernetes-এ Pod ক্ষণস্থায়ী — একটি Pod মারা গেলে তার Log-ও হারিয়ে যায়, যদি কেন্দ্রীয়ভাবে সংগ্রহ না করা হয়। Cloud-এ সেবা ব্যবহার করলে Provider-এর Managed টুল (যেমন CloudWatch, Azure Monitor) থাকে — সেগুলো না জানলে আপনি Cloud-এর সুবিধা নিতে পারবেন না, আবার সেগুলোর ওপর পুরোপুরি নির্ভর করলেও Vendor Lock-in হয়। TechNova Docker থেকে Kubernetes-এ যাচ্ছে, আর কিছু কাজ Cloud-এ — তিনটি পরিবেশের Observability একসঙ্গে জানা দরকার।

২৩.৩ পূর্বশর্ত জ্ঞান

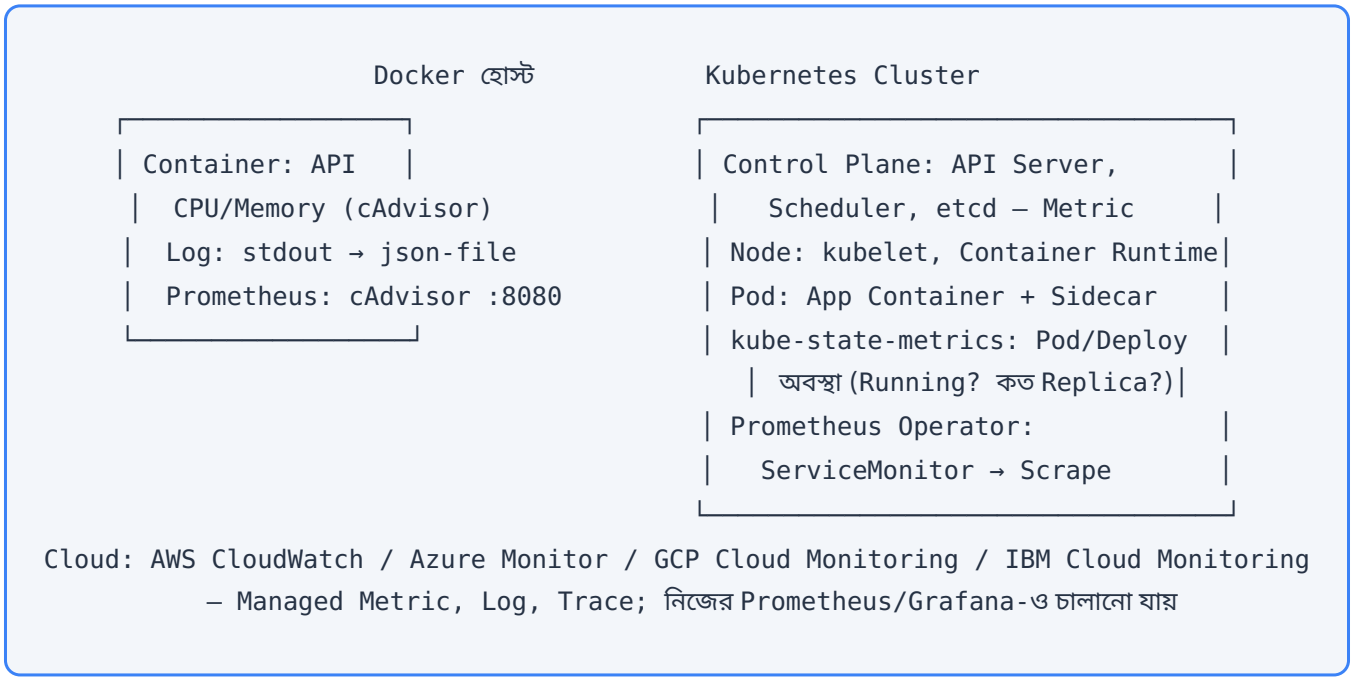
Docker Container-এর মৌলিক ধারণা; Kubernetes-এর Pod, Deployment, Service-এর নাম শুনে থাকা; Prometheus ও Grafana-র ধারণা (আগের অধ্যায়গুলো); Cloud (AWS/Azure/GCP) কী তা সাধারণভাবে জানা। YAML পড়তে পারা।

২৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: পর্যবেক্ষণ সবসময় তিনটি প্রশ্নের উত্তর দেয় — (১) জিনিসটি চলছে কি? (২) তার সম্পদ কতটুকু ব্যবহার হচ্ছে? (৩) সে কী বলছে (Log)? Docker-এ "জিনিসটি" হলো Container; Kubernetes-এ Pod, Node, Control Plane — তিনটি স্তর; Cloud-এ Managed সেবা — যার ভেতরে আপনি ঢুকতে পারেন না, Provider-এর টুল দিয়েই দেখতে হয়। প্রতিটি স্তরে একই তিনটি প্রশ্ন, শুধু টুল বদলায়। Pod ক্ষণস্থায়ী বলে Log ও Metric কেন্দ্রীয়ভাবে সংগ্রহ করা বাধ্যতামূলক — Pod মারা গেলে তার প্রমাণও মারা যায়।

> **CONCEPT:** হোস্ট পর্যবেক্ষণ করা মানে বাড়ি দেখা; Container পর্যবেক্ষণ করা মানে বাড়ির প্রতিটি ঘর দেখা; Kubernetes পর্যবেক্ষণ করা মানে পুরো অ্যাপার্টমেন্ট ভবন — প্রতিটি ফ্ল্যাট (Pod), লিফট (Control Plane), ও বিদ্যুৎ (Node) — একসঙ্গে দেখা।

২৩.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



২৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Docker Observability

Docker-এ প্রতিটি Container-এর CPU, Memory, Network ও Disk I/O **cAdvisor** (Container Advisor) দিয়ে মাপা যায় — Prometheus তা Scrape করে। `docker stats` দিয়ে তাৎক্ষণিক দেখা যায়, কিন্তু ইতিহাসের জন্য Prometheus দরকার। Log: Container-এর `stdout/stderr` Docker-এর Logging Driver দিয়ে সংগ্রহ করা হয় — `json-file` ডিফল্ট, কিন্তু প্রোডাকশনে কেন্দ্রীয় Log (Loki/Elasticsearch)-এ পাঠানো উচিত। Container-এর Healthcheck (`HEALTHCHECK`) দিয়ে সেবা সাড়া দিচ্ছে কি না জানা যায়। মনে রাখুন: Container মুছে গেলে তার স্থানীয় Log-ও মুছে যায়।

Kubernetes Observability — তিনটি স্তর

Control Plane: API Server, Scheduler, Controller Manager, etcd — এগুলোর Metric (যেমন API অনুরোধের latency, etcd-র স্বাস্থ্য) Cluster-এর স্বাস্থ্য বলে। **Node:** kubelet ও Container Runtime-এর Metric — Node-এর CPU/Memory/ডিস্ক, Pod-এর সংখ্যা। **Pod/Container:** প্রতিটি Pod-এর CPU/Memory ব্যবহার, Restart সংখ্যা, Readiness/Liveness Probe-এর ফল। **kube-state-metrics** Kubernetes API থেকে Object-এর অবস্থা (যেমন Deployment-এ কত Replica চাওয়া হয়েছে, কত চলছে) Metric আকারে দেয় — এটি অ্যাপের Metric নয়, Cluster-এর অবস্থার Metric। **Prometheus Operator** হলো Kubernetes-এ Prometheus চালানোর Operator — **ServiceMonitor** Custom Resource দিয়ে বলে দেয় কোন Service-এর Metric Scrape করতে হবে, হাতে Prometheus কনফিগারেশন না লিখে।

Cloud Platforms — Managed Monitoring (উচ্চ-স্তর)

প্রতিটি বড় Cloud Provider-এর Managed Monitoring আছে — **AWS CloudWatch** (Metric, Log, Alarm, Trace — X-Ray), **Azure Monitor** (Metric, Log, Application Insights), **Google Cloud Monitoring/Logging/Trace** (পূর্বে Stackdriver), **IBM Cloud Monitoring/Logging**। সাধারণ ধারণা একই: Provider-এর সেবা (যেমন VM, Database, Load Balancer) স্বয়ংক্রিয়ভাবে Metric পাঠায়; আপনি Log কেন্দ্রীয়ভাবে দেখেন; Alarm বসান। Managed সেবার সুবিধা:

চালানোর ঝামেলা নেই, স্কেল করে। সীমা: খরচ ব্যবহার অনুযায়ী, Vendor-এর টুলেই আটকে থাকা, ও নিজের Prometheus/Grafana-র মতো নমনীয়তা কম। অনেক দল Hybrid করে: Cloud-এর Managed Metric + নিজের Prometheus/Grafana/Tempo। এই অধ্যায়ে কোনো মূল্য, নির্দিষ্ট সুবিধার তালিকা বা উপলব্ধতার দাবি করা হয়নি — সেগুলো Provider-এর সরকারি ডকুমেন্টেশন (official docs) থেকে যাচাই করুন, কারণ তা প্রায়ই বদলায়।

পরিবেশ	Metric উৎস	Log উৎস	মনে রাখার কথা
Docker	cAdvisor, docker stats	stdout → Logging Driver	Container মুছলে Log হারায়
Kubernetes	kubelet, kube-state-metrics	Pod Log → কেন্দ্রীয় সংগ্রহ	Pod ক্ষণস্থায়ী
Cloud (AWS/Azure/GCP/IBM)	Managed Monitoring	Cloud Log সেবা	Official Docs যাচাই করুন

২৩.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# Docker — Container-এর তাৎক্ষণিক অবস্থা (নমুনা আউটপুট — চিত্রণমূলক)
docker stats --no-stream
# --no-stream : একবার দেখিয়ে থামা; CPU %, Memory ব্যবহার, Network I/O দেখায়
docker logs --tail 50 technova-api
# --tail 50 : শেষ ৫০ লাইন; Container-এর Log দেখা
docker inspect technova-api --format '{{.State.Health.Status}}'
# Healthcheck-এর অবস্থা: healthy / unhealthy / starting
```

```
# Kubernetes — Pod ও Node-এর অবস্থা (নমুনা আউটপুট — চিত্রণমূলক)
kubectl top pods -n technova
# Pod-এর CPU/Memory ব্যবহার (Metrics Server থাকতে হয়)
kubectl top nodes
kubectl get pods -n technova -o wide
kubectl logs -n technova deployment/technova-api --tail=50
# Pod মুছে গেলে Log হারায় — তাই কেন্দ্রীয় Log সংগ্রহ (যেমন Loki) দরকার
```

২৩.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

Prometheus Operator-এ ServiceMonitor-ধরনের YAML — TechNova API-র Metric Scrape করার ঘোষণা:

```
# servicemonitor-technova.yaml — Prometheus Operator (Kubernetes)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: technova-api
  namespace: technova
  labels: { app: technova-api }
spec:
  selector: { matchLabels: { app: technova-api } }  # 1. কোন Service-এর Metric নেবে
  namespaceSelector: { matchNames: ["technova"] }   # 2. কোন Namespace-এ খুঁজবে
  endpoints:
    - port: metrics   # 3. Service-এর কোন পোর্টে /metrics আছে
```

```
      interval: 30s   # 4. কতক্ষণ পরপর Scrape করবে
      path: /metrics  # 5. Metric-এর পথ
# এই YAML প্রয়োগ করলে Operator স্বয়ংক্রিয়ভাবে Prometheus কনফিগারেশনে Scrape যোগ করে
```

```
# Docker Compose — cAdvisor + Prometheus (ল্যাব, মূল অংশ)
services:
  cadvisor: { image: gcr.io/cadvisor/cadvisor:latest, ports: ["8080:8080"],
    volumes: ["/:/rootfs:ro", "/var/run:/var/run:ro", "/sys:/sys:ro", "/var/lib/docker/:/var/
lib/docker:ro"] }
  prometheus: { image: prom/prometheus:latest, ports: ["9090:9090"],
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml"] }
# prometheus.yml-এ scrape_configs-এ cadvisor (203.0.113.10:8080) যোগ করুন
```

২৩.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova Kubernetes-এ যাওয়ার পর একটি Pod বারবার Restart হচ্ছিল — `CrashLoopBackOff`। হোস্টের Metric ঠিক ছিল। `kubectl get pods`-এ Restart সংখ্যা বাড়ছিল, `kubectl logs --previous` দিয়ে আগের (মৃত) Container-এর Log দেখা গেল: Memory Limit ছাড়িয়ে OOMKilled হয়েছে — অ্যাপের মেমরি সীমা (limit) খুব কম দেওয়া হয়েছিল। সমাধান: Limit বাড়ানো ও Memory Leak তদন্ত (অধ্যায় ২১)। শিক্ষা: Kubernetes-এ "Pod Restart" নিজেই একটি Alert-যোগ্য সংকেত — kube-state-metrics থেকে `kube_pod_container_status_restarts_total` বাড়লে Alert দিন। আরেকটি পরিস্থিতি: Cloud-এ Managed Database-এর ধীর Query — Cloud-এর Log সেবায় Query Log দেখে Index-এর অভাব ধরা পড়ল, নিজের সার্ভারে নয়।

> **WARNING:** Kubernetes-এ Pod-এর Log শুধু Pod-এ থাকে না — কেন্দ্রীয় Log সংগ্রহ না থাকলে তদন্তের প্রমাণ হারিয়ে যায়। প্রোডাকশনে যাওয়ার আগেই Log সংগ্রহ (Loki/Cloud Log) বসান, পরে নয়।

২৩.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. Docker-এ cAdvisor চালিয়ে Prometheus-এ Container Metric দেখুন।
2. একটি Container-এর Memory Limit কমিয়ে OOMKilled-এর আচরণ পর্যবেক্ষণ করুন (ল্যাব)।
3. Kubernetes ল্যাব Cluster-এ (kind/minikube) `kubectl top pods` চালান।
4. kube-state-metrics বসিয়ে Deployment-এর Desired বনাম Available Replica দেখুন।
5. ServiceMonitor YAML প্রয়োগ করে একটি Service-এর Metric Prometheus-এ আসা দেখুন।
6. একটি Cloud Provider-এর সরকারি ডকুমেন্টেশন খুলে তার Managed Monitoring-এর মূল পাতাটি পড়ুন ও নোট করুন — মূল্য বা সুবিধা অনুমান করবেন না।

২৩.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল:** শুধু Node Metric দেখে Pod-এর সমস্যা খোঁজা — Node ঠিক থাকলেও Pod-এর Limit-এ আটকে যেতে পারে।
- **ভুল ধারণা:** "Kubernetes-এ Log এমনিই থেকে যায়।" — Pod মুছলে Log হারায়; কেন্দ্রীয় সংগ্রহ দরকার।
- **ভুল:** Request/Limit না দেওয়া — Pod যেকোনো সম্পদ নিয়ে অন্য Pod-কে চাপে ফেলে।
- **ভুল:** Cloud-এর Managed টুলের মূল্য বা সুবিধা অনুমান করা — Official Docs যাচাই না করে সিদ্ধান্ত নেবেন না।
- **ভুল:** Control Plane Metric না দেখা — API Server ধীর হলে পুরো Cluster-এর Deploy থেমে যায়।

২৩.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Kubernetes Metric-এ Pod নাম, Namespace ও Label থাকে — তাতে সংবেদনশীল তথ্য রাখবেন না। kube-state-metrics ও Prometheus-এর প্রবেশাধিকার সীমিত রাখুন — Cluster-এর ভেতরের তথ্য বাইরে যাবে না। Cloud Log-এ ব্যক্তিগত তথ্য যেতে পারে — Log-এ Redaction ও Retention নীতি Cloud-এও প্রযোজ্য। Container Image-এ Secret রাখবেন না — Secret Manager বা Kubernetes Secret ব্যবহার করুন (প্লেসহোল্ডার: CHANGE-ME-...)।

২৩.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Kubernetes-এ Metric-এর পরিমাণ Pod সংখ্যার সঙ্গে বাড়ে — High Cardinality Label (যেমন Pod ID প্রতিটি Metric-এ) Prometheus Storage বাড়ায়। kube-state-metrics হালকা, কিন্তু বড় Cluster-এ Scrape Interval ঠিক করুন। Cloud Managed Monitoring-এর খরচ Metric/Log-এর পরিমাণ অনুযায়ী হয় — Sampling ও Retention দিয়ে নিয়ন্ত্রণ করুন; নির্দিষ্ট মূল্য এখানে দেওয়া হয়নি, Provider-এর Official Docs দেখুন।

২৩.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Pod বারবার Restart	OOMKilled / Liveness Probe ব্যর্থ	kubectl describe pod; logs --previous দেখুন
Metric আসে না	ServiceMonitor Selector মেলেনি	Service Label ও ServiceMonitor Selector মিলিয়ে দেখুন
kubectl top কাজ করে না	Metrics Server নেই	Metrics Server ইনস্টল আছে কি না দেখুন
Pod-এর Log পাওয়া যায় না	Pod মুছে গেছে, কেন্দ্রীয় Log নেই	কেন্দ্রীয় Log সংগ্রহ বসান
Cloud Metric দেখা যায় না	অনুমতি (IAM/Role) নেই	Provider-এর Docs অনুযায়ী অনুমতি যাচাই করুন

২৩.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি Container-এ Request/Limit দিন; কেন্দ্রীয় Log সংগ্রহ বসান; Pod Restart-এ Alert দিন; Control Plane, Node ও Pod — তিন স্তরই দেখুন; Cloud সিদ্ধান্ত Official Docs দিয়ে যাচাই করুন। **অ্যান্টি-প্যাটার্ন:** শুধু হোস্ট Metric দেখা; Pod Log-এর ওপর নির্ভর করা; Limit না দেওয়া; Cloud মূল্য অনুমান করা।

২৩.১৬ অধ্যায়ের সারাংশ

Docker-এ cAdvisor ও কেন্দ্রীয় Log; Kubernetes-এ Control Plane, Node ও Pod — তিন স্তরের Metric, kube-state-metrics ও Prometheus Operator-এর ServiceMonitor; Cloud-এ Managed Monitoring, Log ও Tracing — উচ্চ-স্তরে, Official Docs-ভিত্তিক। Pod ক্ষণস্থায়ী — তাই কেন্দ্রীয় সংগ্রহই Observability-র ভিত্তি।

২৩.১৭ মূল প্রযুক্তিগত পরিভাষা

cAdvisor: Container-এর CPU/Memory/Network মাপার টুল।

kube-state-metrics: Kubernetes Object-এর অবস্থা Metric আকারে দেওয়ার সেবা।

Prometheus Operator: Kubernetes-এ Prometheus চালানো ও কনফিগার করার Operator।

ServiceMonitor: কোন Service-এর Metric Scrape করতে হবে তা ঘোষণা করা Custom Resource।

Control Plane: Kubernetes Cluster পরিচালনাকারী উপাদান (API Server, Scheduler, etcd)।

kubelet: প্রতিটি Node-এ চলা Agent, Pod চালায় ও রিপোর্ট করে।

OOMKilled: Memory Limit ছাড়ালে Container বন্ধ হয়ে যাওয়া।

CrashLoopBackOff: Pod বারবার Crash করে Restart-এর অপেক্ষায় থাকা অবস্থা।

Managed Monitoring: Cloud Provider-এর পরিচালিত পর্যবেক্ষণ সেবা।

Metrics Server: Kubernetes-এ `kubectl top` -এর জন্য Metric সরবরাহকারী।

Liveness/Readiness Probe: Pod চলছে কি না ও ট্রাফিক নেওয়ার উপযুক্ত কি না তা পরীক্ষা।

Request/Limit: Pod-এর জন্য চাওয়া ও সর্বোচ্চ সম্পদ (CPU/Memory)।

২৩.১৮ জ্ঞান যাচাই — প্রশ্ন

1. Docker-এ Container মুছে গেলে তার Log কেন হারিয়ে যায়? সমাধান কী?
2. Kubernetes-এ Control Plane, Node ও Pod — তিন স্তরে কী কী দেখতে হয়?
3. kube-state-metrics কী মাপে? এটি অ্যাপের Metric থেকে কীভাবে আলাদা?
4. ServiceMonitor কী করে? হাতে Prometheus কনফিগারেশন লেখার চেয়ে এর সুবিধা কী?
5. Pod বারবার Restart হচ্ছে — প্রথম তিনটি পরীক্ষা কী?

6. Cloud Managed Monitoring-এর সুবিধা ও সীমা কী?
7. Request/Limit না দিলে কী সমস্যা হয়?
8. Cloud-এর মূল্য বা সুবিধা নিয়ে সিদ্ধান্ত নেওয়ার আগে কী যাচাই করবেন?

২৩.১৯ ব্যবহারিক অনুশীলন

1. cAdvisor + Prometheus দিয়ে একটি Container-এর CPU/Memory Graph বানান।
2. kind/minikube-তে একটি Deployment চালিয়ে `kubectl top pods` ও Restart সংখ্যা দেখুন।
3. ServiceMonitor YAML লিখে একটি Service-এর Metric Scrape করুন।
4. একটি Pod-এর Memory Limit কমিয়ে OOMKilled পর্যবেক্ষণ করুন ও কারণ লিখুন।
5. একটি Cloud Provider-এর Official Docs থেকে Managed Monitoring-এর মূল ধারণা নোট করুন।
6. Pod Restart-এর জন্য একটি Alert Rule-এর খসড়া লিখুন।

অধ্যায় ২৪ • PART V — Production Observability Engineering

Chapter 24 — Enterprise Observability Architecture, Reliability, and Production Operations

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: **SLI/SLO** নির্বাচন ও **Error Budget** ব্যাখ্যা করতে; **Golden Signals**, **RED** ও **USE** পদ্ধতি প্রয়োগ করতে; Retention, খরচ, Sampling ও Data Privacy নীতি বানাতে; Observability প্ল্যাটফর্মের নিজের High Availability ও ব্যর্থতার ধরন বিশ্লেষণ করতে; এবং Runbook ও Governance দিয়ে প্রোডাকশন অপারেশন সাজাতে।

২৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

ছোট সিস্টেমে Prometheus + Grafana বসালেই হয়; Enterprise-এ প্রশ্নগুলো বদলে যায়: শত শত সার্ভিসের Telemetry কোথায় যাবে, কতদিন রাখা হবে, খরচ কে দেবে, কে কোন ডেটা দেখতে পারবে, আর পর্যবেক্ষণ ব্যবস্থা নিজে নষ্ট হলে কী হবে? TechNova বড় হচ্ছে — এখনই এই নকশা না করলে এক বছর পর Telemetry-র খরচ সার্ভারের খরচ ছাড়িয়ে যাবে, আর Incident-এর সময় Dashboard-ই খুলবে না। এই অধ্যায় Volume 10-এর সমাপ্তি: সবকিছু একত্র করে একটি টেকসই Enterprise আর্কিটেকচার।

২৪.৩ পূর্বশর্ত জ্ঞান

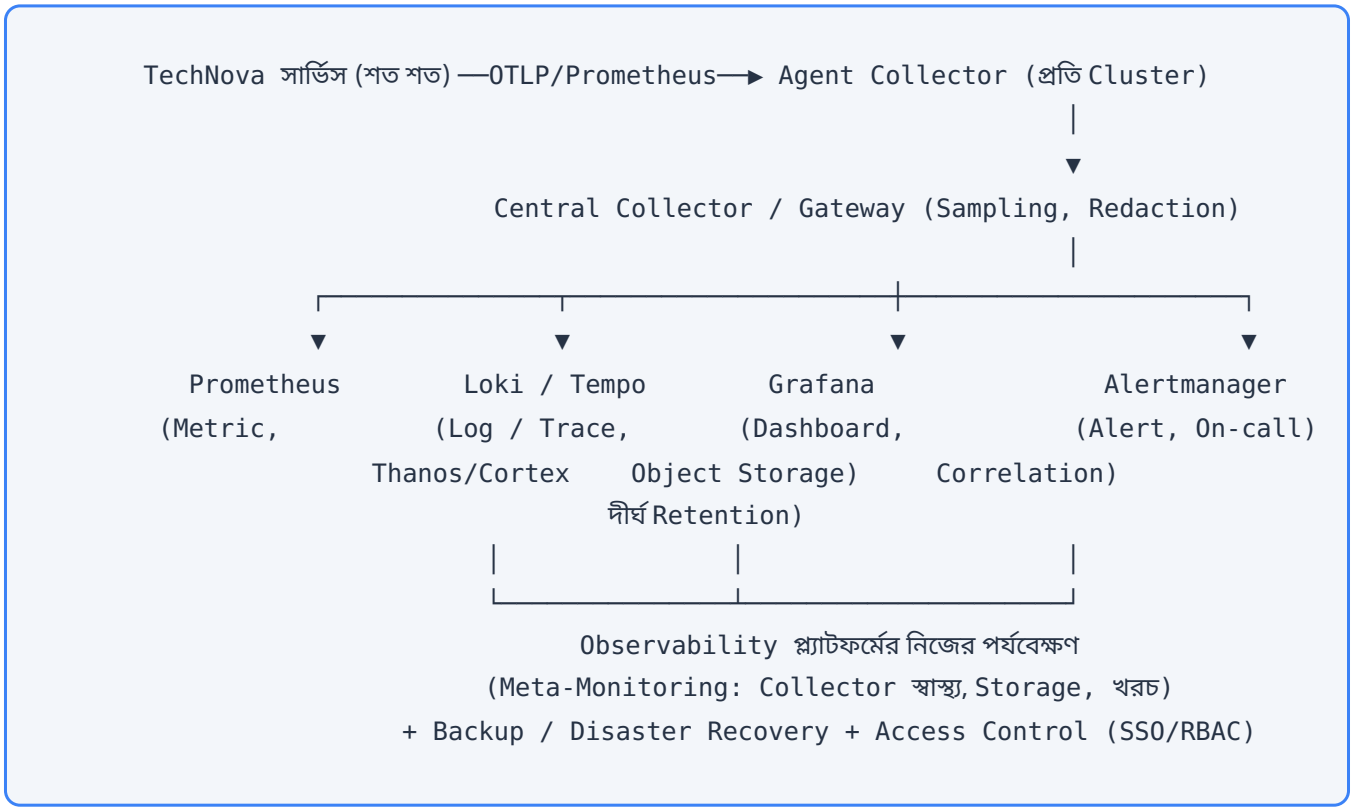
Volume 10-এর আগের অধ্যায়গুলো — Metric, Log, Trace, Alerting, APM, Kubernetes Observability; SLI/SLO-র নাম শুনে থাকা; বাজেট ও Retention-এর সাধারণ ধারণা। এন্টারপ্রাইজ অভিজ্ঞতা লাগবে না।

২৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

প্রথম নীতি: Observability-ও একটি সেবা — তার নিজের নির্ভরযোগ্যতা, খরচ ও গ্রাহক আছে। গ্রাহক হলো Developer ও SRE; তাদের প্রশ্ন: "সেবা ঠিক আছে কি?" উত্তর দেওয়ার জন্য SLI (Service Level Indicator — কী মাপবেন, যেমন সফল অনুরোধের হার), SLO (Service Level Objective — লক্ষ্য, যেমন ৯৯.৯% সফল), ও Error Budget (কতটুকু ব্যর্থতা সহ্য করা যায়, যেমন ০.১%) — এই তিনটি দরকার। Error Budget শেষ হয়ে গেলে নতুন ফিচার থামিয়ে নির্ভরযোগ্যতায় কাজ করতে হয় — এটি ব্যবসা ও ইঞ্জিনিয়ারিং-এর মধ্যে চুক্তি। আর Observability প্ল্যাটফর্ম নিজে ব্যর্থ হলে আপনি অন্ধ হয়ে যান — তাই তার নিজের High Availability, Backup ও ব্যর্থতার পরিকল্পনা (failure mode) থাকতে হয়।

> **CONCEPT:** SLI হলো থার্মোমিটার (কী মাপছেন), SLO হলো সুস্থতার সীমা (কতটুকু জ্বর সহ্য করবেন), Error Budget হলো অসুস্থতার ছুটি (কতদিন খারাপ থাকা যাবে)। ছুটি শেষ হলে বিশ্রাম নিতে হয় — নতুন কাজ নয়।

২৪.৫ অভ্যন্তরীণ আর্কিটেকচার ও চিত্র



২৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

SLI/SLO নির্বাচন ও Error Budget

SLI বেছে নিন ব্যবহারকারীর দৃষ্টিকোণ থেকে: Availability (সফল অনুরোধের হার), Latency (p95 কত দ্রুত), Freshness (ডেটা কত নতুন), Correctness (উত্তর সঠিক কি না)। ভালো SLI-র উদাহরণ: "Checkout API-র ৯৯.৯% অনুরোধ ৫০০ মিলিসেকেন্ডের মধ্যে সফল হবে।" **SLO** হলো সেই লক্ষ্যের সংখ্যা (৯৯.৯%) ও সময়সীমা (৩০ দিন)। **Error Budget** = ১০০% − SLO = ০.১% — এই সময়ে ব্যর্থতা অনুমোদিত। Budget শেষ হলে Feature Freeze: নতুন ডিপ্লয় থামিয়ে নির্ভরযোগ্যতা ঠিক করা হয়। Budget বেশি থাকলে দ্রুত ফিচার দেওয়া যায়। SLO খুব বেশি (৯৯.৯৯৯%) রাখলে খরচ আকাশ ছোঁবে — ব্যবসার প্রয়োজনে ঠিক করুন, প্রতিযোগিতায় নয়।

Golden Signals, RED ও USE

Golden Signals (Google SRE): Latency, Traffic, Errors, Saturation — যেকোনো সেবার স্বাস্থ্যের চারটি সংকেত। **RED** (Request-ভিত্তিক সেবায়): Rate (প্রতি সেকেন্ডে অনুরোধ), Errors (ব্যর্থতার হার), Duration (সময়কাল)। **USE** (সম্পদ-ভিত্তিক, যেমন ডিস্ক/সার্ভার): Utilization (কতটুকু ব্যবহৃত), Saturation (কতটুকু অপেক্ষায়), Errors (ত্রুটি)। Dashboard বানানোর সময়: সেবার জন্য RED, সম্পদের জন্য USE, আর সব মিলিয়ে Golden Signals — এই তিনটি কাঠামো মনে রাখলে Dashboard এলোমেলো হয় না।

Retention Strategy ও খরচ ব্যবস্থাপনা

সব ডেটা সমান দামে রাখবেন না: **Hot** (গত ৭ দিন, দ্রুত খোঁজা যায়), **Warm** (৩০ দিন, ধীর), **Cold** (৯০ দিন+, Object Storage-এ সস্তা, প্রয়োজনে ফেরানো যায়)। Metric-এ Downsampling: পুরোনো Metric-এর বিস্তারিত কমিয়ে রাখা (যেমন ১ মিনিটের বদলে ১ ঘণ্টার গড়)। খরচের প্রধান চালক: Cardinality (Label-এর সংখ্যা), Sampling হার, ও Retention —

এই তিনটি নিয়ন্ত্রণ করুন। প্রতি দলকে তাদের Telemetry খরচ দেখান (Showback) — তাহলে অপ্রয়োজনীয় Metric নিজেরাই কমায়।

Sampling, Privacy ও Redaction

Sampling নীতি লিখিত রাখুন: সাধারণ Trace ১০% Head, Error ও ধীর Trace ১০০% Tail। **Data Privacy:** Log/Trace-এ ব্যক্তিগত তথ্য (ইমেইল, ফোন, ঠিকানা, স্বাস্থ্য তথ্য) ও Secret (Token, Password) আসতে পারে — Collector-এ Redaction Processor দিয়ে মুছে ফেলুন, সংরক্ষণের আগেই। Retention-এর পর ডেটা মুছে ফেলার নীতি রাখুন। কোন দেশের ডেটা কোথায় সংরক্ষিত হয় (Data Residency) — তা আইনি প্রয়োজন হতে পারে।

Access Control ও Governance

কে কোন ডেটা দেখতে পারে: SSO (Single Sign-On) দিয়ে প্রমাণীকরণ, RBAC (Role-Based Access Control) দিয়ে ভূমিকা — Developer শুধু নিজের দলের সার্ভিস, SRE সব, বাইরের Vendor কিছুই না। Tenant-ভিত্তিক বিচ্ছিন্নতা পরীক্ষা করুন। Governance মানে লিখিত নীতি: Naming Convention (Metric/Label-এর নাম), Dashboard-এর মালিক, Alert-এর মালিক, Retention নীতি, ও নিয়মিত পর্যালোচনা। নীতি না থাকলে প্রতিটি দল আলাদা নামে Metric বানায় — খুঁজে পাওয়া যায় না।

Observability প্ল্যাটফর্মের নিজের নির্ভরযোগ্যতা

পর্যবেক্ষণ ব্যবস্থা নষ্ট হলে আপনি অন্ধ — তাই তার নিজের High Availability দরকার: Prometheus-এর Replica, Collector-এর একাধিক Instance, Storage-এর Replication, ও Grafana-র Backup। **Meta-Monitoring:** Observability প্ল্যাটফর্মের নিজের Metric পর্যবেক্ষণ করা — Collector-এর Dropped Span, Prometheus-এর Storage, Grafana-র Uptime। **ব্যর্থতার ধরন (Failure Mode):** Collector বন্ধ হলে অ্যাপ থামবে না (Buffer/Drop করবে); Prometheus বন্ধ হলে Alert থামবে — তাই Alertmanager-এর জন্য আলাদা স্বাস্থ্য পরীক্ষা; Grafana বন্ধ হলে Dashboard দেখা যাবে না — কিন্তু Alert চলবে (Alertmanager আলাদা)। প্রতিটি ব্যর্থতার জন্য Runbook লিখুন: "Prometheus বন্ধ হলে কী করবেন" — Incident-এর সময় তা খুঁজতে হবে না।

পদ্ধতি	কী মাপে	কখন ব্যবহার করবেন
Golden Signals	Latency, Traffic, Errors, Saturation	যেকোনো সেবার সামগ্রিক স্বাস্থ্য
RED	Rate, Errors, Duration	Request-ভিত্তিক সেবা (API)
USE	Utilization, Saturation, Errors	সম্পদ (ডিস্ক, CPU, নেটওয়ার্ক)
SLI/SLO	ব্যবহারকারীর অভিজ্ঞতা	লক্ষ্য ও Error Budget ঠিক করতে

২৪.৭ কনফিগারেশন ও কমান্ড — লাইন ধরে ধরে ব্যাখ্যা

```
# SLO-ভিত্তিক Alert — Error Budget শেষ হওয়ার আগেই সতর্কতা (PromQL, ধারণাগত)
# SLI: সফল অনুরোধের হার; SLO: 99.9% (30 দিন); Error Budget: 0.1%
# Multi-window Alert: ১ ঘণ্টায় Error হার বেশি + ৬ ঘণ্টায়ও বেশি — তবেই Alert
# expr (সংক্ষেপ): sum(rate(http_requests_total{status!~"5.."}[1h])) /
sum(rate(http_requests_total[1h])) < 0.999
# for: 5m — ক্ষণিকের স্পাইকে নয়; Severity: sev-2 (Budget দ্রুত শেষ হচ্ছে)
# এই ধরনের Alert-কে Burn Rate Alert বলে — Budget কত দ্রুত পুড়ছে তা মাপে
```

```
# Prometheus Retention ও Storage পরীক্ষা (নমুনা আউটপুট — চিত্রণমূলক)
# prometheus --storage.tsdb.retention.time=30d --storage.tsdb.retention.size=50GB
# retention.time: কতদিন রাখবে; retention.size: সর্বোচ্চ আকার — যেটি আগে শেষ হয়
df -h /var/lib/prometheus   # Storage কতটুকু ব্যবহৃত হচ্ছে দেখুন
# (নমুনা আউটপুট — চিত্রণমূলক)
# /dev/sda1  100G  62G  38G  62% /var/lib/prometheus
```

২৪.৮ সম্পূর্ণ কনফিগারেশন উদাহরণ

TechNova-র Enterprise Retention ও Redaction নীতির উদাহরণ (Collector Processor + Prometheus): Collector-এ `attributes/redact` দিয়ে `http.request.header.authorization`, `user.email`, `db.statement`-এর সংবেদনশীল অংশ মোছা; Prometheus-এ Retention ৩০ দিন, Thanos/Cortex-ধরনের দীর্ঘমেয়াদি Storage-এ Object Storage; Loki-তে Log Retention ১৪ দিন, Tempo-তে Trace Retention ৭ দিন (Error Trace ৩০ দিন)। এই সংখ্যাগুলো উদাহরণ — নিজের খরচ ও আইনি প্রয়োজন অনুযায়ী ঠিক করুন, এবং নীতিটি লিখিত Runbook-এ রাখুন।

২৪.৯ বাস্তব প্রোডাকশন পরিস্থিতি

TechNova-র Prometheus একদিন বন্ধ হয়ে গেল — ডিস্ক পূর্ণ। Alertmanager-ও Prometheus-এর ওপর নির্ভরশীল বলে Alert থামল; Grafana Dashboard খালি। সৌভাগ্যবশত সেবা চলছিল, কিন্তু দল অন্ধ ছিল ২ ঘণ্টা। Postmortem-এ তিনটি শিক্ষা: (১) Prometheus-এর Storage-এর জন্য আলাদা Alert — কিন্তু সেটি অন্য Prometheus (Meta-Monitoring) থেকে আসতে হবে, নিজের থেকে নয়; (২) Retention Size সীমা দেওয়া — ডিস্ক পূর্ণ হওয়ার আগেই পুরোনো ডেটা মুছে যাবে; (৩) Grafana-র জন্য Read Replica — Dashboard দেখা বন্ধ হবে না। এরপর থেকে Observability প্ল্যাটফর্মের নিজের স্বাস্থ্য Dashboard-এ প্রথমে দেখা হয় — সেবার Dashboard-এর আগে।

> **REAL WORLD:** "আমাদের Monitoring ঠিক আছে" — এই দাবি যাচাই করুন: Prometheus বন্ধ করে দেখুন Alert আসে কি না। যদি না আসে, তবে Monitoring-এর Monitoring নেই — সবচেয়ে বিপজ্জনক অবস্থা, কারণ আপনি জানেন না আপনি অন্ধ।

২৪.১০ ধাপে ধাপে ব্যবহারিক বাস্তবায়ন

1. একটি সেবার জন্য SLI বেছে নিন (যেমন Checkout সফল হার) ও SLO লিখুন (৯৯.৯%, ৩০ দিন)।
2. Error Budget হিসাব করুন ও Burn Rate Alert-এর খসড়া লিখুন।
3. একটি Dashboard-কে Golden Signals / RED দিয়ে পুনরায় সাজান।
4. Retention নীতি লিখুন: Metric ৩০ দিন, Log ১৪ দিন, Trace ৭ দিন — ও কেন।
5. Collector-এ একটি Redaction Rule যোগ করে সংবেদনশীল Attribute মুছুন।
6. Observability প্ল্যাটফর্মের নিজের স্বাস্থ্য Dashboard বানান (Collector, Prometheus Storage, Grafana Uptime)।
7. "Prometheus বন্ধ হলে কী করবেন" — একটি Runbook লিখুন।

২৪.১১ সাধারণ ভুল ও ভুল ধারণা

- **ভুল ধারণা:** "SLO ৯৯.৯৯৯% রাখলেই ভালো।" — খরচ আকাশ ছোঁবে, আর ব্যবহারকারী পার্থক্য টের পাবেন না। ব্যবসার প্রয়োজনে SLO ঠিক করুন।

- **ভুল:** সব ডেটা একই Retention-এ রাখা — খরচ বাড়ে, দরকারি ডেটা খুঁজে পাওয়া যায় না।
- **ভুল:** Observability প্ল্যাটফর্মের নিজের পর্যবেক্ষণ না করা — অন্ধ হয়ে যাওয়া।
- **ভুল:** Error Budget শেষ হলেও ফিচার চালিয়ে যাওয়া — চুক্তি ভাঙা, দল পুড়ে যায়।
- **ভুল:** High Cardinality Label (User ID) Metric-এ ব্যবহার — Storage ও খরচ বিস্ফোরিত হয়।

২৪.১২ নিরাপত্তা ও গোপনীয়তা বিবেচনা

Enterprise-এ Telemetry-তে ব্যক্তিগত তথ্য, ব্যবসায়িক গোপন তথ্য ও Secret থাকতে পারে। নীতি: সংগ্রহের আগেই Redact করুন; প্রবেশাধিকার RBAC দিয়ে সীমিত করুন; Audit Log রাখুন — কে কোন Trace/Log দেখল; Data Residency মেনে চলুন; Retention-এর পর ডেটা মুছে ফেলুন ও তা প্রমাণ করুন। Vendor-কে Telemetry পাঠালে তাদের Data Processing নীতি পড়ুন — আপনার ডেটা কোথায় যায়, তা জানা আপনার দায়িত্ব।

২৪.১৩ কর্মক্ষমতা ও খরচ বিবেচনা

Enterprise খরচের তিনটি চালক: Cardinality, Sampling, Retention। Cardinality কমান: Label-এ User ID, Order ID, IP রাখবেন না। Sampling ঠিক করুন: ১০% সাধারণ, ১০০% Error। Retention স্তরে ভাগ করুন: Hot/Warm/Cold। খরচ প্রতি দলে ভাগ করে দেখান (Showback) — অপ্রয়োজনীয় Metric কমে যাবে। Managed সেবা ব্যবহার করলে খরচ ব্যবহার অনুযায়ী — Provider-এর Official Docs থেকে যাচাই করুন, অনুমান নয়।

২৪.১৪ সমস্যা সমাধান (Troubleshooting)

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Prometheus ডিস্ক পূর্ণ	Retention বেশি, Cardinality বেশি	Retention Size দিন; High Cardinality Label খুঁজুন
Dashboard ধীর	ভারী Query, বেশি সময়সীমা	Recording Rule ব্যবহার করুন; সময়সীমা কমান
Alert আসে না, Prometheus বন্ধ	Meta-Monitoring নেই	আলাদা স্বাস্থ্য পরীক্ষা বসান
খরচ হঠাৎ বেড়েছে	নতুন High Cardinality Metric	কোন Metric বেড়েছে তা খুঁজুন; Label কমান
Trace-এ ব্যক্তিগত তথ্য	Redaction নেই	Collector-এ Redaction Processor যোগ করুন

TROUBLESHOOTING: খরচ বাড়লে প্রথমে Cardinality দেখুন: Prometheus-এ `topk(10, count by (__name__)({__name__=~".+"}))`-ধরনের Query দিয়ে কোন Metric সবচেয়ে বেশি Series তৈরি করছে তা খুঁজুন — সাধারণত একটি ভুল Label-ই দায়ী।

PRO TIP: প্রতি মাসে একটি "Observability Review" করুন: কোন Dashboard কেউ দেখে না, কোন Alert-এ কেউ সাড়া দেয় না, কোন Metric-এর খরচ বেশি — তিনটি তালিকা করে পরিষ্কার করুন। Observability-ও রক্ষণাবেক্ষণ চায়।

২৪.১৫ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ব্যবহারকারীর দৃষ্টিকোণ থেকে SLI; বাস্তবসম্মত SLO; Error Budget মেনে চলা; Hot/Warm/Cold Retention; সংগ্রহের আগেই Redaction; RBAC; Meta-Monitoring; প্রতিটি ব্যর্থতার Runbook; নিয়মিত Observability Review। **অ্যান্টি-প্যাটার্ন:** অবাস্তব SLO; সব ডেটা চিরকাল রাখা; High Cardinality Label; প্ল্যাটফর্মের নিজের পর্যবেক্ষণ না করা; নীতি ছাড়া Telemetry সংগ্রহ।

২৪.১৬ অধ্যায়ের সারাংশ

Enterprise Observability মানে SLI/SLO/Error Budget দিয়ে লক্ষ্য ঠিক করা, Golden Signals/RED/USE দিয়ে Dashboard সাজানো, Retention ও Sampling দিয়ে খরচ নিয়ন্ত্রণ করা, Redaction ও RBAC দিয়ে গোপনীয়তা রক্ষা করা, আর প্ল্যাটফর্মের নিজের High Availability ও Meta-Monitoring দিয়ে অন্ধ হওয়া এড়ানো। Volume 10-এর যাত্রা এখানেই শেষ — Metric, Log, Trace, Alert ও APM এখন একটি টেকসই ব্যবস্থা।

২৪.১৭ মূল প্রযুক্তিগত পরিভাষা

SLI: Service Level Indicator — সেবার মান মাপার সূচক, যেমন সফল অনুরোধের হার।

SLO: Service Level Objective — SLI-র লক্ষ্যমাত্রা, যেমন ৯৯.৯% সফলতা।

Error Budget: SLO-র অনুমোদিত ব্যর্থতার পরিমাণ (১০০% − SLO)।

Golden Signals: Latency, Traffic, Errors, Saturation — সেবার স্বাস্থ্যের চার সংকেত।

RED Method: Rate, Errors, Duration — Request-ভিত্তিক সেবার পর্যবেক্ষণ পদ্ধতি।

USE Method: Utilization, Saturation, Errors — সম্পদের পর্যবেক্ষণ পদ্ধতি।

Burn Rate: Error Budget কত দ্রুত শেষ হচ্ছে তার হার।

Retention: Telemetry ডেটা কতদিন সংরক্ষণ করা হবে।

Downsampling: পুরোনো Metric-এর বিস্তারিত কমিয়ে সংরক্ষণ করা।

Meta-Monitoring: পর্যবেক্ষণ ব্যবস্থার নিজের স্বাস্থ্য পর্যবেক্ষণ করা।

RBAC: Role-Based Access Control — ভূমিকা অনুযায়ী প্রবেশাধিকার।

Data Residency: ডেটা কোন ভৌগোলিক স্থানে সংরক্ষিত হয় তা নিয়ন্ত্রণ।

২৪.১৮ জ্ঞান যাচাই — প্রশ্ন

1. SLI, SLO ও Error Budget-এর সম্পর্ক কী? একটি উদাহরণ দিন।
2. Error Budget শেষ হলে কী করা উচিত? কেন?
3. Golden Signals-এর চারটি সংকেত কী কী?
4. RED ও USE পদ্ধতির পার্থক্য কী? কোনটি কখন ব্যবহার করবেন?
5. Hot/Warm/Cold Retention কী? এটি খরচ কীভাবে কমায়?
6. Meta-Monitoring কেন দরকার? Prometheus বন্ধ হলে কী হয়?
7. High Cardinality Label খরচ কেন বাড়ায়? উদাহরণ দিন।
8. Telemetry-তে ব্যক্তিগত তথ্য থাকলে সংগ্রহের আগে কী করবেন?

২৪.১৯ ব্যবহারিক অনুশীলন

1. TechNova Checkout-এর জন্য SLI, SLO (৯৯.৯%) ও Error Budget লিখুন।
2. একটি API-র Dashboard-কে RED পদ্ধতিতে পুনরায় সাজান।
3. Retention নীতি লিখুন: Metric/Log/Trace — প্রতিটির সময়সীমা ও কারণ।
4. Collector-এ Redaction Rule লিখে একটি সংবেদনশীল Attribute মুছুন।
5. Observability প্ল্যাটফর্মের নিজের স্বাস্থ্য Dashboard-এর খসড়া বানান।
6. "Prometheus বন্ধ হলে কী করবেন" — একটি Runbook লিখুন।
7. এক মাসের Observability Review-এর তালিকা (অব্যবহৃত Dashboard/Alert/Metric) বানানোর পরিকল্পনা লিখুন।